



Compute Express Link™ (CXL™)

Specification

August 2023

Revision 3.1

LEGAL NOTICE FOR THIS PUBLICLY-AVAILABLE SPECIFICATION FROM COMPUTE EXPRESS LINK CONSORTIUM, INC.

© 2019-2023 COMPUTE EXPRESS LINK CONSORTIUM, INC. ALL RIGHTS RESERVED.

This CXL Specification Revision 3.1 (this “**CXL Specification**” or this “**document**”) is owned by and is proprietary to Compute Express Link Consortium, Inc., a Delaware nonprofit corporation (sometimes referred to as “**CXL**” or the “**CXL Consortium**” or the “**Company**”) and/or its successors and assigns.

NOTICE TO USERS WHO ARE MEMBERS OF THE CXL CONSORTIUM:

If you are a Member of the CXL Consortium (sometimes referred to as a “**CXL Member**”), and even if you have received this publicly-available version of this CXL Specification after agreeing to CXL Consortium’s Evaluation Copy Agreement (a copy of which is available <https://www.computeexpresslink.org/download-the-specification>, each such CXL Member must also be in compliance with all of the following CXL Consortium documents, policies and/or procedures (collectively, the “**CXL Governing Documents**”) in order for such CXL Member’s use and/or implementation of this CXL Specification to receive and enjoy all of the rights, benefits, privileges and protections of CXL Consortium membership: (i) CXL Consortium’s Intellectual Property Policy; (ii) CXL Consortium’s Bylaws; (iii) any and all other CXL Consortium policies and procedures; and (iv) the CXL Member’s Participation Agreement.

NOTICE TO NON-MEMBERS OF THE CXL CONSORTIUM:

If you are **not** a CXL Member and have received this publicly-available version of this CXL Specification, your use of this document is subject to your compliance with, and is limited by, all of the terms and conditions of the CXL Consortium’s Evaluation Copy Agreement (a copy of which is available at <https://www.computeexpresslink.org/download-the-specification>).

In addition to the restrictions set forth in the CXL Consortium’s Evaluation Copy Agreement, any references or citations to this document must acknowledge the Compute Express Link Consortium, Inc.’s sole and exclusive copyright ownership of this CXL Specification. The proper copyright citation or reference is as follows: “© 2019-2023 COMPUTE EXPRESS LINK CONSORTIUM, INC. ALL RIGHTS RESERVED.” When making any such citation or reference to this document you are not permitted to revise, alter, modify, make any derivatives of, or otherwise amend the referenced portion of this document in any way without the prior express written permission of the Compute Express Link Consortium, Inc.

Except for the limited rights explicitly given to a non-CXL Member pursuant to the explicit provisions of the CXL Consortium’s Evaluation Copy Agreement which governs the publicly-available version of this CXL Specification, nothing contained in this CXL Specification shall be deemed as granting (either expressly or impliedly) to any party that is **not** a CXL Member: (i) any kind of license to implement or use this CXL Specification or any portion or content described or contained therein, or any kind of license in or to any other intellectual property owned or controlled by the CXL Consortium, including without limitation any trademarks of the CXL Consortium.; or (ii) any benefits and/or rights as a CXL Member under any CXL Governing Documents.

LEGAL DISCLAIMERS FOR ALL PARTIES:

THIS DOCUMENT AND ALL SPECIFICATIONS AND/OR OTHER CONTENT PROVIDED HEREIN IS PROVIDED ON AN “**AS IS**” BASIS. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, COMPUTE EXPRESS LINK CONSORTIUM, INC. (ALONG WITH THE CONTRIBUTORS TO THIS DOCUMENT) HEREBY DISCLAIM ALL REPRESENTATIONS, WARRANTIES AND/OR COVENANTS, EITHER EXPRESS OR IMPLIED, STATUTORY OR AT COMMON LAW, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, VALIDITY, AND/OR NON-INFRINGEMENT.

In the event this CXL Specification makes any references (including without limitation any incorporation by reference) to another standard’s setting organization’s or any other party’s (“**Third Party**”) content or work, including without limitation any specifications or standards of any such Third Party (“**Third Party Specification**”), you are hereby notified that your use or implementation of any Third Party Specification: (i) is not governed by any of the CXL Governing Documents; (ii) may require your use of a Third Party’s patents, copyrights or other intellectual property rights, which in turn may require you to independently obtain a license or other consent from that Third Party in order to have full rights to implement or use that Third Party Specification; and/or (iii) may be governed by the intellectual property policy or other policies or procedures of the Third Party which owns the Third Party Specification. Any trademarks or service marks of any Third Party which may be referenced in this CXL Specification is owned by the respective owner of such marks.

NOTICE TO ALL PARTIES REGARDING THE PCI-SIG UNIQUE VALUE PROVIDED IN THIS CXL SPECIFICATION:

NOTICE TO USERS: THE UNIQUE VALUE THAT IS PROVIDED IN THIS CXL SPECIFICATION IS FOR USE IN VENDOR DEFINED MESSAGE FIELDS, DESIGNATED VENDOR SPECIFIC EXTENDED CAPABILITIES, AND ALTERNATE PROTOCOL NEGOTIATION ONLY AND MAY NOT BE USED IN ANY OTHER MANNER, AND A USER OF THE UNIQUE VALUE MAY NOT USE THE UNIQUE VALUE IN A MANNER THAT (A) ALTERS, MODIFIES, HARMS OR DAMAGES THE TECHNICAL FUNCTIONING, SAFETY OR SECURITY OF THE PCI-SIG ECOSYSTEM OR ANY PORTION THEREOF, OR (B) COULD OR WOULD REASONABLY BE DETERMINED TO ALTER, MODIFY, HARM OR DAMAGE THE TECHNICAL FUNCTIONING, SAFETY OR SECURITY OF THE PCI-SIG ECOSYSTEM OR ANY PORTION THEREOF (FOR PURPOSES OF THIS NOTICE, “**PCI-SIG ECOSYSTEM**” MEANS THE PCI-SIG SPECIFICATIONS, MEMBERS OF PCI-SIG AND THEIR ASSOCIATED PRODUCTS AND SERVICES THAT INCORPORATE ALL OR A PORTION OF A PCI-SIG SPECIFICATION AND EXTENDS TO THOSE PRODUCTS AND SERVICES INTERFACING WITH PCI-SIG MEMBER PRODUCTS AND SERVICES).

Contents

1.0	Introduction	48
1.1	Audience	48
1.2	Terminology/Acronyms	48
1.3	Reference Documents	59
1.4	Motivation and Overview	60
1.4.1	CXL	60
1.4.2	Flex Bus	63
1.5	Flex Bus Link Features	65
1.6	Flex Bus Layering Overview	65
1.7	Document Scope	66
2.0	CXL System Architecture	69
2.1	CXL Type 1 Device	70
2.2	CXL Type 2 Device	70
2.2.1	Back-Invalidate Snoop Coherence for HDM-DB	71
2.2.2	Bias-based Coherency Model for HDM-D Memory	71
2.2.2.1	Host Bias	72
2.2.2.2	Device Bias	72
2.2.2.3	Mode Management	73
2.3	CXL Type 3 Device	74
2.4	Multi Logical Device (MLD)	75
2.4.1	LD-ID for CXL.io and CXL.mem	75
2.4.1.1	LD-ID for CXL.mem	75
2.4.1.2	LD-ID for CXL.io	75
2.4.2	Pooled Memory Device Configuration Registers	76
2.4.3	Pooled Memory and Shared FAM	77
2.4.4	Coherency Models for Shared FAM	77
2.5	Multi-Headed Device	79
2.5.1	LD Management in MH-MLDs	80
2.6	CXL Device Scaling	80
2.7	CXL Fabric	81
2.8	Global FAM (G-FAM) Type 3 Device	81
2.9	Manageability Overview	81
3.0	CXL Transaction Layer	83
3.1	CXL.io	83
3.1.1	CXL.io Endpoint	84
3.1.2	CXL Power Management VDM Format	84
3.1.2.1	Credit and PM Initialization	88
3.1.3	CXL Error VDM Format	89
3.1.4	Optional PCIe Features Required for CXL	90
3.1.5	Error Propagation	90
3.1.6	Memory Type Indication on ATS	91
3.1.7	Deferrable Writes	91
3.1.8	PBR TLP Header (PTH)	91
3.1.8.1	Transmitter Rules Summary	92
3.1.8.2	Receiver Rules Summary	92
3.1.9	VendPrefixLO	94
3.1.10	CXL DevLoad (CDL) Field in UIO Completions	95
3.1.11	CXL Fabric-related VDMs	95
3.1.11.1	Host Management Transaction Flows of GFD	97
3.1.11.2	Downstream Proxy Command (DPCmd) VDM	99
3.1.11.3	Upstream Command Pull (UCPull) VDM	100

3.1.11.4	Downstream Command Request (DCReq, DCReq-Last, DCReq-Fail) VDMs	101
3.1.11.5	Upstream Command Response (UCRsp, UCRsp-Last, UCRsp-Fail) VDMs	102
3.1.11.6	GFD Async Message (GAM) VDM	102
3.1.11.7	Route Table Update (RTUpdate) VDM	103
3.1.11.8	Route Table Update Response (RTUpdateAck, RTUpdateNak) VDMs	103
3.2	CXL.cache	104
3.2.1	Overview	104
3.2.2	CXL.cache Channel Description	105
3.2.2.1	Channel Ordering	105
3.2.2.2	Channel Crediting	105
3.2.3	CXL.cache Wire Description	106
3.2.3.1	D2H Request	106
3.2.3.2	D2H Response	107
3.2.3.3	D2H Data	108
3.2.3.4	H2D Request	108
3.2.3.5	H2D Response	109
3.2.3.6	H2D Data	110
3.2.4	CXL.cache Transaction Description	110
3.2.4.1	Device-attached Memory Flows for HDM-D/HDM-DB	110
3.2.4.2	Device to Host Requests	111
3.2.4.3	Device to Host Response	123
3.2.4.4	Host to Device Requests	124
3.2.4.5	Host to Device Response	125
3.2.5	Cacheability Details and Request Restrictions	127
3.2.5.1	GO-M Responses	127
3.2.5.2	Device/Host Snoop-GO-Data Assumptions	127
3.2.5.3	Device/Host Snoop/WritePull Assumptions	127
3.2.5.4	Snoop Responses and Data Transfer on CXL.cache Evicts	128
3.2.5.5	Multiple Snoops to the Same Address	128
3.2.5.6	Multiple Reads to the Same Cacheline	128
3.2.5.7	Multiple Evicts to the Same Cacheline	128
3.2.5.8	Multiple Write Requests to the Same Cacheline	128
3.2.5.9	Multiple Read and Write Requests to the Same Cacheline	128
3.2.5.10	Normal Global Observation (GO)	129
3.2.5.11	Relaxed Global Observation (FastGO)	129
3.2.5.12	Evict to Device-attached Memory	129
3.2.5.13	Memory Type on CXL.cache	129
3.2.5.14	General Assumptions	129
3.2.5.15	Buried Cache State Rules	130
3.2.5.16	H2D Req Targeting Device-attached Memory	131
3.3	CXL.mem	132
3.3.1	Introduction	132
3.3.2	CXL.mem Channel Description	133
3.3.2.1	Direct P2P CXL.mem for Accelerators	134
3.3.2.2	Snoop Handling with Direct P2P CXL.mem	134
3.3.3	Back-Invalidate Snoop	135
3.3.4	QoS Telemetry for Memory	136
3.3.4.1	QoS Telemetry Overview	136
3.3.4.2	Reference Model for Host/Peer Support of QoS Telemetry	137
3.3.4.3	Memory Device Support for QoS Telemetry	138
3.3.5	M2S Request (Req)	148
3.3.6	M2S Request with Data (RwD)	152
3.3.6.1	Trailer Present for RwD (256B Flit)	154
3.3.7	M2S Back-Invalidate Response (BIRsp)	154
3.3.8	S2M Back-Invalidate Snoop (BISnp)	155

3.3.8.1	Rules for Block Back-Invalidate Snoops	156
3.3.9	S2M No Data Response (NDR)	157
3.3.10	S2M Data Response (DRS)	158
3.3.10.1	Trailer Present for DRS (256B Flit)	159
3.3.11	Forward Progress and Ordering Rules	160
3.3.11.1	Buried Cache State Rules for HDM-D/HDM-DB	160
3.4	Transaction Ordering Summary	162
3.5	Transaction Flows to Device-attached Memory	165
3.5.1	Flows for Back-Invalidate Snoops on CXL.mem	165
3.5.1.1	Notes and Assumptions	165
3.5.1.2	BISnp Blocking Example	166
3.5.1.3	Conflict Handling	167
3.5.1.4	Block Back-Invalidate Snoops	169
3.5.2	Flows for Type 1 Devices and Type 2 Devices	171
3.5.2.1	Notes and Assumptions	171
3.5.2.2	Requests from Host	172
3.5.2.3	Requests from Device in Host and Device Bias	179
3.5.3	Type 2 Memory Flows and Type 3 Memory Flows	184
3.5.3.1	Speculative Memory Read	184
3.6	Flows to HDM-H in a Type 3 Device	185
4.0	CXL Link Layers	187
4.1	CXL.io Link Layer	187
4.2	CXL.cache and CXL.mem 68B Flit Mode Common Link Layer	188
4.2.1	Introduction	188
4.2.2	High-Level CXL.cachemem Flit Overview	190
4.2.3	Slot Format Definition	197
4.2.3.1	H2D and M2S Formats	198
4.2.3.2	D2H and S2M Formats	203
4.2.4	Link Layer Registers	208
4.2.5	68B Flit Packing Rules	208
4.2.6	Link Layer Control Flit	210
4.2.7	Link Layer Initialization	214
4.2.8	CXL.cachemem Link Layer Retry	215
4.2.8.1	LLR Variables	215
4.2.8.2	LLCRD Forcing	217
4.2.8.3	LLR Control Flits	218
4.2.8.4	RETRY Framing Sequences	219
4.2.8.5	LLR State Machines	220
4.2.8.6	Interaction with vLSM Retrain State	224
4.2.8.7	CXL.cachemem Flit CRC	224
4.2.9	Viral	226
4.3	CXL.cachemem Link Layer 256B Flit Mode	226
4.3.1	Introduction	226
4.3.2	Flit Overview	226
4.3.3	Slot Format Definition	231
4.3.3.1	Implicit Data Slot Decode	245
4.3.3.2	Trailer Decoder	246
4.3.4	256B Flit Packing Rules	247
4.3.5	Credit Return	249
4.3.6	Link Layer Control Messages	252
4.3.6.1	Link Layer Initialization	254
4.3.6.2	Viral Injection and Containment	254
4.3.6.3	Late Poison	255
4.3.6.4	Link Integrity and Data Encryption (IDE)	256
4.3.7	Credit Return Forcing	256
4.3.8	Latency Optimizations	256

4.3.8.1	Empty Flit	257
5.0	CXL ARB/MUX.....	258
5.1	vLSM States	259
5.1.1	Additional Rules for Local vLSM Transitions.....	262
5.1.2	Rules for vLSM State Transitions across Link.....	262
5.1.2.1	General Rules	262
5.1.2.2	Entry to Active Exchange Protocol	263
5.1.2.3	Status Synchronization Protocol	263
5.1.2.4	State Request ALMP	265
5.1.2.5	L0p Support	270
5.1.2.6	State Status ALMP.....	272
5.1.2.7	Unexpected ALMPs (68B Flit Mode Only).....	273
5.1.3	Applications of the vLSM State Transition Rules for 68B Flit Mode	274
5.1.3.1	Initial Link Training	274
5.1.3.2	Status Exchange Snapshot Example	277
5.1.3.3	L1 Abort Example.....	278
5.2	ARB/MUX Link Management Packets	279
5.2.1	ARB/MUX Bypass Feature.....	282
5.3	Arbitration and Data Multiplexing/Demultiplexing.....	282
6.0	Flex Bus Physical Layer	283
6.1	Overview	283
6.2	Flex Bus.CXL Framing and Packet Layout.....	284
6.2.1	Ordered Set Blocks and Data Blocks	284
6.2.2	68B Flit Mode	285
6.2.2.1	Protocol ID[15:0].....	285
6.2.2.2	x16 Packet Layout.....	286
6.2.2.3	x8 Packet Layout	289
6.2.2.4	x4 Packet Layout	291
6.2.2.5	x2 Packet Layout	291
6.2.2.6	x1 Packet Layout	291
6.2.2.7	Special Case: CXL.io — When a TLP Ends on a Flit Boundary ..	292
6.2.2.8	Framing Errors.....	292
6.2.3	256B Flit Mode	293
6.2.3.1	256B Flit Format	294
6.2.3.2	CRC Corruption for Containment with 256B Flits	300
6.3	256B Flit Mode Retry Buffers.....	301
6.4	Link Training.....	301
6.4.1	PCIe Mode vs. Flex Bus.CXL Mode Selection.....	301
6.4.1.1	Hardware Autonomous Mode Negotiation.....	302
6.4.1.2	Virtual Hierarchy vs. Restricted CXL Device Negotiation	307
6.4.1.3	256B Flit Mode.....	308
6.4.1.4	Flit Mode and VH Negotiation	309
6.4.1.5	Flex Bus.CXL Negotiation with Maximum Supported Link Speed of 8 GT/s or 16 GT/s	310
6.4.1.6	Link Width Degradation and Speed Downgrade	310
6.5	68B Flit Mode: Recovery.Idle and Config.Idle Transitions to L0	310
6.6	L1 Abort Scenario.....	310
6.7	68B Flit Mode: Exit from Recovery	311
6.8	Retimers and Low Latency Mode.....	311
6.8.1	68B Flit Mode: SKP Ordered Set Frequency and L1/Recovery Entry	312
7.0	Switching	315
7.1	Overview	315
7.1.1	Single VCS	315
7.1.2	Multiple VCS.....	316
7.1.3	Multiple VCS with MLD Ports	317

Contents

7.1.4	vPPB Ordering	317
7.2	Switch Configuration and Composition	318
7.2.1	CXL Switch Initialization Options	318
7.2.1.1	Static Initialization	319
7.2.1.2	Fabric Manager Boots First	320
7.2.1.3	Fabric Manager and Host Boot Simultaneously	322
7.2.2	Sideband Signal Operation	323
7.2.3	Binding and Unbinding	324
7.2.3.1	Binding and Unbinding of a Single Logical Device Port	324
7.2.3.2	Binding and Unbinding of a Pooled Device.....	326
7.2.4	PPB and vPPB Behavior for MLD Ports	329
7.2.4.1	MLD Type 1 Configuration Space Header	330
7.2.4.2	MLD PCI*-compatible Configuration Registers	330
7.2.4.3	MLD PCIe Capability Structure	330
7.2.4.4	MLD PPB Secondary PCIe Capability Structure.....	333
7.2.4.5	MLD Physical Layer 16.0 GT/s Extended Capability.....	333
7.2.4.6	MLD Physical Layer 32.0 GT/s Extended Capability.....	334
7.2.4.7	MLD Lane Margining at the Receiver Extended Capability	334
7.2.5	MLD ACS Extended Capability	335
7.2.6	MLD PCIe Extended Capabilities	335
7.2.7	MLD Advanced Error Reporting Extended Capability	335
7.2.8	MLD DPC Extended Capability	336
7.2.9	Switch Mailbox CCI	337
7.3	CXL.io, CXL.cachemem Decode and Forwarding	338
7.3.1	CXL.io	338
7.3.1.1	CXL.io Decode	338
7.3.1.2	RCD Support	338
7.3.2	CXL.cache.....	338
7.3.3	CXL.mem.....	339
7.3.3.1	CXL.mem Request Decode.....	339
7.3.3.2	CXL.mem Response Decode.....	339
7.3.4	FM-owned PPB CXL Handling	339
7.4	CXL Switch PM	339
7.4.1	CXL Switch ASPM L1.....	339
7.4.2	CXL Switch PCI-PM and L2	339
7.4.3	CXL Switch Message Management	339
7.5	CXL Switch RAS	340
7.6	Fabric Manager Application Programming Interface	340
7.6.1	CXL Fabric Management.....	341
7.6.2	Fabric Management Model	341
7.6.3	CCI Message Format and Transport Protocol	342
7.6.3.1	Transport Details for MLD Components	343
7.6.4	CXL Switch Management.....	343
7.6.4.1	Initial Configuration.....	343
7.6.4.2	Dynamic Configuration.....	344
7.6.4.3	MLD Port Management.....	344
7.6.5	MLD Component Management	344
7.6.6	Management Requirements for System Operations	345
7.6.6.1	Initial System Discovery.....	345
7.6.6.2	CXL Switch Discovery	346
7.6.6.3	MLD and Switch MLD Port Management.....	346
7.6.6.4	Event Notifications	346
7.6.6.5	Binding Ports and LDs on a Switch.....	347
7.6.6.6	Unbinding Ports and LDs on a Switch	347
7.6.6.7	Hot-Add and Managed Hot-Removal of Devices	347
7.6.6.8	Surprise Removal of Devices.....	348

7.6.7	Fabric Management Application Programming Interface.....	348
7.6.7.1	Physical Switch Command Set	349
7.6.7.2	Virtual Switch Command Set	355
7.6.7.3	MLD Port Command Set	359
7.6.7.4	MLD Component Command Set.....	364
7.6.7.5	Multi-Headed Device Command Set	371
7.6.7.6	DCD Management Command Set.....	374
7.6.8	Fabric Management Event Records	384
7.6.8.1	Physical Switch Event Records	384
7.6.8.2	Virtual CXL Switch Event Records	386
7.6.8.3	MLD Port Event Records	387
7.7	CXL Fabric Architecture	387
7.7.1	CXL Fabric Use Case Examples.....	388
7.7.1.1	Machine-learning Accelerators.....	388
7.7.1.2	HPC/Analytics Use Case	389
7.7.1.3	Composable Systems.....	389
7.7.2	Global-Fabric-Attached Memory (G-FAM).....	390
7.7.2.1	Overview	390
7.7.2.2	Host Physical Address View	391
7.7.2.3	G-FAM Capacity Management	392
7.7.2.4	G-FAM Request Routing, Interleaving, and Address Translations	394
7.7.2.5	G-FAM Access Protection	398
7.7.2.6	Global Memory Access Endpoint	400
7.7.2.7	Event Notifications from GFDs.....	401
7.7.3	Global Integrated Memory (GIM).....	402
7.7.3.1	Host GIM Physical Address View	403
7.7.3.2	Use Cases	404
7.7.3.3	Transaction Flows and Rules for GIM.....	405
7.7.3.4	Restrictions with Host-to-Host UIO Usages	407
7.7.4	Non-GIM Usages with VendPrefixLO	408
7.7.5	HBR and PBR Switch Configurations.....	408
7.7.5.1	PBR Forwarding Dependencies, Loops, and Deadlocks	410
7.7.6	PBR Switching Details.....	412
7.7.6.1	Virtual Hierarchies Spanning a Fabric.....	412
7.7.6.2	PBR Message Routing across the Fabric.....	413
7.7.6.3	PBR Message Routing within a Single PBR Switch.....	415
7.7.6.4	PBR Switch vDSP/vUSP Bindings and Connectivity	416
7.7.6.5	PID Use Models and Assignments	418
7.7.6.6	CXL Switch Message Format Conversion	419
7.7.6.7	HBR Switch Port Processing of CXL Messages	423
7.7.6.8	PBR Switch Port Processing of CXL Messages	425
7.7.6.9	PPB and vPPB Behavior of PBR Link Ports	427
7.7.7	Inter-Switch Links (ISLs)	433
7.7.7.1	.io Deadlock Avoidance on ISLs/PBR Fabric.....	433
7.7.8	PBR TLP Header (PTH) Rules.....	436
7.7.9	PBR Support for UIO Direct P2P to HDM	436
7.7.9.1	FAST Decoder Use for UIO Direct P2P to G-FAM.....	437
7.7.9.2	LDST Decoder Use for UIO Direct P2P to LD-FAM	437
7.7.9.3	ID-Based Re-Router for UIO Completions with LD-FAM.....	438
7.7.9.4	LDST and ID-Based Re-Router Access Protection	439
7.7.10	PBR Support for Direct P2P CXL.mem for Accelerators.....	439
7.7.10.1	Message Routing for Direct P2P CXL.mem Accesses with GFD.	440
7.7.10.2	Message Routing for Direct P2P CXL.mem Accesses with MLD.	440
7.7.11	PBR Link Events and Messages	441
7.7.11.1	PBR Link Fundamentals.....	442
7.7.11.2	CXL VDMs	443
7.7.11.3	Single VH Events.....	443

7.7.11.4	Shared Link Events.....	447
7.7.11.5	Switch Reported Events	448
7.7.11.6	PBR Link CCI Message Format and Transport Protocol	449
7.7.12	PBR Fabric Management	450
7.7.12.1	Fabric Boot and Initialization.....	450
7.7.12.2	PBR Fabric Discovery	451
7.7.12.3	Assigning and Binding PIDs	452
7.7.12.4	Reporting Fabric Route Performance via CDAT.....	452
7.7.12.5	Configuring CacheID in PBR Fabric.....	453
7.7.12.6	Dynamic Fabric Changes	453
7.7.13	PBR Switch Command Set.....	454
7.7.13.1	Identify PBR Switch (Opcode 5700h)	454
7.7.13.2	Fabric Crawl Out (Opcode 5701h).....	455
7.7.13.3	Get PBR Link Partner Info (Opcode 5702h)	457
7.7.13.4	Get PID Target List (Opcode 5703h)	458
7.7.13.5	Configure PID Assignment (Opcode 5704h)	459
7.7.13.6	Get PID Binding (Opcode 5705h).....	460
7.7.13.7	Configure PID Binding (Opcode 5706h)	461
7.7.13.8	Get Table Descriptors (Opcode 5707h).....	462
7.7.13.9	Get DRT (Opcode 5708h)	463
7.7.13.10	Set DRT (Opcode 5709h)	464
7.7.13.11	Get RGT (Opcode 570Ah)	464
7.7.13.12	Set RGT (Opcode 570Bh)	466
7.7.13.13	Get LDST/IDT Capabilities (Opcode 570Ch)	466
7.7.13.14	Set LDST/IDT Configuration (Opcode 570Dh).....	467
7.7.13.15	Get LDST Segment Entries (Opcode 570Eh).....	468
7.7.13.16	Set LDST Segment Entries (Opcode 570Fh)	469
7.7.13.17	Get LDST IDT DPID Entries (Opcode 5710h)	470
7.7.13.18	Set LDST IDT DPID Entries (Opcode 5711h)	471
7.7.13.19	Get Completer ID-Based Re-Router Entries (Opcode 5712h) ..	472
7.7.13.20	Set Completer ID-Based Re-Router Entries (Opcode 5713h)...	473
7.7.13.21	Get LDST Access Vector (Opcode 5714h).....	474
7.7.13.22	Get VCS LDST Access Vector (Opcode 5715h).....	475
7.7.13.23	Configure VCS LDST Access (Opcode 5716h)	475
7.7.14	Global Memory Access Endpoint Command Set	476
7.7.14.1	Identify GAE (Opcode 5800h)	476
7.7.14.2	Get PID Interrupt Vector (Opcode 5801h).....	477
7.7.14.3	Get PID Access Vectors (Opcode 5802h)	478
7.7.14.4	Get FAST/IDT Capabilities (Opcode 5803h).....	479
7.7.14.5	Set FAST/IDT Configuration (Opcode 5804h)	480
7.7.14.6	Get FAST Segment Entries (Opcode 5805h)	481
7.7.14.7	Set FAST Segment Entries (Opcode 5806h)	483
7.7.14.8	Get IDT DPID Entries (Opcode 5807h)	483
7.7.14.9	Set IDT DPID Entries (Opcode 5808h).....	484
7.7.14.10	Proxy GFD Management Command (Opcode 5809h)	485
7.7.14.11	Get Proxy Thread Status (Opcode 580Ah).....	486
7.7.14.12	Cancel Proxy Thread (Opcode 580Bh)	487
7.7.15	Global Memory Access Endpoint Management Command Set.....	487
7.7.15.1	Identify VCS GAE (Opcode 5900h).....	488
7.7.15.2	Get VCS PID Access Vectors (Opcode 5901h).....	488
7.7.15.3	Configure VCS PID Access (Opcode 5902h).....	489
7.7.15.4	Get VendPrefixL0 State (Opcode 5903h)	490
7.7.15.5	Set VendPrefixL0 State (Opcode 5904h).....	490
8.0	Control and Status Registers	492
8.1	Configuration Space Registers.....	493
8.1.1	PCIe Designated Vendor-Specific Extended Capability (DVSEC) ID Assignment	493
8.1.2	CXL Data Object Exchange (DOE) Type Assignment.....	494

8.1.3	PCIe DVSEC for CXL Devices	494
8.1.3.1	DVSEC CXL Capability (Offset 0Ah)	496
8.1.3.2	DVSEC CXL Control (Offset 0Ch)	497
8.1.3.3	DVSEC CXL Status (Offset 0Eh)	498
8.1.3.4	DVSEC CXL Control2 (Offset 10h)	498
8.1.3.5	DVSEC CXL Status2 (Offset 12h)	499
8.1.3.6	DVSEC CXL Lock (Offset 14h)	500
8.1.3.7	DVSEC CXL Capability2 (Offset 16h)	500
8.1.3.8	DVSEC CXL Range Registers	500
8.1.3.9	DVSEC CXL Capability3 (Offset 38h)	506
8.1.4	Non-CXL Function Map DVSEC	507
8.1.4.1	Non-CXL Function Map Register 0 (Offset 0Ch)	508
8.1.4.2	Non-CXL Function Map Register 1 (Offset 10h)	508
8.1.4.3	Non-CXL Function Map Register 2 (Offset 14h)	508
8.1.4.4	Non-CXL Function Map Register 3 (Offset 18h)	508
8.1.4.5	Non-CXL Function Map Register 4 (Offset 1Ch)	509
8.1.4.6	Non-CXL Function Map Register 5 (Offset 20h)	509
8.1.4.7	Non-CXL Function Map Register 6 (Offset 24h)	509
8.1.4.8	Non-CXL Function Map Register 7 (Offset 28h)	509
8.1.5	CXL Extensions DVSEC for Ports	510
8.1.5.1	CXL Port Extension Status (Offset 0Ah)	510
8.1.5.2	Port Control Extensions (Offset 0Ch)	511
8.1.5.3	Alternate Bus Base (Offset 0Eh)	512
8.1.5.4	Alternate Bus Limit (Offset 0Fh)	512
8.1.5.5	Alternate Memory Base (Offset 10h)	512
8.1.5.6	Alternate Memory Limit (Offset 12h)	512
8.1.5.7	Alternate Prefetchable Memory Base (Offset 14h)	513
8.1.5.8	Alternate Prefetchable Memory Limit (Offset 16h)	513
8.1.5.9	Alternate Memory Prefetchable Base High (Offset 18h)	513
8.1.5.10	Alternate Prefetchable Memory Limit High (Offset 1Ch)	513
8.1.5.11	CXL RCRB Base (Offset 20h)	513
8.1.5.12	CXL RCRB Base High (Offset 24h)	514
8.1.6	GPF DVSEC for CXL Port	514
8.1.6.1	GPF Phase 1 Control (Offset 0Ch)	515
8.1.6.2	GPF Phase 2 Control (Offset 0Eh)	515
8.1.7	GPF DVSEC for CXL Device	516
8.1.7.1	GPF Phase 2 Duration (Offset 0Ah)	516
8.1.7.2	GPF Phase 2 Power (Offset 0Ch)	517
8.1.8	PCIe DVSEC for Flex Bus Port	517
8.1.9	Register Locator DVSEC	517
8.1.9.1	Register Offset Low (Offset: Varies)	518
8.1.9.2	Register Offset High (Offset: Varies)	519
8.1.10	MLD DVSEC	519
8.1.10.1	Number of LD Supported (Offset 0Ah)	520
8.1.10.2	LD-ID Hot Reset Vector (Offset 0Ch)	520
8.1.11	Table Access DOE	520
8.1.11.1	Read Entry	521
8.1.12	Memory Device Configuration Space Layout	521
8.1.12.1	PCI Header - Class Code Register (Offset 09h)	521
8.1.12.2	Memory Device PCIe Capabilities and Extended Capabilities ...	522
8.1.13	Switch Mailbox CCI Configuration Space Layout	522
8.1.13.1	PCI Header - Class Code Register (Offset 09h)	522
8.2	Memory Mapped Registers	522
8.2.1	RCD Upstream Port and RCH Downstream Port Registers	524
8.2.1.1	RCH Downstream Port RCRB	524
8.2.1.2	RCD Upstream Port RCRB	526
8.2.1.3	Flex Bus Port DVSEC	528
8.2.2	Accessing Component Registers	533

8.2.3	Component Register Layout and Definition	533
8.2.4	CXL.cache and CXL.mem Registers.....	534
8.2.4.1	CXL Capability Header Register (Offset 00h)	536
8.2.4.2	CXL RAS Capability Header (Offset: Varies)	537
8.2.4.3	CXL Security Capability Header (Offset: Varies).....	537
8.2.4.4	CXL Link Capability Header (Offset: Varies)	537
8.2.4.5	CXL HDM Decoder Capability Header (Offset: Varies)	537
8.2.4.6	CXL Extended Security Capability Header (Offset: Varies).....	538
8.2.4.7	CXL IDE Capability Header (Offset: Varies)	538
8.2.4.8	CXL Snoop Filter Capability Header (Offset: Varies)	538
8.2.4.9	CXL Timeout and Isolation Capability Header (Offset: Varies) .	539
8.2.4.10	CXL.cachemem Extended Register Capability (Offset: Varies) .	539
8.2.4.11	CXL BI Route Table Capability Header (Offset: Varies).....	539
8.2.4.12	CXL BI Decoder Capability Header (Offset: Varies).....	539
8.2.4.13	CXL Cache ID Route Table Capability Header (Offset: Varies) .	540
8.2.4.14	CXL Cache ID Decoder Capability Header (Offset: Varies)	540
8.2.4.15	CXL Extended HDM Decoder Capability Header (Offset: Varies).....	541
8.2.4.16	CXL Extended Metadata Capability Header (Offset: Varies)	541
8.2.4.17	CXL RAS Capability Structure.....	541
8.2.4.18	CXL Security Capability Structure	549
8.2.4.19	CXL Link Capability Structure.....	550
8.2.4.20	CXL HDM Decoder Capability Structure.....	556
8.2.4.21	CXL Extended Security Capability Structure	569
8.2.4.22	CXL IDE Capability Structure	570
8.2.4.23	CXL Snoop Filter Capability Structure.....	574
8.2.4.24	CXL Timeout and Isolation Capability Structure	574
8.2.4.25	CXL.cachemem Extended Register Capability	580
8.2.4.26	CXL BI Route Table Capability Structure.....	581
8.2.4.27	CXL BI Decoder Capability Structure	583
8.2.4.28	CXL Cache ID Route Table Capability Structure	585
8.2.4.29	CXL Cache ID Decoder Capability Structure	588
8.2.4.30	CXL Extended HDM Decoder Capability Structure.....	589
8.2.4.31	CXL Extended Metadata Capability Register	590
8.2.5	CXL ARB/MUX Registers.....	591
8.2.5.1	ARB/MUX PM Timeout Control Register (Offset 00h).....	591
8.2.5.2	ARB/MUX Uncorrectable Error Status Register (Offset 04h)	592
8.2.5.3	ARB/MUX Uncorrectable Error Mask Register (Offset 08h)	592
8.2.5.4	ARB/MUX Arbitration Control Register for CXL.io (Offset 180h)	592
8.2.5.5	ARB/MUX Arbitration Control Register for CXL.cache and CXL.mem (Offset 1C0h)	593
8.2.6	BAR Virtualization ACL Register Block	593
8.2.6.1	BAR Virtualization ACL Size Register (Offset 00h)	594
8.2.7	CPMU Register Interface	594
8.2.7.1	Per CPMU Registers	595
8.2.7.2	Per Counter Unit Registers	598
8.2.8	CXL Device Register Interface.....	601
8.2.8.1	CXL Device Capabilities Array Register (Offset 00h)	602
8.2.8.2	CXL Device Capability Header Register (Offset: Varies).....	602
8.2.8.3	Device Status Registers (Offset: Varies)	603
8.2.8.4	Mailbox Registers (Offset: Varies).....	603
8.2.8.5	Memory Device Capabilities	609
8.2.8.6	Switch Mailbox CCI Capability	611
8.2.9	Component Command Interface.....	611
8.2.9.1	Information and Status Command Set	615
8.2.9.2	Events	618
8.2.9.3	Firmware Update	646
8.2.9.4	Timestamp	651

8.2.9.5	Logs	652
8.2.9.6	Features	669
8.2.9.7	Maintenance	674
8.2.9.8	PBR Component Command Set	689
8.2.9.9	Memory Device Command Sets	692
8.2.9.10	FM API Commands	766
9.0	Reset, Initialization, Configuration, and Manageability	772
9.1	CXL Boot and Reset Overview	772
9.1.1	General	772
9.1.2	Comparing CXL and PCIe Behavior	773
9.1.2.1	Switch Behavior	773
9.2	CXL Device Boot Flow	774
9.3	CXL System Reset Entry Flow	774
9.4	CXL Device Sleep State Entry Flow	775
9.5	Function Level Reset (FLR)	776
9.6	Cache Management	777
9.7	CXL Reset	778
9.7.1	Effect on the Contents of the Volatile HDM	779
9.7.2	Software Actions	779
9.7.3	CXL Reset and Request Retry Status (RRS)	780
9.8	Global Persistent Flush (GPF)	780
9.8.1	Host and Switch Responsibilities	780
9.8.2	Device Responsibilities	781
9.8.3	Energy Budgeting	783
9.9	Hot-Plug	785
9.10	Software Enumeration	788
9.11	RCD Enumeration	788
9.11.1	RCD Mode	788
9.11.2	PCIe Software View of an RCH and RCD	789
9.11.3	System Firmware View of an RCH and RCD	789
9.11.4	OS View of an RCH and RCD	789
9.11.5	System Firmware-based RCD Enumeration Flow	790
9.11.6	RCD Discovery	790
9.11.7	eRCDs with Multiple Flex Bus Links	792
9.11.7.1	Single CPU Topology	792
9.11.7.2	Multiple CPU Topology	793
9.11.8	CXL Devices Attached to an RCH	794
9.12	CXL VH Enumeration	796
9.12.1	CXL Root Ports	797
9.12.2	CXL Virtual Hierarchy	797
9.12.3	Enumerating CXL RPs and DSPs	798
9.12.4	eRCD Connected to a CXL RP or DSP	799
9.12.4.1	Boot time Reconfiguration of CXL RP or DSP to Enable an eRCD	799
9.12.5	CXL eRCD below a CXL RP and DSP - Example	803
9.12.6	Mapping of Link and Protocol Registers in CXL VH	804
9.13	Software View of HDM	806
9.13.1	Memory Interleaving	806
9.13.1.1	Legal Interleaving Configurations: 12-way, 6-way, and 3-way	810
9.13.2	CXL Memory Device Label Storage Area	810
9.13.2.1	Overall LSA Layout	811
9.13.2.2	Label Index Blocks	812
9.13.2.3	Common Label Properties	814
9.13.2.4	Region Labels	815

Contents

9.13.2.5	Namespace Labels.....	816
9.13.2.6	Vendor-specific Labels	817
9.13.3	Dynamic Capacity Device (DCD)	817
9.13.3.1	DCD Management By FM.....	821
9.13.3.2	Setting up Memory Sharing	822
9.13.3.3	Extent List Tracking.....	822
9.13.4	Capacity or Performance Degradation	823
9.14	Back-Invalidate Configuration	823
9.14.1	Discovery	824
9.14.2	Configuration	824
9.14.3	Mixed Configurations	827
9.14.3.1	BI-capable Type 2 Device.....	828
9.14.3.2	Type 2 Device Fallback Modes.....	828
9.14.3.3	BI-capable Type 3 Device.....	829
9.15	Cache ID Configuration and Routing.....	829
9.15.1	Host Capabilities	829
9.15.2	Downstream Port Decode Functionality	829
9.15.3	Upstream Switch Port Routing Functionality	830
9.15.4	Host Bridge Routing Functionality.....	831
9.16	UIO Direct P2P to HDM.....	833
9.16.1	Processing of UIO Direct P2P to HDM Messages.....	834
9.16.1.1	UIO Address Match (DSP and Root Port).....	834
9.16.1.2	UIO Address Match (CXL.mem Device).....	835
9.17	Direct P2P CXL.mem for Accelerators	835
9.17.1	Peer SLD Configuration.....	836
9.17.2	Peer MLD Configuration	836
9.17.3	Peer GFD Configuration.....	836
9.18	CXL OS Firmware Interface Extensions.....	836
9.18.1	CXL Early Discovery Table (CEDT)	836
9.18.1.1	CEDT Header	837
9.18.1.2	CXL Host Bridge Structure (CHBS).....	837
9.18.1.3	CXL Fixed Memory Window Structure (CFMWS).....	838
9.18.1.4	CXL XOR Interleave Math Structure (CXIMS).....	841
9.18.1.5	RCEC Downstream Port Association Structure (RDPAS)	842
9.18.1.6	CXL System Description Structure (CSDS)	842
9.18.2	CXL _OSC.....	843
9.18.2.1	Rules for Evaluating _OSC.....	845
9.18.3	CXL Root Device Specific Methods (_DSM).....	846
9.18.3.1	_DSM Function for Retrieving QTG ID	847
9.19	Manageability Model for CXL Devices.....	848
9.20	Component Command Interface	849
9.20.1	CCI Properties	849
9.20.2	MCTP-based CCI Properties	850
10.0	Power Management	851
10.1	Statement of Requirements	851
10.2	Policy-based Runtime Control - Idle Power - Protocol Flow	851
10.2.1	General	851
10.2.2	Package-level Idle (C-state) Entry and Exit Coordination	851
10.2.2.1	PMReq Message Generation and Processing Rules.....	852
10.2.3	PkgC Entry Flows	853
10.2.4	PkgC Exit Flows	855
10.2.5	CXL Physical Layer Power Management States	856
10.3	CXL Power Management.....	856
10.3.1	CXL PM Entry Phase 1.....	857
10.3.2	CXL PM Entry Phase 2.....	858

Contents

10.3.3	CXL PM Entry Phase 3.....	860
10.3.4	CXL Exit from ASPM L1	861
10.3.5	L0p Negotiation for 256B Flit Mode	862
10.4	CXL.io Link Power Management	862
10.4.1	CXL.io ASPM Entry Phase 1 for 256B Flit Mode	862
10.4.2	CXL.io ASPM L1 Entry Phase 1 for 68B Flit Mode.....	862
10.4.3	CXL.io ASPM L1 Entry Phase 2	863
10.4.4	CXL.io ASPM Entry Phase 3	863
10.5	CXL.cache + CXL.mem Link Power Management.....	863
11.0	CXL Security	864
11.1	CXL IDE Overview	864
11.2	CXL.io IDE.....	866
11.3	CXL.cachemem IDE	866
11.3.1	CXL.cachemem IDE Architecture in 68B Flit Mode	867
11.3.2	CXL.cachemem IDE Architecture in 256B Flit Mode	870
11.3.3	Encrypted PCRC.....	876
11.3.4	Cryptographic Keys and IV	878
11.3.5	CXL.cachemem IDE Modes	878
11.3.5.1	Discovery of Integrity Modes and Settings	879
11.3.5.2	Negotiation of Operating Mode and Settings.....	879
11.3.5.3	Rules for MAC Aggregation	879
11.3.6	Early MAC Termination	882
11.3.7	Handshake to Trigger the Use of Keys.....	885
11.3.8	Error Handling	885
11.3.9	Switch Support	886
11.3.10	IDE Termination Handshake	887
11.4	CXL.cachemem IDE Key Management (CXL_IDE_KM).....	887
11.4.1	CXL_IDE_KM Protocol Overview	889
11.4.2	Secure Messaging Layer Rules	889
11.4.3	CXL_IDE_KM Common Data Structures.....	890
11.4.4	Discovery Messages	891
11.4.5	Key Programming Messages	892
11.4.6	Activation/Key Refresh Messages	895
11.4.7	Get Key Messages.....	898
11.5	CXL Trusted Execution Environments Security Protocol (TSP)	901
11.5.1	Overview	901
11.5.2	Scope.....	902
11.5.3	Threat Model	902
11.5.3.1	Definitions.....	902
11.5.3.2	Assumptions.....	903
11.5.3.3	Threats and Mitigations.....	905
11.5.4	Reference Architecture	905
11.5.4.1	Architectural Scope	905
11.5.4.2	Determining TSP Support	906
11.5.4.3	CMA/SPDM.....	906
11.5.4.4	Authentication and Attestation	908
11.5.4.5	TE State Changes and Access Control	908
11.5.4.6	Memory Encryption	915
11.5.4.7	Transport Security.....	919
11.5.4.8	Configuration.....	920
11.5.4.9	Component Command Interfaces	927
11.5.4.10	Dynamic Capacity	927
11.5.5	TSP Requests and Responses.....	928
11.5.5.1	TSP Request Overview	928
11.5.5.2	TSP Response Overview	929

11.5.5.3	Request Response and CMA/SPDM Sessions.....	930
11.5.5.4	Version	930
11.5.5.5	Target Capabilities	931
11.5.5.6	Target Configuration.....	934
11.5.5.7	Optional Explicit TE State Change Requests and Responses....	943
11.5.5.8	Optional Target-based Memory Encryption Requests and Responses.....	945
11.5.5.9	Optional Delayed Completion Requests and Responses.....	953
11.5.5.10	Error Response	955
12.0	Reliability, Availability, and Serviceability	956
12.1	Supported RAS Features	956
12.2	CXL Error Handling	956
12.2.1	Protocol and Link Layer Error Reporting	956
12.2.1.1	RCH Downstream Port-detected Errors	957
12.2.1.2	RCD Upstream Port-detected Errors	958
12.2.1.3	RCD RCiEP-detected Errors.....	959
12.2.1.4	Header Log and Handling of Multiple Errors.....	960
12.2.2	CXL Root Ports, Downstream Switch Ports, and Upstream Switch Ports	961
12.2.3	CXL Device Error Handling	961
12.2.3.1	CXL.cache and CXL.mem Errors	962
12.2.3.2	Memory Error Logging and Signaling Enhancements	963
12.2.3.3	CXL Device Error Handling Flows	964
12.3	Isolation on CXL.cache and CXL.mem.....	964
12.3.1	CXL.cache Transaction Layer Behavior during Isolation	965
12.3.2	CXL.mem Transaction Layer Behavior during Isolation	966
12.4	CXL Viral Handling.....	966
12.4.1	Switch Considerations.....	966
12.4.2	Device Considerations	967
12.5	Maintenance	968
12.6	CXL Error Injection	968
13.0	Performance Considerations.....	969
13.1	Performance Recommendations.....	969
13.2	Performance Monitoring	971
14.0	CXL Compliance Testing.....	976
14.1	Applicable Devices under Test (DUTs)	976
14.2	Starting Configuration/Topology (Common for All Tests)	976
14.2.1	Test Topologies	977
14.2.1.1	Single Host, Direct Attached SLD EP (SHDA).....	977
14.2.1.2	Single Host, Switch Attached SLD EP (SHSW)	977
14.2.1.3	Single Host, Fabric Managed, Switch Attached SLD EP (SHSW-FM).....	978
14.2.1.4	Dual Host, Fabric Managed, Switch Attached SLD EP (DHSW-FM).....	979
14.2.1.5	Dual Host, Fabric Managed, Switch Attached MLD EP (DHSW-FM-MLD)	980
14.2.1.6	Cascaded Switch Topologies	981
14.3	CXL.io and CXL.cache Application Layer/Transaction Layer Testing	982
14.3.1	General Testing Overview	982
14.3.2	Algorithms	983
14.3.3	Algorithm 1a: Multiple Write Streaming.....	983
14.3.4	Algorithm 1b: Multiple Write Streaming with Bogus Writes.....	984
14.3.5	Algorithm 2: Producer Consumer Test.....	985
14.3.6	Test Descriptions	986
14.3.6.1	Application Layer/Transaction Layer Tests	986
14.4	Link Layer Testing	991

Contents

14.4.1	RSVD Field Testing CXL.cachemem.....	991
14.4.1.1	Device Test	991
14.4.1.2	Host Test	991
14.4.2	CRC Error Injection RETRY_PHY_REINIT.....	992
14.4.3	CRC Error Injection RETRY_ABORT	993
14.5	ARB/MUX	994
14.5.1	Reset to Active Transition.....	994
14.5.2	ARB/MUX Multiplexing	994
14.5.3	Active to L1.x Transition (If Applicable).....	995
14.5.4	L1.x State Resolution (If Applicable)	996
14.5.5	Active to L2 Transition	997
14.5.6	L1 to Active Transition (If Applicable)	997
14.5.7	Reset Entry	998
14.5.8	Entry into L0 Synchronization	998
14.5.9	ARB/MUX Tests Requiring Injection Capabilities.....	999
14.5.9.1	ARB/MUX Bypass	999
14.5.9.2	PM State Request Rejection	999
14.5.9.3	Unexpected Status ALMP.....	1000
14.5.9.4	ALMP Error.....	1000
14.5.9.5	Recovery Re-entry	1001
14.5.10	L0p Feature	1001
14.5.10.1	Positive ACK for L0p	1001
14.5.10.2	Force NAK for L0p Request.....	1002
14.6	Physical Layer.....	1002
14.6.1	Tests Applicable to 68B Flit Mode	1002
14.6.1.1	Protocol ID Checks	1002
14.6.1.2	NULL Flit	1003
14.6.1.3	EDS Token	1003
14.6.1.4	Correctable Protocol ID Error	1004
14.6.1.5	Uncorrectable Protocol ID Error	1004
14.6.1.6	Unexpected Protocol ID.....	1004
14.6.1.7	Recovery.Idle/Config.Idle Transition to L0	1005
14.6.1.8	Uncorrectable Mismatched Protocol ID Error	1005
14.6.2	Drift Buffer (If Applicable)	1006
14.6.3	SKP OS Scheduling/Alternation (If Applicable).....	1006
14.6.4	SKP OS Exiting the Data Stream (If Applicable).....	1007
14.6.5	Link Initialization Resolution	1007
14.6.6	Hot Add Link Initialization Resolution	1008
14.6.7	Link Speed Advertisement.....	1009
14.6.8	Link Speed Degradation - CXL Mode	1010
14.6.9	Link Speed Degradation below 8 GT/s.....	1010
14.6.10	Tests Requiring Injection Capabilities.....	1010
14.6.10.1	TLP Ends on Flit Boundary	1010
14.6.10.2	Failed CXL Mode Link Up	1011
14.6.11	Link Initialization in Standard 256B Flit Mode.....	1011
14.6.12	Link Initialization in Latency-Optimized 256B Flit Mode.....	1012
14.6.13	Sync Header Bypass (If Applicable)	1012
14.7	Switch Tests.....	1013
14.7.1	Introduction to Switch Types	1013
14.7.2	Compliance Testing	1013
14.7.2.1	HBR Switch Assumptions.....	1013
14.7.2.2	PBR Switch Assumptions	1015
14.7.3	Unmanaged HBR Switch.....	1016
14.7.4	Reset Propagation	1017
14.7.4.1	Host PERST# Propagation	1017
14.7.4.2	LTSSM Hot Reset	1018

Contents

14.7.4.3	Secondary Bus Reset (SBR) Propagation	1020
14.7.5	Managed Hot-Plug - Adding a New Endpoint Device	1024
14.7.5.1	Managed Add of an SLD Component	1024
14.7.5.2	Managed Add of an MLD Component (HBR Switch Only)	1025
14.7.5.3	Managed Add of an MLD Component to an SLD Port (HBR Switch Only)	1025
14.7.6	Managed Hot-Plug Removal of an Endpoint Device.....	1026
14.7.6.1	Managed Removal of an SLD Component from a VCS (HBR Switch).....	1026
14.7.6.2	Managed Removal of an SLD Component (PBR Switch).....	1026
14.7.6.3	Managed Removal of an MLD Component from a Switch (HBR Switch Only)	1026
14.7.6.4	Removal of a Device from an Unbound Port	1027
14.7.7	Bind/Unbind and Port Access Operations	1027
14.7.7.1	Binding and Granting Port Access of Pooled Resources to Hosts	1027
14.7.7.2	Unbinding Resources from Hosts without Removing the Endpoint Devices	1029
14.7.8	Error Injection	1030
14.7.8.1	AER Error Injection.....	1030
14.8	Configuration Register Tests	1032
14.8.1	Device Presence	1032
14.8.2	CXL Device Capabilities.....	1033
14.8.3	DOE Capabilities	1034
14.8.4	DVSEC Control Structure.....	1035
14.8.5	DVSEC CXL Capability.....	1036
14.8.6	DVSEC CXL Control	1036
14.8.7	DVSEC CXL Lock	1037
14.8.8	DVSEC CXL Capability2.....	1038
14.8.9	Non-CXL Function Map DVSEC	1038
14.8.10	CXL Extensions DVSEC for Ports Header.....	1039
14.8.11	Port Control Override.....	1040
14.8.12	GPF DVSEC Port Capability	1041
14.8.13	GPF Port Phase 1 Control	1041
14.8.14	GPF Port Phase 2 Control	1042
14.8.15	GPF DVSEC Device Capability	1042
14.8.16	GPF Device Phase 2 Duration	1043
14.8.17	Flex Bus Port DVSEC Capability Header.....	1044
14.8.18	DVSEC Flex Bus Port Capability.....	1044
14.8.19	Register Locator	1045
14.8.20	MLD DVSEC Capability Header	1045
14.8.21	MLD DVSEC Number of LD Supported	1046
14.8.22	Table Access DOE	1047
14.8.23	PCI Header - Class Code Register.....	1047
14.9	Reset and Initialization Tests	1048
14.9.1	Warm Reset Test	1048
14.9.2	Cold Reset Test	1048
14.9.3	Sleep State Test	1049
14.9.4	Function Level Reset Test.....	1049
14.9.5	CXL Range Setup Time	1049
14.9.6	FLR Memory	1050
14.9.7	CXL_Reset Test	1050
14.9.8	Global Persistent Flush (GPF).....	1052
14.9.8.1	Host and Switch Test.....	1053
14.9.8.2	Device Test	1053
14.9.9	Hot-Plug Test	1054

Contents

14.9.10	Device to Host Cache Viral Injection	1054
14.9.11	Device to Host Mem Viral Injection	1055
14.10	Power Management Tests	1055
14.10.1	Pkg-C Entry (Device Test)	1055
14.10.2	Pkg-C Entry Reject (Device Test)	1056
14.10.3	Pkg-C Entry (Host Test)	1057
14.11	Security	1057
14.11.1	Component Measurement and Authentication	1057
14.11.1.1	DOE CMA Instance	1057
14.11.1.2	FLR while Processing DOE CMA Request	1058
14.11.1.3	OOB CMA while in Fundamental Reset.....	1058
14.11.1.4	OOB CMA while Function Gets FLR.....	1059
14.11.1.5	OOB CMA during Conventional Reset	1060
14.11.2	Link Integrity and Data Encryption CXL.io IDE.....	1060
14.11.2.1	CXL.io Link IDE Streams Functional	1061
14.11.2.2	CXL.io Link IDE Streams Aggregation.....	1061
14.11.2.3	CXL.io Link IDE Streams PCRC	1062
14.11.2.4	CXL.io Selective IDE Stream Functional	1063
14.11.2.5	CXL.io Selective IDE Streams Aggregation	1063
14.11.2.6	CXL.io Selective IDE Streams PCRC	1064
14.11.3	CXL.cachemem IDE	1065
14.11.3.1	CXL.cachemem IDE Capability (SHDA, SHSW)	1065
14.11.3.2	Establish CXL.cachemem IDE (SHDA) in Standard 256B Flit Mode.....	1065
14.11.3.3	Establish CXL.cachemem IDE (SHSW).....	1066
14.11.3.4	Establish CXL.cachemem IDE (SHDA) Latency-Optimized 256B Flit Mode.....	1067
14.11.3.5	Establish CXL.cachemem IDE (SHDA) 68B Flit Mode.....	1068
14.11.3.6	Locally Generate IV (SHDA).....	1069
14.11.3.7	Data Encryption – Decryption and Integrity Testing with Containment Mode for MAC Generation and Checking	1070
14.11.3.8	Data Encryption – Decryption and Integrity Testing with Skid Mode for MAC Generation and Checking	1070
14.11.3.9	Key Refresh.....	1071
14.11.3.10	Early MAC Termination.....	1071
14.11.3.11	Error Handling	1072
14.11.4	Certificate Format/Certificate Chain	1077
14.11.5	Security RAS	1078
14.11.5.1	CXL.io Poison Inject from Device	1078
14.11.5.2	CXL.cache Poison Inject from Device	1079
14.11.5.3	CXL.cache CRC Inject from Device.....	1080
14.11.5.4	CXL.mem Poison Injection	1082
14.11.5.5	CXL.mem CRC Injection	1082
14.11.5.6	Flow Control Injection	1083
14.11.5.7	Unexpected Completion Injection	1084
14.11.5.8	Completion Timeout Injection	1086
14.11.5.9	Memory Error Injection and Logging	1087
14.11.5.10	CXL.io Viral Inject from Device.....	1088
14.11.5.11	CXL.cache Viral Inject from Device	1089
14.11.6	Security Protocol and Data Model	1091
14.11.6.1	SPDM GET_VERSION	1091
14.11.6.2	SPDM GET_CAPABILITIES	1092
14.11.6.3	SPDM NEGOTIATE_ALGORITHMS	1093
14.11.6.4	SPDM GET_DIGESTS	1094
14.11.6.5	SPDM GET_CERTIFICATE.....	1095
14.11.6.6	SPDM CHALLENGE.....	1096
14.11.6.7	SPDM GET_MEASUREMENTS Count.....	1097
14.11.6.8	SPDM GET_MEASUREMENTS All	1098

14.11.6.9	SPDM GET_MEASUREMENTS Repeat with Signature	1099
14.11.6.10	SPDM CHALLENGE Sequences	1100
14.11.6.11	SPDM ErrorCode Unsupported Request	1102
14.11.6.12	SPDM Major Version Invalid	1103
14.11.6.13	SPDM ErrorCode UnexpectedRequest	1103
14.12	Reliability, Availability, and Serviceability	1104
14.12.1	RAS Configuration	1105
14.12.1.1	AER Support	1105
14.12.1.2	CXL.io Poison Injection from Device to Host	1106
14.12.1.3	CXL.cache Poison Injection	1107
14.12.1.4	CXL.cache CRC Injection	1109
14.12.1.5	CXL.mem Link Poison Injection	1111
14.12.1.6	CXL.mem CRC Injection	1111
14.12.1.7	Flow Control Injection	1112
14.12.1.8	Unexpected Completion Injection	1113
14.12.1.9	Completion Timeout	1113
14.12.1.10	CXL.mem Media Poison Injection	1114
14.12.1.11	CXL.mem LSA Poison Injection	1114
14.12.1.12	CXL.mem Device Health Injection	1115
14.13	Memory Mapped Registers	1115
14.13.1	CXL Capability Header	1115
14.13.2	CXL RAS Capability Header	1116
14.13.3	CXL Security Capability Header	1116
14.13.4	CXL Link Capability Header	1117
14.13.5	CXL HDM Decoder Capability Header	1117
14.13.6	CXL Extended Security Capability Header	1118
14.13.7	CXL IDE Capability Header	1118
14.13.8	CXL HDM Decoder Capability Register	1119
14.13.9	CXL HDM Decoder Commit	1119
14.13.10	CXL HDM Decoder Zero Size Commit	1120
14.13.11	CXL Snoop Filter Capability Header	1120
14.13.12	CXL Device Capabilities Array Register	1121
14.13.13	Device Status Registers Capabilities Header Register	1122
14.13.14	Primary Mailbox Registers Capabilities Header Register	1122
14.13.15	Secondary Mailbox Registers Capabilities Header Register	1122
14.13.16	Memory Device Status Registers Capabilities Header Register	1123
14.13.17	CXL Timeout and Isolation Capability Header	1123
14.13.18	CXL.cachemem Extended Register Header	1124
14.13.19	CXL BI Route Table Capability Header	1125
14.13.20	CXL BI Decoder Capability Header	1125
14.13.21	CXL Cache ID Route Table Header	1126
14.13.22	CXL Cache ID Decoder Capability Header	1126
14.13.23	CXL Extended HDM Decoder Capability Header	1127
14.14	Memory Device Tests	1127
14.14.1	DVSEC CXL Range 1 Size Low Registers	1127
14.14.2	DVSEC CXL Range 2 Size Low Registers	1128
14.15	Sticky Register Tests	1129
14.15.1	Sticky Register Test	1129
14.16	Device Capability and Test Configuration Control	1131
14.16.1	CXL Device Test Capability Advertisement	1131
14.16.2	Debug Capabilities in Device	1132
14.16.2.1	Error Logging	1132
14.16.2.2	Event Monitors	1133
14.16.3	Compliance Mode DOE	1134
14.16.3.1	Compliance Mode Capability	1134
14.16.3.2	Compliance Mode Status	1135

Figures

14.16.3.3	Compliance Mode Halt All	1136
14.16.3.4	Compliance Mode Multiple Write Streaming.....	1136
14.16.3.5	Compliance Mode Producer-Consumer.....	1137
14.16.3.6	Test Algorithm 1b Multiple Write Streaming with Bogus Writes	1138
14.16.3.7	Inject Link Poison.....	1139
14.16.3.8	Inject CRC	1140
14.16.3.9	Inject Flow Control.....	1140
14.16.3.10	Toggle Cache Flush	1141
14.16.3.11	Inject MAC Delay	1141
14.16.3.12	Insert Unexpected MAC.....	1142
14.16.3.13	Inject Viral.....	1142
14.16.3.14	Inject ALMP in Any State	1143
14.16.3.15	Ignore Received ALMP	1144
14.16.3.16	Inject Bit Error in Flit	1144
14.16.3.17	Inject Memory Device Poison	1145
A	Taxonomy.....	1148
A.1	Accelerator Usage Taxonomy	1148
A.2	Bias Model Flow Example – From CPU	1149
A.3	CPU Support for Bias Modes.....	1150
A.3.1	Remote Snoop Filter	1150
A.3.2	Directory in Accelerator-attached Memory	1150
A.4	Giant Cache Model.....	1151
B	Unordered I/O to Support Peer-to-Peer Directly to HDM-DB.....	1152
C	Memory Protocol Tables	1153
C.1	HDM-D and HDM-DB Requests	1155
C.1.1	Forward Flows for HDM-D.....	1159
C.1.2	BISnp for HDM-DB	1161
C.2	HDM-H Requests	1163
C.3	HDM-D/HDM-DB RxD	1165
C.4	HDM-H RxD	1166

Figures

1-1	Conceptual Diagram of Device Attached to Processor via CXL.....	60
1-2	Fan-out and Pooling Enabled by Switches.....	61
1-3	Direct Peer-to-Peer Access to an HDM Memory by PCIe/CXL Devices without Going through the Host	62
1-4	Shared Memory across Multiple Virtual Hierarchies	62
1-5	CPU Flex Bus Port Example	63
1-6	Flex Bus Usage Model Examples.....	64
1-7	Remote Far Memory Usage Model Example.....	64
1-8	CXL Downstream Port Connections	65
1-9	Conceptual Diagram of Flex Bus Layering	66
2-1	CXL Device Types	69
2-2	Type 1 Device - Device with Cache.....	70
2-3	Type 2 Device - Device with Memory	71
2-4	Type 2 Device - Host Bias	72
2-5	Type 2 Device - Device Bias	73
2-6	Type 3 Device - HDM-H Memory Expander	74
2-7	Head-to-LD Mapping in MH-SLDs	79
2-8	Head-to-LD Mapping in MH-MLDs.....	80

Figures

3-1	Flex Bus Layers - CXL.io Transaction Layer Highlighted	83
3-2	CXL Power Management Messages Packet Format - Non-Flit Mode	85
3-3	CXL Power Management Messages Packet Format - Flit Mode.....	85
3-4	Power Management Credits and Initialization	88
3-5	CXL EFN Messages Packet Format - Non-Flit Mode.....	90
3-6	CXL EFN Messages Packet Format - Flit Mode.....	90
3-7	ATS 64-bit Request with CXL Indication - Non-Flit Mode	91
3-8	Valid .io TLP Formats on PBR Links	94
3-9	Host Management Transaction Flows of GFD	97
3-10	CXL.cache Channels	105
3-11	CXL.cache Read Behavior	112
3-12	CXL.cache Read0 Behavior	113
3-13	CXL.cache Device to Host Write Behavior	114
3-14	CXL.cache WrInv Transaction	115
3-15	WOWrInv/F with FastGO/ExtCmp	116
3-16	CXL.cache Read0-Write Semantics	117
3-17	CXL.cache Snoop Behavior	124
3-18	CXL.mem Channels for Devices	133
3-19	CXL.mem Channels for Hosts	134
3-20	Flows for Back-Invalidate Snoops on CXL.mem Legend	166
3-21	Example BISnp with Blocking of M2S Req.....	167
3-22	BISnp Early Conflict	168
3-23	BISnp Late Conflict	169
3-24	Block BISnp with Block Response	170
3-25	Block BISnp with Cacheline Response	171
3-26	Flows for Type 1 Devices and Type 2 Devices Legend	172
3-27	Example Cacheable Read from Host	172
3-28	Example Read for Ownership from Host	173
3-29	Example Non Cacheable Read from Host	174
3-30	Example Ownership Request from Host - No Data Required	174
3-31	Example Flush from Host	175
3-32	Example Weakly Ordered Write from Host	176
3-33	Example Write from Host with Invalid Host Caches.....	177
3-34	Example Write from Host with Valid Host Caches.....	178
3-35	Example Device Read to Device-attached Memory (HDM-D)	179
3-36	Example Device Read to Device-attached Memory (HDM-DB).....	180
3-37	Example Device Write to Device-Attached Memory in Host Bias (HDM-D).....	181
3-38	Example Device Write to Device-attached Memory in Host Bias (HDM-DB).....	182
3-39	Example Device Write to Device-attached Memory	183
3-40	Example Host to Device Bias Flip (HDM-D)	184
3-41	Example MemSpecRd	185
3-42	Read from Host to HDM-H.....	185
3-43	Write from Host to All HDM Regions	186
4-1	Flex Bus Layers - CXL.io Link Layer Highlighted.....	187
4-2	Flex Bus Layers - CXL.cache + CXL.mem Link Layer Highlighted	189
4-3	CXL.cachemem Protocol Flit Overview.....	190
4-4	CXL.cachemem All Data Flit Overview	191
4-5	Example of a Protocol Flit from Device to Host	192
4-6	H0 - H2D Req + H2D Rsp.....	198
4-7	H1 - H2D Data Header + H2D Rsp + H2D Rsp	198

Figures

4-8	H2 - H2D Req + H2D Data Header	199
4-9	H3 - 4 H2D Data Header	199
4-10	H4 - M2S RwD Header	199
4-11	H5 - M2S Req	200
4-12	H6 - MAC	200
4-13	G0 - H2D/M2S Data	200
4-14	G0 - M2S Byte Enable	201
4-15	G1 - 4 H2D Rsp	201
4-16	G2 - H2D Req + H2D Data Header + H2D Rsp	201
4-17	G3 - 4 H2D Data Header + H2D Rsp	202
4-18	G4 - M2S Req + H2D Data Header	202
4-19	G5 - M2S RwD Header + H2D Rsp	202
4-20	H0 - D2H Data Header + 2 D2H Rsp + S2M NDR	203
4-21	H1 - D2H Req + D2H Data Header	203
4-22	H2 - 4 D2H Data Header + D2H Rsp	204
4-23	H3 - S2M DRS Header + S2M NDR	204
4-24	H4 - 2 S2M NDR	204
4-25	H5 - 2 S2M DRS Header	205
4-26	H6 - MAC	205
4-27	G0 - D2H/S2M Data	205
4-28	G0 - D2H Byte Enable	206
4-29	G1 - D2H Req + 2 D2H Rsp	206
4-30	G2 - D2H Req + D2H Data Header + D2H Rsp	206
4-31	G3 - 4 D2H Data Header	207
4-32	G4 - S2M DRS Header + 2 S2M NDR	207
4-33	G5 - 2 S2M NDR	207
4-34	G6 - 3 S2M DRS Header	208
4-35	LLCRD Flit Format (Only Slot 0 is Valid; Others are Reserved)	212
4-36	RETRY Flit Format (Only Slot 0 is Valid; Others are Reserved)	213
4-37	IDE Flit Format (Only Slot 0 is Valid; Others are Reserved)	213
4-38	INIT Flit Format (Only Slot 0 is Valid; Others are Reserved)	213
4-39	Retry Buffer and Related Pointers	218
4-40	CXL.cachemem Replay Diagram	224
4-41	Standard 256B Flit	227
4-42	Latency-Optimized (LOpt) 256B Flit	227
4-43	256B Packing: Slot and Subset Definition	231
4-44	256B Packing: G0/H0/HS0 HBR Messages	232
4-45	256B Packing: G0/H0 PBR Messages	232
4-46	256B Packing: G1/H1/HS1 HBR Messages	233
4-47	256B Packing: G1/H1 PBR Messages	233
4-48	256B Packing: G2/H2/HS2 HBR Messages	234
4-49	256B Packing: G2/H2 PBR Messages	234
4-50	256B Packing: G3/H3/HS3 HBR Messages	235
4-51	256B Packing: G3/H3 PBR Messages	235
4-52	256B Packing: G4/H4/HS4 HBR Messages	236
4-53	256B Packing: G4/H4 PBR Messages	236
4-54	256B Packing: G5/H5/HS5 HBR Messages	237
4-55	256B Packing: G5/H5 PBR Messages	237
4-56	256B Packing: G6/H6/HS6 HBR Messages	238
4-57	256B Packing: G6/H6 PBR Messages	238

Figures

4-58	256B Packing: G7/H7/HS7 HBR Messages	239
4-59	256B Packing: G7/H7 PBR Messages	239
4-60	256B Packing: G12/H12/HS12 HBR Messages	240
4-61	256B Packing: G12/H12 PBR Messages	240
4-62	256B Packing: G13/H13/HS13 HBR Messages	241
4-63	256B Packing: G13/H13 PBR Messages	241
4-64	256B Packing: G14/H14/HS14 HBR Messages	242
4-65	256B Packing: G14/H14 PBR Messages	242
4-66	256B Packing: G15/H15/HS15 HBR Messages	243
4-67	256B Packing: G15/H15 PBR Messages	243
4-68	256B Packing: Implicit Data	244
4-69	256B Packing: Implicit Trailer RWD	244
4-70	256B Packing: Implicit Trailer DRS	244
4-71	256B Packing: Byte-Enable Trailer for D2H Data	245
4-72	Header Slot Decode Example	246
4-73	DRS Trailer Slot Decode Example	247
4-74	256B Packing: H8/HS8 Link Layer Control Message Slot Format	254
4-75	Viral Error Message Injection Standard 256B Flit	255
4-76	Viral Error Message Injection LOpt 256B Flit	255
5-1	Flex Bus Layers - CXL ARB/MUX Highlighted	258
5-2	Entry to Active Protocol Exchange	263
5-3	Example Status Exchange	264
5-4	CXL Entry to Active Example Flow	266
5-5	CXL Entry to PM State Example	267
5-6	Successful PM Entry following PM Retry	268
5-7	PM Abort before Downstream Port PM Acceptance	268
5-8	PM Abort after Downstream Port PM Acceptance	269
5-9	Example of a PMNAK Flow	270
5-10	CXL Recovery Exit Example Flow	272
5-11	CXL Exit from PM State Example	273
5-12	Both Upstream Port and Downstream Port Hide Recovery Transitions from ARB/MUX	274
5-13	Both Upstream Port and Downstream Port Notify ARB/MUX of Recovery Transitions	275
5-14	Downstream Port Hides Initial Recovery, Upstream Port Does Not	276
5-15	Upstream Port Hides Initial Recovery, Downstream Port Does Not	277
5-16	Snapshot Example during Status Synchronization	278
5-17	L1 Abort Example	279
5-18	ARB/MUX Link Management Packet Format	279
5-19	ALMP Byte Positions in Standard 256B Flit	280
5-20	ALMP Byte Positions in Latency-Optimized 256B Flit	280
6-1	Flex Bus Layers - Physical Layer Highlighted	283
6-2	Flex Bus x16 Packet Layout	287
6-3	Flex Bus x16 Protocol Interleaving Example	288
6-4	Flex Bus x8 Packet Layout	289
6-5	Flex Bus x8 Protocol Interleaving Example	290
6-6	Flex Bus x4 Packet Layout	291
6-7	CXL.io TLP Ending on Flit Boundary Example	292
6-8	Standard 256B Flit	294
6-9	CXL.io Standard 256B Flit	294

Figures

6-10	Standard 256B Flit Applied to Physical Lanes (x16)	296
6-11	Latency-Optimized 256B Flit	297
6-12	CXL.io Latency-Optimized 256B Flit	299
6-13	Different Methods for Generating 6-Byte CRC	300
6-14	Flex Bus Mode Negotiation during Link Training (Sample Flow)	306
6-15	NULL Flit with EDS and Sync Header Bypass Optimization	313
6-16	NULL Flit with EDS and 128b/130b Encoding	314
7-1	Example of a Single VCS	315
7-2	Example of a Multiple VCS with SLD Ports	316
7-3	Example of a Multiple Root Switch Port with Pooled Memory Devices	317
7-4	Static CXL Switch with Two VCSs	319
7-5	Example of CXL Switch Initialization when FM Boots First	320
7-6	Example of CXL Switch after Initialization Completes	321
7-7	Example of Switch with Fabric Manager and Host Boot Simultaneously	322
7-8	Example of Simultaneous Boot after Binding	323
7-9	Example of Binding and Unbinding of an SLD Port	324
7-10	Example of CXL Switch Configuration after an Unbind Command	325
7-11	Example of CXL Switch Configuration after a Bind Command	326
7-12	Example of a CXL Switch before Binding of LDs within Pooled Device	327
7-13	Example of a CXL Switch after Binding of LD-ID 1 within Pooled Device	328
7-14	Example of a CXL Switch after Binding of LD-IDs 0 and 1 within Pooled Device	329
7-15	Multi-function Upstream vPPB.....	337
7-16	Single-function Mailbox CCI.....	337
7-17	CXL Switch with a Downstream Link Auto-negotiated to Operate in RCD Mode	338
7-18	Example of Fabric Management Model	341
7-19	CCI Message Format	342
7-20	Tunneling Commands to an MLD through a CXL Switch	343
7-21	Example of MLD Management Requiring Tunneling	345
7-22	Tunneling Commands to an LD in an MLD.....	360
7-23	Tunneling Commands to an LD in an MLD through a CXL Switch.....	360
7-24	Tunneling Commands to the LD Pool CCI in a Multi-Headed Device	361
7-25	High-level CXL Fabric Diagram	388
7-26	ML Accelerator Use Case	389
7-27	HPC/Analytics Use Case	389
7-28	Sample System Topology for Composable Systems.....	390
7-29	Example Host Physical Address View	392
7-30	Example HPA Mapping to DMPs	393
7-31	G-FAM Request Routing, Interleaving, and Address Translations	395
7-32	Memory Access Protection Levels	399
7-33	GFD Dynamic Capacity Access Protections	400
7-34	PBR Fabric Providing LD-FAM and G-FAM Resources.....	401
7-35	PBR Fabric Providing Only G-FAM Resources	401
7-36	CXL Fabric Example with Multiple Host Domains and Memory Types	403
7-37	Example Host Physical Address View with GFD and GIM	403
7-38	Example Multi-host CXL Cluster with Memory on Host and Device Exposed as GIM.	404
7-39	Example ML Cluster Supporting Cross-domain Access through GIM	405
7-40	GIM Access Flows Using FASTs	405
7-41	GIM Access Flows without FASTs.....	406
7-42	Example Supported Switch Configurations	409
7-43	Example PBR Mesh Topology	410

Figures

7-44	Example Routing Scheme for a Mesh Topology.....	411
7-45	Physical Topology and Logical View	413
7-46	Example PBR Fabric	417
7-47	ISL Message Class Sub-channels.....	433
7-48	Deadlock Avoidance Mechanism on ISL	435
7-49	Update-FC DLLP Format on ISL	436
7-50	Example Topology with Direct P2P CXL.mem with GFD.....	440
7-51	Example Topology with Direct P2P CXL.mem with MLD.....	441
7-52	Single VH	444
7-53	Shared Link Events	447
7-54	Tunneling Commands to Remote Devices	455
7-55	Tunneling Commands to Remote Devices with No Assigned PID	456
8-1	PCIe DVSEC for CXL Devices	495
8-2	Non-CXL Function Map DVSEC	507
8-3	CXL Extensions DVSEC for Ports	510
8-4	GPF DVSEC for CXL Port	514
8-5	GPF DVSEC for CXL Device	516
8-6	Register Locator DVSEC with 3 Register Block Entries	517
8-7	MLD DVSEC	519
8-8	RCD and RCH Memory Mapped Register Regions	524
8-9	RCH Downstream Port RCRB	525
8-10	RCD Upstream Port RCRB	527
8-11	PCIe DVSEC for Flex Bus Port	529
8-12	CXL Device Registers.....	601
8-13	Mailbox Registers	605
9-1	PMREQ/RESETPREP Propagation by CXL Switch.....	774
9-2	CXL Device Reset Entry Flow	775
9-3	CXL Device Sleep State Entry Flow	776
9-4	PCIe Software View of an RCH and RCD	789
9-5	One CPU Connected to a Dual-Headed RCD by Two Flex Bus Links	792
9-6	Two CPUs Connected to One CXL Device by Two Flex Bus Links.....	793
9-7	CXL Device Remaps Upstream Port and Component Registers	795
9-8	CXL Device that Does Not Remap Upstream Port and Component Registers	796
9-9	CXL Root Port/DSP State Diagram.....	799
9-10	eRCD MMIO Address Decode - Example	801
9-11	eRCD Configuration Space Decode - Example	802
9-12	Physical Topology - Example	803
9-13	Software View	804
9-14	CXL Link/Protocol Register Mapping in a CXL VH	805
9-15	CXL Link/Protocol Registers in a CXL Switch	805
9-16	One-level Interleaving at Switch - Example	808
9-17	Two-level Interleaving	808
9-18	Three-level Interleaving Example	809
9-19	Overall LSA Layout	811
9-20	Fletcher64 Checksum Algorithm in C	812
9-21	Sequence Numbers in Label Index Blocks	813
9-22	Extent List Example (No Sharing).....	818
9-23	Shared Extent List Example.....	818
9-24	DCD DPA Space Example	819
9-25	UIO Direct P2P to Interleaved HDM	833

Figures

10-1	PkgC Entry Flow Initiated by Device - Example	853
10-2	PkgC Entry Flows for CXL Type 3 Device - Example	854
10-3	PkgC Exit Flows - Triggered by Device Access to System Memory	855
10-4	PkgC Exit Flows - Execution Required by Processor.....	856
10-5	CXL Link PM Phase 1 for 256B Flit Mode.....	857
10-6	CXL Link PM Phase 1 for 68B Flit Mode.....	858
10-7	CXL Link PM Phase 2	859
10-8	CXL PM Phase 3.....	861
11-1	68B Flit: CXL.cachemem IDE Showing Aggregation of 5 Flits.....	867
11-2	68B Flit: CXL.cachemem IDE Showing Aggregation across 5 Flits where One Flit Contains MAC Header in Slot 0	868
11-3	68B Flit: More-detailed View of a 5-Flit MAC Epoch Example	869
11-4	68B Flit: Mapping of AAD Bytes for the Example Shown in Figure 11-3	870
11-5	256B Flit: Handling of Slot 0 when it Carries H8.....	871
11-6	256B Flit: Handling of Slot 0 when it Does Not Carry H8.....	871
11-7	256B Flit: Handling of Slot 15	872
11-8	Mapping of Integrity-only Protected Bits to AAD - Case 1	872
11-9	Mapping of Integrity-only Protected Bits to AAD - Case 2	872
11-10	Mapping of Integrity-only Protected Bits to AAD - Case 3	873
11-11	Standard 256B Flit - Mapping to AAD and P bits when Slot 0 carries H8.....	873
11-12	Standard 256B Flit - Mapping to AAD and P bits when Slot 0 Does Not Carry H8....	874
11-13	Latency-Optimized 256B Flit - Mapping to AAD and P Bits when Slot 0 Carries H8..	875
11-14	Latency-Optimized 256B Flit - Mapping to AAD and P Bits when Slot 0 Does Not Carry H8	876
11-15	Inclusion of the PCRC Mechanism in the AES-GCM Advanced Encryption Function ..	877
11-16	Inclusion of the PCRC Mechanism in the AES-GCM Advanced Decryption Function..	877
11-17	MAC Epochs and MAC Transmission in Case of Back-to-Back Traffic (a) Earliest MAC Header Transmit (b) Latest MAC Header Transmit in the Presence of Multi-Data Header	880
11-18	Example of MAC Header Being Received in the First Flit of the Current MAC Epoch.	881
11-19	Early Termination and Transmission of Truncated MAC Flit.....	883
11-20	CXL.cachemem IDE Transmission with Truncated MAC Flit.....	883
11-21	Link Idle Case after Transmission of Aggregation Flit Count Number of Flits	884
11-22	Various Interface Standards that are Referenced by this Specification and their Lineage	888
11-23	Active and Pending Key State Transitions	899
11-24	Reference Architecture	906
11-25	CMA/SPDM, CXL IDE, and CXL TSP Message Relationship	907
11-26	CMA/SPDM Sessions Creation Sequence	908
11-27	Optional Explicit In-band TE State Change Architecture	912
11-28	CKID-based Memory Encryption Utilizing CKID Base	917
11-29	Range-based Memory Encryption	919
11-30	Target TSP Security States	920
12-1	RCH Downstream Port Detects Error	958
12-2	RCD Upstream Port Detects Error	959
12-3	RCD RCiEP Detects Error	960
12-4	CXL Memory Error Reporting Enhancements	963
14-1	Example Test Topology.....	976
14-2	Example SHDA Topology	977
14-3	Example Single Host, Switch Attached, SLD EP (SHSW) Topology	977

Tables

14-4	Example SHSW-FM Topology	978
14-5	Example DHSW-FM Topology	979
14-6	Example DHSW-FM-MLD Topology	980
14-7	Example Topology for Two PBR Switches	981
14-8	Example Topology for a PBR Switch and an HBR Switch	982
14-9	Representation of False Sharing between Cores (on Host) and CXL Devices	983
14-10	Flow Chart of Algorithm 1a	984
14-11	Flow Chart of Algorithm 1b	985
14-12	Execute Phase for Algorithm 2	986
14-13	Compliance Testing Topology for an HBR Switch with a Single Host	1014
14-14	Compliance Testing Topology for an HBR Switch with Two Hosts	1014
14-15	Compliance Testing Topology for Two PBR Switches.....	1015
14-16	Compliance Testing Topology for a PBR Switch and an HBR Switch	1016
14-17	LTSSM Hot Reset Propagation to SLDs (PBR+HBR Switch)	1019
14-18	Secondary Bus Reset (SBR) Hot Reset Propagation to SLDs (PBR+HBR Switch) ...	1022
14-19	PCIe DVSEC for Test Capability	1131
A-1	Profile D - Giant Cache Model	1151

Tables

1-1	Terminology/Acronyms	48
1-2	Reference Documents	59
2-1	LD-ID Link Local TLP Prefix	76
2-2	MLD PCIe Registers	76
3-1	CXL Power Management Messages - Data Payload Field Definitions	86
3-2	PMREQ Field Definitions	88
3-3	Optional PCIe Features Required for CXL	90
3-4	PBR TLP Header (PTH) Format	93
3-5	NOP-TLP Header Format	93
3-6	Local Prefix Header Format	93
3-7	VendPrefixL0 on Non-MLD Edge HBR Links	94
3-8	PBR VDM	95
3-9	CXL Fabric Vendor Defined Messages	96
3-10	GAM VDM Payload	102
3-11	RTUpdate VDM Payload	103
3-12	CXL.cache Channel Crediting Summary	106
3-13	CXL.cache - D2H Request Fields.....	106
3-14	Non Temporal Encodings	107
3-15	CXL.cache - D2H Response Fields.....	107
3-16	CXL.cache - D2H Data Header Fields	108
3-17	CXL.cache - H2D Request Fields	108
3-18	CXL.cache - H2D Response Fields	109
3-19	RSP_PRE Encodings	109
3-20	Cache State Encoding for H2D Response	110
3-21	CXL.cache - H2D Data Header Fields	110
3-22	CXL.cache - Device to Host Requests	117
3-23	D2H Request (Targeting Non Device-attached Memory) Supported H2D Responses	121
3-24	D2H Request (Targeting Device-attached Memory) Supported Responses	122

Tables

3-25	D2H Response Encodings	123
3-26	CXL.cache – Mapping of H2D Requests to D2H Responses	125
3-27	H2D Response Opcode Encodings	125
3-28	Allowed Opcodes for D2H Requests per Buried Cache State	131
3-29	Impact of DevLoad Indication on Host/Peer Request Rate Throttling	137
3-30	Recommended Host/Peer Adjustment to Request Rate Throttling	138
3-31	Factors for Determining IntLoad	139
3-32	Additional Factors for Determining DevLoad in MLDs.....	144
3-33	Additional Factors for Determining DevLoad in MLDs/GFDs	146
3-34	M2S Request Fields	148
3-35	M2S Req Memory Opcodes	149
3-36	Metadata Field Definition	150
3-37	Meta0-State Value Definition (HDM-D/HDM-DB Devices)	150
3-38	Snoop Type Definition	151
3-39	M2S Req Usage	151
3-40	M2S RwD Fields.....	152
3-41	M2S RwD Memory Opcodes	153
3-42	M2S RwD Usage	154
3-43	RwD Trailers	154
3-44	M2S BIRsp Fields.....	155
3-45	M2S BIRsp Memory Opcodes	155
3-46	S2M BISnp Fields.....	156
3-47	S2M BISnp Opcodes	156
3-48	Block (Blk) Enable Encoding in Address[7:6]	157
3-49	S2M NDR Fields.....	157
3-50	S2M NDR Opcodes	157
3-51	DevLoad Definition	158
3-52	S2M DRS Fields	158
3-53	S2M DRS Opcodes	159
3-54	DRS Trailers	159
3-55	Allowed Opcodes for HDM-D/HDM-DB Req and RwD Messages per Buried Cache State	161
3-56	Upstream Ordering Summary	162
3-57	Downstream Ordering Summary	163
3-58	Device In-Out Ordering Summary	164
3-59	Host In-Out Ordering Summary	165
4-1	CXL.cachemem Link Layer Flit Header Definition	193
4-2	Type Encoding.....	193
4-3	Legal Values of Sz and BE Fields	194
4-4	CXL.cachemem Credit Return Encodings	195
4-5	ReqCrd/DataCrd/RspCrd Channel Mapping	195
4-6	Slot Format Field Encoding	196
4-7	H2D/M2S Slot Formats	196
4-8	D2H/S2M Slot Formats	197
4-9	CXL.cachemem Link Layer Control Types	210
4-10	CXL.cachemem Link Layer Control Details	211
4-11	Control Flits and Their Effect on Sender and Receiver States	219
4-12	Local Retry State Transitions	221
4-13	Remote Retry State Transition	223
4-14	256B G-Slot Formats	228

Tables

4-15	256B H-Slot Formats	229
4-16	256B HS-Slot Formats	230
4-17	Trailer Size and Modes Supported per Channel	246
4-18	128B Group Maximum Message Rates	248
4-19	Credit Returned Encoding	250
4-20	256B Flit Mode Control Message Details	253
5-1	vLSM States Maintained per Link Layer Interface	259
5-2	ARB/MUX Multiple vLSM Resolution Table	260
5-3	ARB/MUX State Transition Table	261
5-4	vLSM State Resolution after Status Exchange	264
5-5	ALMP Byte 1 Encoding	280
5-6	ALMP Byte 2 and 3 Encodings for vLSM ALMP	281
5-7	ALMP Byte 2 and 3 Encodings for L0p Negotiation ALMP	281
6-1	Flex Bus.CXL Link Speeds and Widths for Normal and Degraded Mode	284
6-2	Flex Bus.CXL Protocol IDs	285
6-3	Protocol ID Framing Errors	293
6-4	256B Flit Mode vs. 68B Flit Mode Operation	293
6-5	256B Flit Header	295
6-6	Flit Type[1:0]	295
6-7	Latency-Optimized Flit Processing for CRC Scenarios	298
6-8	Byte Mapping for Input to PCIe 8B CRC Generation	300
6-9	Modified TS1/TS2 Ordered Set for Flex Bus Mode Negotiation	302
6-10	Additional Information on Symbols 8-9 of Modified TS1/TS2 Ordered Set	303
6-11	Additional Information on Symbols 12-14 of Modified TS1/TS2 Ordered Sets	303
6-12	VH vs. RCD Link Training Resolution	307
6-13	Flit Mode and VH Negotiation	309
6-14	Rules of Enable Low-latency Mode Features	311
6-15	Sync Header Bypass Applicability and Ordered Set Insertion Rate.....	311
7-1	CXL Switch Sideband Signal Requirements	323
7-2	MLD Type 1 Configuration Space Header.....	330
7-3	MLD PCI-compatible Configuration Registers.....	330
7-4	MLD PCIe Capability Structure	330
7-5	MLD Secondary PCIe Capability Structure.....	333
7-6	MLD Physical Layer 16.0 GT/s Extended Capability	333
7-7	MLD Physical Layer 32.0 GT/s Extended Capability	334
7-8	MLD Lane Margining at the Receiver Extended Capability.....	334
7-9	MLD ACS Extended Capability	335
7-10	MLD Advanced Error Reporting Extended Capability	336
7-11	MLD PPB DPC Extended Capability.....	337
7-12	CXL Switch Message Management.....	340
7-13	CXL Switch RAS.....	340
7-14	CCI Message Format	342
7-15	FM API Command Sets	349
7-16	Identify Switch Device Response Payload	350
7-17	Get Physical Port State Request Payload	350
7-18	Get Physical Port State Response Payload	351
7-19	Get Physical Port State Port Information Block Format	351
7-20	Physical Port Control Request Payload	353
7-21	Send PPB CXL.io Configuration Request Input Payload	353

Tables

7-22	Send PPB CXL.io Configuration Request Output Payload	353
7-23	Get Domain Validation SV State Response Payload	354
7-24	Set Domain Validation SV Request Payload.....	354
7-25	Get VCS Domain Validation SV State Request Payload	355
7-26	Get VCS Domain Validation SV State Response Payload	355
7-27	Get Domain Validation SV Request Payload	355
7-28	Get Domain Validation SV Response Payload	355
7-29	Virtual Switch Command Set Requirements	355
7-30	Get Virtual CXL Switch Info Request Payload	356
7-31	Get Virtual CXL Switch Info Response Payload	356
7-32	Get Virtual CXL Switch Info VCS Information Block Format	357
7-33	Bind vPPB Request Payload	358
7-34	Unbind vPPB Request Payload.....	359
7-35	Generate AER Event Request Payload	359
7-36	MLD Port Command Set Requirements.....	360
7-37	Tunnel Management Command Request Payload	362
7-38	Tunnel Management Command Response Payload	362
7-39	Send LD CXL.io Configuration Request Payload	363
7-40	Send LD CXL.io Configuration Response Payload	363
7-41	Send LD CXL.io Memory Request Payload.....	364
7-42	Send LD CXL.io Memory Request Response Payload.....	364
7-43	MLD Component Command Set Requirements	364
7-44	Get LD Info Response Payload	365
7-45	Get LD Allocations Request Payload.....	365
7-46	Get LD Allocations Response Payload.....	366
7-47	LD Allocations List Format	366
7-48	Set LD Allocations Request Payload	367
7-49	Set LD Allocations Response Payload	367
7-50	Payload for Get QoS Control Response, Set QoS Control Request, and Set QoS Control Response	367
7-51	Get QoS Status Response Payload.....	369
7-52	Payload for Get QoS Allocated BW Request.....	369
7-53	Payload for Get QoS Allocated BW Response	369
7-54	Payload for Set QoS Allocated BW Request, and Set QoS Allocated BW Response ..	370
7-55	Payload for Get QoS BW Limit Request	370
7-56	Payload for Get QoS BW Limit Response	370
7-57	Payload for Set QoS BW Limit Request, and Set QoS BW Limit Response	371
7-58	Get Multi-Headed Info Request Payload	372
7-59	Get Multi-Headed Info Response Payload	372
7-60	Get Head Info Request Payload.....	372
7-61	Get Head Info Response Payload.....	373
7-62	Get Head Info Head Information Block Format	373
7-63	Get DCD Info Response Payload	374
7-64	Get Host DC Region Configuration Request Payload	375
7-65	Get Host DC Region Configuration Response Payload	375
7-66	DC Region Configuration	376
7-67	Set DC Region Configuration Request and Response Payload	377
7-68	Get DC Region Extent Lists Request Payload.....	378
7-69	Get DC Region Extent Lists Response Payload.....	378
7-70	Initiate Dynamic Capacity Add Request Payload	380

Tables

7-71	Initiate Dynamic Capacity Release Request Payload	381
7-72	Dynamic Capacity Add Reference Request Payload	382
7-73	Dynamic Capacity Remove Reference Request Payload	383
7-74	Dynamic Capacity List Tags Request Payload	383
7-75	Dynamic Capacity List Tags Response Payload	383
7-76	Dynamic Capacity Tag Information	384
7-77	Physical Switch Events Record Format	385
7-78	Virtual CXL Switch Event Record Format	386
7-79	MLD Port Event Records Payload	387
7-80	Differences between LD-FAM and G-FAM	393
7-81	Fabric Segment Size Table	396
7-82	Segment Table Intlv[3:0] Field Encoding	396
7-83	Segment Table Gran[3:0] Field Encoding	397
7-84	PBR Fabric Decoding and Routing, by Message Class	414
7-85	Optional Architected Dynamic Routing Modes	416
7-86	Summary of CacheID Field	420
7-87	Summary of HBR Switch Routing for CXL.cache Message Classes	420
7-88	Summary of PBR Switch Routing for CXL.cache Message Classes	421
7-89	Summary of LD-ID Field	421
7-90	Summary of BI-ID Field	422
7-91	Summary of HBR Switch Routing for CXL.mem Message Classes	422
7-92	Summary of PBR Switch Routing for CXL.mem Message Classes	423
7-93	HBR Switch Port Processing Table for CXL.io	424
7-94	HBR Switch Port Processing Table for CXL.cache	424
7-95	HBR Switch Port Processing Table for CXL.mem	425
7-96	PBR Switch Port Processing Table for CXL.io	426
7-97	PBR Switch Port Processing Table for CXL.cache	426
7-98	PBR Switch Port Processing Table for CXL.mem	427
7-99	ISL Type 1 Configuration Space Header	428
7-100	ISL PCI-compatible Configuration Space Header.....	428
7-101	ISL PCIe Capability Structure.....	428
7-102	ISL Secondary PCIe Extended Capability	430
7-103	ISL Physical Layer 16.0 GT/s Extended Capability.....	431
7-104	ISL Physical Layer 32.0 GT/s Extended Capability.....	431
7-105	ISL Physical Layer 64.0 GT/s Extended Capability.....	432
7-106	ISL Lane Margining at the Receiver Extended Capability	432
7-107	PBR Fabric .io Ordering Table, Non-UIO	434
7-108	PBR Fabric .io Ordering Table, UIO	434
7-109	Link Partner Info Payload	449
7-110	Far End Device Type Detection	451
7-111	Identify PBR Switch Response Payload	455
7-112	Fabric Crawl Out Request Payload	457
7-113	Fabric Crawl Out Response Payload	457
7-114	Get PBR Link Partner Info Request Payload.....	458
7-115	Get PBR Link Partner Info Response Payload.....	458
7-116	Link Partner Info Format	458
7-117	Get PID Target List Request Payload.....	459
7-118	Get PID Target List Response Payload.....	459
7-119	Target List Format	459
7-120	Configure PID Assignment Request Payload	460

Tables

7-121	PID Assignment	460
7-122	Get PID Binding Request Payload	460
7-123	Get PID Binding Response Payload	461
7-124	Configure PID Binding Request Payload	461
7-125	Get Table Descriptors Request Payload	462
7-126	Get Table Descriptors Response Payload	462
7-127	Get Table Descriptor Format	463
7-128	Get DRT Request Payload	463
7-129	Get DRT Response Payload	463
7-130	DRT Entry Format	464
7-131	Set DRT Request Payload	464
7-132	Get RGT Request Payload	465
7-133	Get RGT Response Payload	465
7-134	RGT Entry Format	465
7-135	Set RGT Request Payload	466
7-136	Get LDST/IDT Capabilities Request Payload	466
7-137	Get LDST/IDT Capabilities Response Payload	467
7-138	Set LDST/IDT Configuration Request Payload	468
7-139	Get LDST Segment Entries Request Payload	468
7-140	Get LDST Segment Entries Response Payload	469
7-141	LDST Segment Entry Format	469
7-142	Set LDST Segment Entries Request Payload	470
7-143	Get LDST IDT DPID Entries Request Payload	471
7-144	Get LDST IDT DPID Entries Response Payload	471
7-145	Set LDST IDT DPID Entries Request Payload	472
7-146	Get Completer ID-Based Re-Router Entries Request Payload	472
7-147	Get Completer ID-Based Re-Router Entries Response Payload	473
7-148	Completer ID-Based Re-Router Entry	473
7-149	Set Completer ID-Based Re-Router Entries Request Payload	474
7-150	Get LDST Access Vector Request Payload	474
7-151	Get LDST Access Vector Response Payload	474
7-152	LDST Access Vector	475
7-153	Get VCS LDST Access Vector Request Payload	475
7-154	Configure VCS LDST Access Request Payload	476
7-155	Identify GAE Request Payload	476
7-156	Identify GAE Response Payload	477
7-157	vPPB Global Memory Support Info	477
7-158	Get PID Interrupt Vector Request Payload	478
7-159	Get PID Interrupt Vector Response Payload	478
7-160	PID Interrupt Vector	478
7-161	Get PID Access Vectors Request Payload	479
7-162	Get PID Access Vectors Response Payload	479
7-163	PID Access Vector	479
7-164	Get FAST/IDT Capabilities Request Payload	480
7-165	Get FAST/IDT Capabilities Response Payload	480
7-166	vPPB PID List Entry Format	480
7-167	Set FAST/IDT Configuration Request Payload	481
7-168	Get FAST Segment Entries Request Payload	482
7-169	Get FAST Segment Entries Response Payload	482
7-170	FAST Segment Entry Format	482

Tables

7-171	Set FAST Segment Entries Request Payload.....	483
7-172	Get IDT DPID Entries Request Payload.....	484
7-173	Get IDT DPID Entries Response Payload.....	484
7-174	Set IDT DPID Entries Request Payload	485
7-175	Proxy GFD Management Command Request Payload.....	486
7-176	Proxy GFD Management Command Response Payload.....	486
7-177	Get Proxy Thread Status Request Payload	486
7-178	Get Proxy Thread Status Response Payload	487
7-179	Cancel Proxy Thread Request Payload.....	487
7-180	Cancel Proxy Thread Response Payload.....	487
7-181	Identify VCS GAE Request Payload	488
7-182	Get VCS PID Access Vectors Request Payload	489
7-183	Configure VCS PID Access Request Payload	489
7-184	Get VendPrefixL0 State Request Payload	490
7-185	Get VendPrefixL0 State Response Payload	490
7-186	Set VendPrefixL0 State Request Payload	491
8-1	Register Attributes	492
8-2	CXL DVSEC ID Assignment	493
8-3	CXL DOE Type Assignment	494
8-4	PCIe DVSEC CXL Devices - Header	495
8-5	Non-CXL Function Map DVSEC - Header	507
8-6	CXL Extensions DVSEC for Ports - Header	510
8-7	GPF DVSEC for CXL Port - Header	515
8-8	GPF DVSEC for CXL Device - Header	516
8-9	Register Locator DVSEC - Header	518
8-10	Designated Vendor Specific Register Block Header	519
8-11	MLD DVSEC - Header	520
8-12	Coherent Device Attributes - Data Object Header	520
8-13	Read Entry Request	521
8-14	Read Entry Response.....	521
8-15	Memory Device PCIe Capabilities and Extended Capabilities.....	522
8-16	PCI Header - Class Code Register (Offset 09h) for Switch Mailbox CCI	522
8-17	CXL Memory Mapped Register Regions	523
8-18	RCH Downstream Port PCIe Capabilities and Extended Capabilities	525
8-19	RCD Upstream Port PCIe Capabilities and Extended Capabilities	528
8-20	PCIe DVSEC Header Register Settings for Flex Bus Port	529
8-21	CXL Subsystem Component Register Ranges	534
8-22	CXL_Capability_ID Assignment	535
8-23	CXL.cache and CXL.mem Architectural Register Discovery	536
8-24	CXL.cache and CXL.mem Architectural Register Header Example (Primary Range)	536
8-25	CXL.cache and CXL.mem Architectural Register Header Example (Any Extended Range)	536
8-26	Device Trust Level	550
8-27	CXL.mem Read Response - Error Cases	559
8-28	CXL Extended Security Structure Entry Count (Offset 00h)	569
8-29	Root Port n Security Policy Register (Offset 8*n-4)	569
8-30	Root Port n ID Register (Offset 8*n).....	570
8-31	BAR Virtualization ACL Register Block Layout.....	593
8-32	CPMU Register Layout (Version=1)	594

Tables

8-33	Filter ID and Values	600
8-34	Command Return Codes	607
8-35	CXL Memory Device Capabilities Identifiers	610
8-36	Switch Mailbox CCI Capabilities Identifiers	611
8-37	Generic Component Command Opcodes	613
8-38	Identify Output Payload	615
8-39	Background Operation Status Output Payload	616
8-40	Get Response Message Limit Output Payload	616
8-41	Set Response Message Limit Input Payload.....	617
8-42	Set Response Message Limit Output Payload	617
8-43	Common Event Record Format	618
8-44	Component Identifier Format	620
8-45	General Media Event Record	621
8-46	DRAM Event Record	624
8-47	Memory Module Event Record	628
8-48	Memory Sparing Event Record	629
8-49	Vendor Specific Event Record	630
8-50	Dynamic Capacity Event Record	631
8-51	Dynamic Capacity Extent	632
8-52	Get Event Records Input Payload	633
8-53	Get Event Records Output Payload	634
8-54	Clear Event Records Input Payload	635
8-55	Get Event Interrupt Policy Output Payload.....	636
8-56	Set Event Interrupt Policy Input Payload	638
8-57	Payload for Get MCTP Event Interrupt Policy Output, Set MCTP Event Interrupt Policy Input, and Set MCTP Event Interrupt Policy Output	639
8-58	Event Notification Input Payload	640
8-59	Enhanced Event Notification Input Payload	641
8-60	GFD Notification Type Values	642
8-61	GFD Notification Type Triggers	643
8-62	Enhanced Event Notification Input Payload	645
8-63	Get GAM Buffer Response Payload.....	646
8-64	Set GAM Buffer Address Request Payload.....	646
8-65	Get FW Info Output Payload	647
8-66	Transfer FW Input Payload	650
8-67	Activate FW Input Payload	651
8-68	Get Timestamp Output Payload	652
8-69	Set Timestamp Input Payload	652
8-70	Get Supported Logs Output Payload	653
8-71	Get Supported Logs Supported Log Entry	653
8-72	Get Log Input Payload	654
8-73	Get Log Output Payload	654
8-74	CEL Output Payload.....	654
8-75	CEL Entry Structure	655
8-76	Component State Dump Log Population Methods and Triggers	656
8-77	Component State Dump Log Format	657
8-78	DDR5 Error Check Scrub (ECS) Log	658
8-79	Media Test Capability Log Output Payload	659
8-80	Media Test Capability Log Common Header	659
8-81	Media Test Capability Log Entry Structure	660

Tables

8-82	Media Test Results Short Log	662
8-83	Media Test Results Short Log Entry Common Header	662
8-84	Media Test Results Short Log Entry Structure	663
8-85	Media Test Results Long Log	664
8-86	Media Test Results Long Log Entry Common Header	664
8-87	Media Test Results Long Log Entry Structure	664
8-88	Error Signature	665
8-89	Get Log Capabilities Input Payload	667
8-90	Get Log Capabilities Output Payload	667
8-91	Clear Log Input Payload.....	668
8-92	Populate Log Input Payload	668
8-93	Get Supported Logs Sub-List Input Payload	669
8-94	Get Supported Logs Sub-List Output Payload	669
8-95	Get Supported Features Input Payload.....	670
8-96	Get Supported Features Output Payload.....	670
8-97	Get Supported Features Supported Feature Entry	670
8-98	Feature Attribute(s) Value after Reset	671
8-99	Get Feature Input Payload	672
8-100	Get Feature Output Payload	672
8-101	Set Feature Input Payload.....	674
8-102	Perform Maintenance Input Payload	675
8-103	sPPR Maintenance Input Payload	677
8-104	hPPR Maintenance Input Payload	677
8-105	Memory Sparing Input Payload	679
8-106	Device Built-in Test Input Payload	680
8-107	Test Parameters	680
8-108	Common Configuration Parameters for Media Test Subclass	681
8-109	Test Parameters Entry Media Test Subclass	681
8-110	Maintenance Operation: Classes, Subclasses, and Feature UUIDs	682
8-111	Common Maintenance Operation Feature Format	683
8-112	Supported Feature Entry for the sPPR Feature	684
8-113	sPPR Feature Readable Attributes	684
8-114	sPPR Feature Writable Attributes	685
8-115	Supported Feature Entry for the hPPR Feature	686
8-116	hPPR Feature Readable Attributes	686
8-117	hPPR Feature Writable Attributes	687
8-118	Supported Feature Entry for the Memory Sparing Feature	688
8-119	Memory Sparing Feature Readable Attributes	688
8-120	Memory Sparing Feature Writable Attributes	689
8-121	Identify PBR Component Response Payload	690
8-122	Claim Ownership Request Payload.....	691
8-123	Claim Ownership Response Payload.....	691
8-124	Read CDAT Request Payload.....	692
8-125	Read CDAT Response Payload.....	692
8-126	CXL Memory Device Command Opcodes	693
8-127	Identify Memory Device Output Payload	696
8-128	Get Partition Info Output Payload.....	698
8-129	Set Partition Info Input Payload	699
8-130	Get LSA Input Payload.....	699
8-131	Get LSA Output Payload.....	700

Tables

8-132	Set LSA Input Payload	700
8-133	Get Health Info Output Payload	701
8-134	Get Alert Configuration Output Payload	703
8-135	Set Alert Configuration Input Payload	705
8-136	Get Shutdown State Output Payload	706
8-137	Set Shutdown State Input Payload	706
8-138	Get Poison List Input Payload	708
8-139	Get Poison List Output Payload	708
8-140	Media Error Record	709
8-141	Inject Poison Input Payload	710
8-142	Clear Poison Input Payload	711
8-143	Get Scan Media Capabilities Input Payload	711
8-144	Get Scan Media Capabilities Output Payload	712
8-145	Scan Media Input Payload	713
8-146	Get Scan Media Results Output Payload	714
8-147	Media Operation Input Payload	716
8-148	DPA Range Format	717
8-149	Media Operations Classes and Subclasses	717
8-150	Discovery Operation-specific Arguments	717
8-151	Media Operations Output Payload – Discovery Operation	717
8-152	Supported Operations List Entries	718
8-153	Get Security State Output Payload	718
8-154	Set Passphrase Input Payload	719
8-155	Disable Passphrase Input Payload	720
8-156	Unlock Input Payload	720
8-157	Passphrase Secure Erase Input Payload	721
8-158	Security Send Input Payload	722
8-159	Security Receive Input Payload	723
8-160	Security Receive Output Payload	723
8-161	Get SLD QoS Control Output Payload and Set SLD QoS Control Input Payload	724
8-162	Get SLD QoS Status Output Payload	725
8-163	Get Dynamic Capacity Configuration Input Payload	725
8-164	Get Dynamic Capacity Configuration Output Payload	725
8-165	DC Region Configuration	726
8-166	Get Dynamic Capacity Extent List Input Payload	727
8-167	Get Dynamic Capacity Extent List Output Payload	727
8-168	Add Dynamic Capacity Response Input Payload	728
8-169	Updated Extent	728
8-170	Release Dynamic Capacity Input Payload	729
8-171	Identify GFD Response Payload	730
8-172	Get GFD Status Response Payload	732
8-173	Get GFD DC Region Configuration Request Payload	733
8-174	Get GFD DC Region Configuration Response Payload	734
8-175	GFD DC Region Configuration	734
8-176	Set GFD DC Region Configuration Request Payload	735
8-177	Get GFD DC Region Extent Lists Request Payload	736
8-178	Get GFD DC Region Extent Lists Response Payload	736
8-179	Get GFD DMP Configuration Request Payload	737
8-180	Get GFD DMP Configuration Response Payload	737
8-181	GFD DMP Configuration	737

Tables

8-182	Set GFD DMP Configuration Request Payload	738
8-183	GFD Dynamic Capacity Add Request Payload	741
8-184	GFD Dynamic Capacity Add Response Payload	741
8-185	Initiate Dynamic Capacity Release Request Payload	743
8-186	GFD Dynamic Capacity Release Response Payload	744
8-187	GFD Dynamic Capacity Add Reference Request Payload.....	745
8-188	GFD Dynamic Capacity Remove Reference Request Payload.....	745
8-189	GFD Dynamic Capacity List Tags Request Payload	746
8-190	GFD Dynamic Capacity List Tags Response Payload	746
8-191	GFD Dynamic Capacity Tag Information	746
8-192	Get GFD SAT Entry Request Payload	747
8-193	Get GFD SAT Entry Response Payload	747
8-194	GFD SAT Entry Format	747
8-195	Set GFD SAT Entry Request Payload	748
8-196	GFD SAT Update Format	749
8-197	QoS Payload for Get GFD QoS Control Response, Set GFD QoS Control Request, and Set GFD QoS Control Response	749
8-198	Get GFD QoS Status Response Payload	750
8-199	Payload for Get GFD QoS BW Limit Request.....	751
8-200	Payload for Get GFD QoS BW Limit Response.....	751
8-201	Payload for Set GFD BW Limit Request and Set GFD QoS BW Limit Response	752
8-202	Get GDT Configuration Request Payload	752
8-203	Get GDT Configuration Response Payload	753
8-204	GDT Entry Format	753
8-205	Set GDT Configuration Request Payload	754
8-206	Supported Feature Entry for the Device Patrol Scrub Control Feature	755
8-207	Device Patrol Scrub Control Feature Readable Attributes	756
8-208	Device Patrol Scrub Control Feature Writable Attributes	756
8-209	Supported Feature Entry for the DDR5 ECS Control Feature	757
8-210	DDR5 ECS Control Feature Readable Attributes	758
8-211	DDR5 ECS Control Feature Writable Attributes	759
8-212	Supported Feature Entry for the Advanced Programmable Corrected Volatile Memory Error Threshold Feature	759
8-213	Advanced Programmable Corrected Volatile Memory Error Threshold Feature Readable Attributes	760
8-214	Advanced Programmable Corrected Volatile Memory Error Threshold Feature Writable Attributes	764
8-215	CXL FM API Command Opcodes	767
9-1	Event Sequencing for Reset and Sx Flows	773
9-2	CXL Switch Behavior Message Aggregation Rules	773
9-3	GPF Energy Calculation Example	784
9-4	Memory Decode Rules in Presence of One CPU/Two Flex Bus Links	793
9-5	Memory Decode Rules in Presence of Two CPU/Two Flex Bus Links	794
9-6	12-Way Device-level Interleave at IGB	810
9-7	6-Way Device-level Interleave at IGB	810
9-8	3-Way Device-level Interleave at IGB	810
9-9	Label Index Block Layout	813
9-10	Region Label Layout	815
9-11	Namespace Label Layout	816
9-12	Vendor Specific Label Layout	817

Tables

9-13	Downstream Port Handling of BISnp	824
9-14	Downstream Port Handling of BIRsp	824
9-15	CXL Type 2 Device Behavior in Fallback Operation Mode	828
9-16	Downstream Port Handling of D2H Request Messages	830
9-17	Downstream Port Handling of H2D Response Message and H2D Request Message	830
9-18	Handling of UIO Accesses	834
9-19	CEDT Header	837
9-20	CEDT Structure Types	837
9-21	CHBS Structure	838
9-22	CFMWS Structure	838
9-23	CXIMS Structure	841
9-24	RDPAS Structure	842
9-25	CSDS Structure	843
9-26	_OSC Capabilities Buffer DWORDs	843
9-27	Interpretation of CXL _OSC Support Field	844
9-28	Interpretation of CXL _OSC Control Field, Passed in via Arg3	844
9-29	Interpretation of CXL _OSC Control Field, Returned Value	845
9-30	_DSM Definitions for CXL Root Device	846
9-31	_DSM for Retrieving QTG, Inputs, and Outputs	848
10-1	Runtime-Control - CXL vs. PCIe Control Methodologies	851
10-2	PMReq(), PMRsp(), and PMGo() Encoding	853
11-1	Mapping of PCIe IDE to CXL.io	866
11-2	CXL_IDE_KM Request Header	890
11-3	CXL_IDE_KM Successful Response Header	890
11-4	CXL_IDE_KM Generic Error Conditions	891
11-5	CXL_QUERY Request	891
11-6	CXL_QUERY Processing Errors	891
11-7	Successful CXL_QUERY_RESP Response	891
11-8	CXL_KEY_PROG Request	893
11-9	CXL_KEY_PROG Processing Errors	894
11-10	CXL_KP_ACK Response	894
11-11	CXL_K_SET_GO Request	896
11-12	CXL_K_SET_GO Error Conditions	896
11-13	CXL_K_SET_STOP Request	897
11-14	CXL_K_SET_STOP Error Conditions	897
11-15	CXL_K_GOSTOP_ACK Response	897
11-16	CXL_GETKEY Request	898
11-17	CXL_GETKEY Processing Error	898
11-18	CXL_GETKEY_ACK Response	899
11-19	Security Threats and Mitigations	905
11-20	Target Behavior for Implicit TE State Changes	910
11-21	Target Behavior for Explicit In-band TE State Changes	913
11-22	Target Behavior for Write Access Control	913
11-23	Target Behavior for Read Access Control	914
11-24	Target Behavior for Invalid CKID Ranges	918
11-25	Target Behavior for Verifying CKID Type	918
11-26	TSP Request Overview	928
11-27	TSP Response Overview	929
11-28	TSP Request Response and CMA/SPDM Sessions	930
11-29	Get Target TSP Version	931

Tables

11-30	Get Target TSP Version Response.....	931
11-31	Get Target Capabilities	932
11-32	Get Target Capabilities Response	932
11-33	Explicit In-band TE State Granularity Entry.....	935
11-34	Set Target Configuration	935
11-35	Set Target Configuration Response.....	938
11-36	Get Target Configuration.....	938
11-37	Get Target Configuration Response	939
11-38	Get Target Configuration Report	941
11-39	Get Target Configuration Report Response	941
11-40	TSP Report	942
11-41	Lock Target Configuration	943
11-42	Lock Target Configuration Response	943
11-43	Memory Range	944
11-44	Set Target TE State.....	945
11-45	Set Target TE State Response.....	945
11-46	Set Target CKID Specific Key.....	946
11-47	Set Target CKID Specific Key Response.....	946
11-48	Set Target CKID Random Key	947
11-49	Set Target CKID Random Key Response.....	948
11-50	Clear Target CKID Key.....	949
11-51	Clear Target CKID Key Response.....	949
11-52	Set Target Range Specific Key	950
11-53	Set Target Range Specific Key Response	950
11-54	Set Target Range Random Key	951
11-55	Set Target Range Random Key Response	952
11-56	Clear Target Range Key	953
11-57	Clear Target Range Key Response	953
11-58	Delayed Response.....	953
11-59	Check Target Delayed Completion	954
11-60	Get Target TE State Change Completion Response	954
11-61	Error Response	955
11-62	Error Response — Error Code, Error Data, Extended Error Data	955
12-1	CXL RAS Features	956
12-2	Device-specific Error Reporting and Nomenclature Guidelines	962
13-1	CXL Performance Attributes	969
13-2	Recommended Latency Targets for Selected CXL Transactions	970
13-3	Recommended Maximum Link Layer Latency Targets	970
13-4	CPMU Counter Units	971
13-5	Events under CXL Vendor ID	972
14-1	CRC Error Injection RETRY_PHY_REINIT: Cache CRC Injection Request	992
14-2	CRC Error Injection RETRY_ABORT: Cache CRC Injection Request	993
14-3	Link Initialization Resolution Table	1007
14-4	Hot Add Link Initialization Resolution Table	1008
14-5	Inject MAC Delay Setup	1073
14-6	Inject Unexpected MAC Setup.....	1074
14-7	CXL.io Poison Inject from Device: I/O Poison Injection Request.....	1078
14-8	CXL.io Poison Inject from Device: Multi-Write Streaming Request	1078
14-9	CXL.cache Poison Inject from Device: Cache Poison Injection Request.....	1079

Tables

14-10	CXL.cache Poison Inject from Device: Multi-Write Streaming Request	1080
14-11	CXL.cache CRC Inject from Device: Cache CRC Injection Request	1081
14-12	CXL.cache CRC Inject from Device: Multi-Write Streaming Request	1081
14-13	CXL.mem Poison Injection: Mem-Poison Injection Request	1082
14-14	CXL.mem CRC Injection: MEM CRC Injection Request	1083
14-15	Flow Control Injection: Flow Control Injection Request	1083
14-16	Flow Control Injection: Multi-Write Streaming Request	1084
14-17	Unexpected Completion Injection: Unexpected Completion Injection Request	1085
14-18	Unexpected Completion Injection: Multi-Write Streaming Request	1085
14-19	Completion Timeout Injection: Completion Timeout Injection Request	1086
14-20	Completion Timeout Injection: Multi-Write Streaming Request	1086
14-21	Memory Error Injection and Logging: Poison Injection Request	1087
14-22	Memory Error Injection and Logging: Multi-Write Streaming Request	1087
14-23	CXL.io Viral Inject from Device: I/O Viral Injection Request	1088
14-24	CXL.io Viral Inject from Device: Multi-Write Streaming Request	1089
14-25	CXL.cache Viral Inject from Device: Cache Viral Injection Request	1090
14-26	CXL.cache Viral Inject from Device: Multi-Write Streaming Request	1090
14-27	Register 1: CXL.cachemem LinkLayerErrorInjection	1104
14-28	Register 2: CXL.io LinkLayer Error Injection	1105
14-29	Register 3: Flex Bus LogPHY Error Injections	1105
14-30	CXL.io Poison Injection from Device to Host: I/O Poison Injection Request	1106
14-31	CXL.io Poison Injection from Device to Host: Multi-Write Streaming Request	1106
14-32	Device to Host Poison Injection: Cache Poison Injection Request	1107
14-33	Device to Host Poison Injection: Multi-Write Streaming Request	1108
14-34	Host to Device Poison Injection: Cache Poison Injection Request	1109
14-35	Device to Host CRC Injection: Cache Poison Injection Request	1109
14-36	Host to Device CRC Injection: Cache Poison Injection Request	1110
14-37	DVSEC Registers	1131
14-38	DVSEC CXL Test Lock (Offset 0Ah)	1131
14-39	DVSEC CXL Test Configuration Base Low (Offset 14h)	1132
14-40	DVSEC CXL Test Configuration Base High (Offset 18h)	1132
14-41	Register 9: ErrorLog1 (Offset 40h)	1132
14-42	Register 10: ErrorLog2 (Offset 48h)	1132
14-43	Register 11: ErrorLog3 (Offset 50h)	1132
14-44	Register 12: EventCtrl (Offset 60h)	1133
14-45	Register 13: EventCount (Offset 68h)	1133
14-46	Compliance Mode – Data Object Header	1134
14-47	Compliance Mode Return Values	1134
14-48	Compliance Mode Availability Request	1134
14-49	Compliance Mode Availability Response	1134
14-50	Compliance Options Value Descriptions	1135
14-51	Compliance Mode Status	1135
14-52	Compliance Mode Status Response	1136
14-53	Compliance Mode Halt All	1136
14-54	Compliance Mode Halt All Response	1136
14-55	Enable Multiple Write Streaming Algorithm on the Device	1136
14-56	Compliance Mode Multiple Write Streaming Response	1137
14-57	Enable Producer-Consumer Algorithm on the Device	1137
14-58	Compliance Mode Producer-Consumer Response	1138
14-59	Enable Algorithm 1b, Write Streaming with Bogus Writes	1138

Tables

14-60	Algorithm1b Response	1139
14-61	Enable Poison Injection into	1139
14-62	Poison Injection Response.....	1139
14-63	Enable CRC Error into Traffic	1140
14-64	CRC Injection Response.....	1140
14-65	Enable Flow Control Injection	1140
14-66	Flow Control Injection Response	1140
14-67	Enable Cache Flush Injection	1141
14-68	Cache Flush Injection Response	1141
14-69	MAC Delay Injection	1141
14-70	MAC Delay Response	1141
14-71	Unexpected MAC Injection	1142
14-72	Unexpected MAC Injection Response	1142
14-73	Enable Viral Injection	1142
14-74	Flow Control Injection Response	1143
14-75	Inject ALMP Request	1143
14-76	Inject ALMP Response	1143
14-77	Ignore Received ALMP Request	1144
14-78	Ignore Received ALMP Response	1144
14-79	Inject Bit Error in Flit Request.....	1144
14-80	Inject Bit Error in Flit Response.....	1144
14-81	Memory Device Media Poison Injection Request	1145
14-82	Memory Device Media Poison Injection Response	1145
14-83	Memory Device LSA Poison Injection Request	1145
14-85	Inject Memory Device Health Enable Memory Device Health Injection	1146
14-84	Memory Device LSA Poison Injection Response	1146
14-86	Device Health Injection Response.....	1147
A-1	Accelerator Usage Taxonomy.....	1148
C-1	Field Encoding Abbreviations	1153
C-2	Optional Codes	1154
C-3	HDM-D/HDM-DB Memory Requests	1155
C-4	HDM-D Request Forward Sub-table	1159
C-5	HDM-DB BISnp Flow	1161
C-6	HDM-H Memory Request	1163
C-7	HDM-D/HDM-DB Memory Rwd	1165
C-8	HDM-H Memory Rwd	1166



Revision History

Revision	Description	Date
3.1	- General spec updates - Merged CXL 3.0 errata (G3-G23) - Direct P2P CXL.mem for accelerators - Extended Metadata - General typo, grammar, punctuation, and formatting fixes - Changed "R/requestor" to "R/requester" to align with PCIe Base Specification - Added terms and abbreviations to Table 1-1 Chapter 3.0, "CXL Transaction Layer" and Chapter 4.0, "CXL Link Layers" - General clarifications/cleanup Chapter 6.0, "Flex Bus Physical Layer" - Added clarification on CRC corruption for CXL.cachemem viral/late poison and CXL.io nullify/poison Chapter 7.0, "Switching" - CXL PBR fabric decoding and routing defined - CXL PBR fabric FM APIs defined - CXL fabric switch reset flows defined - CXL fabric deadlock avoidance rules defined - UIO direct peer to peer support in CXL fabric defined - GFD details refined - Global Integrated Memory concept defined Chapter 8.0, "Control and Status Registers" and Chapter 9.0, "Reset, Initialization, Configuration, and Manageability" - Addressed ambiguities in CXL Header log definition - Clarified device handling of unrecognized CCI input payload size and feature input payload size - Defined the ability to abort a background operation - Expanded visibility and control over CXL memory device RAS – Memory sparing, media testing, memory scrub and DDR5 ECS - Improved visibility into CXL memory device errors – Advanced programmable corrected volatile error thresholds, added enums to describe detailed memory error causes, leveraged DMTF PLDM standard-based FRU/sub-FRU identification (Component ID field) - Ability to sanitize and zero media via media operations command - Support for setting up one writer-multiple reader type memory sharing Section 11.5, "CXL Trusted Execution Environments Security Protocol (TSP)" - Trusted Execution Environment Security Protocol – Addition of basic security threat model, behaviors and interfaces for utilizing direct attached memory devices with confidential computing scenarios. This addition defines secure CMA/SPDM request and response payloads to retrieve the device's security capabilities, configure security features on the device, and lock the device to prevent runtime configuration changes that could alter the device's behavior or leak trusted data. These optional TSP security features can be utilized with CXL IDE and/or PCIe TDISP but are not dependent on either. Chapter 14.0, "CXL Compliance Testing" - Updated test prerequisites and dependencies for uniformity - Moved Test 14.6.1.9 to its correct location at 14.6.13 - ECN fix for Test 14.7.5.3	August 7, 2023

Revision	Description	Date
3.0	- General spec updates 256B Flit mode updates - Back-Invalidate updates - Multiple CXL.cache devices per VCS - Fabric - Intra-VH P2P using UIO - Memory sharing - Merged CXL 2.0 ECNs and errata - Scrubbed terminology removing references to CXL 1.1 device/host, CXL 2.0 device/switch/host etc. and replaced with feature-specific link mode terminology (VH, RCD, eRCD, etc.) - Removed Symmetric Memory - Major additions on the SW side - Multi-headed MLD configuration - Dynamic Capacity Device (DCD) - Performance Monitoring - Chapter 7.0, "Switching" - G-FAM architecture definition completed - PBR switch SW view defined at a high level - PBR switch routing of CXL.mem/CXL.cache defined - PBR TLP header defined to route CXL.io traffic through the PBR fabric - DCD management architecture defined - Chapter 4.0, "CXL Link Layers" - Updates to 256B Packing Rules - Update BI-ID from 8 to 12 bits - Added clarification and examples for Late-Poison/Viral in 256B flit - Chapter 3.0, "CXL Transaction Layer" - Added PBR TLP header definition for CXL.io - Cleanup on HDM vs Device type terms (HDM-H, HDM-D, HDM-DB) - Update BI-ID from 8 to 12 bits - Added examples for BISnp - Chapter 14.0, "CXL Compliance Testing" - Re-ordered and organized link layer initialization tests to 68B and 256B modes - Updated switch tests to include routing types (HBR, PBR) - Added security tests to differentiate between 68B and 256B modes - Added Timeout and Isolation capability tests - Corrected DOE response for CXL.mem poison injection requests - Chapter 5.0, "CXL ARB/MUX" - Corrections and implementation notes on the different timers for PM/L0p handshakes - Implementation note clarifying there is a common ALMP for both PCI-PM and ASPM handshakes - Added clarity on Physical Layer LTSSM Recovery transitions vs Active to Retrain arc for vLSM - Chapter 6.0, "Flex Bus Physical Layer" - Corrections and updates to 6B CRC scheme as well as updated description on how to generate 6B CRC code from 8B CRC - Attached the generator matrix and system Verilog reference code for 6B CRC generation using the two methods described in the text - Flit Type encoding updates - Support for PBR flit negotiation	August 1, 2022

Revision	Description	Date
3.0	• Updates to Chapter 7.0, "Switching", Chapter 8.0, "Control and Status Registers", Chapter 12.0, "Reliability, Availability, and Serviceability", and Chapter 13.0, "Performance Considerations" — Performance Monitor interface — BI registers, discovery and configuration — Cache ID registers, discovery configuration — Create more room for CXL.cache/CXL.mem registers, raised maximum HDM decoders count for switches and RP — FM API over mailbox interface — Merged Features ECN and maintenance ECN • CXL 3.0 IDE updates for 256B Flit mode • 256B Flit Slot Packing Rules and Credit flow control extensively updated • First inclusion for Port Based Routing (PBR) Switching architecture including message definition and new slot packing • Physical Layer: NOP hint optimization added, link training resolution table for CXL 3.0, clean up of nomenclature • ARB/MUX: Corner case scenarios around L0p and Recovery and EIOSQ before ACK • Compliance Mode DOE is now required (was optional) • Compliance DVSEC interface is removed • Test 14.6.5 test updated, now requires Analyzer, and comprehends a retimer • Test 14.8.3 invalid verify step removed • Test 14.11.3.11 bug fix, test title and pass criteria are updated • Test 14.11.3.11.2 test corrected • Test 14.11.3.11.3 test corrected • General typo, grammar, punctuation, and formatting fixes • Added terms and abbreviations to Table 1-1 • Clarified mention of AES-GCM and its supporting document, NIST Special Publication 800-38D • Added mention of DSP0236, DSP0237, and DSP0238 in Section 11.4.1 • Adjusted Chapter 11.0, "CXL Security" section heading levels • In Chapter 14.0, "CXL Compliance Testing", clarified prerequisites, test equipment, and SPD-M-related command styles, and updated bit ranges and values to match CXL 3.0 v0.7 updates	August 1, 2022

Revision	Description	Date
3.0	• Chapter 6.0, "Flex Bus Physical Layer" updates for some error cases. • Flex Bus Port DVSEC updates for CXL 3.0 feature negotiation. • ALMP updates for L0p • Chapter 5.0, "CXL ARB/MUX" and Chapter 10.0, "Power Management" updates for CXL 3.0 PM negotiation flow. • Incremental changes to Back-Invalidation Snoop (BISnp) in CXL.mem including flows, ordering rules, and field names. • Added channel description to CXL.mem protocol covering BISnp and Symmetric Memory. • Added Shared FAM overview • Integrated following CXL 2.0 ECNs: – Compliance DOE 1B – Compliance DOE return value – Error Isolation on CXL.cache and CXL.mem – CXL.cachemem IDE Establishment Flow – Mailbox Ready Time – NULL CXL Capability ID – QoS Telemetry Compliance Testcases – Vendor Specific Extension to Register Locator DVSEC – Devices Operating in CXL 1.1 mode with no RCRB – Component State Dump Log – 3, 6, 12, and 16-way Memory Interleaving • Incorporated Compute Express Link (CXL) 2.0 Errata F1-F34. • Incorporated the following CXL 2.0 ECNs: "Compliance DOE 1B", "Memory Device Error Injection", and "CEDT CFMWS & QTG_DSM" • Updated Chapter 3.0, "CXL Transaction Layer", Chapter 4.0, "CXL Link Layers", Chapter 5.0, "CXL ARB/MUX", and Chapter 6.0, "Flex Bus Physical Layer" with CXL 3.0 feature support (CXL 3.0 flit format; Back-Invalidation Snoops; Symmetric Memory flows; Cache Scaling; Additional fields in CXL 2.0 flit format to enable CXL 3.0 features on devices without CXL 3.0 flit support; updated ARB/MUX flows and ALMP definitions for CXL 3.0) • Enhanced I/O feature description is in separate appendix for now – this will be merged into Chapter 3.0, "CXL Transaction Layer" in a future release.	August 1, 2022

Revision	Description	Date
2.0	- Incorporated Errata for the Compute Express Link Specification Revision 1.1. Renamed L1.1 - L1.4 to L1.0 - L1.3 in the ARB/MUX chapter to make consistent with PCI Express* (PCIe*). - Added new chapter Chapter 7.0, "Switching". Added CXL Integrity and Data Encryption definition to the Security chapter. Added support for Hot-Plug, persistent memory, memory error reporting, and telemetry. - Removed the Platform Architecture chapter - Change to CXL.mem QoS Telemetry definition to use message-based load passing from Device to host using newly defined 2-bit DevLoad field passed in all S2M messages. Chapter 3.0, "CXL Transaction Layer" - Update to ordering tables to clarify reasoning for 'Y' (bypassing). Chapter 4.0, "CXL Link Layers" - Updates for QoS and IDE support. ARB/MUX chapter clarifications around vLSM transitions and ALMPS status synchronization handshake and resolution. Chapter 6.0, "Flex Bus Physical Layer" - Updates around retimer detection, additional check during alternate protocol negotiation, and clarifications around CXL operation without 32 GT/s support. Major compliance chapter update for CXL 2.0 features. - Add Register, mailbox command, and label definitions for the enumeration and management of both volatile and persistent memory CXL devices. Chapter 7.0, "Switching" - Incorporated 0.7 draft review feedback Chapter 8.0, "Control and Status Registers" - Updated DVSEC ID 8 definition to be more scalable, deprecated Error DOE in favor of CXL Memory Configuration Interface ECR, updated HDM decoder definition to introduce DPASkip, Added CXL Snoop filter capability structure, Merged CXL Memory Configuration Interface ECR, incorporated 0.7 draft review feedback Chapter 9.0, "Reset, Initialization, Configuration, and Manageability" - Aligned device reset terminology with PCIe, Moved MEFN into different class of CXL VDMs, removed cold reset section, replaced eFLR section with CXL Reset, additional clarifications regarding GPF behavior, added firmware flow for detecting retimer mismatch in CXL 1.1 system, added Memory access/config access/error reporting flows that describe CXL 1.1 device below a switch, added section that describes memory device label storage, added definition of CEDT ACPI table, incorporated 0.7 draft review feedback Chapter 12.0, "Reliability, Availability, and Serviceability" - Added detailed flows that describe how a CXL 1.1 device, Downstream Port and Upstream Port detected errors are logged and signaled, clarified that CXL 2.0 device must keep track of poison received, updated memory error reporting section per CXL Memory Configuration Interface ECR, incorporated 0.7 draft review feedback - Added Appendix B, "Memory Protocol Tables" to define legal CXL.mem request/response messages and device state for Type 2 and Type 3 devices. - Updated to address member feedback. - Incorporated PCRC updates. - Incorporated QoS (Synchronous Load Reporting) changes. - Updated viral definition to cover the switch behavior.	October 26, 2020

Revision	Description	Date
1.1	- Added Reserved and ALMP terminology definition to Terminology/Acronyms table and also alphabetized the entries. - Completed update to CXL* terminology (mostly figures); removed disclaimer re: old terminology. - General typo fixes. - Added missing figure caption in Transaction Layer chapter. - Modified description of Deferrable Writes in Section 3.1.7 to be less restrictive. - Added clarification in Section 3.2.5.14 that ordering between CXL. - CXL.io traffic and CXL.cache traffic must be enforced by the device (e.g., between MSIs and D2H memory writes). - Removed ExtCmp reference in ItoMWr & MemWr. - Flit organization clarification: updated Figure 4-2 and added example with Figure 4-4. - Fixed typo in Packing Rules MDH section with respect to H4. - Clarified that Advanced Error Reporting (AER) is required for CXL. - Clarification on data interleave rules for CXL.mem in Section 3.3.11. - Updated Table 5-3, "ARB/MUX State Transition Table" to add missing transitions and to correct transition conditions. - Updated Section 5.1.2 to clarify rules for ALMP state change handshakes and to add rule around unexpected ALMPs. - Updated Section 5.2.1 to clarify that ALMPs must be disabled when multiple protocols are not enabled. - Updates to ARB/MUX flow diagrams. - Fixed typos in the Physical Layer interleave example figures (LCRC at the end of the TLPs instead of IDLEs). - Updated Table 6-3 to clarify Protocol ID error detection and handling. - Added Section 6.7 to clarify behavior out of recovery. - Increased the HDM size granularity from 1MB to 256MB (defined in the Flex Bus* Device DVSEC in Control and Status Registers chapter). - Updated Viral Status in the Flex Bus Device DVSEC to RWS (from RW). - Corrected the RCRB BAR definition so fields are RW instead of RWO. - Corrected typo in Flex Bus Port DVSEC size value. - Added entry to Table 8-21 to clarify that upper 7K of the 64K MEMBAR0 region is reserved. - Corrected Table 9-1 so the PME-Turn_Off/Ack handshake is used consistently as a warning for both PCIe and CXL mode. - Update Section 10.2.3 and Section 10.2.4 to remove references to EA and L2. - Updated Section 12.2.3 to clarify device handling of non-function errors. - Added additional latency recommendations to cover CXL.mem flows to Chapter 13.0; also changed wording to clarify that the latency guidelines are recommendations and not requirements. - Added compliance test chapter.	June, 2019
1.0	Initial release.	March, 2019

§ §

1.0 Introduction

1.1 Audience

The information in this document is intended for anyone designing or architecting any hardware or software associated with Compute Express Link (CXL) or Flex Bus.

1.2 Terminology/Acronyms

Refer to PCI Express* (PCIe*) Base Specification for additional terminology and acronym definitions beyond those listed in [Table 1-1](#).

Table 1-1. Terminology/Acronyms (Sheet 1 of 11)

Term/Acronym	Definition
AAD	Additional Authentication Data - data that is integrity protected but not encrypted
Accelerator Acc	Devices that may be used by software running on Host processors to offload or perform any type of compute or I/O task. Examples of accelerators include programmable agents (e.g., GPU/GPGPU), fixed-function agents, or reconfigurable agents such as FPGAs.
ACL	Access Control List
ACPI	Advanced Configuration and Power Interface
ACS	Access Control Services as defined in PCIe Base Specification
ADF	All-Data Flit
AER	Advanced Error Reporting as defined in PCIe Base Specification
AES-GCM	Advanced Encryption Standard-Galois Counter Mode as defined in NIST* publication [AES-GCM]
AIC	Add In Card
Ak	Acknowledgment (bit/field name)
ALMP	ARB/MUX Link Management Packet
ARB/MUX	Arbiter/Multiplexer
ARI	Alternate Routing ID as defined in PCIe Base Specification.
ASI	Advanced Switching Interconnect
ASL	ACPI Source Language as defined in ACPI Specification
ASPM	Active State Power Management
ATS	Address Translation Services as defined in PCIe Base Specification
Attestation	Process of providing a digital signature for a set of measurements from a device and verifying those signatures and measurements on a host.
Authentication	Process of determining whether an entity is who or what it claims to be.
BAR	Base Address Register as defined in PCIe Base Specification
_BBN	Base Bus Number as defined in ACPI Specification
BCC	Base Class Code as defined in PCIe Base Specification
BDF	Bus Device Function

Table 1-1. Terminology/Acronyms (Sheet 2 of 11)

Term/Acronym	Definition
BE	Byte Enable as defined in PCIe Base Specification
BEI	BAR Equivalent Indicator
BEP	Byte-Enables Present
BI	Back-Invalidate
Bias Flip	Bias refers to coherence tracking of HDM-D* memory regions by the owning device to indicate that the host may have a cache copy. The Bias Flip is a process used to change the bias state that indicates the host has a cached copy of the line (Bias=Host) by invalidating the cache state for the corresponding address(es) in the host such that the device has exclusive access (Bias=Device).
BIR	BAR Indicator Register as defined in PCIe Base Specification
BIRsp	Back-Invalidate Response
BISnp	Back-Invalidate Snoop
BMC	Baseboard Management Controller
BME	Bus Master Enable
BW	Bandwidth
CA	Certificate Authority
	Congestion Avoidance
CAM	Content Addressable Memory
CAS	Column Address Strobe
_CBR	CXL Host Bridge Register Info as defined in ACPI Specification
CCI	Component Command Interface
CCID	CXL Cache ID
CDAT	Coherent Device Attribute Table, a table that describes performance characteristics of a CXL device or a CXL switch.
CDL	CXL DevLoad
CEDT	CXL Early Discovery Table
CEL	Command Effects Log
CFMWS	CXL Fixed Memory Window Structure
CHBCR	CXL Host Bridge Component Registers
CHBS	CXL Host Bridge Structure
_CID	Compatible ID as defined in ACPI Specification
CIE	Correctable Internal Error
CIKMA	CXL.cachemem IDE Key Management Agent
CKID	Context Key Identifier passed in the protocol flit for identifying security keys utilized for memory encryption using TSP.
CMA	Component Measurement and Authentication as defined in PCIe Base Specification
Coh	Coherency
Cold reset	As defined in PCIe Base Specification
Comprehensive Trust	Security model in which every device available to the TEE is presumed to be trusted by all TEEs in the system.
CPMU	CXL Performance Monitoring Unit
CRD	Credit Return(ed)
CQID	Command Queue ID
CSDS	CXL System Description Structure

Table 1-1. Terminology/Acronyms (Sheet 3 of 11)

Term/Acronym	Definition
CSR	Configuration Space register
CT	Crypto Timeout
CVME	Corrected Volatile Memory Error, a corrected error associated with volatile memory.
CXIMS	CXL XOR Interleave Math Structure
CXL	Compute Express Link, a low-latency, high-bandwidth link that supports dynamic protocol muxing of coherency, memory access, and I/O protocols, thus enabling attachment of coherent accelerators or memory devices.
CXL.cache	Agent coherency protocol that supports device caching of Host memory.
CXL.cachemem	CXL.cache/CXL.mem
CXL.io	PCIe-based non-coherent I/O protocol with enhancements for accelerator support.
CXL.mem	Memory access protocol that supports device-attached memory.
CXL Memory Device	CXL device with a specific Class Code as defined in Section 8.1.12.1 .
D2H	Device to Host
DAPM	Deepest Allowable Power Management state
DC	Dynamic Capacity
DCD	Dynamic Capacity Device
DCOH	Device Coherency agent on the device that is responsible for resolving coherency with respect to device caches and managing Bias states.
DDR	Double Data Rate
DH	Data Header
DHSW-FM	Dual Host, Fabric Managed, Switch Attached SLD EP
DHSW-FM-MLD	Dual Host, Fabric Managed, Switch Attached MLD EP
DLLP	Data Link Layer Packet as defined in PCIe Base Specification
DM	Data Mask
DMP	Device Media Partition
DMTF	Distributed Management Task Force
DOE	Data Object Exchange as defined in PCIe Base Specification
Domain	Set of host ports and devices within a single coherent HPA space
Downstream ES	Within the context of a single VH, an Edge Switch other than the Host ES.
Downstream Port	Physical port that can be a root port or a downstream switch port or an RCH Downstream Port
DPA	Device Physical Address. DPA forms a device-scoped flat address space. An LD-FAM device presents a distinct DPA space per LD. A G-FAM device presents the same DPA space to all hosts. The CXL HDM decoders or GFD decoders map HPA into DPA space.
DPC	Downstream Port Containment as defined in PCIe Base Specification
DPID	Destination PID
DRS	Data Response
DRT	DPID Routing Table
DSAR	Downstream Acceptance Rules
_DSM	Device Specific Method as defined in ACPI Specification
DSM	Device Security Manager. A logical entity in a device that can be admitted into the TCB for a TVM and enforces security policies on the device.
DSMAD	Device Scoped Memory Affinity Domain as defined in Coherent Device Attribute Table (CDAT) Specification

Table 1-1. Terminology/Acronyms (Sheet 4 of 11)

Term/Acronym	Definition
DSMAS	Device Scoped Memory Affinity Structure as defined in Coherent Device Attribute Table (CDAT) Specification
DSP	Downstream Switch Port
DTLB	Data Translation Lookaside Buffer
DUT	Device Under Test
DVSEC	Designated Vendor-Specific Extended Capability as defined in PCIe Base Specification
ECC	Error-correcting Code
ECN	Engineering Change Notice
ECS	Error Check Scrub
ECRC	End-to-End CRC
EDB	End Bad as defined in PCIe Base Specification
Edge DSP	PBR downstream switch port that connects to a device (including a GFD) or an HBR USP
Edge Port	PBR switch port suitable for connecting to an RP, device, or HBR USP.
Edge Switch (ES)	Within the context of a single VH, a PBR switch that contains one or more Edge Ports.
Edge USP	PBR upstream switch port that connects to a root port
eDPC	Enhanced Downstream Port Control as defined in PCIe Base Specification
EDS	End of Data Stream
EFN	Event Firmware Notification
EIEOS	Electrical Idle Exit Ordered Set as defined in PCIe Base Specification
EIOS	Electrical Idle Ordered Set as defined in PCIe Base Specification
EIOSQ	EIOS Sequence as defined in PCIe Base Specification
EMD	Extended Metadata
EMV	Extended MetaValue
ENIW	Encoded Number of Interleave Ways
EP	Endpoint
eRCD	Exclusive Restricted CXL Device (formerly known as CXL 1.1 only Device). eRCD is a CXL device component that can operate only in RCD mode.
eRCH	Exclusive Restricted CXL Host (formerly known as CXL 1.1 only host). eRCH is a CXL host component that can operate only in RCD mode.
evPPB	Egress vPPB
Explicit Access Control	TE State ownership tracking where the state change occurs explicitly utilizing a TE State change message that is independent of each memory transaction.
Fabric Port (FPort)	PBR switch port that connects to another PBR switch port.
FAM	Fabric-Attached Memory. HDM within a Type 2 or Type 3 device that can be made accessible to multiple hosts concurrently. Each HDM region can either be pooled (dedicated to a single host) or shared (accessible concurrently by multiple hosts).
FAST	Fabric Address Segment Table
FC	Flow Control
FCBP	Flow Control Backpressured
FEC	Forwarded Error Correction
Flex Bus	A flexible high-speed port that is configured to support either PCIe or CXL.
Flex Bus.CXL	CXL protocol over a Flex Bus interconnect.
Flit	Link Layer Unit of Transfer

Table 1-1. Terminology/Acronyms (Sheet 5 of 11)

Term/Acronym	Definition
FLR	Function Level Reset
FM	The Fabric Manager is an entity separate from the switch or host firmware that controls aspects of the system related to binding and management of pooled ports and devices.
FMLD	Fabric Manager-owned Logical Device
FM-owned PPB	Link that contains traffic from multiple VCSs or an unbound physical port.
Fundamental Reset	As defined in PCIe Base Specification
FW	Firmware
GAM	GFD Async Message
GAE	Global Memory Access Endpoint
GDT	GFD Decoder Table
G-FAM	Global FAM. Highly scalable form of FAM that is presented to hosts without using Logical Devices (LDs). Like LD-FAM, G-FAM presents HDM that can be pooled or shared.
GFD	G-FAM device
GFD decoder	HPA-to-DPA translation mechanism inside a GFD
GI	Generic Affinity
GIM	Global Integrated Memory
GMV	Global Memory Mapping Vector
GO	Global Observation. This is used in the context of coherence protocol as a message to know when data is guaranteed to be observed by all agents in the coherence domain for either a read or a write.
GPF	Global Persistent Flush
H2D	Host to Device
HA	Home Agent. This is the agent on the host that is responsible for resolving system wide coherency for a given address.
HBIG	Host Bridge Interleave Granularity
HBM	High-Bandwidth Memory
HBR	Hierarchy Based Routing
HBR link	Link operating in a mode, where all flits are in HBR format
HBR switch	Switch that supports only HBR
HDM	Host-managed Device Memory. Device-attached memory that is mapped to system coherent address space and accessible to the Host using standard write-back semantics. Memory located on a CXL device can be mapped as either HDM or PDM.
HDM-H	Host-only Coherent HDM region type. Used only for Type 3 Devices.
HDM-D	Device Coherent HDM region type. Used only for Type 2 devices that rely on CXL.cache to manage coherence with the host.
HDM-DB	Device Coherent using Back-Invalidate HDM region type. Can be used by Type 2 devices or Type 3 devices.
HID	Hardware ID as defined in ACPI Specification
HMAT	Heterogeneous Memory Attribute Table as defined in ACPI Specification
Host ES	Within the context of a single VH, the single Edge Switch connected to the RP.
Hot Reset	As defined in PCIe Base Specification
HPA	Host Physical Address
HPC	High-Performance Computing
hPPR	Hard Post Package Repair
HW	Hardware

Table 1-1. Terminology/Acronyms (Sheet 6 of 11)

Term/Acronym	Definition
IDE	Integrity and Data Encryption
IDT	Interleave DPID Table
IG	Interleave Granularity
IGB	Interleave Granularity in number of bytes
Intermediate Fabric Switch	Within the context of a single VH, a PBR switch that forwards VH traffic but is not a Host ES or downstream ES.
IOMMU	I/O Memory Management Unit as defined in PCIe Base Specification
Implicit Access Control	TE State ownership tracking where the state change occurs implicitly as part of executing each memory transaction.
Initiator	Defined by TSP as a host or accelerator device that issues TSP requests.
IP2PM	Independent Power Manager to (Master) Power Manager, PM messages from the device to the host.
ISL	Inter-Switch Link. PBR link that connects Fabric Ports.
ISP	Interleave Set Position
IV	Initialization Vector
IW	Interleave Ways
IWB	Implicit Writeback
LAV	LDST Access Vector
LD	Logical Device. Entity that represents a CXL Endpoint that is bound to a VCS. An SLD contains one LD. An MLD contains multiple LDs.
LDST	LD-FAM Segment Table
LD-FAM	FAM that is presented to hosts via Logical Devices (LDs).
Link Layer Clock	CXL.cachemem link layer clock rate usually defined by the flit rate during normal operation.
LLC	Last Level Cache
LLCRD	Link Layer Credit
LLCTRL	Link Layer Control
LLR	Link Layer Retry
LLRB	Link Layer Retry Buffer
LOpt	Latency-Optimized
LRSM	Local Retry State Machine
LRU	Least recently used
LSA	Label Storage Area
LSM	Link State Machine
LTR	Latency Tolerance Reporting
LTSSM	Link Training and Status State Machine as defined in PCIe Base Specification
LUN	Logical Unit Number
M2S	Master to Subordinate
MAC	Message Authentication Code; also referred to as Authentication Tag or Integrity value.
MAC epoch	Set of flits which are aggregated together for MAC computation.
MB	Megabyte - 2^{20} bytes (1,048,576 bytes)
	Mailbox
MC	Memory Controller
MCA	Machine Check Architecture

Table 1-1. Terminology/Acronyms (Sheet 7 of 11)

Term/Acronym	Definition
MCTP	Management Component Transport Protocol
MDH	Multi-Data Header
Measurement	Representation of firmware/software or configuration data on a device.
MEC	Multiple Event Counting
MEFN	Memory Error Firmware Notification. An EFN that is used to report memory errors.
Memory Group	Set of DC blocks that can be accessed by the same set of requesters.
Memory Sparing	Repair function that replaces a portion of memory (the “spared memory”) with a portion of functional memory at that same DPA.
MESI	Modified, Exclusive, Shared, and Invalid cache coherence protocol
MGT	Memory Group Table
MH-MLD	Multi-headed MLD. CXL component that contains multiple CXL ports, each presenting an MLD or SLD. The ports must correctly operate when connected to any combination of common or different hosts. The FM API is used to configure each LD as well as the overall MH-MLD. Currently, MH-MLDs are architected only for Type 3 LDs. MH-MLDs are considered a specialized type of MLD and, as such, are subject to all functional and behavioral requirements of MLDs.
MH-SLD	Multi-headed SLD. CXL component that contains multiple CXL ports, each presenting an SLD. The ports must correctly operate when connected to any combination of common or different hosts. The FM API is used to configure each SLD as well as the overall MH-SLD. Currently, MH-SLDs are architected only for Type 3 LDs.
ML	Machine Learning
MLD	Multi-Logical Device. CXL component that contains multiple LDs, out of which one LD is reserved for configuration via the FM API, and each remaining LD is suitable for assignment to a different host. Currently MLDs are architected only for Type 3 LDs.
MLD Port	An MLD Port is one that has linked up with an MLD Component. The port is natively bound to an FM-owned PPB inside the switch.
MMIO	Memory Mapped I/O
MR	Multi-Root
MRL	Manually-operated Retention Latch as defined in PCIe Base Specification
MS0	Meta0-State
MSB	Most Significant Bit
MSE	Memory Space Enable
MSI/MSI-X	Message Signaled Interrupt as defined in PCIe Base Specification
N/A	Not Applicable
Native Width	This is the maximum possible expected negotiated link width.
NDR	No Data Response
NG	Number of Groups
NIB	Number of Bitmap Entries
NIW	Number of Interleave Ways
NOP	No Operation
NT	Non-Temporal
NVM	Non-volatile Memory
NXM	Non-existent Memory
OBFF	Optimized Buffer Flush/Fill as defined in PCIe Base Specification
OHC	Orthogonal Header Content as defined in PCIe Base Specification
OOB	Out of Band

Table 1-1. Terminology/Acronyms (Sheet 8 of 11)

Term/Acronym	Definition
_OSC	Operating System Capabilities as defined in ACPI Specification
OSCKID	CKID memory encryption key configured for use with non-TEE data.
OSPM	Operating System-directed configuration and Power Management as defined in ACPI Specification
PA	Physical Address
P2P	Peer to peer
PBR	Port Based Routing
PBR link	Link where all flits are in PBR format
PBR switch	Switch that supports PBR and has PBR enabled
P/C	Producer-Consumer
PCIe	PCI Express
PCRC	CRC-32 calculated on the flit plaintext content. Encrypted PCRC is used to provide robustness against hard and soft faults internal to the encryption and decryption engines.
PDM	Private Device memory. Device-attached memory that is not mapped to system address space or directly accessible to Host as cacheable memory. Memory located on PCIe devices is of this type. Memory located on a CXL device can be mapped as either PDM or HDM.
Peer	Peer device in the context of peer-to-peer (P2P) accesses between devices.
Persistent Memory Device	Device that retains content across power cycling. A CXL Memory device can advertise "Persistent Memory" capability as long as it supports the minimum set of requirements described in Section 8.2.9.9 . The platform owns the final responsibility of determining whether a memory device can be used as Persistent Memory. This determination is beyond the scope of CXL specification.
PI	Programming Interface as defined in PCIe Base Specification
PID	PBR ID
PIF	PID Forward
PLDM	Platform-Level Data Model
PM	Power Management
PME	Power Management Event
PM2IP	(Master) Power Manager to Independent Power Manager, PM messages from the host to the device.
PPB	PCI*-to-PCI Bridge inside a CXL switch that is FM-owned. The port connected to a PPB can be disconnected, or connected to a PCIe component or connected to a CXL component.
PPR	Post Package Repair
PrimarySession	CMA/SPDM session established between the TSM or TSM RoT and the DSM. Utilized for configuring and locking the device and setting and clearing memory encryption keys using TSP.
PSK	CMA/SPDM-defined pre-shared key.
PTH	PBR TLP Header
QoS	Quality of Service
QTG	QoS Throttling Group, the group of CXL.mem target resources that are throttled together in response to QoS telemetry (see Section 3.3.3). Each QTG is identified using a number that is known as QTG ID. QTG ID is a positive integer.
Rank	Set of memory devices on a channel that together execute a transaction.
RAS	Reliability Availability and Serviceability
RC	Root Complex
RCD	Shorthand for a Device that is operating in RCD mode (formerly known as CXL 1.1 Device)
RCD mode	Restricted CXL Device mode (formerly known as CXL 1.1 mode). The CXL operating mode with a number of restrictions. These restrictions include lack of Hot-Plug support and 68B Flit mode being the only supported flit mode. See Section 9.11.1 for the complete list of these restrictions.
RCEC	Root Complex Event Collector, collects errors from PCIe RCIEPs, as defined in PCIe Base Specification.

Table 1-1. Terminology/Acronyms (Sheet 9 of 11)

Term/Acronym	Definition
RCH	Restricted CXL Host. CXL Host that is operating in RCD mode (formerly known as CXL 1.1 Host).
RCIEP	Root Complex Integrated Endpoint
RCRB	Root Complex Register Block as defined in PCIe Base Specification
RDPAS	RCEC Downstream Port Association Structure
Reserved	The contents, states, or information are not defined at this time. Reserved register fields must be read only and must return 0 (all 0s for multi-bit fields) when read. Reserved encodings for register and packet fields must not be used. Any implementation dependent on a Reserved field value or encoding will result in an implementation that is not CXL-spec compliant. The functionality of such an implementation cannot be guaranteed in this or any future revision of this specification. Flit, Slot, and message reserved bits should be cleared to 0 by the sender and the receiver should ignore them.
RFM	Refresh Management
RGT	Routing Group Table
RP	Root Port
RPID	Requester PID. In the context of a GFD, the SPID from an incoming request.
RRS	Request Retry Status
RRSM	Remote Retry State Machine
RSMT	Root System Description Table as defined in ACPI Specification
RSVD or RV	Reserved
RT	Route Table
RTT	Round-Trip Time
RwD	Request with Data
S2M	Subordinate to Master
SAT	SPID Access Table
SBR	Secondary Bus Reset as defined in PCIe Base Specification
SCC	Sub-Class Code as defined in PCIe Base Specification
SDC	Silent Data Corruption
SDS	Start of Data Stream
SecondarySession	One or more optional CMA/SPDM sessions established between a host entity and the DSM. Utilized for setting and clearing memory encryption keys using TSP.
Selective Trust	Security model in which each TEE selects which devices it shall include in its TCB. The device trusted by one TEE may not be trusted by other TEEs within the system.
SF	Snoop Filter
SFSC	Security Features for SCSI Commands
Sharer	Entity that is sharing data with another entity.
SHDA	Single Host, Direct Attached SLD EP
SHSW	Single Host, Switch Attached SLD EP
SHSW-FM	Single Host, Fabric Managed, Switch Attached SLD EP
Sideband	Signal used for device detection, configuration, and Hot-Plug in PCIe connectors, as defined in PCIe Base Specification
SLD	Single Logical Device
Smart I/O	Enhanced I/O with additional protocol support.
SP	Security Protocol
SPDM	Security Protocol and Data Model
SPDM over DOE	Security Protocol and Data Model over Data Object Exchange as defined in PCIe Base Specification

Table 1-1. Terminology/Acronyms (Sheet 10 of 11)

Term/Acronym	Definition
SPID	Source PID
sPPR	Soft Post Package Repair
SRAT	System Resource Affinity Table as defined in ACPI Specification
SRIS	Separate Reference Clocks with Independent Spread Spectrum Clocking as defined in PCIe Base Specification
SV	Secret Value
SVM	Shared Virtual Memory
SW	Software
Target	Defined by TSP as a memory expander device that receives TSP requests.
TC	Traffic Class
TCB	Trusted Computing Base - refers to the set of hardware, software and/or firmware entities that security assurances depend upon
TDISP	PCI-SIG-defined TEE Device Interface Security Protocol
TEE	Trusted Execution Environment
TEE Intent	TEE or non-TEE intent of a memory request from an initiator defined by the opcodes utilized in the transaction.
TE State	TEE Exclusive State maintained by the device for each page or cacheline to enforce access control.
TLP	Transaction Layer Packet as defined in PCIe Base Specification
TMAC	Truncated Message Authentication Code
TRP	Trailer Present
TSM	TEE Security Manager. Logical entity within a host processor that is the TCB for a TVM and enforces security policies on the host.
TSM RoT	TEE Security Manager Root of Trust. The entity on the host that establishes the CMA/SPDM PrimarySession.
TSP	CXL-defined TEE Security Protocol. The collection of requirements and interfaces that allow memory devices to be utilized for confidential computing.
TVM	TEE Virtual Machine. A TEE that has the property of a virtual machine. A TVM does not need to trust the hypervisor that hosts the TVM.
TVMCKID	CKID memory encryption key configured for use with TEE data.
UEFI	Unified Extensible Firmware Interface
UIE	Uncorrectable Internal Error
UIG	Upstream Interleave Granularity
UIO	Unordered Input/Output
UIW	Upstream Interleave Ways
Upstream Port	Physical port that can be an upstream switch port, or an Endpoint port, or an RCD Upstream Port.
UQID	Unique Queue ID
UR	Unsupported Request
USAR	Upstream Acceptance Rules
USP	Upstream Switch Port
UTC	Coordinated Universal Time
UUID	Universally Unique IDentifier as defined in the IETF RFC 4122 Specification
VA	Virtual Address
VC	Virtual Channel

Table 1-1. Terminology/Acronyms (Sheet 11 of 11)

Term/Acronym	Definition
VCS	Virtual CXL Switch. Includes entities within the physical switch belonging to a single VH. It is identified using the VCS ID.
VDM	Vendor Defined Message
VDSP	Downstream vPPB in a Host ES that is bound to one vUSP within a specific Downstream ES.
VH	Virtual Hierarchy. Everything from the CXL RP down, including the CXL RP, CXL PPBs, and CXL Endpoints. Hierarchy ID means the same as PCIe.
VH Mode	A mode of operation where CXL RP is the root of the hierarchy
VID	Vendor ID
vLSM	Virtual Link State Machine
VM	Virtual Machine
VMM	Virtual Machine Manager
vPPB	Virtual PCI-to-PCI Bridge inside a CXL switch that is host-owned. Can be bound to a port that is either disconnected, connected to a PCIe component or connected to a CXL component.
VTV	VendPrefixL0 Target Vector
vUSP	Upstream vPPB in a Downstream ES that is bound to one vDSP within a specific Host ES.
Warm Reset	As defined in PCIe Base Specification
XSDT	Extended System Description Table as defined in ACPI Specification

1.3 Reference Documents

Table 1-2. Reference Documents

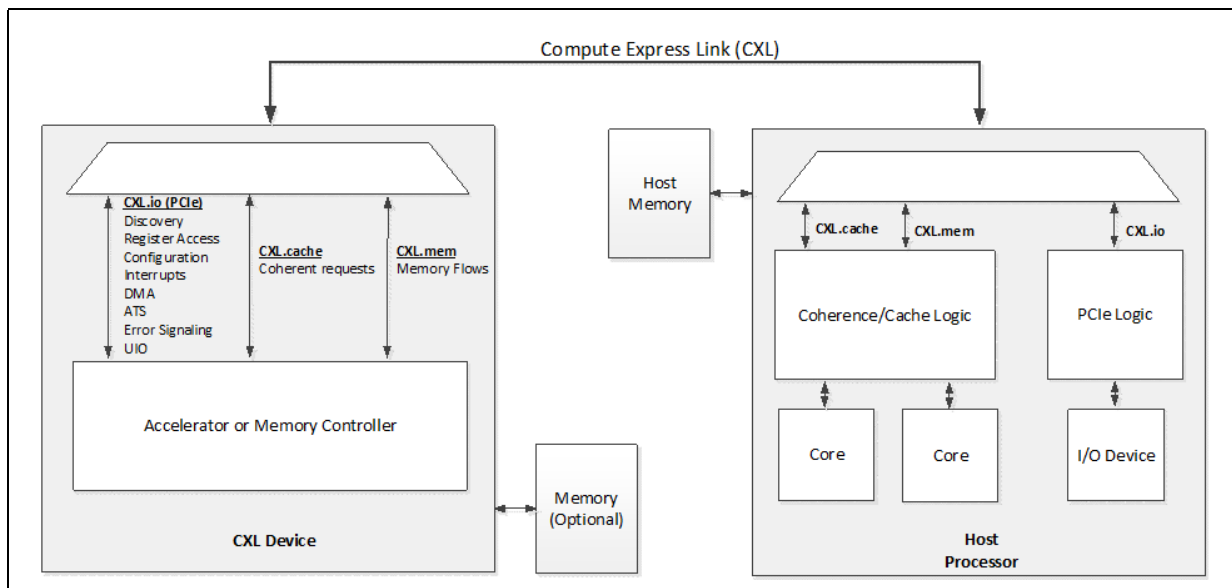
Document	Chapter Reference	Document No./Location
PCI Express Base Specification Revision 6.0	N/A	www.pcisig.com
PCI Firmware Specification (revision 3.3 or later)	Various	www.pcisig.com
ACPI Specification (version 6.5 or later)	Various	www.uefi.org
Coherent Device Attribute Table (CDAT) Specification (version 1.04 or later)	Various	www.uefi.org/acpi
RFC 4122 A Universally Unique IDentifier UUID URN Namespace	Various	www.ietf.org/rfc/rfc4122
UEFI Specification (version 2.10 or later)	Various	www.uefi.org
CXL Fabric Manager API over MCTP Binding Specification (DSP0234) (version 1.0.0 or later)	Chapter 7	www.dmtf.org/dsp/DSP0234
Management Component Transport Protocol (MCTP) Base Specification (DSP0236) (version 1.3.1 or later)	Chapters 7, 8, 11	www.dmtf.org/dsp/DSP0236
Management Component Transport Protocol (MCTP) SMBus/I2C Transport Binding Specification (DSP0237) (version 1.2.0 or later)	Chapter 11	www.dmtf.org/dsp/DSP0237
Management Component Transport Protocol (MCTP) PCIe VDM Transport Binding Specification (DSP0238) (version 1.2.0 or later)	Chapters 7, 11	www.dmtf.org/dsp/DSP0238
Security Protocol and Data Model (SPDM) Specification (DSP0274) (version 1.2.0 or later)	Chapters 11, 14	www.dmtf.org/dsp/DSP0274
Security Protocol and Data Model (SPDM) over MCTP Binding Specification (DSP0275) (version 1.0.0 or later)	Chapter 11	www.dmtf.org/dsp/DSP0275
Secured Messages using SPDM over MCTP Binding Specification (DSP0276) (version 1.0.0 or later)	Chapter 11	www.dmtf.org/dsp/DSP0276
Secured Messages using SPDM Specification (DSP0277) (version 1.0.0 or later)	Chapter 11	www.dmtf.org/dsp/DSP0277
CXL Type 3 Component Command Interface over MCTP Binding Specification (DSP0281) (version 1.0.0 or later)	Chapters 7, 9	www.dmtf.org/dsp/DSP0281
NIST Special Publication 800-38D Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC	Chapters 8, 11	nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
JEDEC DDR5 Specification, JESD79-5 (5B, version 1.2 or later)	Chapters 8, 13	www.jedec.org
Security Features for SCSI Commands (SFSC)	Chapter 8	webstore.ansi.org
Unordered IO (UIO) ECN to PCI Express Base Specification Revision 6.0	N/A	members.pcisig.com/wg/PCI-SIG/document/19388

1.4 Motivation and Overview

1.4.1 CXL

CXL is a dynamic multi-protocol technology designed to support accelerators and memory devices. CXL provides a rich set of protocols that include I/O semantics similar to PCIe (i.e., CXL.io), caching protocol semantics (i.e., CXL.cache), and memory access semantics (i.e., CXL.mem) over a discrete or on-package link. CXL.io is required for discovery and enumeration, error reporting, P2P accesses to CXL memory¹ and host physical address (HPA) lookup. CXL.cache and CXL.mem protocols may be optionally implemented by the particular accelerator or memory device usage model. An important benefit of CXL is that it provides a low-latency, high-bandwidth path for an accelerator to access the system and for the system to access the memory attached to the CXL device. Figure 1-1 is a conceptual diagram that shows a device attached to a Host processor via CXL.

Figure 1-1. Conceptual Diagram of Device Attached to Processor via CXL

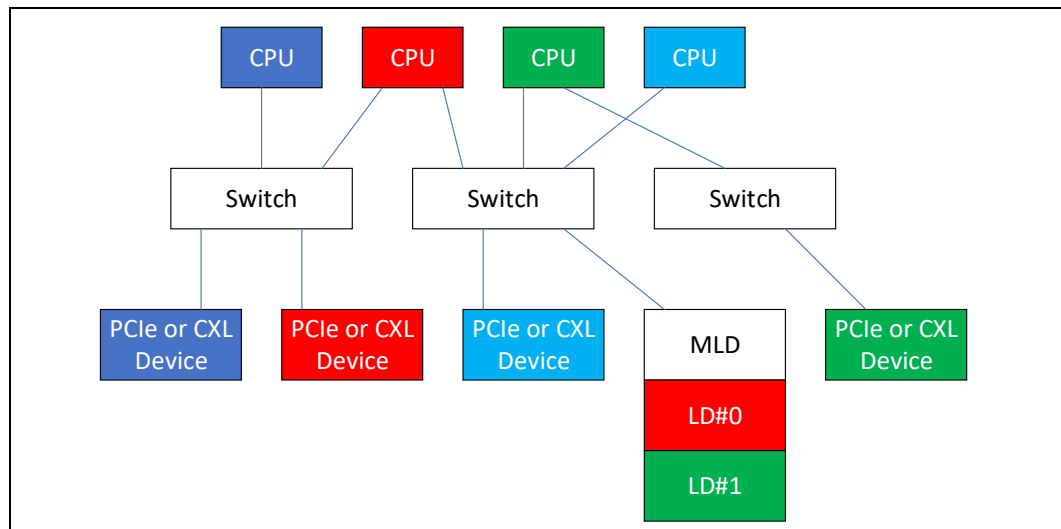


The CXL 2.0 specification enables additional usage models beyond the CXL 1.1 specification, while being fully backward compatible with the CXL 1.1 (and CXL 1.0) specification. It enables managed Hot-Plug, security enhancements, persistent memory support, memory error reporting, and telemetry. The CXL 2.0 specification also enables single-level switching support for fan-out as well as the ability to pool devices across multiple virtual hierarchies, including multi-domain support of memory devices.

Figure 1-2 demonstrates memory and accelerator disaggregation through single level switching, in addition to fan-out, across multiple virtual hierarchies, each represented by a unique color. The CXL 2.0 specification also enables these resources (memory or accelerators) to be off-lined from one domain and on-lined into another domain, thereby allowing the resources to be time-multiplexed across different virtual hierarchies, depending on their resource demand.

1. Peer-to-peer flows to CXL.mem regions are supported using Unordered I/O (UIO) on the CXL.io protocol or with Direct CXL.mem access as described in this specification.

Figure 1-2. Fan-out and Pooling Enabled by Switches



The CXL 3.0 specification doubles the bandwidth while enabling additional usage models beyond the CXL 2.0 specification. The CXL 3.0 specification is fully backward compatible with the CXL 2.0 specification (and hence with the CXL 1.1 and CXL 1.0 specifications). The maximum Data Rate doubles to 64.0 GT/s with PAM-4 signaling, leveraging the PCIe Base Specification PHY along with its CRC and FEC, to double the bandwidth, with provision for an optional Flit arrangement for low latency. Multi-level switching is enabled with the CXL 3.0 specification, supporting up to 4K Ports, to enable CXL to evolve as a fabric extending, including non-tree topologies, to the Rack and Pod level. The CXL 3.0 specification enables devices to perform direct peer-to-peer accesses to HDM memory using UIO (in addition to MMIO memory that existed before) to deliver performance at scale, as shown in [Figure 1-3](#). Snooper Filter support can be implemented in Type 2 and Type 3 devices to enable direct peer-to-peer accesses using the back-invalidate channels introduced in CXL.mem. Shared memory support across multiple virtual hierarchies is provided for collaborative processing across multiple virtual hierarchies, as shown in [Figure 1-4](#).

CXL protocol is compatible with PCIe CEM Form Factor (4.0 and later), all form factors relating to EDSFF SSF-TA-1009 (revision 2.0 and later) and other form factors that support PCIe.

Figure 1-3. Direct Peer-to-Peer Access to an HDM Memory by PCIe/CXL Devices without Going through the Host

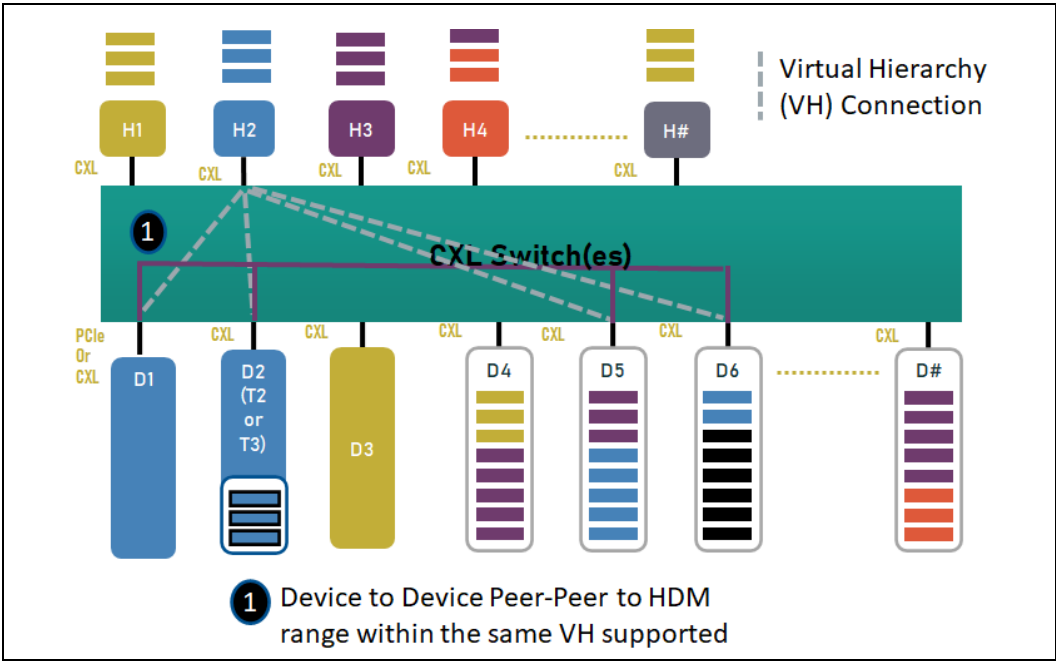
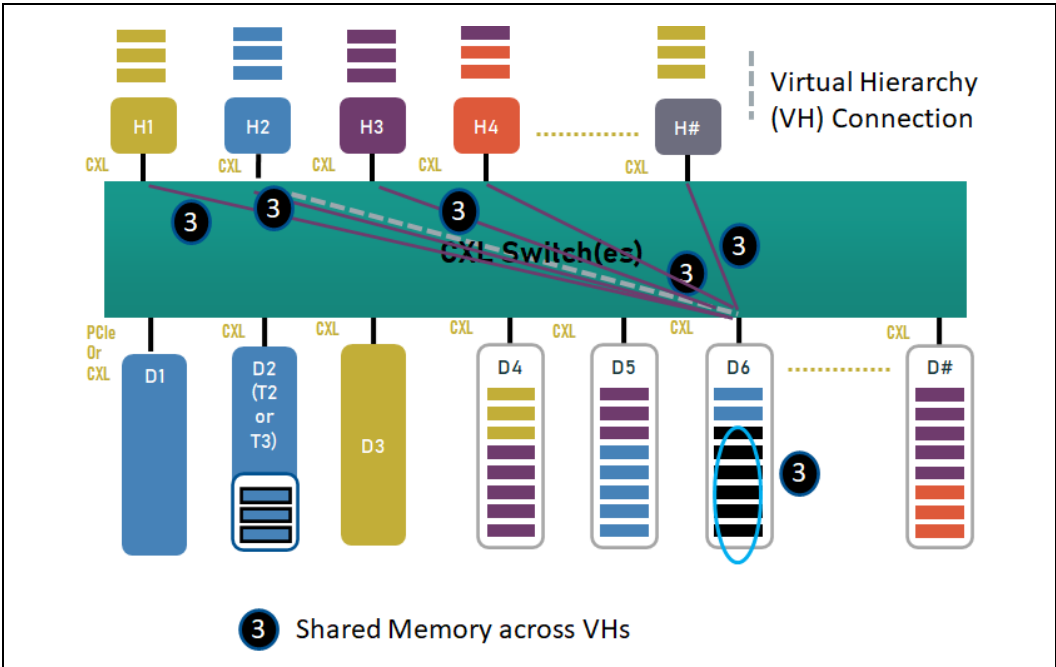


Figure 1-4. Shared Memory across Multiple Virtual Hierarchies



1.4.2 Flex Bus

A Flex Bus port allows designs to choose between providing native PCIe protocol or CXL over a high-bandwidth, off-package link; the selection happens during link training via alternate protocol negotiation and depends on the device that is plugged into the slot. Flex Bus uses PCIe electricals, making it compatible with PCIe retimers and with form factors that support PCIe.

Figure 1-5 provides a high-level diagram of a Flex Bus port implementation, illustrating both a slot implementation and a custom implementation where the device is soldered down on the motherboard. The slot implementation can accommodate either a Flex Bus.CXL card or a PCIe card. One or two optional retimers can be inserted between the CPU and the device to extend the channel length. As illustrated in Figure 1-6, this flexible port can be used to attach coherent accelerators or smart I/O to a Host processor.

Figure 1-7 illustrates how a Flex Bus.CXL port can be used as a memory expansion port.

Figure 1-5. CPU Flex Bus Port Example

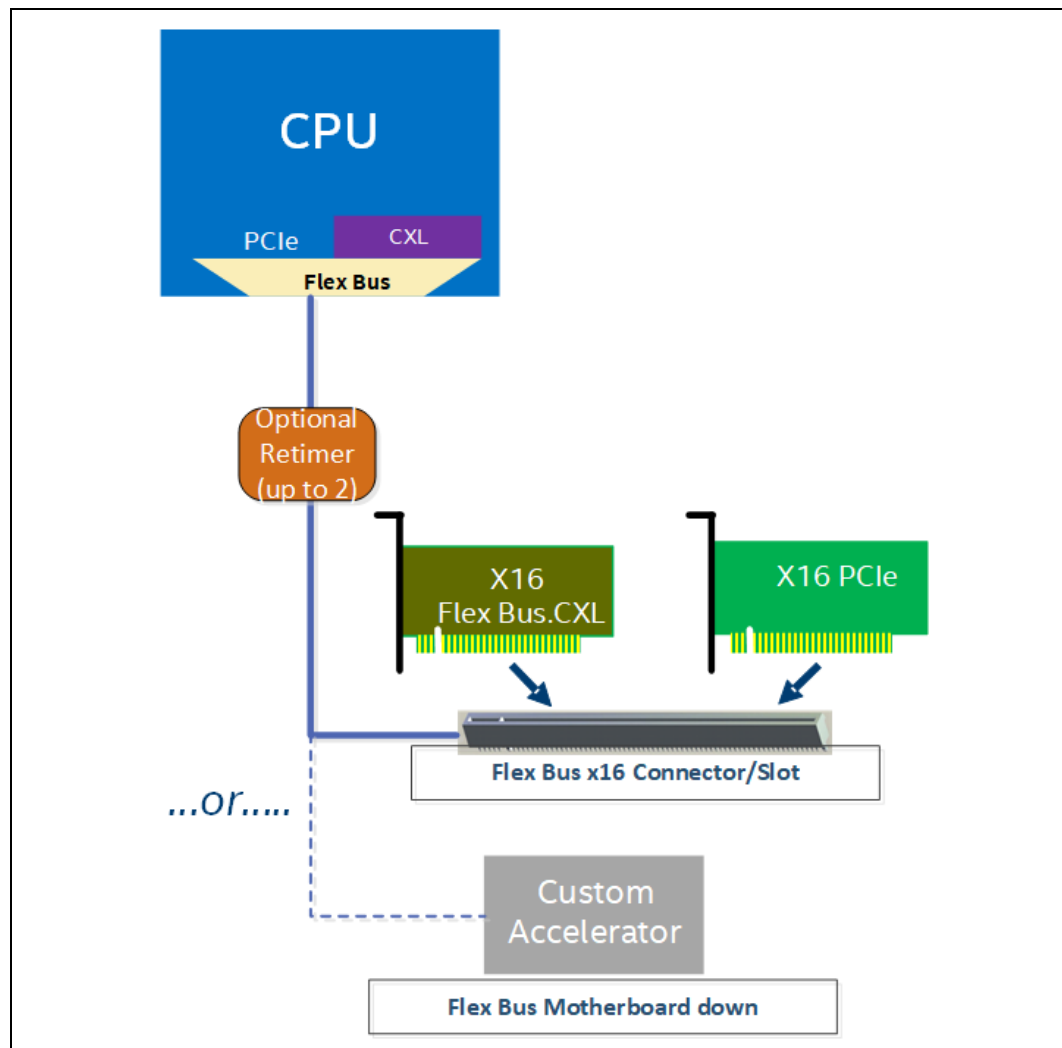


Figure 1-6. Flex Bus Usage Model Examples

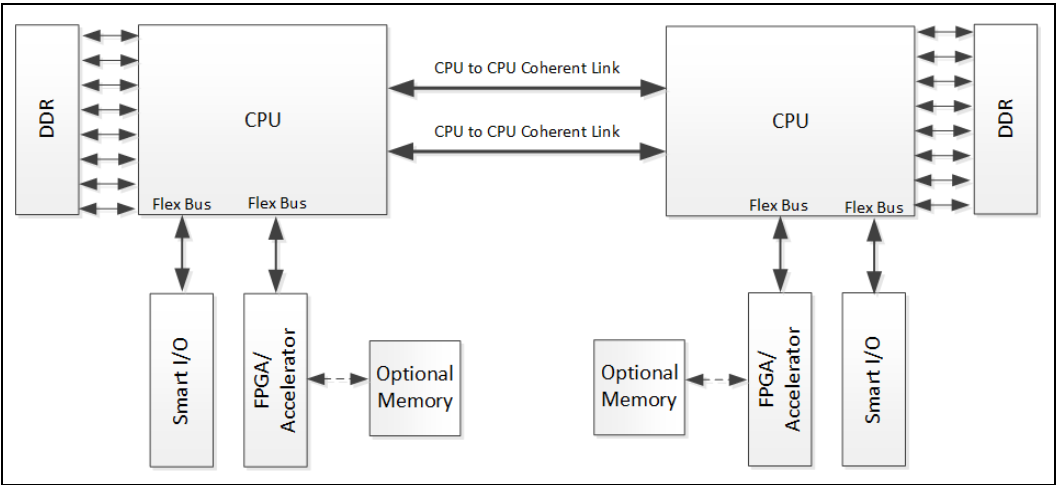


Figure 1-7. Remote Far Memory Usage Model Example

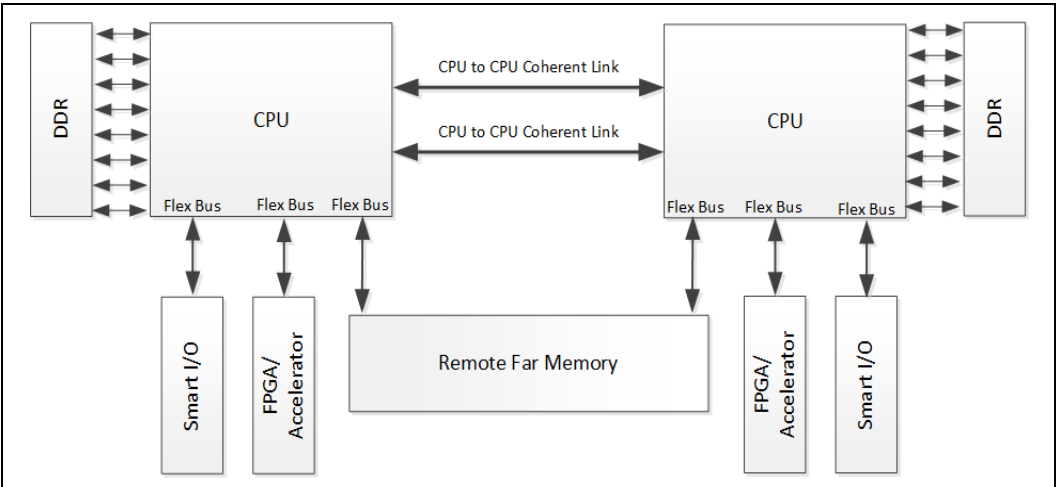
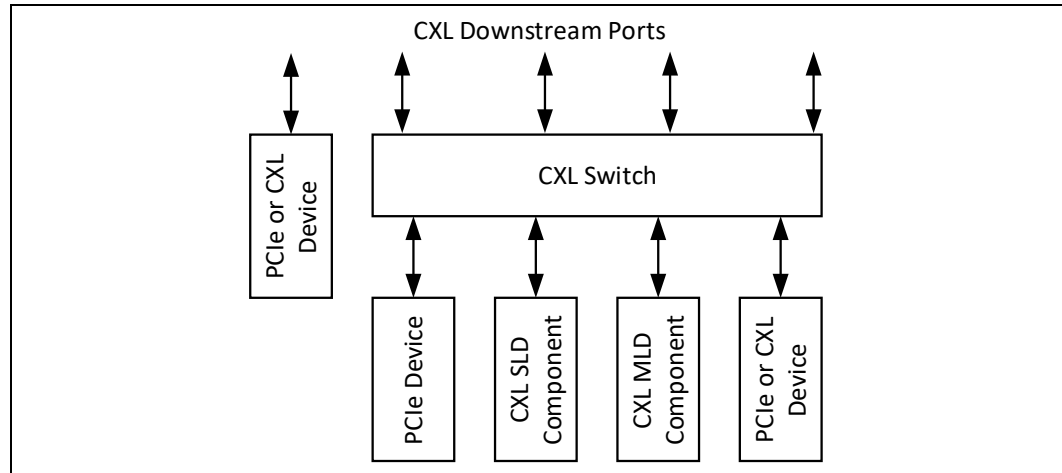


Figure 1-8 illustrates the connections that are supported below a CXL Downstream Port.

Figure 1-8. CXL Downstream Port Connections



1.5 Flex Bus Link Features

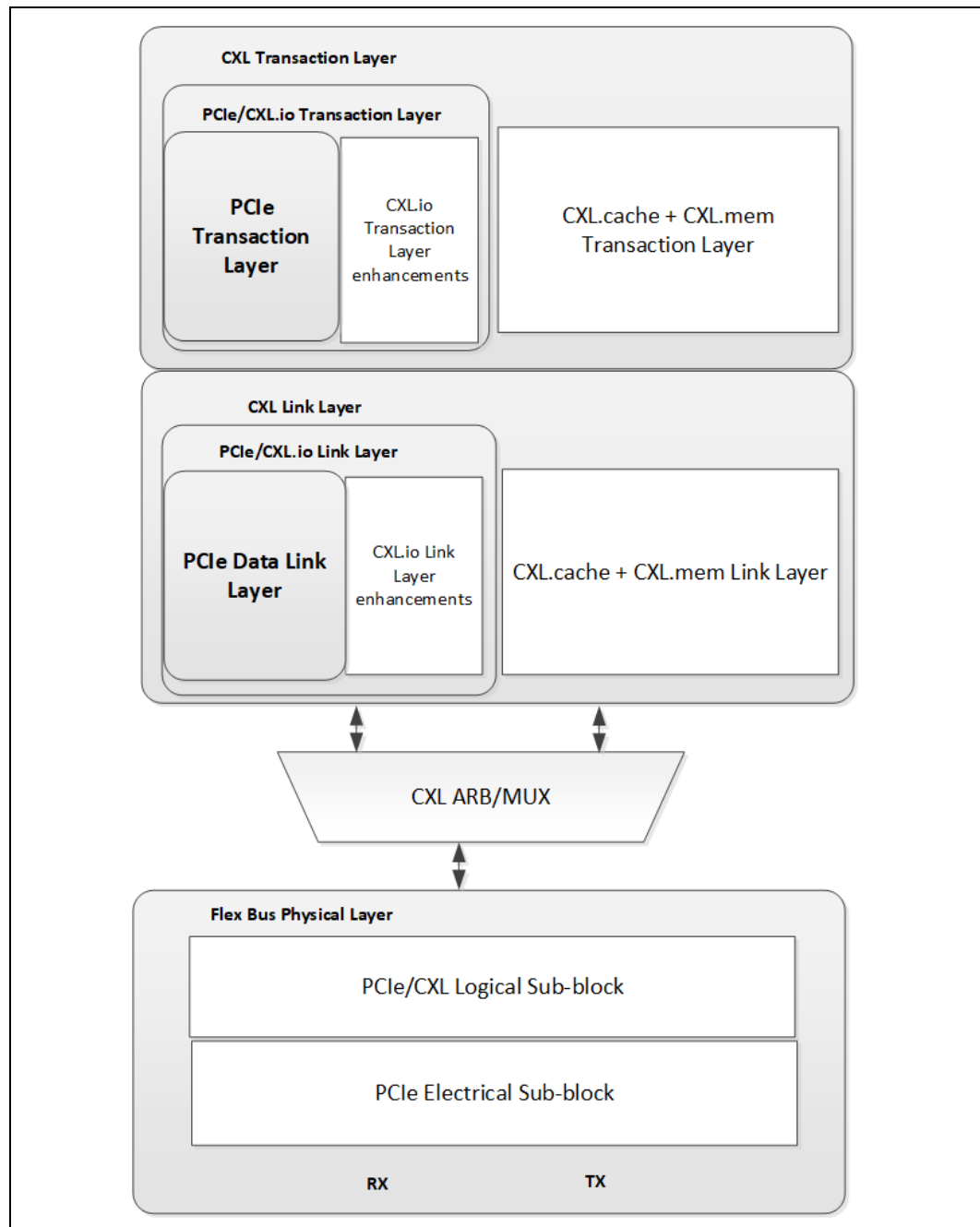
Flex Bus provides a point-to-point interconnect that can transmit native PCIe protocol or dynamic multi-protocol CXL to provide I/O, caching, and memory protocols over PCIe electricals. The primary link attributes include support of the following features:

- Native PCIe mode, full feature support as defined in PCIe Base Specification
- CXL mode, as defined in this specification
- Configuration of PCIe vs. CXL protocol mode
- With PAM4, signaling rate of 64 GT/s, and degraded rates of 32 GT/s, 16 GT/s, or 8 GT/s in CXL mode. Otherwise, signaling rate of 32 GT/s, degraded rate of 16 GT/s or 8 GT/s in CXL mode
- Link width support for x16, x8, x4, x2 (degraded mode), and x1 (degraded mode) in CXL mode
- Bifurcation (aka Link Subdivision) support to x4 in CXL mode

1.6 Flex Bus Layering Overview

Flex Bus architecture is organized as multiple layers, as illustrated in Figure 1-9. The CXL transaction (protocol) layer is subdivided into logic that handles CXL.io and logic that handles CXL.cache and CXL.mem; the CXL link layer is subdivided in the same manner. Note that the CXL.cache and CXL.mem logic are combined within the transaction layer and within the link layer. The CXL link layer interfaces with the CXL ARB/MUX, which interleaves the traffic from the two logic streams. Additionally, the PCIe transaction and data link layers are optionally implemented and, if implemented, are permitted to be converged with the CXL.io transaction and link layers, respectively. As a result of the link training process, the transaction and link layers are configured to operate in either PCIe mode or CXL mode. While a host CPU would most likely implement both modes, an accelerator AIC is permitted to implement only the CXL mode. The logical sub-block of the Flex Bus physical layer is a converged logical physical layer that can operate in either PCIe mode or CXL mode, depending on the results of alternate mode negotiation during link training.

Figure 1-9. Conceptual Diagram of Flex Bus Layering



1.7 Document Scope

This document specifies the functional and operational details of the Flex Bus interconnect and the CXL protocol. It describes the CXL usage model and defines how the transaction, link, and physical layers operate. Reset, power management, and initialization/configuration flows are described. Additionally, RAS behavior is described. Refer to PCIe Base Specification for PCIe protocol details.

The contents of this document are summarized in the following chapter highlights:

- **Chapter 2.0, “CXL System Architecture”** – This chapter describes Type 1, Type 2, and Type 3 devices that might attach to a CPU Root Complex or a CXL switch over a CXL-capable link. For each device profile, a description of the typical workload and system resource usage is provided along with an explanation of which CXL capabilities are relevant for that workload. Bias-based and Back-Invalidate-based coherency models are introduced. This chapter also covers multi-headed devices and G-FAM devices and how they enable memory pooling and memory sharing usages. It also provides a summary of the CXL fabric extensions and its scalability.
- **Chapter 3.0, “CXL Transaction Layer”** – This chapter is divided into subsections that describe details for CXL.io, CXL.cache, and CXL.mem. The CXL.io protocol is required for all implementations, while the other two protocols are optional depending on expected device usage and workload. The transaction layer specifies the transaction types, transaction layer packet formatting, transaction ordering rules, and crediting. The CXL.io protocol is based on the “Transaction Layer Specification” chapter of PCIe Base Specification; any deltas from PCIe Base Specification are described in this chapter. These deltas include PCIe Vendor Defined Messages for reset and power management, modifications to the PCIe ATS request and completion formats to support accelerators. For CXL.cache, this chapter describes the channels in each direction (i.e., request, response, and data), the transaction opcodes that flow through each channel, and the channel crediting and ordering rules. The transaction fields associated with each channel are also described. For CXL.mem, this chapter defines the message classes in each direction, the fields associated with each message class, and the message class ordering rules. Finally, this chapter provides flow diagrams that illustrate the sequence of transactions involved in completing host-initiated and device-initiated accesses to device-attached memory.
- **Chapter 4.0, “CXL Link Layers”** – The link layer is responsible for reliable transmission of the transaction layer packets across the Flex Bus link. This chapter is divided into subsections that describe details for CXL.io and for CXL.cache and CXL.mem. The CXL.io protocol is based on the “Data Link Layer Specification” chapter of PCIe Base Specification; any deltas from PCIe Base Specification are described in this chapter. For CXL.cache and CXL.mem, the 68B flit format and 256B flit format are specified. The flit packing rules for selecting transactions from internal queues to fill the slots in the flit are described. Other features described for 68B flit mode include the retry mechanism, link layer control flits, CRC calculation, and viral and poison.
- **Chapter 5.0, “CXL ARB/MUX”** – The ARB/MUX arbitrates between requests from the CXL link layers and multiplexes the data to forward to the physical layer. On the receiver side, the ARB/MUX decodes the flit to determine the target to forward transactions to the appropriate CXL link layer. Additionally, the ARB/MUX maintains virtual link state machines for every link layer it interfaces with, processing power state transition requests from the local link layers and generating ARB/MUX link management packets to communicate with the remote ARB/MUX.
- **Chapter 6.0, “Flex Bus Physical Layer”** – The Flex Bus physical layer is responsible for training the link to bring it to operational state for transmission of PCIe packets or CXL flits. During operational state, it prepares the data from the CXL link layers or the PCIe link layer for transmission across the Flex Bus link; likewise, it converts data received from the link to the appropriate format to pass on to the appropriate link layer. This chapter describes the deltas from PCIe Base Specification to support the CXL mode of operation. The framing of the CXL flits and the physical layer packet layout for 68B Flit mode as well as 256B Flit mode are described. The mode selection process to decide between CXL mode or PCIe mode, including hardware autonomous negotiation and software-controlled selection is also described. Finally, CXL low-latency modes are described.
- **Chapter 7.0, “Switching”** – This chapter provides an overview of different CXL switching configurations and describes rules for how to configure switches.

Additionally, the Fabric Manager application interface is specified and CXL Fabric is introduced.

- [Chapter 8.0, “Control and Status Registers”](#) – This chapter provides details of the Flex Bus and CXL control and status registers. It describes the configuration space and memory mapped registers that are located in various CXL components. It also describes the CXL Component Command Interface.
- [Chapter 9.0, “Reset, Initialization, Configuration, and Manageability”](#) – This chapter describes the flows for boot, reset entry, and sleep-state entry; this includes the transactions sent across the link to initiate and acknowledge entry as well as steps taken by a CXL device to prepare for entry into each of these states. Additionally, this chapter describes the software enumeration model of both RCD and CXL virtual hierarchy and how the System Firmware view of the hierarchy may differ from the OS view. This chapter discusses the software view of CXL.mem, CXL extensions to Firmware-OS interfaces, and the CXL device manageability model.
- [Chapter 10.0, “Power Management”](#) – This chapter provides details on protocol specific link power management and physical layer power management. It describes the overall power management flow in three phases: protocol specific PM entry negotiation, PM entry negotiation for ARB/MUX interfaces (managed independently per protocol), and PM entry process for the physical layer. The PM entry process for CXL.cache and CXL.mem is slightly different than the process for CXL.io; these processes are described in separate subsections in this chapter.
- [Chapter 11.0, “CXL Security”](#) – This chapter provides details on the CXL Integrity and Data Encryption (CXL IDE) scheme that is used for securing CXL protocol flits that are transmitted across the link, IDE modes, and configuration flows.
- [Chapter 12.0, “Reliability, Availability, and Serviceability”](#) – This chapter describes the RAS capabilities supported by a CXL host, a CXL switch, and a CXL device. It describes how various types of errors are logged and signaled to the appropriate hardware or software error handling agent. It describes the Link Down flow and the error handling expectation. Finally, it describes the error injection requirements.
- [Chapter 13.0, “Performance Considerations”](#) – This chapter describes hardware and software considerations for optimizing performance across the Flex Bus link in CXL mode. It also describes the performance monitoring infrastructure for CXL components.
- [Chapter 14.0, “CXL Compliance Testing”](#) – This chapter describes methodologies for ensuring that a device is compliant with the CXL specification.

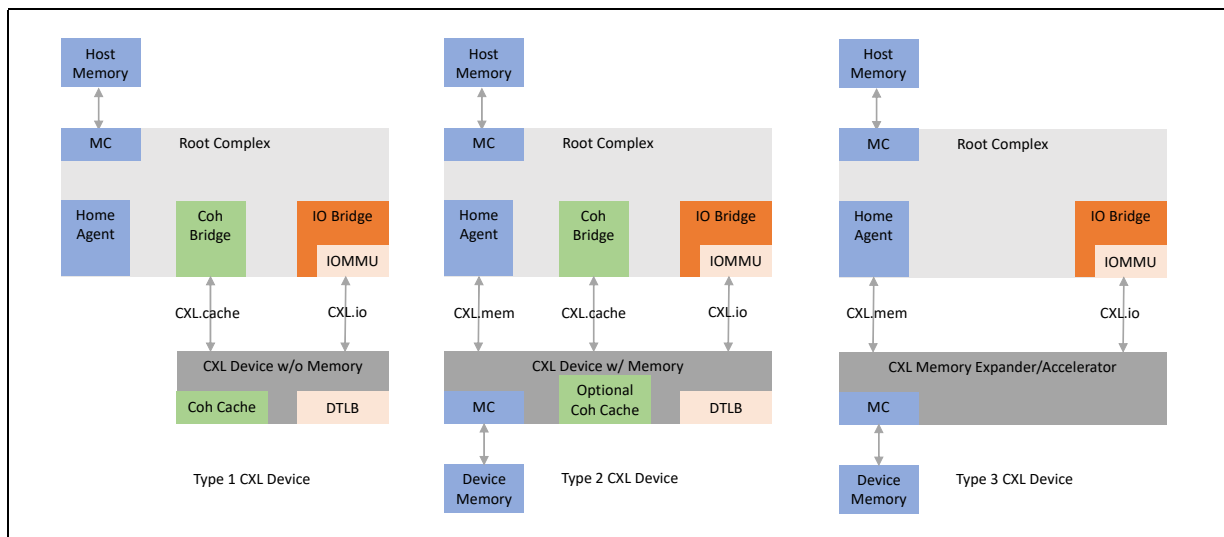


2.0 CXL System Architecture

This chapter describes the performance advantages and main features of CXL. CXL is a high-performance I/O bus architecture that is used to interconnect peripheral devices that can be either traditional non-coherent I/O devices, memory devices, or accelerators with additional capabilities. The types of devices that can attach via CXL and the overall system architecture is described in [Figure 2-1](#).

When Type 2 and Type 3 device memory is exposed to the host, it is referred to as Host-managed Device Memory (HDM). The coherence management of this memory has 3 options: Host-only Coherent (HDM-H), Device Coherent (HDM-D), and Device Coherent using Back-Invalidate Snoop (HDM-DB). The host and device must have a common understanding of the type of HDM for each address region. For additional details, refer to [Section 3.3](#).

Figure 2-1. CXL Device Types



Before diving into the details of each type of CXL device, here's a foreword about where CXL is not applicable.

Traditional non-coherent I/O devices mainly rely on standard Producer-Consumer ordering models and execute against Host-attached memory. For such devices, there is little interaction with the Host except for work submission and signaling on work completion boundaries. Such accelerators also tend to work on data streams or large contiguous data objects. These devices typically do not need the advanced capabilities provided by CXL, and traditional PCIe* is sufficient as an accelerator-attached medium.

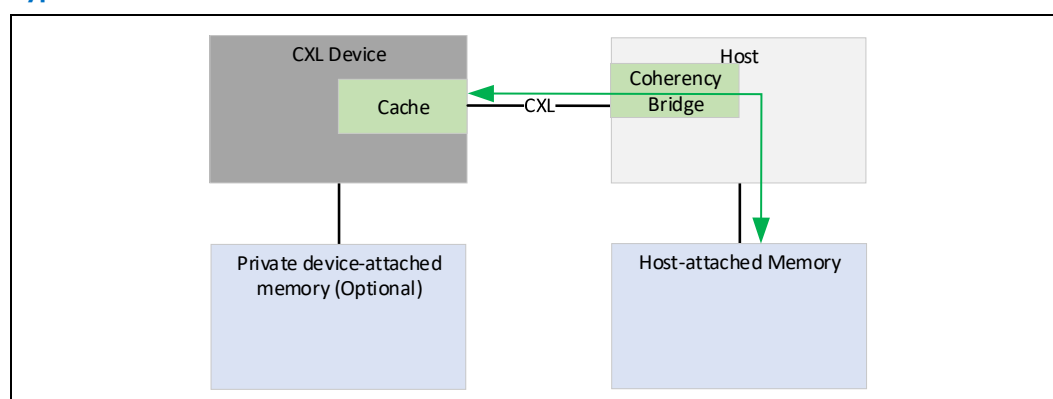
The following sections describe various profiles of CXL devices.

2.1 CXL Type 1 Device

CXL Type 1 Devices have special needs for which having a fully coherent cache in the device becomes valuable. For such devices, standard Producer-Consumer ordering models do not work well. One example of a device with special requirements is to perform complex atomics that are not part of the standard suite of atomic operations present on PCIe.

Basic cache coherency allows an accelerator to implement any ordering model it chooses and allows it to implement an unlimited number of atomic operations. These tend to require only a small capacity cache which can easily be tracked by standard processor snoop filter mechanisms. The size of cache that can be supported for such devices depends on the host's snoop filtering capacity. CXL supports such devices using its optional CXL.cache link over which an accelerator can use CXL.cache protocol for cache coherency transactions.

Figure 2-2. Type 1 Device - Device with Cache

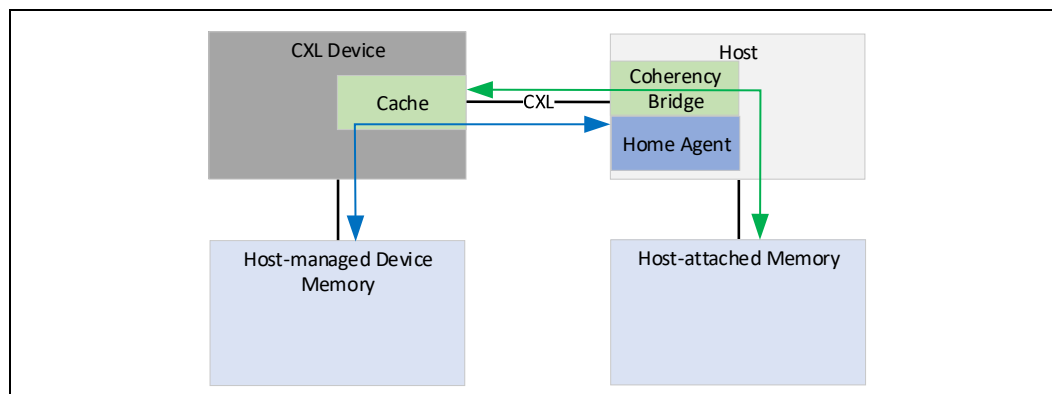


2.2 CXL Type 2 Device

CXL Type 2 are devices that negotiate all three protocols (CXL.cache, CXL.mem, and CXL.io). In addition to fully coherent cache, CXL Type 2 devices also have memory (e.g., DDR, High-Bandwidth Memory (HBM), etc.) attached to the device. These devices execute against memory, but their performance comes from having massive bandwidth between the accelerator and device-attached memory. The main goal for CXL is to provide a means for the Host to push operands into device-attached memory and for the Host to pull results out of device-attached memory such that it does not add software and hardware cost that offsets the benefit of the accelerator. This spec refers to coherent system address mapped device-attached memory as Host-managed Device Memory with Device Managed Coherence (HDM-D/HDM-DB).

There is an important distinction between HDM and traditional I/O and PCIe Private Device Memory (PDM). An example of such a device is a GPGPU with attached GDDR. Such devices have treated device-attached memory as private. This means that the memory is not accessible to the Host and is not coherent with the remainder of the system. It is managed entirely by the device hardware and driver and is used primarily as intermediate storage for the device with large data sets. The obvious disadvantage to a model such as this is that it involves high-bandwidth copies back and forth from the Host memory to device-attached memory as operands are brought in and results are written back. Please note that CXL does not preclude devices with PDM.

Figure 2-3. Type 2 Device - Device with Memory



At a high level, there are two methods of resolving device coherence of HDM. The first uses CXL.cache to manage coherence of the HDM and is referred to as “Device coherent.” The memory region supporting this flow is indicated with the suffix of “D” (HDM-D). The second method uses the dedicated channel in CXL.mem called Back-Invalidate Snoop and is indicated with the suffix “DB” (HDM-DB). The following sections will describe these in more detail.

2.2.1 Back-Invalidate Snoop Coherence for HDM-DB

With HDM-DB for Type 2 and Type 3 devices, the protocol enables new channels in the CXL.mem protocol that allow direct snooping by the device to the host using a dedicated Back-Invalidate Snoop (BISnp) channel. The response channel for these snoops is the Back-Invalidate Response (BIRsp) channel. The channels allow devices the flexibility to manage coherence by using an inclusive snoop filter tracking coherence for individual cachelines that may block new M2S Requests until BISnp messages are processed by the host. All device coherence tracking options described in [Section 2.2.2](#) are also possible when using HDM-DB; however, the coherence flows to the host for the HDM-DB must only use the CXL.mem S2M BISnp channel and not the D2H CXL.cache Request channel. HDM-DB support is required for all devices that implement 256B Flit mode, but the HDM-D flows will be supported for compatibility with 68B Flit mode.

For additional details on the flows used in HDM-DB, see [Section 3.5.1, “Flows for Back-Invalidate Snoops on CXL.mem.”](#)

2.2.2 Bias-based Coherency Model for HDM-D Memory

The Host-managed Device Memory (HDM) attached to a given device is referred to as device-attached memory to denote that it is local to only that device. The Bias-based coherency model defines two states of bias for device-attached memory: Host Bias and Device Bias. When the device-attached memory is in Host Bias state, it appears to the device just as regular Host-attached memory does. That is, if the device needs to access memory, it sends a request to the Host which will resolve coherency for the requested line. On the other hand, when the device-attached memory is in Device Bias state, the device is guaranteed that the Host does not have the line in any cache. As such, the device can access it without sending any transaction (e.g., request, snoops, etc.) to the Host whatsoever. It is important to note that the Host itself sees a uniform view of device-attached memory regardless of the bias state. In both modes, coherency is preserved for device-attached memory.

The main benefits of Bias-based coherency model are:

- Helps maintain coherency for device-attached memory that is mapped to system coherent address space.
- Helps the device access its local attached memory at high bandwidth without incurring significant coherency overheads (e.g., snoops to the Host).
- Helps the Host access device-attached memory in a coherent, uniform manner, just as it would for Host-attached memory.

To maintain Bias modes, a CXL Type 2 Device will:

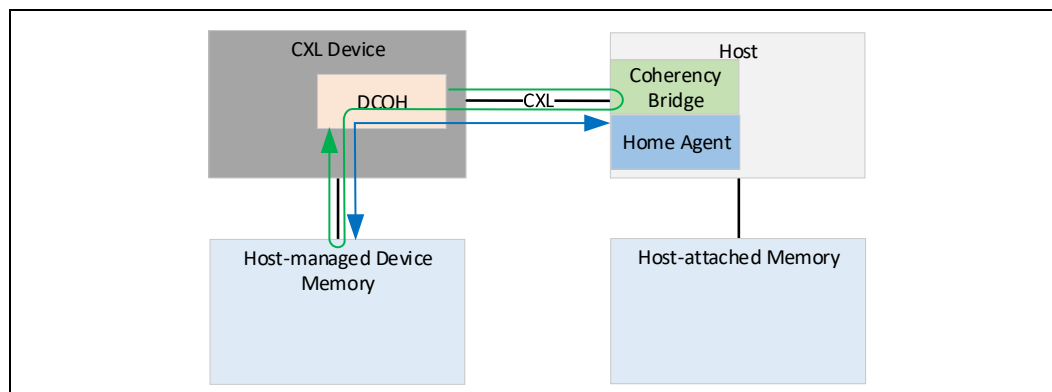
- Implement the Bias Table which tracks page-granularity Bias (e.g., 1 per 4-KB page) which can be cached in the device using a Bias Cache.
- Build support for Bias transitions using a Transition Agent (TA). This essentially looks like a DMA engine for “cleaning up” pages, which essentially means to flush the host’s caches for lines belonging to that page.
- Build support for basic load and store access to accelerator local memory for the benefit of the Host.

The bias modes are described in detail below.

2.2.2.1 Host Bias

Host Bias mode typically refers to the part of the cycle when the operands are being written to memory by the Host during work submission or when results are being read out from the memory after work completion. During Host Bias mode, coherency flows allow for high-throughput access from the Host to device-attached memory (as shown by the bidirectional blue arrow in Figure 2-4 to/from the host-managed device memory, the DCOH in the CXL device, and the Home Agent in the host) whereas device access to device-attached memory is not optimal since they need to go through the host (as shown by the green arrow in Figure 2-4 that loops between the DCOH in the CXL device and the Coherency Bridge in the host, and between the DCOH in the CXL device and the host-managed device memory).

Figure 2-4. Type 2 Device - Host Bias



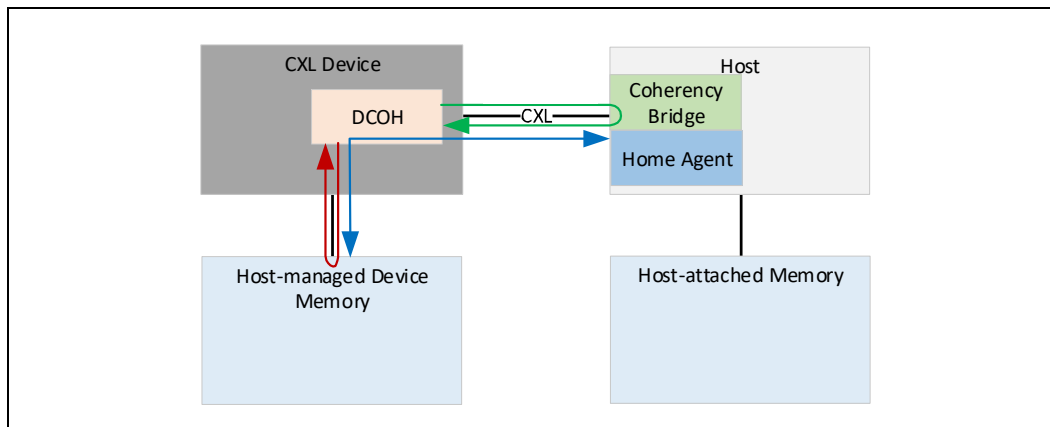
2.2.2.2 Device Bias

Device Bias mode is used when the device is executing the work, between work submission and completion, and in this mode, the device needs high-bandwidth and low-latency access to device-attached memory.

In this mode, device can access device-attached memory without consulting the Host’s coherency engines (as shown by the red arrow in Figure 2-5 that loops between the DCOH in the CXL device and the host-managed device memory). The Host can still access device-attached memory but may be forced to give up ownership by the

accelerator (as shown by the green arrow in Figure 2-5 that loops between the DCOH in the CXL device and the Coherency Bridge in the host). This results in the device seeing ideal latency and bandwidth from device-attached memory, whereas the Host sees compromised performance.

Figure 2-5. Type 2 Device - Device Bias



2.2.2.3 Mode Management

There are two envisioned Bias Mode Management schemes – Software Assisted and Hardware Autonomous. CXL supports both modes. Examples of Bias Flows are present in [Appendix A](#).

While two modes are described below, it is worth noting that devices do not need to implement any bias. In this case, all the device-attached memory degenerates to Host Bias. This means that all accesses to device-attached memory must be routed through the Host. An accelerator is free to choose a custom mix of Software assisted and Hardware autonomous bias management scheme. The Host implementation is agnostic to any of the above choices.

2.2.2.3.1 Software-assisted Bias Mode Management

With Software Assistance, we rely on software to know for a given page, in which state of the work execution flow the page resides. This is useful for accelerators with phased computation with regular access patterns. Based on this, software can best optimize the coherency performance on a page granularity by choosing Host or Device Bias modes appropriately.

Here are some characteristics of Software-assisted Bias Mode Management:

- Software Assistance can be used to have data ready at an accelerator before computation.
- If data is not moved to accelerator memory in advance, it is generally moved on demand based on some attempted reference to the data by the accelerator.
- In an “on-demand” data-fetch scenario, the accelerator must be able to find work to execute, for which data is already correctly placed, or the accelerator must stall.
- Every cycle that an accelerator is stalled eats into its ability to add value over software running on a core.
- Simple accelerators typically cannot hide data-fetch latencies.

Efficient software assisted data/coherency management is critical to the aforementioned class of simple accelerators.

2.2.2.3.2 Hardware Autonomous Bias Mode Management

Software assisted coherency/data management is ideal for simple accelerators, but of lesser value to complex, programmable accelerators. At the same time, the complex problems frequently mapped to complex, programmable accelerators like GPUs present an enormously complex problem to programmers if software assisted coherency/data movement is a requirement. This is especially true for problems that split computation between Host and accelerator or problems with pointer-based, tree-based, or sparse data sets.

The Hardware Autonomous Bias Mode Management, does not rely on software to appropriately manage page level coherency bias. Rather, it is the hardware that makes predictions on the bias mode based on the requester for a given page and adapts accordingly. The main benefits for this model are:

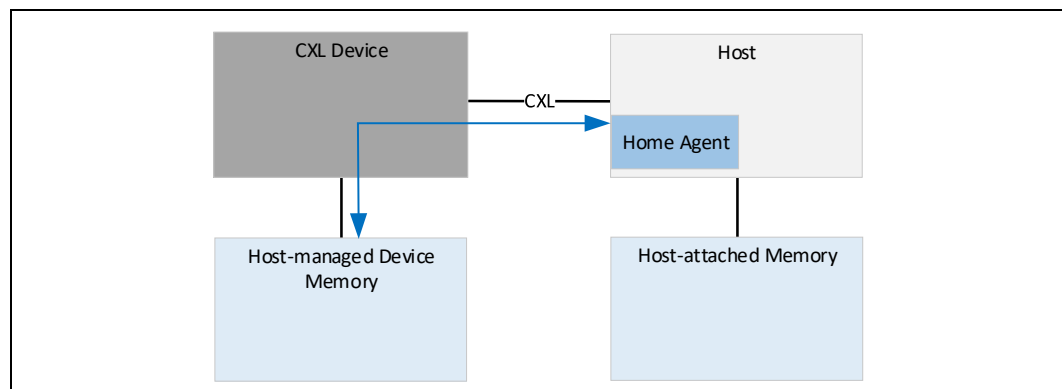
- Provide the same page granular coherency bias capability as in the software assisted model.
- Eliminate the need for software to identify and schedule page bias transitions prior to offload execution.
- Provide hardware support for dynamic bias transition during offload execution.
- Hardware support for this model can be a simple extension to the software-assisted model.
- Link flows and Host support are unaffected.
- Impact limited primarily to actions taken at the accelerator when a Host touches a Device Biased page and vice-versa.
- Note that even though this is an ostensible hardware driven solution, hardware need not perform all transitions autonomously – though it may do so if desired.

It is sufficient if hardware provides hints (e.g., “transition page X to bias Y now”) but leaves the actual transition operations under software control.

2.3 CXL Type 3 Device

A CXL Type 3 Device supports CXL.io and CXL.mem protocols. An example of a CXL Type 3 Device is an HDM-H memory expander for the Host as shown in [Figure 2-6](#).

Figure 2-6. Type 3 Device - HDM-H Memory Expander



Since this is not a traditional accelerator that operates on host memory, the device does not make any requests over CXL.cache. A passive memory expansion device would use the HDM-H memory region and normally do not directly manipulate the memory content while the memory is exposed to the host (exceptions exist for RAS and Security requirements). The device operates primarily over CXL.mem to service

requests sent from the Host. The CXL.io protocol is used for device discovery, enumeration, error reporting and management. The CXL.io protocol is permitted to be used by the device for other I/O-specific application usages. The CXL architecture is independent of memory technology and allows for a range of memory organization possibilities depending on support implemented in the Host. Type 3 device Memory that is exposed as an HDM-DB allows the same use of coherence as described in [Section 2.2.1](#) for Type 2 devices and requires the Type 3 device to include an internal Device Coherence engine (DCOH) in addition to what is shown in [Figure 2-6](#) for HDM-H. HDM-DB memory enables the device to behave as an accelerator (one variation of this is in-memory computing) and also enables direct access from peers using UIO on CXL.io or CXL.mem (see [Section 3.3.2.1](#)).

2.4 Multi Logical Device (MLD)

A Type 3 Multi-Logical Device (MLD) can partition its resources into up to 16 isolated Logical Devices. Each Logical Device is identified by a Logical Device Identifier (LD-ID) in CXL.io and CXL.mem protocols. Each Logical Device visible to a Virtual Hierarchy (VH) operates as a Type 3 device. The LD-ID is transparent to software accessing a VH. MLD components have common Transaction and Link Layers for each protocol across all LDs. Because LD-ID capability exists only in the CXL.io and CXL.mem protocols, MLDs are constrained to only Type 3 devices.

An MLD component has one LD reserved for the Fabric Manager (FM) and up to 16 LDs available for host binding. The FM-owned LD (FMLD) allows the FM to configure resource allocation across LDs and manage the physical link shared with multiple Virtual CXL Switches (VCSs). The FMLD's bus mastering capabilities are limited to generating error messages. Error messages generated by this function must only be routed to the FM.

The MLD component contains one MLD DVSEC (see [Section 8.1.10](#)) that is only accessible by the FM and addressable by requests that carry an LD-ID of FFFFh in CXL LD-ID TLP Prefix. Switch implementations must guarantee that FM is the only entity that is permitted to use the LD-ID of FFFFh.

An MLD component is permitted to use FM API to configure LDs or have statically configured LDs. In both of these configurations the configured LD resource allocation is advertised through MLD DVSEC. In addition, MLD DVSEC LD-ID Hot Reset Vector register of the FMLD is also used by CXL switch to trigger Hot Reset of one or more LDs. See [Section 8.1.10.2](#) for details.

2.4.1 LD-ID for CXL.io and CXL.mem

LD-ID is a 16-bit Logical Device identifier applicable for CXL.io and CXL.mem requests and responses. All requests targeting, and responses returned by, an MLD must include LD-ID.

See [Section 3.3.5](#) and [Section 3.3.6](#) for CXL.mem header formatting to carry the LD-ID field.

2.4.1.1 LD-ID for CXL.mem

CXL.mem supports only the lower 4 bits of LD-ID and therefore can support up to 16 unique LD-ID values over the link. Requests and responses forwarded over an MLD Port are tagged with LD-ID.

2.4.1.2 LD-ID for CXL.io

CXL.io supports carrying 16 bits of LD-ID for all requests and responses forwarded over an MLD Port. LD-ID FFFFh is reserved and is always used by the FM.

CXL.io utilizes the Vendor Defined Local TLP Prefix to carry 16 bits of LD-ID value. The format for Vendor Defined Local TLP Prefix is as follows. CXL LD-ID Vendor Defined Local TLP Prefix uses the VendPrefixL0 Local TLP Prefix type.

Table 2-1. LD-ID Link Local TLP Prefix

+0								+1								+2								+3							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
PCIe Base Specification Defined								LD-ID[15:0]																RSVD							

2.4.2 Pooled Memory Device Configuration Registers

Each LD is visible to software as one or more PCIe Endpoint (EP) Functions. While LD Functions support all the configuration registers, several control registers that impact common link behavior are virtualized and have no direct impact on the link. Each function of an LD must implement the configuration registers as described in PCIe Base Specification. Unless specified otherwise, the scope of the configuration registers is as described in PCIe Base Specification. For example, Memory Space Enable (MSE) bit in the command register controls a function's response to memory space.

Table 2-2 lists the set of register fields that have modified behavior when compared to PCIe Base Specification.

Table 2-2. MLD PCIe Registers (Sheet 1 of 2)

Register/Capability Structure	Capability Register Fields	LD-ID = FFFFh	All Other LDs
BIST Register	All Fields	Supported	Hardwire to all 0s
Device Capabilities Register	Max_Payload_Size_Supported, Phantom Functions Supported, Extended Tag Field Supported, Endpoint L1 Acceptable Latency	Supported	Mirrors LD-ID = FFFFh
	Endpoint L0s Acceptable Latency	Not supported	Not supported
	Captured Slot Power Limit Value, Captured Slot Power Scale	Supported	Mirrors LD-ID = FFFFh
Link Control Register	All Fields applicable to PCIe Endpoint	Supported (FMLD controls the fields) L0s not supported.	Read/Write with no effect
Link Status Register	All Fields applicable to PCIe Endpoint	Supported	Mirrors LD-ID = FFFFh
Link Capabilities Register	All Fields applicable to PCIe Endpoint	Supported	Mirrors LD-ID = FFFFh
Link Control 2 Register	All Fields applicable to PCIe Endpoint	Supported	Mirrors LD-ID = FFFFh RW fields are Read/Write with no effect
Link Status 2 Register	All Fields applicable to PCIe Endpoint	Supported	Mirrors LD-ID = FFFFh
MSI/MSI-X Capability Structures	All registers	Not supported	Each Functions that requires MSI/MSI-X must support it

Table 2-2. MLD PCIe Registers (Sheet 2 of 2)

Register/Capability Structure	Capability Register Fields	LD-ID = FFFFh	All Other LDs
Secondary PCIe Capability Registers	All register sets related to supported speeds (8 GT/s, 16 GT/s, 32 GT/s, 64 GT/s)	Supported	Mirrors LD-ID = FFFFh RO/Hwinit fields are Read/Write with no effect
	Lane Error Status, Local Data Parity Mismatch Status	Supported	Hardwire to all 0s
	Received Modified TS Data1 register, Received Modified TS Data 2 register, Transmitted Modified TS Data1 register, Transmitted Modified TS Data 2 register	Supported	Mirrors LD-ID = FFFFh
Lane Margining		Supported	Not supported
L1 Substates Extended Capability		Not supported	Not supported
Advanced Error Reporting (AER)	Registers that apply to Endpoint functions	Supported	Supported per LD ¹

1. AER – If an event is uncorrectable to the entire MLD, then it must be reported across all LDs. If the event is specific to a single LD, then it must be isolated to that LD.

2.4.3 Pooled Memory and Shared FAM

Host-managed Device Memory (HDM) that is exposed from a device that supports multiple hosts is referred to as Fabric-Attached Memory (FAM). FAM exposed via Logical Devices (LDs) is known as LD-FAM; FAM exposed in a more-scalable manner using Port Based Routing (PBR) links is known as Global-FAM (G-FAM).

FAM where each HDM region is dedicated to a single host interface is known as “pooled memory” or “pooled FAM”. FAM where multiple host interfaces are configured to access a single HDM region concurrently is known as “Shared FAM”, and different Shared FAM regions may be configured to support different sets of host interfaces.

LD-FAM includes several device variants. A Multi-Logical Device (MLD) exposes multiple LDs over a single shared link. A multi-headed Single Logical Device (MH-SLD) exposes multiple LDs, each with a dedicated link. A multi-headed MLD (MH-MLD) contains multiple links, where each link supports either MLD or SLD operation (optionally configurable), and at least one link supports MLD operation. See [Section 2.5, “Multi-Headed Device”](#) for additional details.

G-FAM devices (GFDs) are currently architected with one or more links supporting multiple host/peer interfaces, where the host interface of the incoming CXL.mem or UIO request is determined by its Source PBR ID (SPID) field included in the PBR message (see [Section 7.7.2](#) for additional details).

MH-SLDs and MH-MLDs should be distinguished from arbitrary multi-ported Type 3 components, such as the ones described in [Section 9.11.7.2](#), which supports a multiple CPU topology in a single OS domain.

2.4.4 Coherency Models for Shared FAM

The coherency model for each shared HDM-DB region is designated by the FM as being either multi-host hardware coherency or software-managed coherency.

Multi-host hardware coherency requires MLD hardware to track host coherence state as defined in [Table 3-37](#) for each cacheline to some varying extents, depending upon the MLD's implementation-specific tracking mechanism, which generally can be classified as a snoop filter or full directory. Each host can perform arbitrary atomic operations supported by its Instruction-Set Architecture (ISA) by gaining Exclusive access to a cacheline, performing the atomic operation on it within its cache. The data becomes globally observed using cache coherence and follows normal hardware cache eviction flows. MemWr commands to this region of memory must set the SnpType field to No-Op to prevent deadlock, which requires that the host must acquire ownership using the M2S Request channel before issuing the MemWr resulting in 2 phases to complete a write. This is a requirement for hardware coherency model in Shared FAM and Direct P2P CXL.mem (as compared HDM-DB region that is not shared and assigned to a single host root port and can use single phase snoopable Writes).

Shared FAM may also expose memory as simple HDM-H to the host, but this will only support the software coherence model between hosts.

Software-managed coherency does not require MLD hardware to track host coherence state. Instead, software on each host uses software-specific mechanisms to coordinate software ownership of each cacheline. Software may choose to rely on multi-host hardware coherency in other HDM regions to coordinate software ownership of cachelines in software-managed coherency HDM regions. Other mechanisms for software coordinating cacheline ownership are beyond the scope of this specification.

IMPLEMENTATION NOTE

Software-managed coherency relies on software having mechanisms to invalidate and/or flush cache hierarchies as well as relying on caching agents only to issue writebacks resulting from explicit cacheline modifications performed by local software. For performance optimization, many processors prefetch data without software having any direct control over the prefetch algorithm. For a variety of implementation-specific reasons, some caching agents may spontaneously write back clean cachelines that were prefetched by hardware but never modified by local software (e.g., promoting an E to M state without a store instruction execution). Any clean writeback of a cacheline by caching agents in hosts or devices that only intended to read that cacheline can overwrite updates performed by a host or device that executed writes to the cacheline. This breaks software-managed coherency. Note that a writeback resulting from a zero-length write transaction is not considered a clean writeback. Also note that hosts and/or devices may have an internal cacheline size that is larger than 64 bytes and a writeback could require multiple CXL writes to complete. If any of these CXL writes contain software-modified data, the writeback is not considered clean.

Software-managed coherency schemes are complicated by any host or device whose caching agents generate clean writebacks. A "No Clean Writebacks" capability bit is available for a host in the CXL System Description Structure (CSDS; see [Section 9.18.1.6](#)) or for a device in the DVSEC CXL Capability2 register (see [Section 8.1.3.7](#)), with caching agents to set if it guarantees that they will never generate clean writebacks. For backward compatibility, this bit being cleared does not necessarily indicate that any associated caching agents generate clean writebacks. When this bit is set for all caching agents that may access a Shared FAM range, a software-managed coherency protocol targeting that range can provide reliable results. This bit should be ignored by software for hardware-coherent memory ranges.

2.5 Multi-Headed Device

A Type 3 device with multiple CXL ports is considered a Multi-Headed Device. Each port is referred to as a “head”. There are two types of Multi-Headed Devices that are distinguished by how they present themselves on each head:

- MH-SLD, which present SLDs on all heads
- MH-MLD, which may present MLDs on any of their heads

Management of heads in Multi-Headed Devices follows the model defined for the device presented by that head:

- Heads that present SLDs may support the port management and control features that are available for SLDs
- Heads that present MLDs may support the port management and control features that are available for MLDs

Management of memory resources in Multi-Headed Devices follows the model defined for MLD components because both MH-SLDs and MH-MLDs must support the isolation of memory resources, state, context, and management on a per-LD basis. LDs within the device are mapped to a single head.

- In MH-SLDs, there is a 1:1 mapping between heads and LDs.
- In MH-MLDs, multiple LDs are mapped to at most one head. A head in a Multi-Headed Device shall have at least one and no more than 16 LDs mapped. A head with one LD mapped shall present itself as an SLD and a head with more than one LD mapped shall present itself as an MLD. Each head may have a different number of LDs mapped to it.

Figure 2-7 and Figure 2-8 illustrate the mappings of LDs to heads for MH-SLDs and MH-MLDs, respectively.

Figure 2-7. Head-to-LD Mapping in MH-SLDs

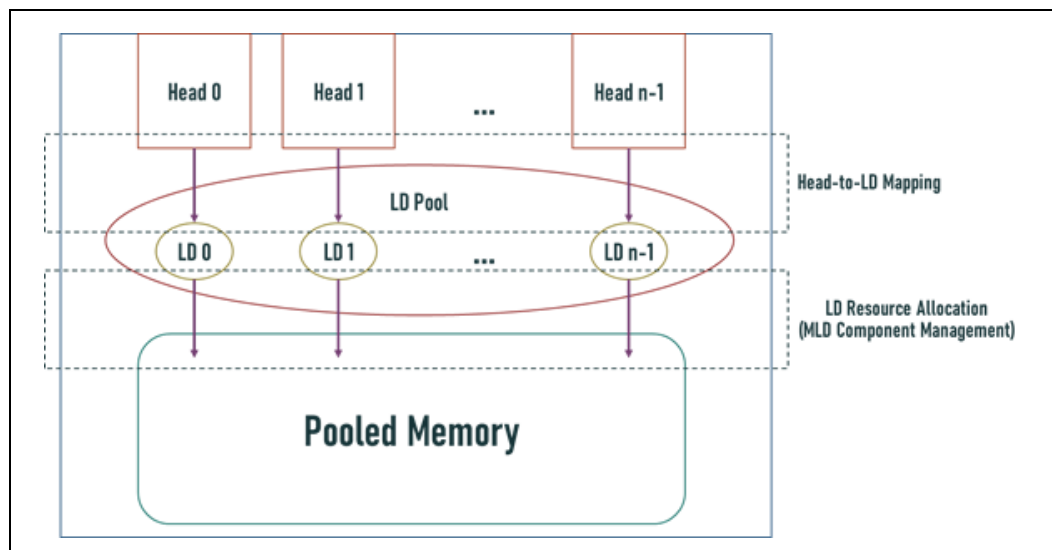
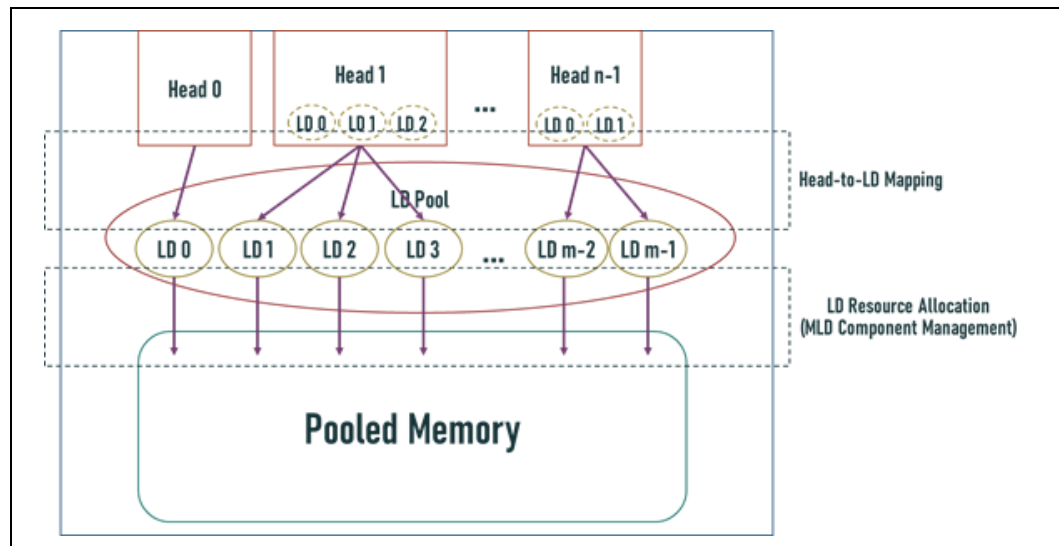


Figure 2-8. Head-to-LD Mapping in MH-MLDs

Multi-Headed Devices shall expose a dedicated Component Command Interface (CCI), the LD Pool CCI, for management of all LDs within the device. The LD Pool CCI may be exposed as an MCTP-based CCI or can be accessed via the Tunnel Management Command command through a head's Mailbox CCI, as detailed in [Section 7.6.7.3.1](#). The LD Pool CCI shall support the Tunnel Management Command for the purpose of tunneling management commands to all LDs within the device.

The number of supported heads reported by a Multi-Headed Device shall remain constant. Devices that support proprietary mechanisms to dynamically reconfigure the number of accessible heads (e.g., dynamic bifurcation of 2 x8 ports into a single x16 head, etc.) shall report the maximum number of supported heads.

2.5.1 LD Management in MH-MLDs

The LD Pool in an MH-MLD may support more than 16 LDs. MLDs exposed via the heads of an MH-MLD use LD-IDs from 0 to n-1 relative to that head, where n is the number of LDs mapped to the head. The MH-MLD maps the LD-IDs received at a head to the device-wide LD index in the MH-MLD's LD pool. The FMLD within each head of an MH-MLD shall expose and manage only the LDs that are mapped to that head.

An LD or FMLD on a head may permit visibility and management of all LDs within the device by using the Tunnel Management command to access the LD Pool CCI, as detailed in [Section 7.6.7.3.1](#).

2.6 CXL Device Scaling

CXL supports the ability to connect up to 16 Type 1 and/or Type 2 devices below a VH. To support this scaling, the Type 2 devices are required to use BISnp channel in the CXL.mem protocol to manage coherence of the HDM region. The BISnp channel introduced in the CXL 3.0 specification definition replaces the use of CXL.cache protocol to manage coherence of the device's HDM region. Type 2 devices that use CXL.cache for HDM-D coherence management are limited to a single device per Host bridge.

2.7 CXL Fabric

CXL Fabric describes features that rely on the Port Based Routing (PBR) messages and flows to enable scalable switching and advanced switching topologies. PBR enables a flexible low-latency architecture supporting up to 4096 PIDs in each fabric. G-FAM device attach (see [Section 2.8](#)) is supported natively into the fabric. Hosts and devices use standard messaging flows translated to and from PBR format through Edge Switches in the fabric. [Section 7.7](#) defines the requirements and use cases.

A CXL Fabric is a collection of one or more switches that are each PBR capable and interconnected with PBR links. A Domain is of a set of Host Ports and Devices within a single coherent Host Physical Address (HPA) space. A CXL Fabric connects one or more Host Ports to the devices within each Domain.

2.8 Global FAM (G-FAM) Type 3 Device

A G-FAM device (GFD) is a Type 3 device that connects to a CXL Fabric using a PBR link and relies on PBR message formats to provide FAM with much-higher scalability compared to LD-FAM devices. The associated FM API documented in [Section 8.2.9.9.10](#) and host mailbox interface details are provided in [Section 7.7.14](#).

Like LD-FAM devices, GFDs can support pooled FAM, Shared FAM, or both. GFDs rely exclusively on the Dynamic Capacity mechanism for capacity management. See [Section 7.7.2.3](#) for details and for other comparisons with LD-FAM devices.

2.9 Manageability Overview

To allow for different types of managed systems, CXL supports multiple types of management interfaces and management interconnects. Some are defined by external standards, while some are defined in the CXL specification.

CXL component discovery, enumeration, and basic configuration are defined by PCI-SIG* and CXL specifications. These functions are accomplished via access to Configuration Space structures and associated MMIO structures.

Security authentication and data integrity/encryption management are defined in PCI-SIG, DMTF, and CXL specifications. The associated management traffic is transported either via Data Object Exchange (DOE) using Configuration Space, or via MCTP-based transports. The latter can operate in-band using PCIe VDMs, or out-of-band using management interconnects such as SMBus, I3C, or dedicated PCIe links.

The Manageability Model for CXL Devices is covered in [Section 9.19](#). Advanced CXL-specific component management is handled using one or more CCIs, which are covered in [Section 9.20](#). CCI commands fall into 4 broad sets:

- Generic Component commands
- Memory Device commands
- FM API commands
- Vendor Specific commands

All 4 sets are covered in [Section 8.2.9](#), specifically:

- Command and capability determination
- Command foreground and background operation
- Event logging, notification, and log retrieval
- Interactions when a component has multiple CCIs

Each command is mandatory, optional, or prohibited, based on the component type and other attributes. Commands can be sent to devices, switches, or both.

CCIs use several transports and interconnects to accomplish their operations. The mailbox mechanism is covered in [Section 8.2.8.4](#), and mailboxes are accessed via an architected MMIO register interface. MCTP-based transports use PCIe VDMs in-band or any of the previously mentioned out-of-band management interconnects. FM API commands can be tunneled to MLDs and GFDs via CXL switches. Configuration and MMIO accesses can be tunneled to LDs within MLDs via CXL switches.

DMTF's Platform-Level Data Model (PLDM) is used for platform monitoring and control, and can be used for component firmware updates. PLDM may use MCTP to communicate with target CXL components.

Given CXL's use of multiple manageability standards and interconnects, it is important to consider interoperability when designing a system that incorporates CXL components.

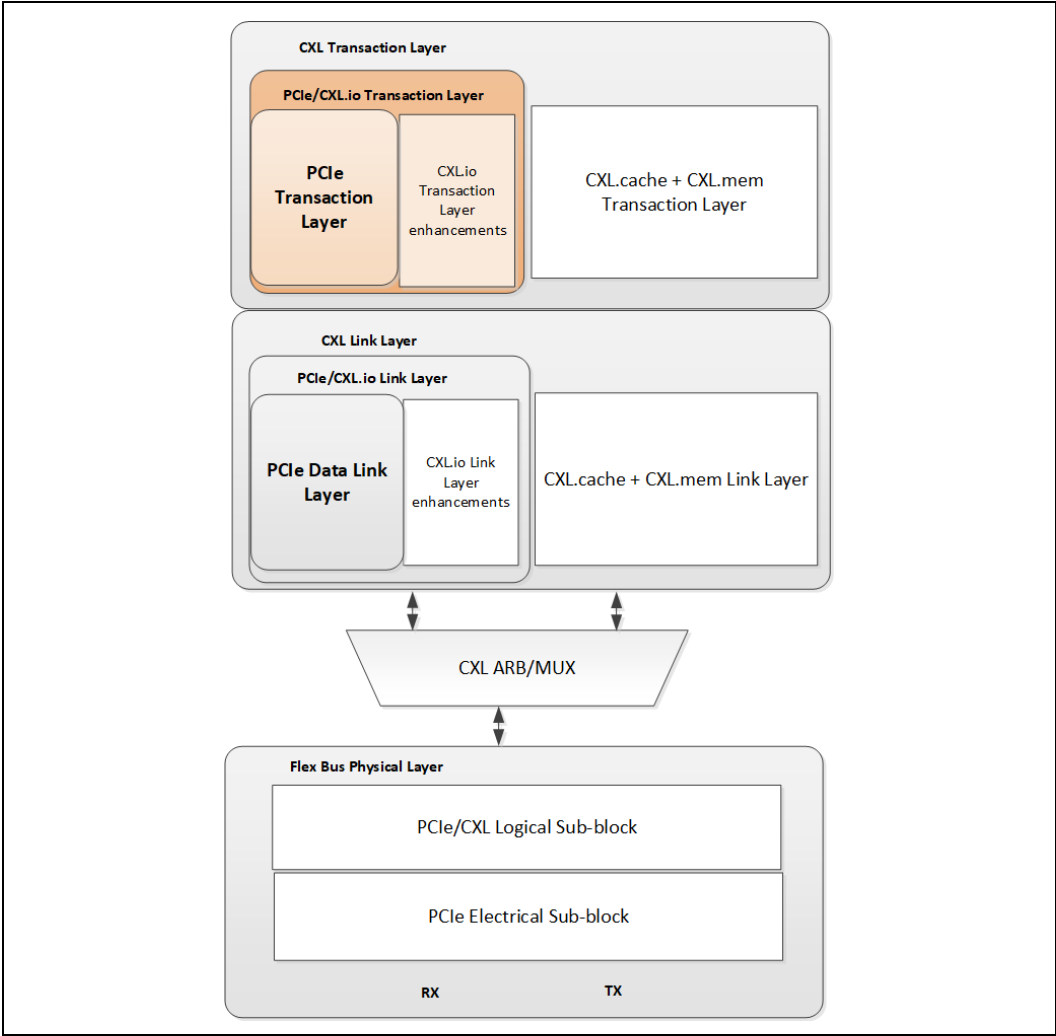
§ §

3.0 CXL Transaction Layer

3.1 CXL.io

CXL.io provides a non-coherent load/store interface for I/O devices. Figure 3-1 shows where the CXL.io transaction layer exists in the Flex Bus layered hierarchy. Transaction types, transaction packet formatting, credit-based flow control, virtual channel management, and transaction ordering rules follow the PCIe* definition; please refer to the “Transaction Layer Specification” chapter of PCIe Base Specification for details. This chapter highlights notable PCIe modes or features that are used for CXL.io.

Figure 3-1. Flex Bus Layers - CXL.io Transaction Layer Highlighted



3.1.1 CXL.io Endpoint

The CXL Alternate Protocol negotiation determines the mode of operation. See [Section 9.11](#) and [Section 9.12](#) for descriptions of how CXL devices are enumerated with the help of CXL.io.

A Function on a CXL device must not generate INTx messages if that Function participates in CXL.cache protocol or CXL.mem protocols. A Non-CXL Function Map DVSEC (see [Section 8.1.4](#)) enumerates functions that do not participate in CXL.cache or CXL.mem. Even though not recommended, these non-CXL functions are permitted to generate INTx messages.

Functions associated with an LD within an MLD component, including non-CXL functions, are not permitted to generate INTx messages.

3.1.2 CXL Power Management VDM Format

The CXL power management messages are sent as PCIe Vendor Defined Type 0 messages with a 4-DWORD data payload. These include the PMREQ, PMRSP, and PMGO messages. [Figure 3-2](#) and [Figure 3-3](#) provide the format for the CXL PM VDMs. The following are the characteristics of these messages:

- Fmt and Type fields are set to indicate message with data. All messages use routing of "Local-Terminate at Receiver." Message Code is set to Vendor Defined Type 0.
- Vendor ID field is set to 1E98h¹.
- Byte 15 of the message header contains the VDM Code and is set to the value of "CXL PM Message" (68h).
- The 4-DWORD Data Payload contains the CXL PM Logical Opcode (e.g., PMREQ, GPF) and any other information related to the CXL PM message. Details of fields within the Data Payload are described in [Table 3-1](#).

If a CXL component receives PM VDM with poison (EP=1), the receiver shall drop such a message. Because the receiver is able to continue regular operation after receiving such a VDM, it shall treat this event as an advisory non-fatal error.

If the receiver Power Management Unit (PMU) does not understand the contents of PM VDM Payload, it shall silently drop that message and shall not signal an uncorrectable error.

-
1. **NOTICE TO USERS:** THE UNIQUE VALUE THAT IS PROVIDED IN THIS CXL SPECIFICATION IS FOR USE IN VENDOR DEFINED MESSAGE FIELDS, DESIGNATED VENDOR SPECIFIC EXTENDED CAPABILITIES, AND ALTERNATE PROTOCOL NEGOTIATION ONLY AND MAY NOT BE USED IN ANY OTHER MANNER, AND A USER OF THE UNIQUE VALUE MAY NOT USE THE UNIQUE VALUE IN A MANNER THAT (A) ALTERS, MODIFIES, HARMS OR DAMAGES THE TECHNICAL FUNCTIONING, SAFETY OR SECURITY OF THE PCI-SIG ECOSYSTEM OR ANY PORTION THEREOF, OR (B) COULD OR WOULD REASONABLY BE DETERMINED TO ALTER, MODIFY, HARM OR DAMAGE THE TECHNICAL FUNCTIONING, SAFETY OR SECURITY OF THE PCI-SIG ECOSYSTEM OR ANY PORTION THEREOF (FOR PURPOSES OF THIS NOTICE, "**PCI-SIG ECOSYSTEM**" MEANS THE PCI-SIG SPECIFICATIONS, MEMBERS OF PCI-SIG AND THEIR ASSOCIATED PRODUCTS AND SERVICES THAT INCORPORATE ALL OR A PORTION OF A PCI-SIG SPECIFICATION AND EXTENDS TO THOSE PRODUCTS AND SERVICES INTERFACING WITH PCI-SIG MEMBER PRODUCTS AND SERVICES).

Figure 3-2. CXL Power Management Messages Packet Format - Non-Flit Mode

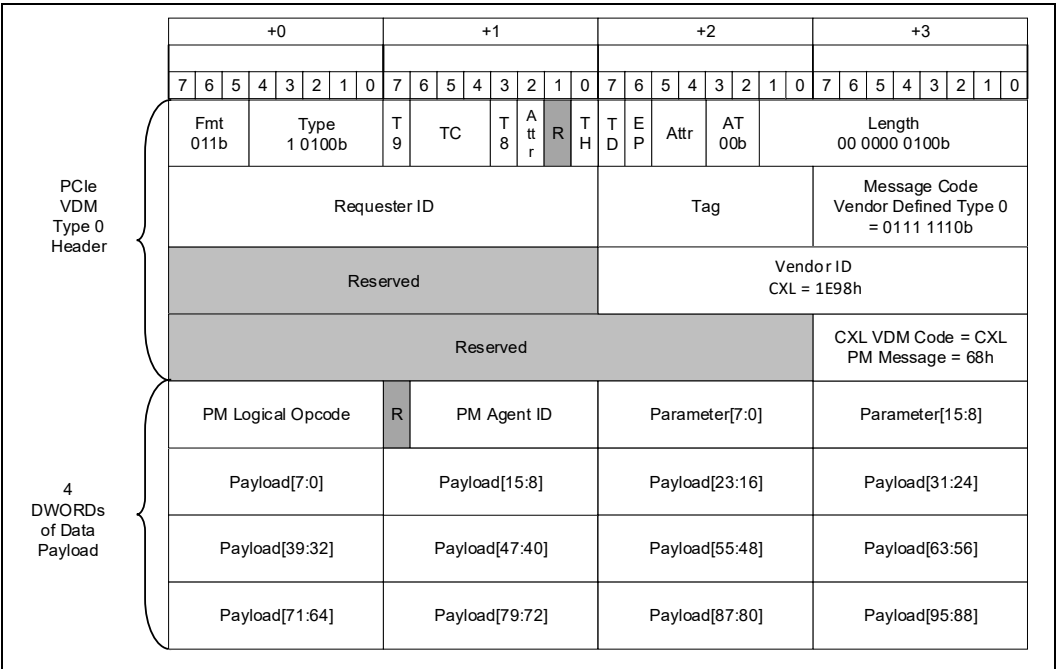


Figure 3-3. CXL Power Management Messages Packet Format - Flit Mode

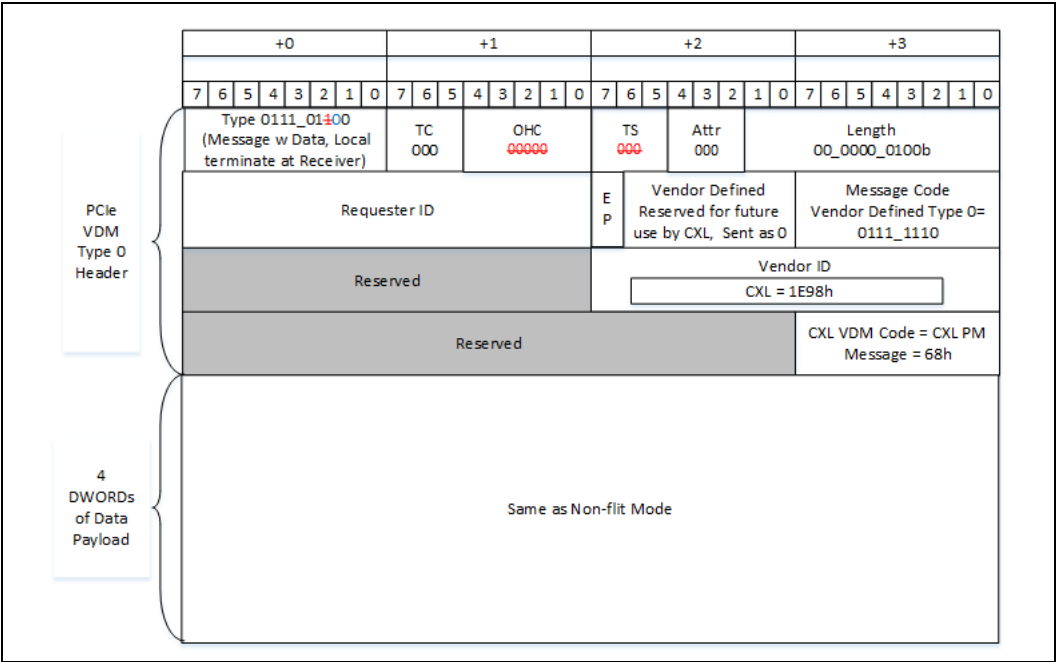


Table 3-1. CXL Power Management Messages - Data Payload Field Definitions (Sheet 1 of 2)

Field	Description	Notes
PM Logical Opcode[7:0]	Power Management Command 00h = AGENT_INFO 02h = RESETPREP 04h = PMREQ (PMRSP and PMGO) 06h = Global Persistent Flush (GPF) - FEh = CREDIT_RTN	
PM Agent ID[6:0]	PM2IP : Reserved. IP2PM : PM agent ID assigned to the device. Host communicates the PM Agent ID to device via the TARGET_AGENT_ID field of the first CREDIT_RTN message.	A device does not consume this value when it receives a message from the Host.
Parameter[15:0]	CREDIT_RTN (PM2IP and IP2PM) : Reserved. AGENT_INFO (PM2IP and IP2PM) - Bit[0]: REQUEST (set) /RESPONSE_N (cleared) - Bits[7:1]: INDEX - Bits[15:8]: Reserved PMREQ (PM2IP and IP2PM) - Bit[0]: REQUEST (set) /RESPONSE_N (cleared) - Bit[2]: GO - Bits[15:3]: Reserved RESETPREP (PM2IP and IP2PM) - Bit[0]: REQUEST (set) /RESPONSE_N (cleared) - Bits[15:1]: Reserved GPF (PM2IP and IP2PM) - Bit[0]: REQUEST (set) /RESPONSE_N (cleared) - Bits[15:1]: Reserved	

Table 3-1. CXL Power Management Messages - Data Payload Field Definitions (Sheet 2 of 2)

Field	Description	Notes
Payload[95:0]	CREDIT_RTN - Bits[7:0]: NUM_CREDITS (PM2IP and IP2PM) - Bits[14:8]: TARGET_AGENT_ID (Valid during the first PM2IP message, reserved in all other cases) - Bit[15]: Reserved AGENT_INFO (Request and Response) If Param.Index == 0: - Bits[7:0]: CAPABILITY_VECTOR - Bit[0]: Always set to indicate support for PM messages defined in CXL 1.1 spec - Bit[1]: Support for GPF messages - Bits[7:2]: Reserved - All other bits are reserved else: All reserved - All bits are reserved RESETPREP (Request and Response) - Bits[7:0]: ResetType 01h = System transition from S0 to S1 03h = System transition from S0 to S3 04h = System transition from S0 to S4 05h = System transition from S0 to S5 10h = System reset - Bits[15:8]: PrepType 00h = General Prep - All other encodings are reserved - Bits[17:16]: Reserved - All other bits are reserved PMREQ - Bits[31:0]: PCIe LTR format (as defined in Bytes 12-15 of PCIe LTR message, see Table 3-2) - All other bits are reserved GPF - Bits[7:0]: GPFTYPE - Bit[0]: Set to indicate that a power failure is imminent. Only valid for Phase 1 request messages. - Bit[1]: Set to indicate device must flush its caches. Only valid for Phase 1 request messages. - Bits[7:2]: Reserved - Bits[15:8]: GPF Status - Bit[8]: Set to indicate that the Cache Flush phase encountered an error. Only valid for Phase 1 responses and Phase 2 requests. - Bits[15:9]: Reserved - Bits[17:16]: Phase 01h = Phase 1 02h = Phase 2 - All other encodings are reserved - All other bits are reserved	CXL Agent must treat the TARGET_AGENT_ID field as reserved when returning credits to Host. Only Index 0 is defined for AGENT_INFO. All other Index values are reserved.

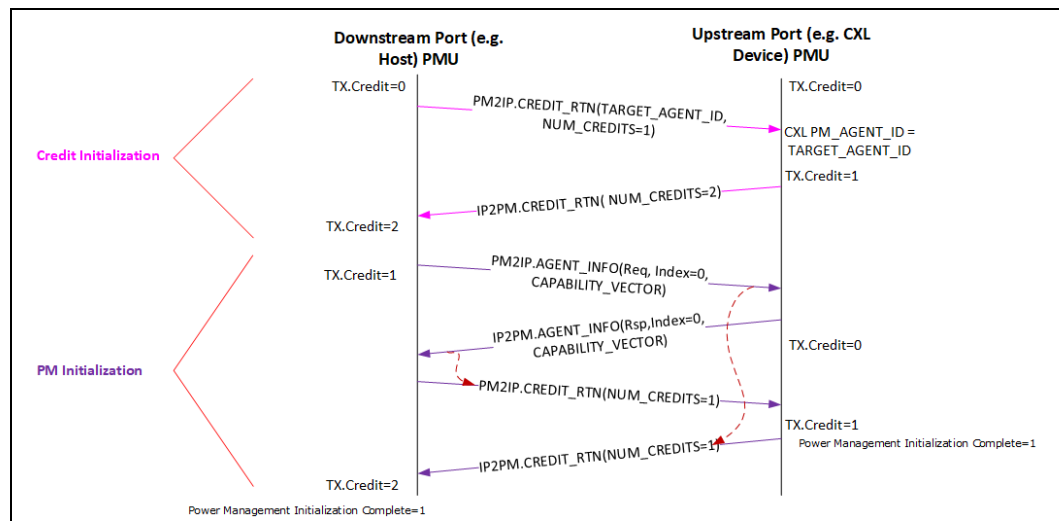
Table 3-2. PMREQ Field Definitions

Payload Bit Position	LTR Field
[31:24]	Snoop Latency[7:0]
[23:16]	Snoop Latency[15:8]
[15:8]	No-Snoop Latency[7:0]
[7:0]	No-Snoop Latency[15:8]

3.1.2.1 Credit and PM Initialization

PM Credits and initialization process is link local. [Figure 3-4](#) illustrates the use of PM2IP.CREDIT_RTN and PM2IP.AGENT_INFO messages to initialize Power Management messaging protocol intended to facilitate communication between the Downstream Port PMU and the Upstream Port PMU. A CXL switch provides an aggregation function for PM messages as described in [Section 9.1.2.1](#).

GPF messages do not require credits and the receiver shall not generate CREDIT_RTN in response to GPF messages.

Figure 3-4. Power Management Credits and Initialization

The CXL Upstream Port PMU must be able to receive and process CREDIT_RTN messages without dependency on any other PM2IP messages. Also, CREDIT_RTN messages do not use a credit. The CREDIT_RTN messages are used to initialize and update the Tx credits on each side, so that flow control can be appropriately managed. During the first CREDIT_RTN message during PM Initialization, the credits being sent via NUM_CREDITS field represent the number of credit-dependent PM messages that the initiator of CREDIT_RTN can receive from the other end. During the subsequent CREDIT_RTN messages, the NUM_CREDITS field represents the number of PM credits that were freed up since the last CREDIT_RTN message in the same direction. The first CREDIT_RTN message is also used by the Downstream Port PMU to assign a PM_AGENT_ID to the Upstream Port PMU. This ID is communicated via the TARGET_AGENT_ID field in the CREDIT_RTN message. The Upstream Port PMU must wait for the CREDIT_RTN message from the Downstream Port PMU before initiating any IP2PM messages.

An Upstream Port PMU must support at least one credit, where a credit implies having sufficient buffering to sink a PM2IP message with 128 bits of payload.

After credit initialization, the Upstream Port PMU must wait for an AGENT_INFO message from the Downstream Port PMU. This message contains the CAPABILITY_VECTOR of the PM protocol of the Downstream Port PMU. Upstream Port PMU must send its CAPABILITY_VECTOR to the Downstream Port PMU in response to the AGENT_INFO Req from the Downstream Port PMU. When there is a mismatch, Downstream Port PMU may implement a compatibility mode to work with a less capable Upstream Port PMU. Alternatively, Downstream Port PMU may log the mismatch and report an error, if it does not know how to reliably function with a less capable Upstream Port PMU.

There is an expectation from the Upstream Port PMU that it restores credits to the Downstream Port PMU as soon as a message is received. Downstream Port PMU can have multiple messages in flight, if it was provided with multiple credits. Releasing credits in a timely manner provides better performance for latency sensitive flows.

The following list summarizes the rules that must be followed by an Upstream Port PMU:

- Upstream Port PMU must wait to receive a PM2IP.CREDIT_RTN message before initiating any IP2PM messages.
- Upstream Port PMU must extract TARGET_AGENT_ID field from the first PM2IP message received from the Downstream Port PMU and use that as its PM_AGENT_ID in future messages.
- Upstream Port PMU must implement enough resources to sink and process any CREDIT_RTN messages without dependency on any other PM2IP or IP2PM messages or other message classes.
- Upstream Port PMU must implement at least one credit to sink a PM2IP message.
- Upstream Port PMU must return any credits to the Downstream Port PMU as soon as possible to prevent blocking of PM message communication over CXL Link.
- Upstream Port PMU are recommended to not withhold a credit for longer than 10 us.

3.1.3

CXL Error VDM Format

The CXL Error Messages are sent as PCIe Vendor Defined Type 0 messages with no data payload. Presently, this class includes a single type of message, namely Event Firmware Notification (EFN). When EFN is utilized to report memory errors, it is referred to as Memory Error Firmware Notification (MEFN). [Figure 3-5](#) and [Figure 3-6](#) provide the format for EFN messages.

The following are the characteristics of the EFN message:

- Fmt and Type fields are set to indicate message with no data.
- The message is sent using routing of "Routed to Root Complex." It is always initiated by a device.
- Message Code is set to Vendor Defined Type 0.
- Vendor ID field is set to 1E98h.
- Byte 15 of the message header contains the VDM Code and is set to the value of "CXL Error Message" (00h).
- Bytes 8, 9, 12, and 13 are cleared to all 0s.
- Bits[7:4] of Byte 14 are cleared to 0h. Bits[3:0] of Byte 14 are used to communicate the Firmware Interrupt Vector (abbreviated as FW Interrupt Vector in [Figure 3-5](#) and [Figure 3-6](#)).

Figure 3-5. CXL EFN Messages Packet Format - Non-Flit Mode

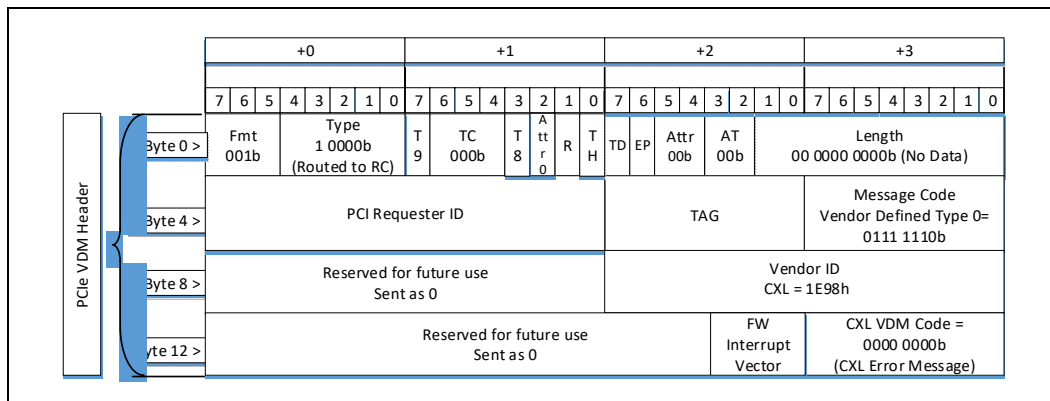
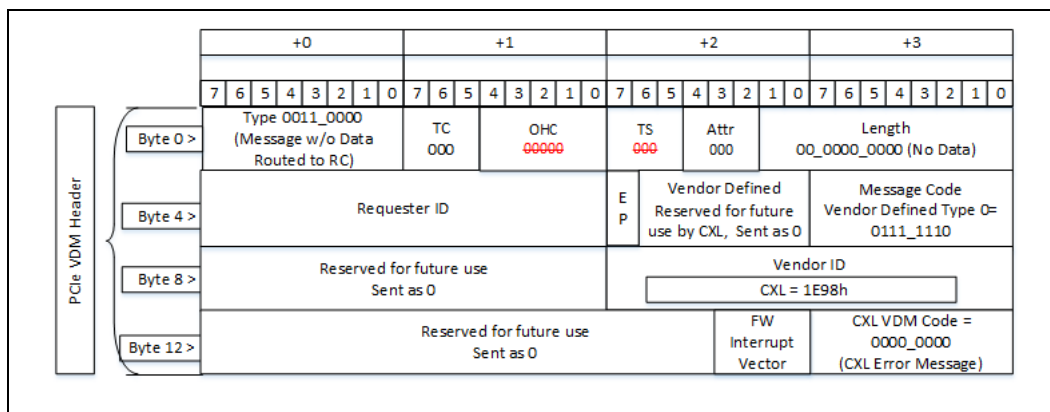


Figure 3-6. CXL EFN Messages Packet Format - Flit Mode



Encoding of the FW Interrupt Vector field is Host specific and thus not defined by the CXL specification. A Host may support more than one type of Firmware environment and this field may be used to indicate to the Host which one of these environments is to process this message.

3.1.4 Optional PCIe Features Required for CXL

Table 3-3 lists optional features per PCIe Base Specification that are required for CXL.

Table 3-3. Optional PCIe Features Required for CXL

Optional PCIe Feature	Notes
Data Poisoning by transmitter	
ATS	Only required if CXL.cache is present (e.g., only for Type 1 and Type 2 devices, but not for Type 3 devices)
Advanced Error Reporting (AER)	

3.1.5 Error Propagation

CXL.cache and CXL.mem errors detected by the device are propagated Upstream over the CXL.io traffic stream. These errors are logged as correctable and uncorrectable internal errors in the PCIe AER registers of the detecting component.

3.1.6 Memory Type Indication on ATS

Requests to certain memory regions can only be issued on CXL.io and cannot be issued on CXL.cache. It is up to the host to decide what these memory regions are. For example, on x86 systems, the host may choose to restrict access only to Uncacheable (UC) type memory over CXL.io. The host indicates such regions by means of an indication on ATS completion to the device.

All CXL functions that issue ATS requests must set the Page Aligned Request bit in the ATS Capability register to 1. In addition, ATS requests sourced from a CXL device must set the CXL Src bit.

Figure 3-7. ATS 64-bit Request with CXL Indication - Non-Flit Mode

ATS Request	+0							+1							+2							+3										
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Byte 0	Fmt 001b			Type 0_0000b				T9	TC			T8	ATT 0	R	R	TD	EP	ATTR			AT 01b		00_00xx_xxx0b									
Byte 4	Requester ID															Tag						Last DW BE 1111b				1st DW BE 1111b						
Byte 8	Untranslated Address [63:32]																															
Byte 12	Untranslated Address [31:12]																		Reserved								CXL Src	R	NW			

DWORD3, Byte 3, Bit 3 in ATS 64-bit request and ATS 32-bit request for both Flit Mode and Non-Flit Mode carries the CXL Src bit. Figure 3-7 shows the position of this bit in ATS 64-bit request (Non-Flit mode). See PCIe Base Specification for the format of the other request messages. The CXL Src bit is defined as follows:

- 0 = Indicates request initiated by a Function that does not support CXL.io Indication on ATS.
- 1 = Indicates request initiated by a Function that supports CXL.io Indication on ATS. All CXL Functions must set this bit.

Note:

This bit is Reserved in the ATS request as defined by PCIe Base Specification.

ATS translation completion from the Host carries the CXL.io bit in the Translation Completion Data Entry. See PCIe Base Specification for the message formats.

The CXL.io bit in the ATS Translation completion is valid when the CXL Src bit in the request is set. The CXL.io bit is as defined as follows:

- 0 = Requests to the page can be issued on all CXL protocols.
- 1 = Requests to the page can be issued by the Function on CXL.io only. It is a violation to issue requests to the page using CXL.cache protocol.

3.1.7 Deferrable Writes

Earlier revisions of this specification captured the “Deferrable Writes” extension to the CXL.io protocol, but this protocol has been adopted by PCIe Base Specification.

3.1.8 PBR TLP Header (PTH)

On PBR links in a PBR fabric, all .io TLPs, with exception of NOP-TLP, carry a fixed 1-DWORD header field called the PBR TLP header (PTH). PBR links are either Inter-Switch Links (ISL) or edge links from PBR switch to G-FAM. See Section 7.7.8 for details of where this header is inserted and deleted when the .io TLP traverses the PBR fabric from source to target.

NOP-TLPs are always transmitted without a preceding PTH. For Non-NOP-TLPs, PTH is always transmitted and it is transmitted on the immediate DWORD preceding the TLP Header base. Local-prefixes, if any, associated with a TLP are always transmitted before the PTH is transmitted. This is pictorially shown in [Figure 3-8](#).

To assist the receiver on a PBR link from disambiguating PTH from an NOP-TLP/Local-Prefix, the PCIe flit mode TLP grammar is modified as follows. Bits[7:6] of the first byte of all DWORDs, from the 1st DWORD of a TLP until a PTH is detected, are encoded as follows:

- 00b = NOP-TLP
- 01b = Rsvd
- 10b = Local Prefix
- 11b = PTH

After the receiver detects a PTH, PCIe TLP grammar rules are applied per PCIe Base Specification until the TLP ends, with the restriction that NOP-TLP and Local prefix cannot be transmitted in this region of the TLP.

3.1.8.1 Transmitter Rules Summary

- For NOP-TLP and Local-Prefix *Type*¹ field encodings, no PTH is pre-pended
- For all other *Type*¹ field encodings, a PTH is pre-pended immediately ahead of the Header base

3.1.8.2 Receiver Rules Summary

- For NOP-TLP, if bits[5:0] are not all 0s, the receiver treats it as a malformed packet and reports the error following the associated error reporting rules
- For a Local Prefix, if bits[5:0] are not one of 00 1101b through 00 1111b, the receiver treats it as a malformed packet and reports the error following the associated error reporting rules
- From beginning of a TLP to when a PTH is detected, receiver silently drops a DWORD if a reserved value of 01b is received for bits[7:6] in the DWORD
- If an NOP-TLP or Local Prefix is received immediately after a PTH, the receiver treats it as a malformed packet and reports the error following the associated error reporting rules

Note: Header queues in PBR switches/devices should be able to handle the additional DWORD of PTH that is needed to be carried between the source and target PBR links.

Note: PTH is included as part of normal link level CRC/FEC calculations/checks on PBR links to ensure reliable PTH delivery over the PBR link. For details regarding the PIF, DSAR, and Hie bits, see [Section 7.7.3.3](#), [Section 7.7.7](#), and [Section 7.7.6.2](#).

On MLD links, in the egress direction, the SPID information in this header is used to generate the LD-ID information on VendPrefixL0 message as defined in [Section 2.4](#). On MLD links, in the ingress direction, LD-ID in the VendPrefixL0 message is used to determine the DPID in the PBR packet.

1. *Type*[7:0] field as defined in PCIe Base Specification for Flit mode.

Table 3-4. PBR TLP Header (PTH) Format

Byte	+0								+1								+2								+3							
Bits	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Byte 0 ->	1	1	Rsvd			H i e	D S A R	P I F	SPID[11:0]										DPID[11:0]													

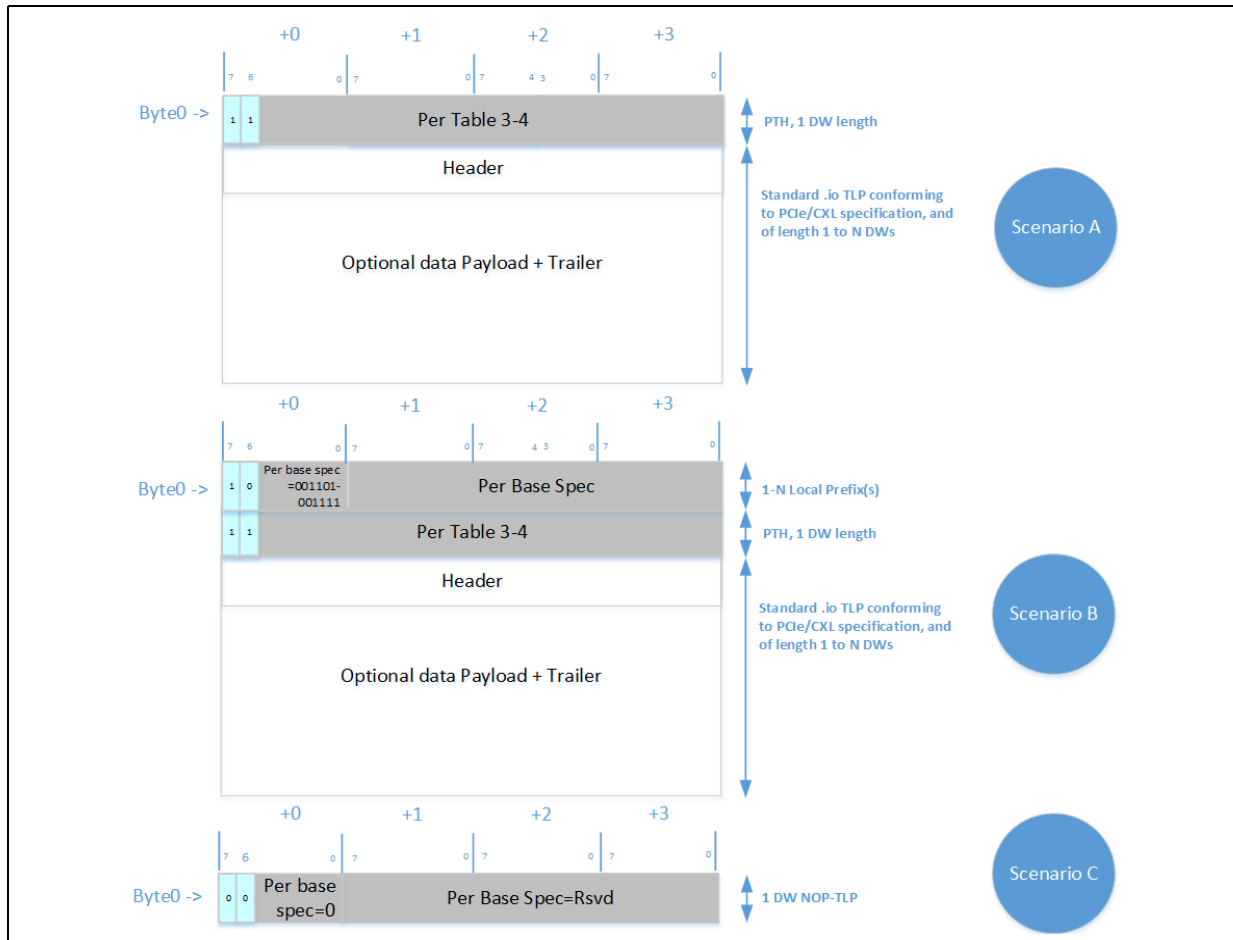
Table 3-5. NOP-TLP Header Format

Byte	+0								+1								+2								+3							
Bits	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Byte 0 ->	0	0	00 0000b						Per PCIe Base Specification																							

Table 3-6. Local Prefix Header Format

Byte	+0								+1								+2								+3							
Bits	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Byte 0 ->	1	0	0	0	1	1	01b/ 10b/ 11b		Per PCIe Base Specification																							

Figure 3-8. Valid .io TLP Formats on PBR Links



3.1.9 VendPrefixL0

Section 2.4.1.2 describes VendPrefixL0 usage on MLD links. For non-MLD HBR links, VendPrefixL0 carries the PBR-ID field to facilitate inter-domain communication between hosts and devices (e.g., GIM; see Section 7.7.3) and other vendor-proprietary usages (see Section 7.7.4). HBR links that use this form of the prefix must be directly attached to a PBR switch. On the switch ingress side, this prefix carries the DPID of the target edge link. On the egress side, this message carries the SPID of the source link that originated the TLP. The prefix format is shown in Table 3-51.

Table 3-7. VendPrefixL0 on Non-MLD Edge HBR Links

Byte	+0								+1								+2								+3							
Bits	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Byte 0 ->	As defined in PCIe Base Specification								PBR-ID[11:0]								Rsvd															

On the switch side, handling of this prefix is disabled by default. The FM can enable this functionality on each edge USP and DSP, via CCI mailbox. The method that the FM uses to determine the set of USPs/DSPs that are capable and trustworthy of enabling this functionality is beyond the scope of this specification.

Note: Edge PCIe links are not precluded from using this prefix for the same purpose described above. However, such usages are beyond the scope of this specification.

See [Section 7.7.3](#) and [Section 7.7.4](#) for transaction flows that involve TLPs with this prefix.

3.1.10 CXL DevLoad (CDL) Field in UIO Completions

To support QoS Telemetry (see [Section 3.3.4](#)) with UIO Direct P2P to HDM (see [Section 7.7.9](#)), UIO Completions contain the 2-bit CDL field, which carries the CXL DevLoad indication from HDM devices that support UIO Direct P2P. If an HDM device supports UIO Direct P2P to HDM, the HDM device shall populate the CDL field with values as defined in [Table 3-51](#). The CDL field exists in UIOWrCpl, UIORdCpl, and UIORdCplD TLPs.

3.1.11 CXL Fabric-related VDMs

In CXL Fabric (described in [Section 7.7](#)), there are many different uses for a CXL VDM. The uses fall into two categories: within a PBR Fabric, and outside a PBR Fabric.

When a VDM has a CXL Vendor ID, bytes 14 and 15 in the VDM header distinguish the use case via a CXL VDM Code and whether the use is within a PBR fabric. If within a PBR fabric, there is also a PBR Opcode. Additionally for PBR Fabric CXL VDMs, many of the traditional PCIe-defined fields such as Requester ID have no meaning and thus are reserved or in some cases, repurposed. See [Table 3-8](#) for a breakdown of the VDM header bytes for PBR Fabric VDMs.

Table 3-8. PBR VDM

Byte	+0								+1								+2								+3								
	7	6	5	4	3		1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	
0	Type 0011 0100b Type 0111 0100b								TC 000b		OHC 0 0000b						TS 000b		Attr 000b		Length 00 0000 0000b Length 00 00xx xxxxb ¹												
4	PCIe Requester ID / Rsvd																E P	Vendor Defined Rsvd				Message Code Vendor Defined Type 0 0111 1110b											
8	Rsvd																Vendor ID CXL = 1E98h																
12	Reserved				CmdSeq				SeqLen: 00h = 256 DWORDs								R S V D	SeqNum				PBR Opcode				CXL VDM Code							

1. x indicates don't care.

[Table 3-8](#) shows two Type encodings, a VDM without data and a VDM with data, both routed as "terminate at receiver". If a payload is not needed, the VDM without data is used. If any payload is required, the VDM with data is used. Because the PBR VDMs use PTH to route, the 'receiver' is the end of the tunnel (i.e., the matching DPID). PBR VDMs with data can have at most 128B (=32 DWORDs) of payload. If the SeqLen is more than 32 DWORDs, multiple VDMs will be needed to convey the entire sequence of VDMs (for UCPull VDM).

Depending on the CXL VDM Code, other fields in the VDM header may have meaning. Use of these additional fields will be defined in the section covering that particular encoding. These fields include:

- **PBR Opcode:** Subclass of PBR Fabric VDMs
- **CmdSeq:** Sequence number of the Host management transaction flow
- **SeqLen:** Length of the VDM sequence, applies to UCPull VDM
- **SeqNum:** Sequential VDM count with wrap if a message requires multiple sequential VDMs

Table 3-9 summarizes the various CXL vendor defined messages, each with a CXL VDM code and PBR Opcode, a message destination, and a brief summary of the message's use. The CXL VDM Code provides the category of VDM, while the PBR Opcode makes distinctions within that category. The remainder of this section deals with GFD management-related VDMs and Route Table Update related VDMs. For details of other VDMs, see [Section 7.7.11](#).

Table 3-9. CXL Fabric Vendor Defined Messages

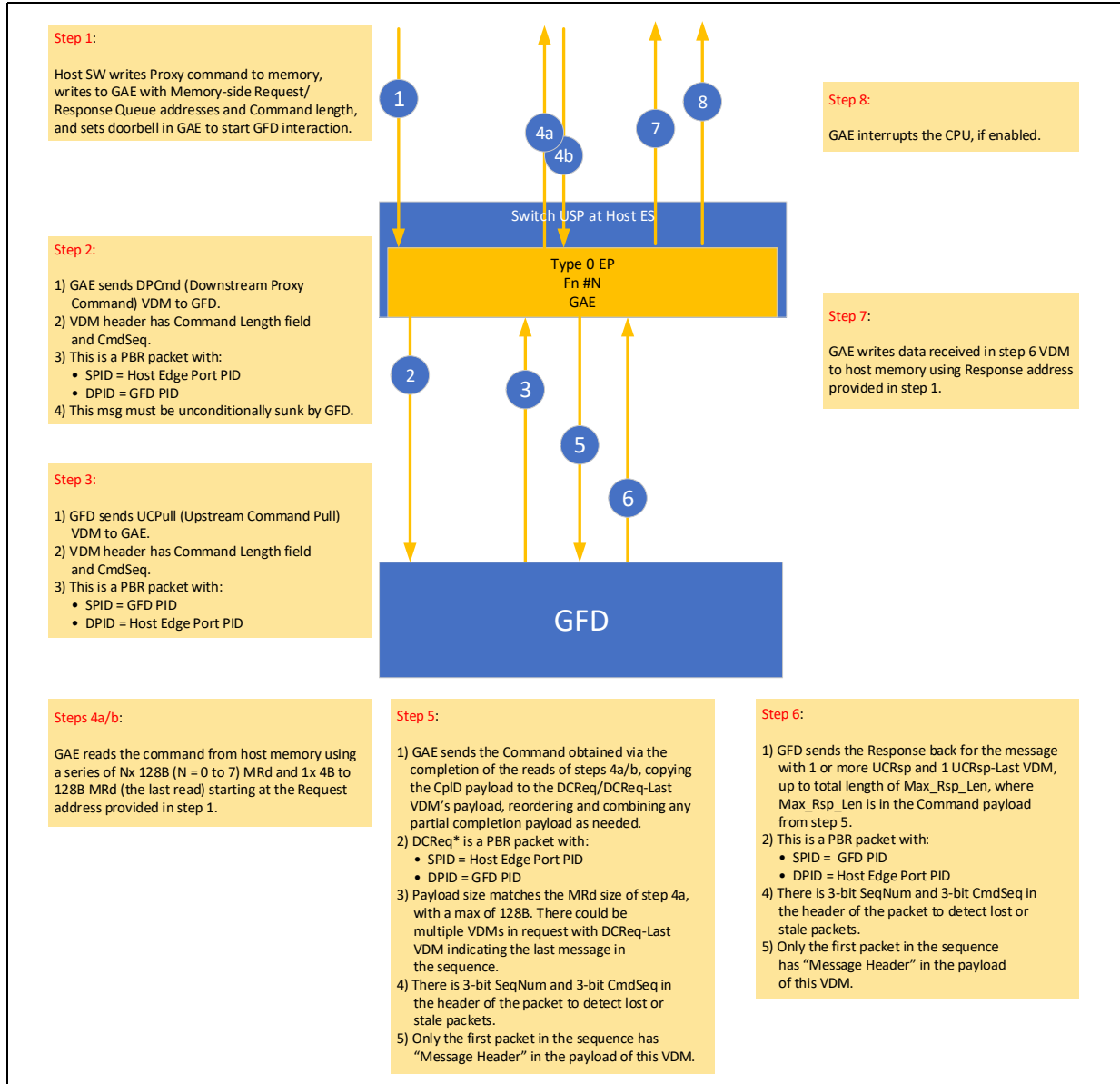
Message	USAR/ DSAR	CXL VDM Code	PBR Opcode	Destination	Payload (DWORDs)	Comment
Assert PERST#	DSAR	80h	0h	vUSP	0	Propagate fundamental reset downstream from vDSP.
Assert Reset	DSAR	80h	1h	vUSP	0	Propagate hot reset downstream from vDSP.
Deassert Reset	DSAR	80h	3h	vUSP	0	Propagate reset deassertion downstream from vDSP.
Link Up	DSAR	80h	4h	vDSP	0	Send upstream to vDSP, changing link state to L0 from detect.
PBR Link Partner Info	DSAR	90h	0h	Link Partner	16	Message with Data. Data saved in recipient.
DPCmd	DSAR	A0h	0	GFD	0	Downstream Proxy Command from GAE.
UCPull	DSAR	A1h	1	Host ES	0	Upstream command pull from GFD.
DCReq	DSAR	A0h	2	GFD	32	Downstream Command Request from GAE.
DCReq-Last	DSAR	A0h	3	GFD	1 – 32	Last DCReq.
UCRsp	DSAR	A1h	4	Host ES	32	Upstream Completion Response from GFD.
UCRsp-Last	DSAR	A1h	5	Host ES	1 – 32	Last UCRsp.
UCRsp-Fail	DSAR	A1h	6	Host ES	0	Failed DCReq receipt.
GAM	DSAR	A1h	7	Host ES	8	GFD log to host.
DCReq-Fail	DSAR	A0h	8	GFD	0	Failed UCPull response.
RTUpdate	DSAR	A1h	10h	Host ES	1 – 8	CacheID bus update from Downstream ES.
RTUpdateAck	DSAR	A1h	12h	Downstream ES	0	Acknowledgment of RTUpdate from Host ES.
RTUpdateNak	DSAR	A1h	13h	Downstream ES	0	Nak for RTUpdate from Host ES.
CXL PM	DSAR	68h	0	Varies	4	CXL Power Management.
CXL Error	USAR	00h	0	Host	0	CXL Error.

Although they exist outside the PBR Fabric, CXL VDM Codes 00h and 68h are listed to show the complete CXL VDM mapping. Their VDM Header is defined by PCI-SIG and thus does not match the fields provided for a PBR VDM header. These two VDMs will pass through the PBR Fabric using a hierarchical route and using the VDM Header originally defined in [Section 3.1.2](#) for CXL PM and in [Section 3.1.3](#) for CXL Error.

3.1.11.1 Host Management Transaction Flows of GFD

Figure 3-9 summarizes the Host Management Transaction Flows of GFD.

Figure 3-9. Host Management Transaction Flows of GFD



The Host ES has one GAE per host port. The GAE and GFD communicate via PID-routed VDMs.

Each GAE has an array of active messages, such that a host can communicate with multiple GFDs in parallel. Host software shall ensure that there is only one host-GFD management flow active per host-GFD pair.

The Host-to-GFD message flow consists of the following steps, after first storing the GFD command in host memory:

1. Host writes to GAE.
 - Writes pointer to GFD command in host memory (for UCPull read)
 - Writes pointer to write responses from GFD in host memory (for UCRsp data)
 - Writes command length
 - Writes CmdSeq
 - Write a mailbox command doorbell register (see [Section 8.2.8.4.4](#)), which causes the GAE to start the host management flow with step 2
2. Host ES creates CXL PBR VDM “DPCmd” with command length that targets the GFD PID and CmdSeq to identify the current command sequence.
 - This is an unsolicited message from the GFD point of view, and the GFD must be able to sink one such message for any supported RPID and drop any message from an unsupported RPID
3. GFD responds with a CXL PBR VDM “UCPull”, pulling the command for the indicated CmdSeq. The GFD response time may be delayed by responding to other doorbells from other RPIDs.
4. GAE converts CXL PBR VDM “UCPull” to one or more PCIe MRd TLPs.
 - a. GAE sends a series of MRds to read the command, starting at the address pointer supplied to the GAE in the Proxy GFD Management Command input payload. Each MRd size is a maximum of 128B. A command larger than 128B shall require multiple MRd to gather the full command. A total of (Nx) 128B MRd (with N from 0 to 7) and 1x (1B to 128B) MRd is needed to read any command of size up to 1024B.
 - b. The host completes each MRd with one or two CplD TLPs.
5. GAE gathers the read completion data in step 4b, re-ordering and combining partial completions as needed, to create a VDM payload. The GAE sends a series of DCReq/DCReq-Last VDMs with the completion data as VDM payload in the order that matches the series of MRd in step 4. The maximum payload for PBR VDMs is 128B (= 32 DWORDs).
 - Each VDM header contains the following:
 - An incrementing SeqNum, to allow detection of missing messages, starting with 0
 - A CmdSeq, to identify the current command for this Host – GFD thread
 - The last VDM in the sequence will be DCReq-Last.
 - VDMs before the last, if needed, will be DCReq.
 - The first VDM in the sequence shall start with a payload that matches the CCI Message header and payload as defined in [Section 7.6.3](#). Subsequent VDM’s payload shall contain only the remaining payload portion of the CCI Message and not repeat the header.
 - A failed command pull shall result in a DCReq-Fail VDM response instead of any DCReq and DCReq-Last.
6. GFD processes the command after it receives the last VDM (the “DCReq-Last”). The GFD shall send UCRsp/UCRsp-Last VDMs in response to the Host ES.
 - Each VDM header contains the following:
 - An incrementing SeqNum, to allow detection of missing messages, starting with 0
 - A CmdSeq, to identify the current command for this Host – GFD thread
 - The last VDM in the sequence will be UCRsp-Last
 - VDMs before the last, if needed, will be UCRsp

- If instead the GFD received DCReq-Fail, a UCRsp-Last shall be sent without processing the (incomplete) command
- 7. GAE converts “UCRsp” and “UCRsp-Last” series to a series of MWr with the payload the same as the VDM payload. The MWr address is supplied to the GAE in the Proxy Command input payload.
 - After the UCRsp-Last payload is written, the GAE mailbox control doorbell described in [Section 8.2.8.4.4](#) is cleared. If the MB Doorbell Interrupt is set, an interrupt will be sent by the GAE to the host.

If at any point the GAE disables the GFD access vector, any incoming UCRsp/UCRsp-Last VDMs from the disabled GFD shall be dropped, and any UCPull shall be replied to with a DML-Fail VDM.

The CmdSeq is used to synchronize the GAE and GFD to be working on the same command sequence. A host may issue a subsequent command with a different CmdSeq to abort a prior command that may not have completed the sequence. Both the GAE (step 3 UCPull and step 6 UCRsp*) and the GFD (step 2 DPCmd and step 5 DCReq*) shall check that the command sequence number is the current one for communication with the partner PID (GAE uses GFD’s PID, and GFD uses GAE’s PID). Any stale command sequence VDM will be dropped and logged. The GFD will always update its current CmdSeq[GAE’s PID] based on the value received in step 2 DPCmd.

The host management flow of a GFD also includes an asynchronous notification from the GFD to inform the host of events in the GFD, using a GAM (GFD Async Message) VDM. The GAM has a payload of up to 32B (8 DWORDs). This payload passes through the GAE to write to an address supplied to the GAE in the Proxy Command input payload. Each GAM write starts at a 32B-aligned offset.

All CXL.io TLPs sent over a PBR link shall have a PTH. The host management flow of GFD VDMs have PTH fields restricted to the following values:

- SPID =
 - From Host ES: Host Edge Port PID
 - From GFD: GFD PID
- DPID =
 - To GFD: GFD PID
 - To Host ES: Host Edge Port PID
- DSAR flag = 1

VDM header fields for GFD Message VDMs:

- CXL VDM code of A0h (to GFD) or A1h (to Host ES)
- PBR Opcode 0 – 8 to indicate the particular VDM
- **CmdSeq**: Holds the command sequence number issued initially in step 2, DPCmd.
- **SeqLen**: Holds the length in DWORDs of the subsequent stage DCReq sequence or UCRsp sequence
- **SeqNum**: Holds the sequence number for multi-VDM command or multi-VDM response, starting at 0h and wrapping after 7h back to 0h
- A list of all the CXL VDMs is provided in [Table 3-8](#)

3.1.11.2 Downstream Proxy Command (DPCmd) VDM

Initiating a Proxy GFD Management Command on the GAE shall cause the Host ES to create a ‘DPCmd’ VDM that targets the GFD.

The 'DPCmd' VDM fields are as follows. PTH holds:

- SPID = Host Edge Port PID
- DPID = GFD PID
- DSAR flag = 1

VDM header fields for 'DPCmd' VDMs:

- CXL VDM Code of A0h
- PBR Opcode 0 (DPCmd) DPC
- **CmdSeq**: Current host management command sequence
- **SeqLen**: Command Length (DWORDs, 1 to 256 DWORD max — value of 00h is 256 DWORDs)

A 'DPCmd' VDM is an unsolicited message from the GFD point of view. A GFD must be able to successfully record every 'DPCmd' VDM that it receives, up to one from each of its registered RPIDs. The 'DPCmd' VDM is a message without data. The SeqLen part of the VDM header holds the command length that will be pulled by the 'UCPull'.

Only one active DPCmd at a time is allowed per Host Edge Port PID/GFD PID pair. A DPCmd is considered active until the GAE receives a UCRsp-Last VDM in response to a DPCmd.

A GFD should receive only a single active DPCmd per Host PID. If a second DPCmd is received from the same Host PID, the first shall be silently aborted. If a second DPCmd is received before the current DPCmd completes, the GFD updates its current command sequence to the new DPCmd CmdSeq and aborts the prior command sequence.

3.1.11.3 Upstream Command Pull (UCPull) VDM

A GFD shall issue a 'UCPull' VDM when it services a received 'DPCmd' VDM. A single UCPull shall be issued for each DPCmd received, with its command length matching the command length of the DPCmd.

The 'UCPull' VDM fields are as follows. PTH holds:

- SPID = GFD PID
- DPID = Host Edge Port PID
- DSAR flag = 1

VDM header fields for 'UCPull' VDMs:

- CXL VDM Code of A1h
- PBR Opcode 1 (UCPull)
- **CmdSeq**: Matching current command sequence from DPCmd
- **SeqLen**: Length of command to pull (1 to 256 DWORDs)

A GAE must be able to successfully service every 'UCPull' VDM that it receives. The GAE advertises a maximum number of outstanding proxy threads, which defines the maximum number of UCPull VDMs that it would need to track.

A 'UCPull' is a message without data and consists of a single VDM (there is no sequence of UCPulls). The SeqLen field in the VDM header contains the targeted command length to pull from host memory via the GAE. The CmdSeq contains the current command sequence.

The CmdSeq should be checked to match the current command sequence for the GFD thread; if the CmdSeq does not match, the UCPull is dropped and logged. The UCPull SeqLen shall exactly match the DPCmd SeqLen. The GAE shall issue one or more MRds to pull the command. The last MRd may be 1 to 32 DWORDs. Any prior MRd shall be for exactly 32 DWORDs. The sum of all the MRd lengths shall be the SeqLen.

3.1.11.4 Downstream Command Request (DCReq, DCReq-Last, DCReq-Fail) VDMs

When the Host ES reads the command from host memory in response to a UCPull VDM, the completions for those reads are then conveyed to the GFD over a sequence of zero or more DCReq VDMs plus exactly one DCReq-Last VDM. Each completion payload is copied directly to the VDM payload. The Host ES is responsible for combining any partial completions together to make a single payload for the VDM. Each MRd issued to the host will result, when the CplDs for that MRd all return, in a single DCReq VDM or DCReq-Last VDM. The order of the DCReq/DCReq-Last VDMs shall match the order of the MRd. The DCReq-Last VDM represents the end of the Downstream Command Request series. Any missing DCReq/DCReq-Last VDMs in the sequence should result in the GFD failing the command.

The 'DCReq' / 'DCReq-Last' / 'DCReq-Fail' VDM fields are as follows. PTH holds:

- SPID = Host Edge Port PID
- DPID = GFD PID
- DSAR flag = 1

VDM header fields for 'DCReq' / 'DCReq-Last' / 'DCReq-Fail' VDMs:

- CXL VDM Code of A0h
- PBR Opcode 2 (DCReq) / PBR Opcode 3 (DCReq-Last) / PBR Opcode 8 (DCReq-Fail)
- **CmdSeq**: Command sequence to be checked by Receiver
- **SeqLen**: Defined only for DCReq-Last, holds the expected length of the Response in the next step (UCRsp); 0 for DCReq and DCReq-Fail
- **SeqNum**:
 - Defined for all DCReq and DCReq-Last VDMs, initialized to 0 at the start of the sequence and incremented for each subsequent VDM; 0 of DCReq-Fail
 - Holds the DCReq* VDM sequence number, starting at 0h and incrementing for each subsequent VDM

Any 'DCReq' VDM shall have a payload of exactly 32 DWORDs. A short command may not have any DCReq VDMs. Every Downstream Command Request sequence shall have exactly one DCReq-Last VDM. The DCReq-Last VDM can have any payload length from 1 to 32 DWORDs.

The DCReq-Last VDM header has SeqLen defined to indicate the next step UCRsp length in DWORDs.

The GFD that is receiving the DCReq* VDMs checks that the CmdSeq matches its current command sequence for that Host Edge Port PID; if the CmdSeq does not match, the DCReq* VDM is dropped and logged.

The DCReq-Fail VDM shall be sent if CmdSeq is correct but the PID of the GFD is not enabled in the host's GAE's GMV and a UCPull from that GFD is received.

3.1.11.5 Upstream Command Response (UCRsp, UCRsp-Last, UCRsp-Fail) VDMs

When a GFD receives a 'DCReq-Last' VDM, the GFD checks that the CmdSeq is the current command sequence for that Host Edge Port PID and that all DCReq VDMs and DCReq-Last VDM were received.

If either check fails, the command sequence stops. If all DCReq are not received, as determined by a missing SeqNum, a UCRsp-Fail VDM shall be sent.

If the checks pass, a GFD will issue a 'UCRsp' VDM after the GFD processes the earlier-received 'DCReq-Last' VDM. The total length of the Response is dictated by the SeqLen provided in the 'DCReq-Last' SeqLen in the VDM header.

There will be zero or more 'UCRsp' VDMs and always exactly one 'UCRsp-Last' VDM, where the 'UCRsp-Last' VDM ends the sequence and is sent last. The sum of the DWORDs of response will match the length requested in the 'DCReq-Last' VDM SeqLen field. Each 'UCRsp' VDM will be 32 DWORDs. The 'UCRsp-Last' VDM can be from 1 to 32 DWORDs. Each 'UCRsp' / 'UCRsp-Last' VDM in the sequence increments the sequence number, starting at 0 and wrapping from 7 back to 0. Any missing UCRsp VDM in the sequence should result in a response error being flagged in the GAE.

The 'UCRsp' / 'UCRsp-Last' / 'UCRsp-Fail' VDM fields are as follows. PTH holds:

- SPID = GFD PID
- DPID = Host Edge Port PID
- DSAR flag = 1

VDM header fields for 'UCRsp' / 'UCRsp-Last' / 'UCRsp-Fail' VDMs:

- CXL VDM Code of A1h
- PBR Opcode 4 (UCRsp) / PBR Opcode 5 (UCRsp-Last) / PBR Opcode 6 (UCRsp-Fail)
- **CmdSeq**: Current command sequence
- **SeqNum**: Holds the UCRsp* VDM sequence number, starting at 0h and incrementing for each subsequent VDM; 0 for UCRsp-Fail

3.1.11.6 GFD Async Message (GAM) VDM

The GAM VDM is used to notify a host of some issue with its use of the GFD. The payload of the GAM should pass through to the host GAM buffer at a 32B-aligned offset. The GAM payload is fixed at 8 DWORDs, as shown in [Table 3-10](#).

Table 3-10. GAM VDM Payload

Byte	+3								+2								+1								+0							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	Memory Req Opcode								GFD Notification								Reserved				PID[11:0]											
4	Reserved				RPID[11:0]												Flags[15:0]															
8	DPA[63:0]																															
12																																
16	HPA[63:0]																															
20																																
24	Timestamp																															
28																																

The GAM payload shall be written by the GAE endpoint to the GAE's circular GAM Buffer as described in [Section 7.7.2.7](#).

The 'GAM' VDM fields are as follows. PTH holds:

- SPID = GFD PID
- DPID = Host Edge Port PID
- DSAR flag = 1

VDM header fields for 'GAM' VDMs:

- CXL VDM Code of A1h
- PBR Opcode 7 (GAM)

3.1.11.7 Route Table Update (RTUpdate) VDM

On a PBR link, the CacheID of a CXL.cache message is replaced with a PID. A table is needed at both the Host ES and Downstream ES to swap between PID and CacheID.

A VDM from the Downstream ES is needed to convey the information, a list of pairs of (PID and CacheID), to the Host ES with a maximum of 16 pairs, corresponding to 8 DWORDs. The flow to the RTUpdate VDM is described in more detail in [Section 7.7.12.5](#).

An RTUpdate VDM is sent from Downstream ES firmware to Host ES firmware. The DPID is the Host PID, allowing for a route to the Host ES. However, the Host ES ingress shall trap on the CXL VDM Code of A1h and handle the VDM in the Host ES.

The 'RTUpdate' VDM fields are as follows. PTH holds:

- SPID = vUSP's fabric port's PID
- DPID = Host Edge Port PID
- DSAR flag = 1

VDM header fields for 'RTUpdate' VDMs:

- CXL VDM Code of A1h
- PBR Opcode 10h (RTUpdate)

[Table 3-11](#) shows the RTUpdate VDM payload format. Note that a value of FFFh for DSP_PID in the payload indicates that the PID is invalid and hence the PID to CacheID information pair needs to be discarded.

Table 3-11. RTUpdate VDM Payload

Byte	+3								+2								+1								+0							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	CacheID[3:0]				DSP PID[11:0]								CacheID[3:0]				DSP PID[11:0]															
4	CacheID[3:0]				DSP PID[11:0]								CacheID[3:0]				DSP PID[11:0]															
...	...																															
28	CacheID[3:0]				DSP PID[11:0]								CacheID[3:0]				DSP PID[11:0]															

3.1.11.8 Route Table Update Response (RTUpdateAck, RTUpdateNak) VDMs

The response to the RTUpdate VDM shall be one of the following:

- RTUpdateAck VDM if the update is successful

- RTUpdateNak VDM if the update is unsuccessful
- RTUpdateNak VDM if a VDM in the sequence was lost

The DPID is set to the vUSP's fabric port's PID, which routes the RTUpdateAck VDM back to the Downstream ES. However, the Downstream ES ingress shall trap on the CXL VDM Code of A1h to direct the VDM to switch firmware.

The Downstream ES, upon receipt of the RTUpdateAck VDM, shall set the commit complete bit in the CacheID table.

The 'RTUpdateAck' / 'RTUpdateNak' VDM fields are as follows. PTH holds:

- SPID = Host Edge Port PID
- DPID = vUSP's fabric port's PID
- DSAR flag = 1

VDM header fields for 'RTUpdateAck' / 'RTUpdateNak' Response VDMs are as follows:

- CXL VDM Code of A1h
- PBR Opcode 12h (RTUpdateAck) / PBR Opcode 13h (RTUpdateNak)

3.2

CXL.cache

3.2.1

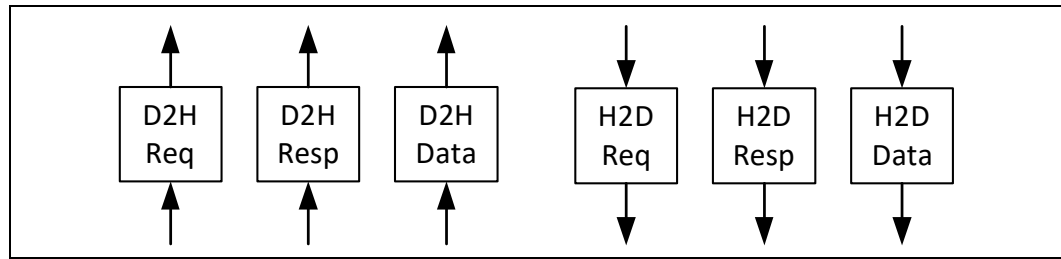
Overview

The CXL.cache protocol defines the interactions between the device and host as a number of requests that each have at least one associated response message and sometimes a data transfer. The interface consists of three channels in each direction: Request, Response, and Data. The channels are named for their direction, D2H for device to host and H2D for host to device, and the transactions they carry, Request, Response, and Data as shown in [Figure 3-10](#). The independent channels allow different kinds of messages to use dedicated wires and achieve both decoupling and a higher effective throughput per wire.

A D2H Request carries new requests from the Device to the Host. The requests typically target memory. Each request will receive zero, one, or two responses and at most one 64-byte cacheline of data. The channel may be back pressured without issue. D2H Response carries all responses from the Device to the Host. Device responses to snoops indicate the state the line was left in the device caches, and may indicate that data is being returned to the Host to the provided data buffer. They may still be blocked temporarily for link layer credits. D2H Data carries all data and byte enables from the Device to the Host. The data transfers can result either from implicit (as a result of snoop) or explicit write-backs (as a result of cache capacity eviction). A full 64-byte cacheline of data is always transferred. D2H Data must make progress or deadlocks may occur. D2H Data may be temporarily blocked for link layer credits, but must not require any other D2H transaction to complete to free the credits.

An H2D Request carries requests from the Host to the Device. These are snoops to maintain coherency. Data may be returned for snoops. The request carries the location of the data buffer to which any returned data should be written. H2D Requests may be back pressured for lack of device resources; however, the resources must free up without needing D2H Requests to make progress. H2D Response carries ordering messages and pulls for write data. Each response carries the request identifier from the original device request to indicate where the response should be routed. For write data pull responses, the message carries the location where the data should be written. H2D Responses can only be blocked temporarily for link layer credits. H2D Data delivers the data for device read requests. In all cases a full 64-byte cacheline of data is transferred. H2D Data transfers can only be blocked temporarily for link layer credits.

Figure 3-10. CXL.cache Channels



3.2.2 CXL.cache Channel Description

3.2.2.1 Channel Ordering

In general, all the CXL.cache channels must work independently of one another to ensure that forward progress is maintained. For example, because requests from the device to the Host to a given address X will be blocked by the Host until it collects all snoop responses for this address X, linking the channels would lead to deadlock.

However, there is a specific instance where ordering between channels must be maintained for the sake of correctness. The Host needs to wait until Global Observation (GO) messages, sent on H2D Response, are observed by the device before sending subsequent snoops for the same address. To limit the amount of buffering needed to track GO messages, the Host assumes that GO messages that have been sent over CXL.cache in a given cycle cannot be passed by snoops sent in a later cycle.

For transactions that have multiple messages on a single channel with an expected order (e.g., WritePull and GO for WrInv) the Device/Host must ensure they are observed correctly using serializing messages (e.g., the Data message between WritePull and GO for WrInv as shown in [Figure 3-14](#)).

3.2.2.2 Channel Crediting

To maintain the modularity of the interface no assumptions can be made on the ability to send a message on a channel because link layer credits may not be available at all times. Therefore, each channel must use a credit for sending any message and collect credit returns from the receiver. During operation, the receiver returns a credit whenever it has processed the message (i.e., freed up a buffer). It is not required that all credits are accounted for on either side, it is sufficient that credit counter saturates when full. If no credits are available, the sender must wait for the receiver to return one.

[Table 3-12](#) describes which channels must drain to maintain forward progress and which can be blocked indefinitely. Additionally, [Table 3-12](#) defines a summary of the forward progress and crediting mechanisms in CXL.cache, but this is not the complete definition. See [Section 3.4](#) for the complete set of the ordering rules that are required for protocol correctness and forward progress.

Table 3-12. CXL.cache Channel Crediting Summary

Channel	Forward Progress Condition	Blocking Condition	Description
D2H Request (Req)	Credited to Host	Can be blocked by all other message classes in CXL.cachemem.	Needs Host buffer, could be held by earlier requests
D2H Response (Rsp)	Pre-allocated	Temporary link layer back pressure is allowed. Host may block waiting for H2D Response to drain.	Headed to specified Host buffer
D2H Data	Pre-allocated	Temporary link layer back pressure is allowed. Host may block for H2D Data to drain.	Headed to specified Host buffer
H2D Request (Req)	Credited to Device	Must make progress. Temporary back pressure is allowed.	May be back pressured temporarily due to lack of availability of D2H Response or D2H Data credits
H2D Response (Rsp)	Pre-allocated	Link layer only, must make progress. Temporary back pressure is allowed.	Headed to specified device buffer
H2D Data	Pre-allocated	Link layer only, must make progress. Temporary back pressure is allowed.	Headed to specified device buffer

3.2.3 CXL.cache Wire Description

The definition of each of the fields for each CXL.cache Channel is provided below. Each message in will support 3 variants: 68B Flit, 256B Flit, and PBR Flit. The use of each of these will be negotiated in the physical layer for each link as defined in [Chapter 6.0](#).

3.2.3.1 D2H Request

Table 3-13. CXL.cache - D2H Request Fields (Sheet 1 of 2)

D2H Request	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The request is valid.
Opcode	5			The opcode specifies the operation of the request. Details in Table 3-22 .
CQID	12			Command Queue ID: The CQID field contains the ID of the tracker entry that is associated with the request. When the response and data are returned for this request, the CQID is sent in the response or data message indicating to the device which tracker entry originated this request. IMPLEMENTATION NOTE CQID usage depends on the round-trip transaction latency and desired bandwidth. A 12-bit ID space allows for 4096 outstanding requests which can saturate link bandwidth for a x16 link at 64 GT/s with average latency of up to 1 μs¹.
NT	1			For cacheable reads, the NonTemporal bit is used as a hint to indicate to the host how it should be cached. Details in Table 3-14 .
CacheID	0	4	0	Logical CacheID of the source of the message. Not supported in 68B flit messages. Not applicable in PBR messages where DPID infers this field.

Table 3-13. CXL.cache - D2H Request Fields (Sheet 2 of 2)

D2H Request	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Address[51:6]	46			Carries the physical address of coherent requests.
SPID	0		12	Source PID
DPID	0		12	Destination PID
RSVD	14	7		
Total	79	76	96	

1. Formula assumed in this calculation is: "Latency Tolerance in ns" = "number of Requests" * (64B per Request) / "Peak Bandwidth in GB/s". Assuming a peak bandwidth of 256 GB/s (raw bidirectional bandwidth of a x16 CXL port at 64 GT/s) results in a latency tolerance of 1024 ns.

Table 3-14. Non Temporal Encodings

NonTemporal	Definition
0	Default behavior. This is Host implementation specific.
1	Requested line should be moved to Least Recently Used (LRU) position

3.2.3.2 D2H Response

Table 3-15. CXL.cache - D2H Response Fields

D2H Response	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The response is valid.
Opcode	5			The opcode specifies the what kind of response is being signaled. Details in Table 3-25 .
UQID	12			Unique Queue ID: This is a reflection of the UQID sent with the H2D Request and indicates which Host entry is the target of the response.
DPID	0		12	Destination PID
RSVD	2	6		
Total	20	24	36	

3.2.3.3 D2H Data

Table 3-16. CXL.cache - D2H Data Header Fields

D2H Data Header	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The Valid signal indicates that this is a valid data message.
UQID	12			Unique Queue ID: This is a reflection of the UQID sent with the H2D Response and indicates which Host entry is the target of the data transfer.
ChunkValid	1	0		In case of a 32B transfer on CXL.cache, this indicates what 32-byte chunk of the cacheline is represented by this transfer. If not set, it indicates the lower 32B and if set, it indicates the upper 32B. This field is ignored for a 64B transfer.
Bogus	1			The Bogus bit indicates that the data associated with this evict message was returned to a snoop after the D2H request was sent from the device, but before a WritePull was received for the evict. This data is no longer the most current, so it should be dropped by the Host.
Poison	1			The Poison bit is an indication that this data chunk is corrupted and should not be used by the Host.
BEP	0	1		Byte-Enables Present: Indication that 5 data slots are included in the message where the 5 th data slot carries the 64-bit Byte Enables. This field is carried as part of the Flit header bits in 68B Flit mode.
DPID	0		12	Destination PID
RSVD	1	8		
Total	17	24	36	

3.2.3.3.1 Byte Enables (68B Flit)

In 68B Flit mode, the presence of data byte enables is indicated in the flit header, but only when one or more of the byte enable bits has a value of 0. In that case, the byte enables are sent as a data chunk as described in [Section 4.2.2](#).

3.2.3.3.2 Byte-Enables Present (256B Flit)

In 256B Flit mode, a BEP (Byte-Enables Present) bit is included with the message header that indicates BE slot is included at the end of the message. The Byte Enable field is 64 bits wide and indicates which of the bytes are valid for the contained data.

3.2.3.4 H2D Request

Table 3-17. CXL.cache – H2D Request Fields (Sheet 1 of 2)

H2D Request	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The Valid signal indicates that this is a valid request.
Opcode	3			The Opcode field indicates the kind of H2D request. Details in Table 3-26 .
Address[51:6]	46			The Address field indicates which cacheline the request targets.

Table 3-17. CXL.cache – H2D Request Fields (Sheet 2 of 2)

H2D Request	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
UQID	12			Unique Queue ID: This indicates which Host entry is the source of the request.
CacheID	0	4	0	Logical CacheID of the destination of the message. Value is assigned by Switch edge ports and not observed by the device. Host implementation may constrain the number of encodings that the Host can support. Not applicable with PBR messages where DPID infers this field.
SPID	0		12	Source PID
DPID	0		12	Destination PID
RSVD	2	6		
Total	64	72	92	

3.2.3.5 H2D Response

Table 3-18. CXL.cache - H2D Response Fields

H2D Response	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The Valid bit indicates that this is a valid response to the device.
Opcode	4			The Opcode field indicates the type of the response being sent. Details in Table 3-27 .
RspData	12			The response Opcode determines how the RspData field is interpreted as shown in Table 3-27 . Thus, depending on Opcode, it can either contain the UQID or the MESI information in bits [3:0] as shown in Table 3-20 .
RSP_PRE	2			RSP_PRE carries performance monitoring information. Details in Table 3-19 .
CQID	12			Command Queue ID: This is a reflection of the CQID sent with the D2H Request and indicates which device entry is the target of the response.
CacheID	0	4	0	Logical CacheID of the destination of the message. This value is returned by the host based on the CacheID sent in the D2H request. Not applicable with PBR messages where DPID infers this field.
DPID	0		12	Destination PID
RSVD	1	5		
Total	32	40	48	

Table 3-19. RSP_PRE Encodings

RSP_PRE[1:0]	Response
00b	Host Cache Miss to Local CPU socket memory
01b	Host Cache Hit
10b	Host Cache Miss to Remote CPU socket memory
11b	Reserved

Table 3-20. Cache State Encoding for H2D Response

Cache State	Encoding
Invalid (I)	0011b
Shared (S)	0001b
Exclusive (E)	0010b
Modified (M)	0110b
Error (Err) ¹	0100b

1. Covers error conditions not covered by poison such as errors in coherence resolution.

3.2.3.6 H2D Data

Table 3-21. CXL.cache - H2D Data Header Fields

H2D Data Header	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The Valid bit indicates that this is a valid data to the device.
CQID	12			Command Queue ID: This is a reflection of the CQID sent with the D2H Request and indicates which device entry is the target of the data transfer.
ChunkValid	1	0		In case of a 32B transfer on CXL.cache, this indicates what 32-byte chunk of the cacheline is represented by this transfer. If not set, it indicates the lower 32B and if set, it indicates the upper 32B. This field is ignored for a 64B transfer.
Poison	1			The Poison bit indicates to the device that this data is corrupted and as such should not be used.
GO-Err	1			The GO-ERR bit indicates to the agent that this data is the result of an error condition and should not be cached or provided as response to snoops. Covers error conditions not covered by poison such as errors in coherence resolution.
CacheID	0	4	0	Logical CacheID of the destination of the message. Host and switch must support this field to set a nonzero value. Not applicable in PBR messages where DPID infers this field.
DPID	0		12	Destination PID
RSVD	8	9		
Total	24	28	36	

3.2.4 CXL.cache Transaction Description

3.2.4.1 Device-attached Memory Flows for HDM-D/HDM-DB

When a CXL Type 2 device exposes memory to the host using Host-managed Device Memory Device-Coherent (HDM-D/HDM-DB), the device is responsible to resolve coherence of HDM between the host and device. CXL defines two protocol options for this:

- CXL.cache Requests which is used for HDM-D
- CXL.mem Back-Invalidate Snoop (BISnp) which is used with HDM-DB

Endpoint devices supporting 256B Flit mode must support BISnp mechanism and can optionally use CXL.cache mechanism when connected to a host that has only 68B flit mode. When using CXL.cache, the host detects the address as coming from the device that owns the region which triggers the special flow that returns Mem*Fwd, in most cases, as captured in [Table 3-24](#).

3.2.4.2 Device to Host Requests

3.2.4.2.1 Device to Host (D2H) CXL.cache Request Semantics

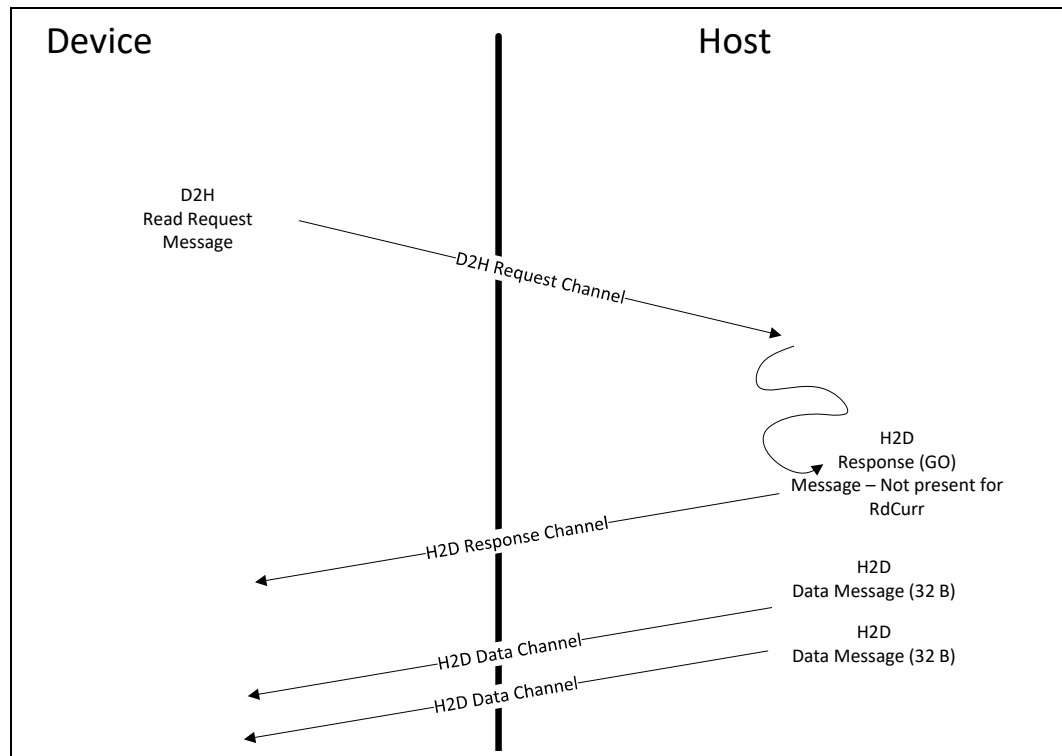
For device to Host requests, there are four different semantics: CXL.cache Read, CXL.cache Read0, CXL.cache Read0/Write, and CXL.cache Write. All device to Host CXL.cache transactions fall into one of these four semantics, though the allowable responses and restrictions for each request type within a given semantic are different.

3.2.4.2.2 CXL.cache Read

CXL.cache Reads must have a D2H request credit and send a request message on the D2H CXL.cache request channel. CXL.cache Read requests require zero or one response (GO) message and data messages totaling a single 64-byte cacheline of data. Both the response, if present, and data messages are directed at the device tracker entry provided in the initial D2H request packet's CQID field. The device entry must remain active until all the messages from the Host have been received. To ensure forward progress, the device must have a reserved data buffer able to accept 64 bytes of data immediately after the request is sent. However, the device may temporarily be unable to accept data from the Host due to prior data returns not draining. Once both the response message and the data messages have been received from the Host, the transaction can be considered complete and the entry deallocated from the device.

[Figure 3-11](#) shows the elements required to complete a CXL.cache Read. Note that the response (GO) message can be received before, after, or between the data messages.

Figure 3-11. CXL.cache Read Behavior

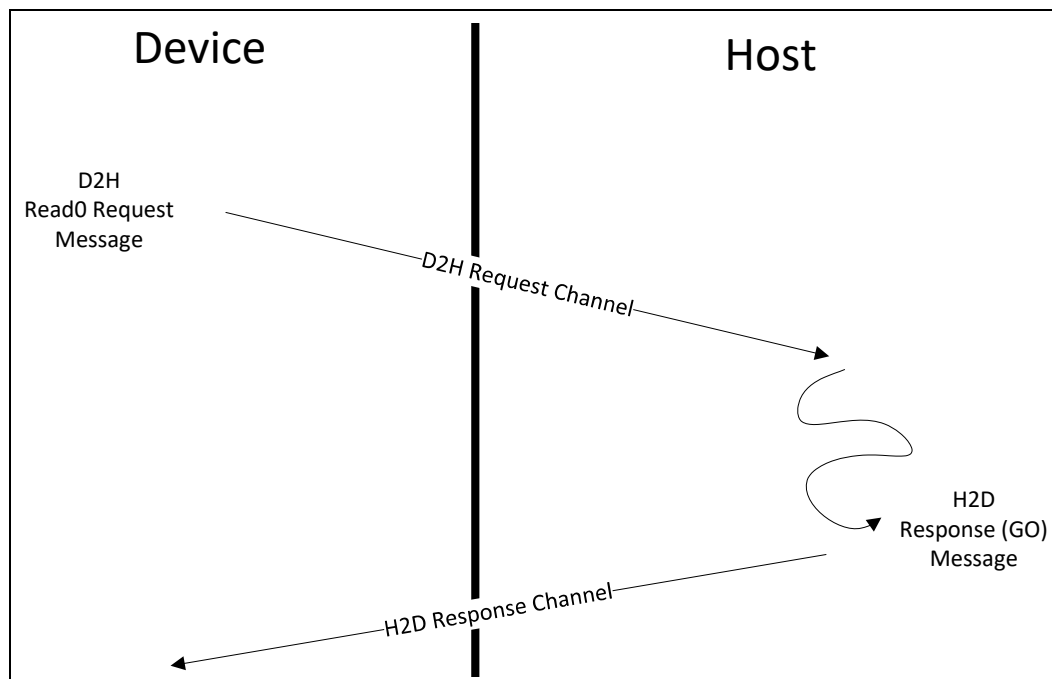


3.2.4.2.3 CXL.cache Read0

CXL.cache Read0 must have a D2H request credit and send a message on the D2H CXL.cache request channel. CXL.cache Read0 requests receive a response message but no data messages. The response message is directed at the device entry indicated in the initial D2H request message's CQID value. Once the GO message is received for these requests, they can be considered complete and the entry deallocated from the device. A data message must not be sent by the Host for these transactions. Most special cycles (e.g., CLFlush) and other miscellaneous requests fall into this category. See Table 3-22 for details.

Figure 3-12 shows the elements required to complete a CXL.cache Read0 transaction.

Figure 3-12. CXL.cache Read0 Behavior



3.2.4.2.4 CXL.cache Write

CXL.cache Write must have a D2H request credit before sending a request message on the D2H CXL.cache request channel. Once the Host has received the request message, it is required to send a GO message and a WritePull message. The WritePull message is not required for CleanEvictNoData. The GO and the WritePull can be a combined message for some requests. The GO message must never arrive at the device before the WritePull, but it can arrive at the same time in the combined message. If the transaction requires posted semantics, then a combined GO-I/WritePull message can be used. If the transaction requires non-posted semantics, then WritePull is issued first followed by the GO-I when the non-posted write is globally observed.

Upon receiving the GO-I message, the device will consider the store done from a memory ordering and cache coherency perspective, giving up snoop ownership of the cacheline (if the CXL.cache message is an Evict).

The WritePull message triggers the device to send data messages to the Host totaling exactly 64 bytes of data, though any number of byte enables can be set.

A CXL.cache write transaction is considered complete by the device once the device has received the GO-I message, and has sent the required data messages. At this point the entry can be deallocated from the device.

The Host considers a write to be done once it has received all 64 bytes of data, and has sent the GO-I response message. All device writes and Evicts fall into the CXL.cache Write semantic.

See [Section 3.2.5.8](#) for more information on restrictions around multiple active write transactions.

Figure 3-13 shows the elements required to complete a CXL.cache Write transaction (that matches posted behavior). The WritePull (or the combined GO_WritePull) message triggers the data messages. There are restrictions on Snoops and WritePulls. See Section 3.2.5.3 for more details.

Figure 3-14 shows a case where the WritePull is a separate message from the GO (for example: strongly ordered uncacheable write).

Figure 3-15 shows the Host FastGO plus ExtCmp responses for weakly ordered write requests.

Figure 3-13. CXL.cache Device to Host Write Behavior

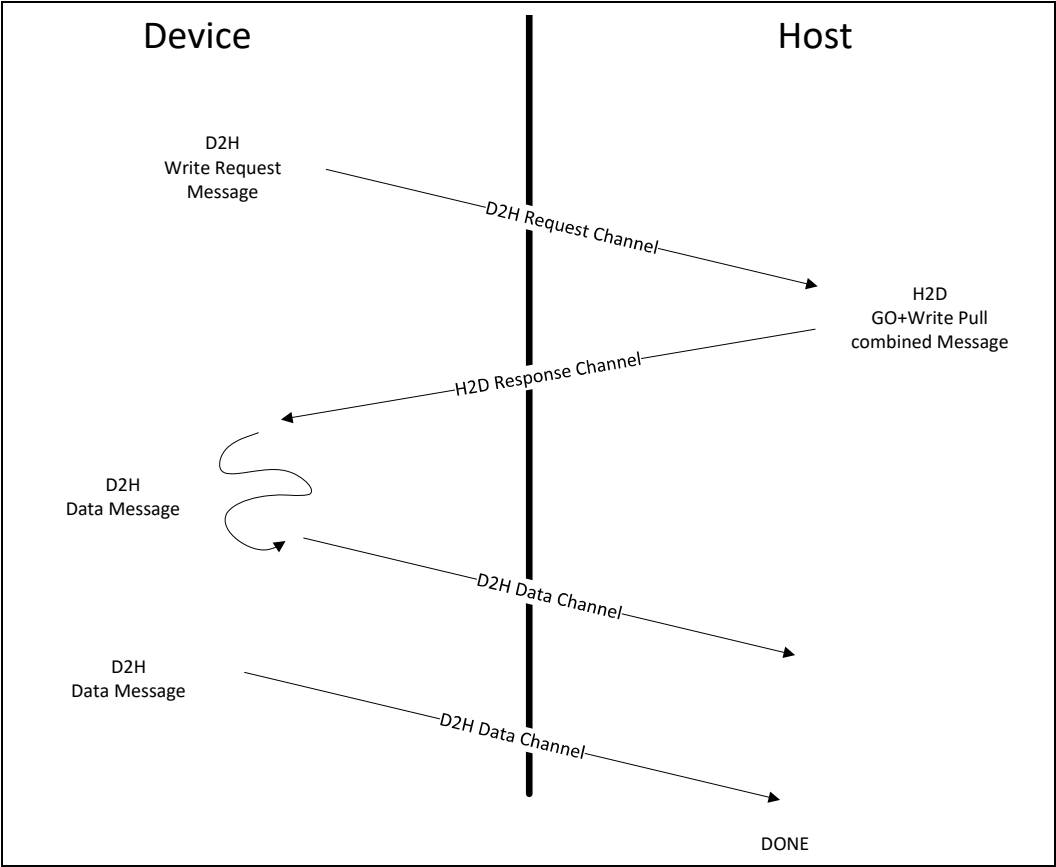


Figure 3-14. CXL.cache WrInv Transaction

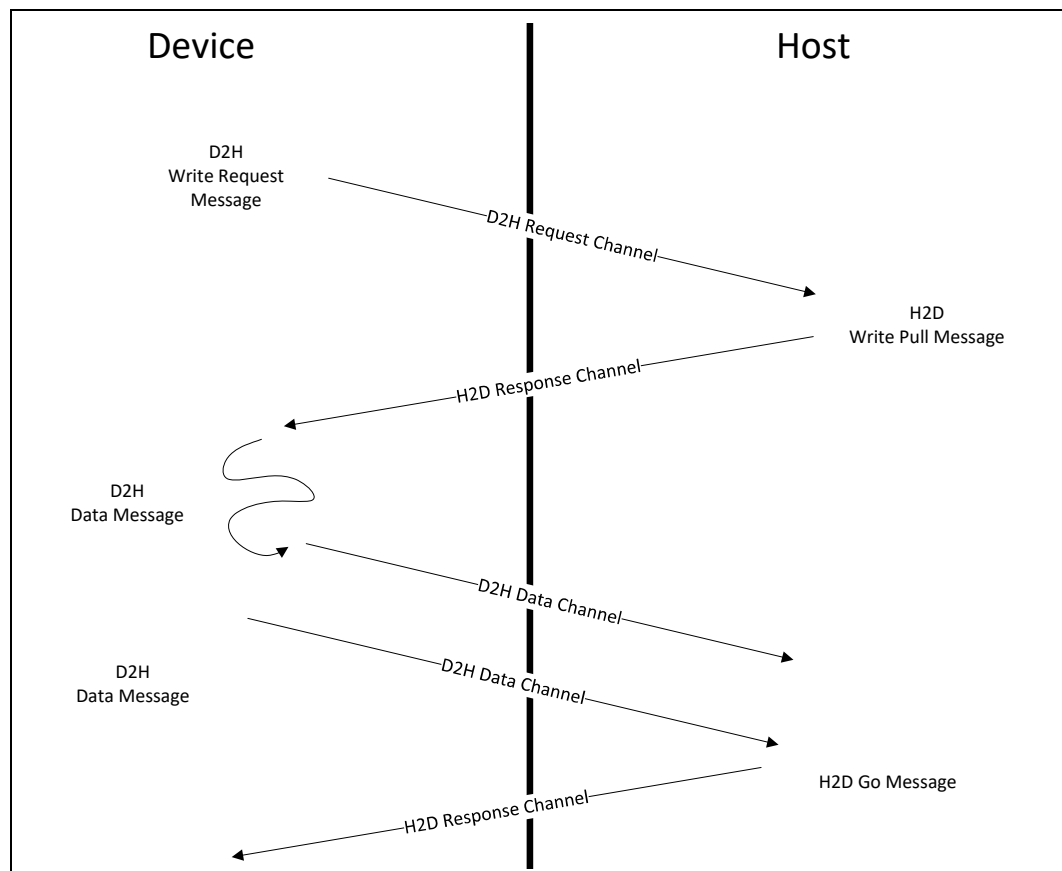
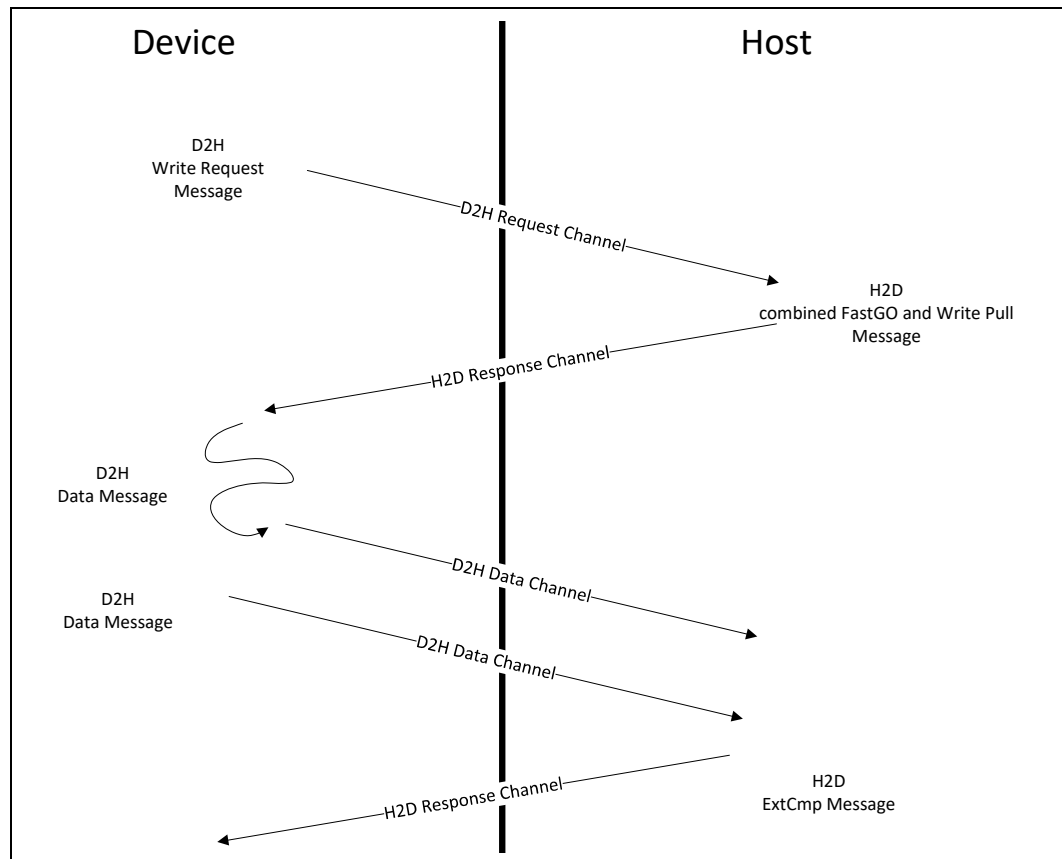


Figure 3-15. WOWrInv/F with FastGO/ExtCmp



3.2.4.2.5 CXL.cache Read0-Write Semantics

CXL.cache Read0-Write requests must have a D2H request credit before sending a request message on the D2H CXL.cache request channel. Once the Host has received the request message, it is required to send one merged GO-I and WritePull message.

The WritePull message triggers the device to send the data messages to the Host, which together transfer exactly 64 bytes of data though any number of byte enables can be set.

A CXL.cache Read0-Write transaction is considered complete by the device once the device has received the GO-I message, and has sent the all required data messages. At this point the entry can be deallocated from the device.

The Host considers a Read0-Write to be done once it has received all 64 bytes of data, and has sent the GO-I response message. ItoMWr falls into the Read0-Write category.

Figure 3-16. CXL.cache Read0-Write Semantics

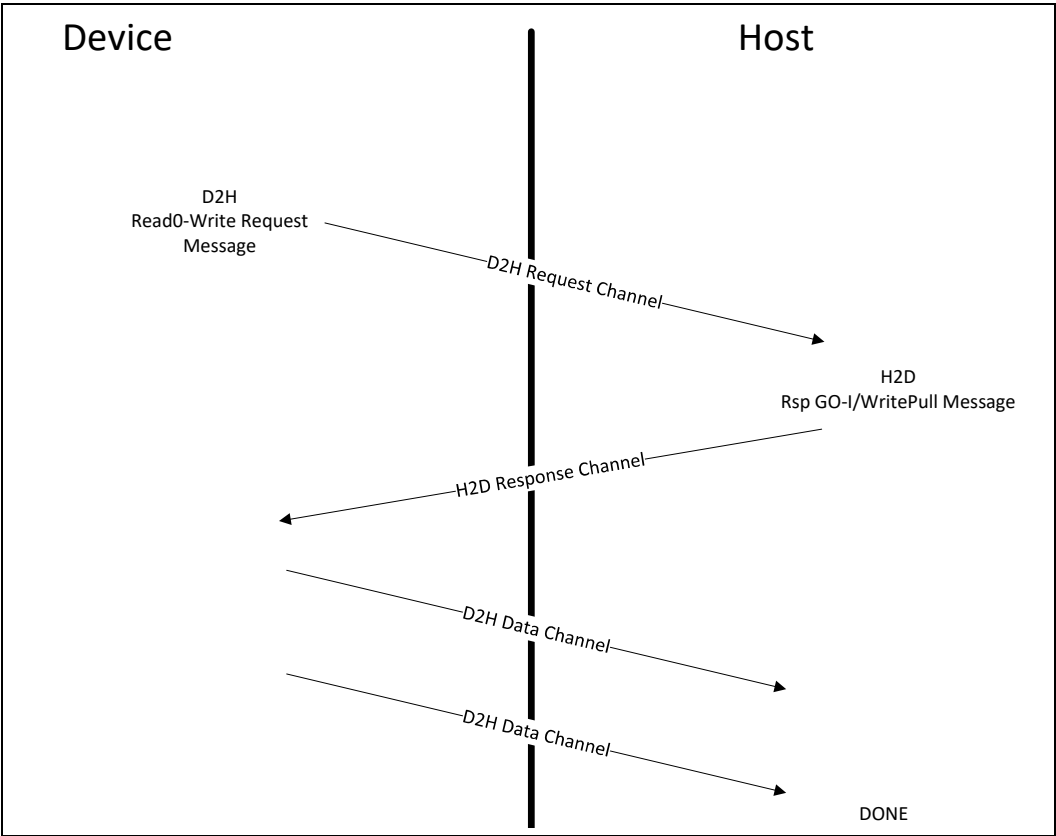


Table 3-22 summarizes all the opcodes that are available from the Device to the Host.

Table 3-22. CXL.cache – Device to Host Requests

CXL.cache Opcode	Semantic	Opcode
RdCurr	Read	0 0001b
RdOwn	Read	0 0010b
RdShared	Read	0 0011b
RdAny	Read	0 0100b
RdOwnNoData	Read0	0 0101b
ItoMWr	Read0-Write	0 0110b
WrCur	Read0-Write	0 0111b
CLFlush	Read0	0 1000b
CleanEvict	Write	0 1001b
DirtyEvict	Write	0 1010b
CleanEvictNoData	Write	0 1011b
WOWrInv	Write	0 1100b
WOWrInvF	Write	0 1101b
WrInv	Write	0 1110b
CacheFlushed	Read0	1 0000b

3.2.4.2.6 RdCurr

These are full cacheline read requests from the device for lines to get the most current data, but not change the existing state in any cache, including in the Host. The Host does not need to track the cacheline in the device that issued the RdCurr. RdCurr gets a data but no GO. The device receives the line in the Invalid state which means that the device gets one use of the line and cannot cache it.

3.2.4.2.7 RdOwn

These are full cacheline read requests from the device for lines to be cached in any writeable state. Typically, RdOwn request receives the line in Exclusive (GO-E) or Modified (GO-M) state. Lines in Modified state must not be dropped, and have to be written back to the Host.

Under error conditions, a RdOwn request may receive the line in Invalid (GO-I) or Error (GO-Err) state. Both return synthesized data of all 1s. The device is responsible for handling the error appropriately.

3.2.4.2.8 RdShared

These are full cacheline read requests from the device for lines to be cached in Shared state. Typically, RdShared request receives the line in Shared (GO-S) state.

Under error conditions, a RdShared request may receive the line in Invalid (GO-I) or Error (GO-Err) state. Both will return synthesized data of all 1s. The device is responsible for handling the error appropriately.

3.2.4.2.9 RdAny

These are full cacheline read requests from the device for lines to be cached in any state. Typically, RdAny request receives the line in Shared (GO-S), Exclusive (GO-E) or Modified (GO-M) state. Lines in Modified state must not be dropped, and have to be written back to the Host.

Under error conditions, a RdAny request may receive the line in Invalid (GO-I) or Error (GO-Err) state. Both return synthesized data of all 1s. The device is responsible for handling the error appropriately.

3.2.4.2.10 RdOwnNoData

These are requests to get exclusive ownership of the cacheline address indicated in the address field. The typical response is Exclusive (GO-E).

Under error conditions, a RdOwnNoData request may receive the line in Error (GO-Err) state. The device is responsible for handling the error appropriately.

Note:

A device that uses this command to write data must be able to update the entire cacheline or may drop the E-state if it is unable to perform the update. There is no support partial M-state data in a device cache. To perform a partial write in the device cache, the device must read the cacheline using RdOwn before merging with the partial write data in the cache.

3.2.4.2.11 ItoMWr

This command requests exclusive ownership of the cacheline address indicated in the address field and atomically writes the cacheline back to the Host. The device guarantees the entire line will be modified, so no data needs to be transferred to the

device. The typical response is GO_WritePull, which is sent once the request is granted ownership. The device must not retain a copy of the line. If a cache exists in the host cache hierarchy before memory, the data should be written there.

If an error occurs, then GO-Err-WritePull is sent instead. The device sends the data to the Host, which drops it. The device is responsible for handling the error as appropriate.

3.2.4.2.12 WrCur

The command behaves like the ItoMWr in that it atomically requests ownership of a cacheline and then writes a full cacheline back to the Fabric. However, it differs from ItoMWr in where the data is written. Only if the command hits in a cache will the data be written there; on a Miss, the data will be written directly to memory. The typical response is GO_WritePull once the request is granted ownership. The device must not retain a copy of the line.

If an error occurs, then GO-Err-WritePull is sent instead. The device sends the data to the Host, which drops it. The device is responsible for handling the error as appropriate.

Note: In earlier revisions of the specification (CXL 2.0 and CXL 1.x), this command was called “MemWr”, but this was a problem because that same message name is used in the CXL.mem protocol, so a new name was selected. The opcode and behavior are unchanged.

3.2.4.2.13 CLFlush

This is a request to the Host to invalidate the cacheline specified in the address field. The typical response is GO-I which is sent from the Host upon completion in memory.

However, the Host may keep tracking the cacheline in Shared state if the Core has issued a Monitor to an address belonging in the cacheline. Thus, the Device that exposes an HDM-D region must not rely on CLFlush/GO-I as a sufficient condition for which to flip a cacheline in the HDM-D region from Host to Device Bias mode. Instead, the Device must initiate RdOwnNoData and receive an H2D Response of GO-E before it updates its Bias Table to Device Bias mode to allow subsequent cacheline access without notifying the Host.

Under error conditions, a CLFlush request may receive the line in the Error (GO-Err) state. The device is responsible for handling the error appropriately.

3.2.4.2.14 CleanEvict

This is a request to the Host to evict a full 64-byte Exclusive cacheline from the device. Typically, CleanEvict receives GO-WritePull or GO-WritePullDrop. The response will cause the device to relinquish snoop ownership of the line. For GO-WritePull, the device will send the data as normal. For GO-WritePullDrop, the device simply drops the data.

Once the device has issued this command and the address is subsequently snooped, but before the device has received the GO-WritePull, the device must set the Bogus field in all D2H Data messages to indicate that the data is now stale.

CleanEvict requests also guarantee to the Host that the device no longer contains any cached copies of this line. Only one CleanEvict from the device may be pending on CXL.cache for any given cacheline address.

CleanEvict is only expected for a host-attached memory range of addresses. For a device-attached memory range, the equivalent operation can be completed internally within the device without sending a transaction to the Host.

3.2.4.2.15 DirtyEvict

This is a request to the Host to evict a full 64-byte Modified cacheline from the device. Typically, DirtyEvict receives GO-WritePull from the Host at which point the device must relinquish snoop ownership of the line and send the data as normal.

Once the device has issued this command and the address is subsequently snooped, but before the device has received the GO-WritePull, the device must set the Bogus field in all D2H Data messages to indicate that the data is now stale.

DirtyEvict requests also guarantee to the Host that the device no longer contains any cached copies of this line. Only one DirtyEvict from the device may be pending on CXL.cache for any given cacheline address.

In error conditions, a GO-Err-WritePull is received. The device sends the data as normal, and the Host drops it. The device is responsible for handling the error as appropriate.

DirtyEvict is only expected for host-attached memory address ranges. For device-attached memory range, the equivalent operation can be completed internally within the device without sending a transaction to the Host.

3.2.4.2.16 CleanEvictNoData

This is a request for the device to update the Host that a clean line is dropped in the device. The sole purpose of this request is to update any snoop filters in the Host and no data is exchanged.

CleanEvictNoData is only expected for host-attached memory address ranges. For device-attached memory range, the equivalent operation can be completed internally within the device without sending a transaction to the Host.

3.2.4.2.17 WOWrInv

This is a weakly ordered write invalidate line request of 0-63 bytes for write combining type stores. Any combination of byte enables may be set.

Typically, WOWrInv receives a FastGO-WritePull followed by an ExtCmp. Upon receiving the FastGO-WritePull the device sends the data to the Host. For host-attached memory, the Host sends the ExtCmp once the write is complete in memory.

FastGO does not provide "Global Observation".

In error conditions, a GO-Err-WritePull is received. The device sends the data as normal, and the Host drops it. The device is responsible for handling the error as appropriate. An ExtCmp is still sent by the Host after the GO-Err in all cases.

3.2.4.2.18 WOWrInvF

Same as WOWrInv (rules and flows), except it is a write of 64 bytes.

3.2.4.2.19 WrInv

This is a write invalidate line request of 0-64 bytes. Typically, WrInv receives a WritePull followed by a GO. Upon getting the WritePull, the device sends the data to the Host. The Host sends GO once the write completes in memory (both, host-attached or device-attached).

In error conditions, a GO-Err is received. The device is responsible for handling the error as appropriate.

3.2.4.2.20 CacheFlushed

This is an indication sent by the device to inform the Host that its caches are flushed, and it no longer contains any cachelines in the Shared, Exclusive or Modified state (a device may exclude addresses that are part its “Device-attached Memory” mapped as HDM-D/HDM-DB). The Host can use this information to clear its snoop filters, block snoops to the device, and return a GO. Once the device receives the GO, it is guaranteed to not receive any snoops from the Host until the device sends the next cacheable D2H Request.

When a CXL.cache device is flushing its cache, the device must wait for all responses for cacheable access before sending the CacheFlushed message. This is necessary because the Host must observe CacheFlushed only after all inflight messages that impact device coherence tracking in the Host are complete.

IMPLEMENTATION NOTE

Snoops may be pending to the device when the Host receives the CacheFlushed command and the Host may complete the CacheFlushed command (sending a GO) while those snoops are outstanding. From the device point of view, this can be observed as receiving snoops after the CacheFlushed message is complete. The device should allow for this behavior without creating long stall conditions on the snoops by waiting for snoop queues to drain before initiating any power state transition (e.g., L1 link state) that could stall snoops.

Table 3-23. D2H Request (Targeting Non Device-attached Memory) Supported H2D Responses

D2H Request	H2D Response										
	WritePull	GO_WritePull	ExtCmp	GO_WritePull_Drop	Fast_GO_WritePull	GO_ERR_WritePull	GO_Err	GO-I	GO-S	GO-E	GO-M
RdCurr											
RdOwn							X	X		X	X
RdShared							X	X	X		
RdAny							X	X	X	X	X
RdOwnNoData							X			X	
ItoMWr		X				X					
WrCur		X				X					
CLFlush							X	X			
CleanEvict		X		X							
DirtyEvict		X				X					
CleanEvictNoData								X			
WOWrInv			X		X	X					
WOWrInvF			X		X	X					
WrInv	X						X	X			
CacheFlushed								X			

For requests that target device-attached memory mapped as HDM-D, if the region is in Device Bias, no transaction is expected on CXL.cache because the Device can internally complete those requests. If the region is in Host Bias, Table 3-24 shows how the device should expect the response. For devices with BISnp channel support in which the memory is mapped as HDM-DB, the resolution of coherence happens separately on the CXL.mem protocol and the “Not Supported” cases in the table are never sent from a device to the device-attached memory address range. The only commands supported on CXL.cache to this address region when BISnp is enabled are ItoMWr, WrCur, and WrInv.

Table 3-24. D2H Request (Targeting Device-attached Memory) Supported Responses

D2H Request	Response on CXL.cache		Response on CXL.mem	
	Without BISnp (HDM-D)	With BISnp (HDM-DB)	Without BISnp (HDM-D)	With BISnp (HDM-DB)
RdCurr	GO-Err Bit set in H2D DH, Synthesized Data with all 1s (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
RdOwn	GO-Err on H2D Response, Synthesized Data with all 1s (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
RdShared	GO-Err on H2D Response, Synthesized Data with all 1s (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
RdAny	GO-Err on H2D Response, Synthesized Data with all 1s (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
RdOwnNoData	GO-Err on H2D Response (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
ItoMWr	Same as host-attached memory ¹		None	
WrCur	Same as host-attached memory ¹		None	
CLFlush	GO-Err on H2D Response (For Error Conditions)	Not Supported	MemRdFwd (For Success Conditions)	Not Supported
CleanEvict	Not Supported			
DirtyEvict	Not Supported			
CleanEvictNoData	Not Supported			
WOWrInv	GO_ERR_WritePull on H2D Response (For Error Conditions)	Not Supported	MemWrFwd (For Success Conditions)	Not Supported
WOWrInvF	GO_ERR_WritePull on H2D Response (For Error Conditions)	Not Supported	MemWrFwd (For Success Conditions)	Not Supported
WrInv	Same as host-attached memory ¹		None	
CacheFlushed	N/A ²		None	

- Flow for these commands follow the same flow as host memory regions and are not expected to check against CXL.mem coherence tracking (Bias Table or Snoop Filter) before issuing. The host will resolve coherence with the device using the CXL.mem protocol.
- There is no address in this command and the host must assume that this applies only to host memory regions (excluding device-attached memory).

CleanEvict, DirtyEvict, and CleanEvictNoData targeting device-attached memory should always be completed internally by the device, regardless of bias state. For D2H Requests that receive a response on CXL.mem, the CQID associated with the CXL.cache request is reflected in the Tag of the CXL.mem MemRdFwd or MemWrFwd command. For MemRdFwd, the caching state of the line is reflected in the MetaValue field as described in Table 3-37.

3.2.4.3 Device to Host Response

Responses are directed at the Host entry indicated in the UQID field in the original H2D request message.

Table 3-25. D2H Response Encodings

Device CXL.cache Rsp	Opcode
RspIHitI	0 0100b
RspVHitV	0 0110b
RspIHitSE	0 0101b
RspSHitSE	0 0001b
RspSFwdM	0 0111b
RspIFwdM	0 1111b
RspVFwdV	1 0110b

3.2.4.3.1 RspIHitI

In general, this is the response that a device provides to a snoop when the line was not found in any caches. If the device returns RspIHitI for a snoop, the Host can assume the line has been cleared from that device.

3.2.4.3.2 RspVHitV

In general, this is the response that a device provides to a snoop when the line was hit in the cache and no state change occurred. If the device returns an RspVHitV for a snoop, the Host can assume a copy of the line is present in one or more places in that device.

3.2.4.3.3 RspIHitSE

In general, this is the response that a device provides to a snoop when the line was hit in a clean state in at least one cache and is now invalid. If the device returns an RspIHitSE for a snoop, the Host can assume the line has been cleared from that device.

3.2.4.3.4 RspSHitSE

In general, this is the response that a device provides to a snoop when the line was hit in a clean state in at least one cache and is now downgraded to shared state. If the device returns an RspSHitSE for a snoop, the Host should assume the line is still in the device.

3.2.4.3.5 RspSFwdM

This response indicates to the Host that the line being snooped is now in S state in the device, after having hit the line in Modified state. The device may choose to downgrade the line to Invalid. This response also indicates to the Host snoop tracking logic that 64 bytes of data is transferred on the D2H CXL.cache Data Channel to the Host data buffer indicated in the original snoop's destination (UQID).

3.2.4.3.6 RspIFwdM

This response indicates to the Host that the line being snooped is now in I state in the device, after having hit the line in Modified state. The Host may now assume the device contains no more cached copies of this line. This response also indicates to the Host

snoop tracking logic that 64 bytes of data will be transferred on the D2H CXL.cache Data Channel to the Host data buffer indicated in the original snoop's destination (UQID).

3.2.4.3.7 RspVFwdV

This response indicates that the device with E or M state (but not S state) is returning the current data to the Host and leaving the state unchanged. The Host must only forward the data to the requester because there is no state information.

3.2.4.4 Host to Device Requests

Snoops from the Host need not gain any credits besides local H2D request credits. The device will always send a Snoop Response message on the D2H CXL.cache Response channel. If the response is of the Rsp*Fwd* format, then the device must respond with 64 bytes of data via the D2H Data channel, directed at the UQID from the original snoop request message. If the response is not Rsp*Fwd*, the Host can consider the request complete upon receiving the snoop response message. The device can stop tracking the snoop once the response has been sent for non-data forwarding cases, or after both the last chunk of data has been sent and the response has been sent.

Figure 3-17 shows the elements required to complete a CXL.cache snoop. Note that the response message can be received by the Host in any relative order with respect to the data messages. The byte enable field is always all 1s for Snoop data transfers.

Figure 3-17. CXL.cache Snoop Behavior

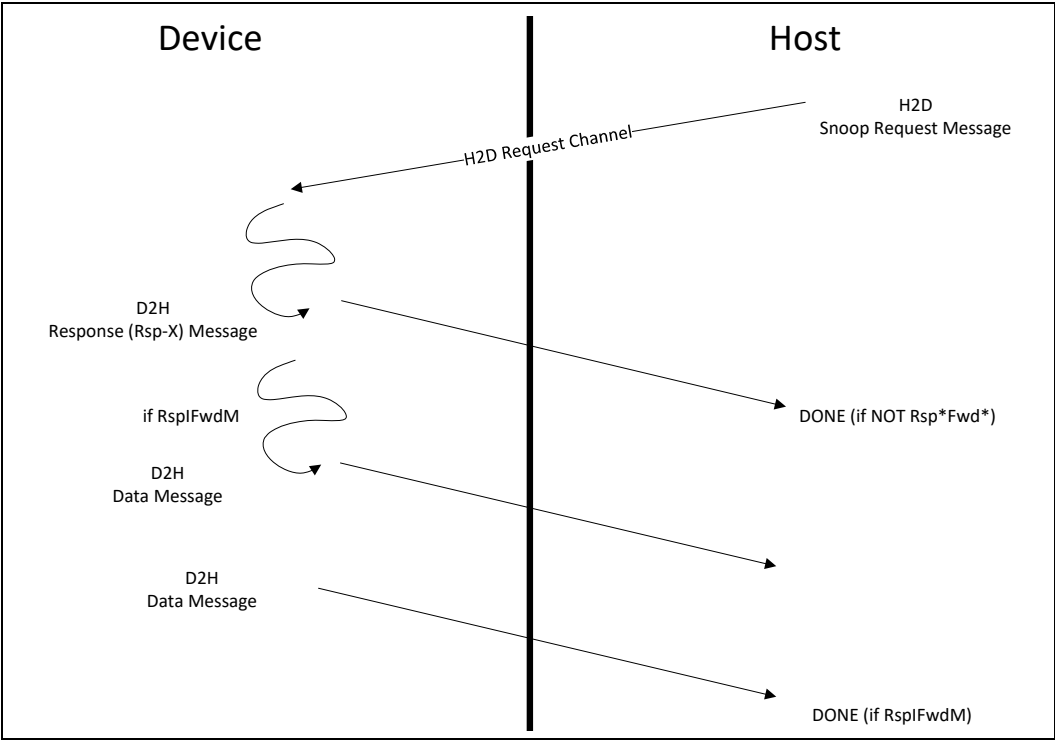


Table 3-26. CXL.cache – Mapping of H2D Requests to D2H Responses

	Opcode	RspIHitI	RspVhitV	RspSHitSE	RspIHitSE	RspSFwdM	RspIFwdM	RspVFwdV
SnpData	001b	X		X		X	X	
SnpInv	010b	X			X		X	
SnpCur	011b	X	X	X		X	X	X

3.2.4.4.1 SnpData

These are snoop requests from the Host for lines that are intended to be cached in either Shared or Exclusive state at the requester (the Exclusive state can be cached at the requester only if all devices respond with RspI). This type of snoop is typically triggered by data read requests. A device that receives this snoop must either invalidate or downgrade all cachelines to Shared state. If the device holds dirty data it must return it to the Host.

3.2.4.4.2 SnpInv

These are snoop requests from the Host for lines that are intended to be granted ownership and Exclusive state at the requester. This type of snoop is typically triggered by write requests. A device that receives this snoop must invalidate all cachelines. If the device holds dirty data it must return it to the Host.

3.2.4.4.3 SnpCur

This snoop gets the current version of the line, but doesn't require change of any cache state in the hierarchy. It is only sent on behalf of the RdCurr request. If the device holds data in Modified state it must return it to the Host. The cache state can remain unchanged in both the device and Host, and the Host should not update its caches. To allow for varied cache implementations, devices are allowed to change cache state as captured in [Table 3-26](#), but it is recommended to not change cache state.

3.2.4.5 Host to Device Response**Table 3-27. H2D Response Opcode Encodings**

H2D Response Class	Encoding	RspData
WritePull	0001b	UQID
GO	0100b	MESI ¹
GO_WritePull	0101b	UQID
ExtCmp	0110b	Don't Care
GO_WritePull_Drop	1000b	UQID
Reserved	1100b	Don't Care
Fast_GO_WritePull	1101b	UQID
GO_ERR_WritePull	1111b	UQID

1. 4-bit MESI encoding is in LSB and the upper bits are Reserved.

3.2.4.5.1 WritePull

This response instructs the device to send the write data to the Host, but not to change the state of the line. This is used for WrInv where the data is needed before the GO-I can be sent. This is because GO-I is the notification that the write was completed.

3.2.4.5.2 GO

The Global Observation (GO) message conveys that read requests are coherent and that write requests are coherent and consistent. It is an indication that the transaction has been observed by the system device and the MESI state that is encoded in the RspType field indicates into which state the data associated with the transaction should be placed for the requester's caches. Details in [Table 3-20](#).

If the Host returns Modified state to the device, then the device is responsible for the dirty data and cannot drop the line without writing it back to the Host.

If the Host returns Invalid or Error state to the device, then the device must use the data at most once and not cache the data. Error responses to reads and cacheable write requests (for example, RdOwn or ItoMWr) will always be the result of an abort condition, so modified data can be safely dropped in the device.

3.2.4.5.3 GO_WritePull

This is a combined GO + WritePull message. No cache state is transferred to the device. The GO+WritePull message is used for write types that do not require a later message to know whether write data is visible.

3.2.4.5.4 ExtCmp

This response indicates that the data that was previously locally ordered (FastGO) has been observed throughout the system. Most importantly, accesses to memory will return the most up-to-date data.

3.2.4.5.5 GO_WritePull_Drop

This message has the same semantics as GO_WritePull, except that the device should not send data to the Host. This response can be sent in place of GO_WritePull when the Host determines that the data is not required. This response will never be sent for partial writes because the byte enables will always need to be transferred.

3.2.4.5.6 Fast_GO_WritePull

Similar to GO_WritePull, but only indicates that the request is locally observed. There will be a later ExtCmp message when the transaction is fully observable in memory. Devices that do not implement the Fast_GO feature may ignore the GO message and wait for the ExtCMP. Data must always be sent for the WritePull. No cache state is transferred to the device.

Locally Observed, in this context, is a host-specific coherence domain that may be a subset of the global coherence domain. An example is a Last Level Cache that the requesting device shares with other CXL.cache devices that are connected below a host-bridge. In that example, local observation is only within the Last Level Cache and not between other Last Level Caches.

3.2.4.5.7 GO_ERR_WritePull

Similar to GO_WritePull, but indicates that there was an error with the transaction that should be handled correctly in the device. Data must be sent to the Host for the WritePull, and the Host will drop the data. No cache state is transferred to the device (assumed Error). An ExtCmp is still sent if it is expected by the originating request.

3.2.5 Cacheability Details and Request Restrictions

These details and restrictions apply to all devices.

3.2.5.1 GO-M Responses

GO-M responses from the host indicate that the device is being granted the sole copy of modified data. The device must cache this data and write it back when it is done.

3.2.5.2 Device/Host Snoop-GO-Data Assumptions

When the host returns a GO response to a device, the expectation is that a snoop arriving to the same address of the request receiving the GO would see the results of that GO. For example, if the host sends GO-E for an RdOwn request followed by a snoop to the same address immediately afterwards, then one would expect the device to transition the line to M state and reply with an RspIFwdM response back to the Host. To implement this principle, the CXL.cache link layer ensures that the device will receive the two messages in separate slots to make the order completely unambiguous.

When the host is sending a snoop to the device, the requirement is that no GO response will be sent to any requests with that address in the device until after the Host has received a response for the snoop and all implicit writeback (IWB) data (dirty data forwarded in response to a snoop) has been received.

When the host returns data to the device for a read type request, and GO for that request has not yet been sent to the device, the host may not send a snoop to that address until after the GO message has been sent. Because the new cache state is encoded in the response message for reads, sending a snoop to an address without having received GO, but after having received data, is ambiguous to the device as to what the snoop response should be in that situation.

Fundamentally, the GO that is associated with a read request also applies to the data returned with that request. Sending data for a read request implies that data is valid, meaning the device can consume it even if the GO has not yet arrived. The GO will arrive later and inform the device what state to cache the line in (if at all) and whether the data was the result of an error condition (e.g., hitting an address region that the device was not allowed to access).

3.2.5.3 Device/Host Snoop/WritePull Assumptions

The device requires that the host cannot have both a WritePull and H2D Snoop active on CXL.cache to a given 64-byte address. The host may not launch a snoop to a 64-byte address until all WritePull data from that address has been received by the host. Conversely, the host may not launch a WritePull for a write until the host has received the snoop response (including data in case of Rsp*Fwd*) for any snoops to the pending writes address. Any violation of these requirements will mean that the Bogus field on the D2H Data channel will be unreliable.

3.2.5.4 Snoop Responses and Data Transfer on CXL.cache Evicts

To snoop cache evictions (for example, DirtyEvict) and maintain an orderly transfer of snoop ownership from the device to the host, cache evictions on CXL.cache must adhere to the following protocol.

If a device Evict transaction has been issued on the CXL.cache D2H request channel, but has not yet processed its WritePull from the host, and a snoop hits the writeback, the device must track this snoop hit if cache state is changed, which excludes the case when SnpCur results in a RspVFwdV response. When the device begins to process the WritePull, if snoop hit is tracked the device must set the Bogus field in all the D2H data messages sent to the host. The intent is to communicate to the host that the request data was already sent as IWB data, so the data from the Evict is potentially stale.

3.2.5.5 Multiple Snoops to the Same Address

The host is only allowed to have one snoop pending at a time per cacheline address per device. The host must wait until it has received both the snoop response and all IWB data (if any) before sending the next snoop to that address.

3.2.5.6 Multiple Reads to the Same Cacheline

Multiple read requests (cacheable or uncacheable) to the same cacheline are allowed only in the following specific cases where host tracking state is consistent regardless of the order requests are processed. The host can freely reorder requests, so the device is responsible for ordering requests when required. For host memory, multiple RdCurr and/or CLFlush are allowed. For these commands the device ends in I-state, so there is no inconsistent state possible for host tracking of a device cache. With Type 2 devices that use HDM-D memory, in addition to RdCurr and/or CLFlush, multiple RdOwnNoData (bias flip requests) are allowed for device-attached memory. This case is allowed because with device-attached memory, the host does not track the device's cache so re-ordering in the host will not create an ambiguous state between the device and the host.

3.2.5.7 Multiple Evicts to the Same Cacheline

Multiple Evicts to the same cacheline are not allowed. All Evict messages from the device provide a guarantee to the host that the evicted cacheline will no longer be present in the device's caches.

Thus, it is a coherence violation to send another Evict for the same cacheline without an intervening cacheable Read/Read0 request to that address.

3.2.5.8 Multiple Write Requests to the Same Cacheline

Multiple WrInv/WOWrInv/ItoMWr/WrCur to the same cacheline are allowed to be outstanding on CXL.cache. The host or switch can freely reorder requests, and the device may receive corresponding H2D Responses in reordered manner. However, it is generally recommended that the device should issue no more than one outstanding Write request for a given cacheline, and order multiple write requests to the same cacheline one after another whenever stringent ordering is warranted.

3.2.5.9 Multiple Read and Write Requests to the Same Cacheline

Multiple RdCur/CLFlush/WrInv/WOWrInv/ItoMWr/WrCur may be issued in parallel from devices to the same cacheline address. Other reads need to issue one at a time (also known as "serialize"). To serialize, the read must not be issued until all other

outstanding accesses to same cacheline address have received GO. Additionally, after the serializing read is issued, no other accesses to the same cacheline address may be issued until it has received GO.

3.2.5.10 Normal Global Observation (GO)

Normal Global Observation (GO) responses are sent only after the host has guaranteed that request will have next ownership of the requested cacheline. GO messages for requests carry the cacheline state permitted through the MESI state or indicate that the data should only be used once and whether an error occurred.

3.2.5.11 Relaxed Global Observation (FastGO)

FastGO is only allowed for requests that do not require strict ordering. The Host may return the FastGO once the request is guaranteed next ownership of the requested cacheline within an implementation dependent sub-domain (e.g., CPU socket), but not necessarily within the system. Requests that receive a FastGO response and require completion messages are usually of the write combining memory type and the ordering requirement is that there will be a final completion (ExtCmp) message indicating that the request is at the stage where it is fully observed throughout the system. To make use of FastGO, devices have specific knowledge of the FastGO boundary of the CXL hierarchy and know the consumer of the data is within that hierarchy; otherwise, they must wait for the ExtCmp to know the data will be visible.

3.2.5.12 Evict to Device-attached Memory

Device Evicts to device-attached memory are not allowed on CXL.cache. Evictions are expected to go directly to the device's own memory; however, a device may use non-Evict writes (e.g., ItoMWr, WrCur) to write data to the host to device-attached memory.

3.2.5.13 Memory Type on CXL.cache

To source requests on CXL.cache, devices need to get the Host Physical Address (HPA) from the Host by means of an ATS request on CXL.io. Due to memory type restrictions, on the ATS completion, the Host indicates to the device if an HPA can only be issued on CXL.io as described in [Section 3.1.6](#). The device is not allowed to issue requests to such HPAs on CXL.cache. For requests that target ranges within the Device's local HDM range, the HPA is permitted to be obtained by means of an ATS request on CXL.io, or by using device-specific means.

3.2.5.14 General Assumptions

1. The Host will NOT preserve ordering of the CXL.cache requests as delivered by the device. The device must maintain the ordering of requests for the case(s) where ordering matters. For example, if D2H memory writes need to be ordered with respect to an MSI (on CXL.io), it is up to the device to implement the ordering. This is made possible by the non-posted nature of all requests on CXL.cache.
2. The order chosen by the Host will be conveyed differently for reads and writes. For reads, a Global Observation (GO) message conveys next ownership of the addressed cacheline; the data message conveys ordering with respect to other transactions. For writes, the GO message conveys both next ownership of the line and ordering with respect to other transactions.
3. The device may cache ownership and internally order writes to an address if a prior read to that address received either GO-E or GO-M.
4. For reads from the device, the Host transfers ownership of the cacheline with the GO message, even if the data response has not yet been received by the device. The device must respond to a snoop to a cacheline which has received GO, but if

data from the current transaction is required (e.g., a RdOwn to write the line) the data portion of the snoop is delayed until the data response is received.

5. The Host must not send a snoop for an address where it has sent a data response for a previous read transaction but has not yet sent the GO. Ordering will ensure that the device observes the GO in this case before any later snoop. Refer to [Section 3.2.5.2](#) for additional details.
6. Write requests (other than Evicts) such as WrInv, WOWrInv*, ItoMWr, and WrCur will never respond to WritePulls with data marked as Bogus.
7. The Host must not send two cacheline data responses to the same device request. The device may assume one-time use ownership (based on the request) and begin processing for any part of a cacheline received by the device before the GO message. Final state information will arrive with the GO message, at which time the device can either cache the line or drop it depending on the response.
8. For a given transaction, H2D Data transfers must come in consecutive packets in natural order with no interleaved transfers from other lines.
9. D2H Data transfer of a cacheline must come in consecutive packets with no interleaved transfers from other lines. The data must come in natural chunk order, that is, 64B transfers must complete the lower 32B half first because snoops are always cacheline aligned.
10. Device snoop responses in D2H Response must not be dependent on any other channel or on any other requests in the device besides the availability of credits in the D2H Response channel. The Host must guarantee that the responses will eventually be serviced and return credits to the device.
11. The Host must not send a second snoop request to an address until all responses, plus data if required, for the prior snoop are collected.
12. H2D Response and H2D Data messages to the device must drain without the need for any other transaction to make progress.
13. The Host must not return GO-M for data that is not actually modified with respect to memory.
14. The Host must not write unmodified data back to memory.
15. Except for WOWrInv and WOWrInF, all other writes are strongly ordered

3.2.5.15 Buried Cache State Rules

Buried Cache state refers to the state of the cacheline registered in the Device's Coherency engine (DCOH) when a CXL.cache request is being sent for that cacheline from the device.

The Buried Cache state rules for a device when issuing CXL.cache requests are as follows:

- Must not issue a Read if the cacheline is buried in Modified, Exclusive, or Shared state.
- Must not issue RdOwnNoData if the cacheline is buried in Modified or Exclusive state. The Device may request for ownership in Exclusive state as an upgrade request from Shared state.
- Must not issue a Read0-Write if the cacheline is buried in Modified, Exclusive, or Shared state.
- All *Evict opcodes must adhere to apropos use case. For example, the Device is allowed to issue DirtyEvict for a cacheline only when it is buried in Modified state. For performance benefits, it is recommended that the Device should not silently drop a cacheline in Exclusive or Shared state and instead use CleanEvict* opcodes toward the Host.

- The CacheFlushed Opcode is not specific to a cacheline, it is an indication to the Host that all the Device's caches are flushed. Thus, the Device must not issue CacheFlushed if there is any cacheline buried in Modified, Exclusive, or Shared state.

Table 3-28 describes which Opcodes in D2H requests are allowed for a given Buried Cache State.

IMPLEMENTATION NOTE

Buried state rules are requirements at the requester's Transaction Layer. It is possible snoops in flight to change the state observed at the host before the host processes the request. An example case is SnpInv sent from the host at the same time as the device issues a CleanEvictNoData from E-state, the snoop will cause the cache state in the device to change to I-state before the CleanEvictNoData is processed in the host, so the host must allow for this degraded cache state in its coherence tracking.

Table 3-28. Allowed Opcodes for D2H Requests per Buried Cache State

D2H Requests		Buried Cache State			
Opcodes	Semantic	Modified	Exclusive	Shared	Invalid
RdCurr	Read				X
RdOwn	Read				X
RdShared	Read				X
RdAny	Read				X
RdOwnNoData	Read0			X	X
ItoMWr	Read0-Write				X
WrCur	Read0-Write				X
CLFlush	Read0				X
CleanEvict	Write		X		
DirtyEvict	Write	X			
CleanEvictNoData	Write		X	X	
WOWrInv	Write				X
WOWrInvF	Write				X
WrInv	Write				X
CacheFlushed	Read0				X

3.2.5.16 H2D Req Targeting Device-attached Memory

H2D Req messages are sent by a host to a device because the host believes that the device may own a cacheline that the device previously received from this same host. The very principle of a Type 2 Device is to provide direct access to Device-attached Memory (i.e., without going through its host). Host coherence for this region is managed by using the M2S Req channel. These statements combined could lead a Type 2 Device to assume that H2D Req messages can never target addresses that belong to the Device-attached memory by design.

However, a host may decide to snoop more cache peers than strictly required, without any other consideration than the cache peer being visible to the host. This type of behavior is allowed by the CXL protocol and can occur for multiple reasons, including coarse tracking and proprietary RAS features. In that context, a host may generate an

H2D Req to a Type 2 device on addresses that belong to the Device-attached Memory. An H2D Req from the host that targets Device-attached memory can cause coherency issues if the device were to respond with data and, more generally speaking, protocol corner cases.

To avoid these issues, both HDM-D Type 2 devices and HDM-DB Type 2 devices are required to:

- Detect H2D Req that target Device-attached Memory
- When detected, unconditionally respond with RspIHitI, disregarding all internal states and without changing any internal states (e.g., don't touch the cache)

3.3

CXL.mem

3.3.1

Introduction

The CXL Memory Protocol is called CXL.mem, and it is a transactional interface between the CPU and Memory. It uses the phy and link layer of CXL when communicating across dies. The protocol can be used for multiple different Memory attach options including when the Memory Controller is located in the Host CPU, when the Memory Controller is within an Accelerator device, or when the Memory Controller is moved to a memory buffer chip. It applies to different Memory types (e.g., volatile, persistent, etc.) and configurations (e.g., flat, hierarchical, etc.) as well.

The CXL.mem provides 3 basic coherence models for CXL.mem Host-managed Device Memory (HDM) address regions exposed by the CXL.mem protocol:

- HDM-H (Host-only Coherent): Used only for Type 3 Devices
- HDM-D (Device Coherent): Used only for legacy Type 2 Devices that rely on CXL.cache to manage coherence with the Host
- HDM-DB (Device Coherent using Back-Invalidate): Can be used by Type 2 Devices or Type 3 Devices

Note:

The view of the address region must be consistent on the CXL.mem path between the Host and the Device.

The coherency engine in the CPU interfaces with the Memory (Mem) using CXL.mem requests and responses. In this configuration, the CPU coherency engine is regarded as the CXL.mem Master and the Mem device is regarded as the CXL.mem Subordinate. The CXL.mem Master is the agent which is responsible for sourcing CXL.mem requests (e.g., reads, writes, etc.) and a CXL.mem Subordinate is the agent which is responsible for responding to CXL.mem requests (e.g., data, completions, etc.).

When the Subordinate maps HDM-D/HDM-DB, CXL.mem protocol assumes the presence of a device coherency engine (DCOH). This agent is assumed to be responsible for implementing coherency related functions such as snooping of device caches based on CXL.mem commands and update of Metadata fields.

Support for memory with Metadata is optional but this needs to be negotiated with the Host in advance. The negotiation mechanisms are beyond the scope of this specification. If Metadata is not supported by device-attached memory, the DCOH will still need to use the Host supplied Metadata updates to interpret the commands. If Metadata is supported by device-attached memory, it can be used by Host to implement a coarse snoop filter for CPU sockets. In the HDM-H address region, the usage is defined by the Host. The protocol allows for 2 bits of Metadata to be stored and returned.

CXL.mem transactions from Master to Subordinate are called "M2S" and transactions from Subordinate to Master are called "S2M".

Within M2S transactions, there are three message classes:

- Request without data - generically called Requests (Req)
- Request with Data - (RwD)
- Back-Invalidate Response - (BIRsp)

Similarly, within S2M transactions, there are three message classes:

- Response without data - generically called No Data Response (NDR)
- Response with data - generically called Data Response (DRS)
- Back-Invalidate Snoop - (BISnp)

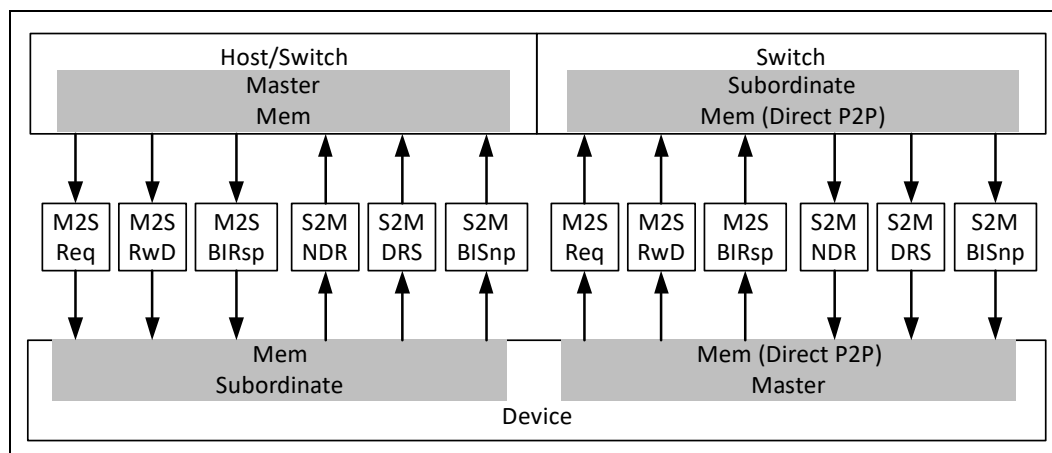
The next sections describe the above message classes and opcodes in detail. Each message in will support 3 variants: 68B Flit, 256B Flit, and PBR Flit. The use of each of these will be negotiated in the physical layer for each link as defined in [Chapter 6.0](#).

3.3.2 CXL.mem Channel Description

In general, the CXL.mem channels work independently of one another to ensure that forward progress is maintained. Details of the specific ordering allowances and requirements between channels are captured in [Section 3.4](#). Within a channel there are no ordering rules, but exceptions to this are described in [Section 3.3.11](#).

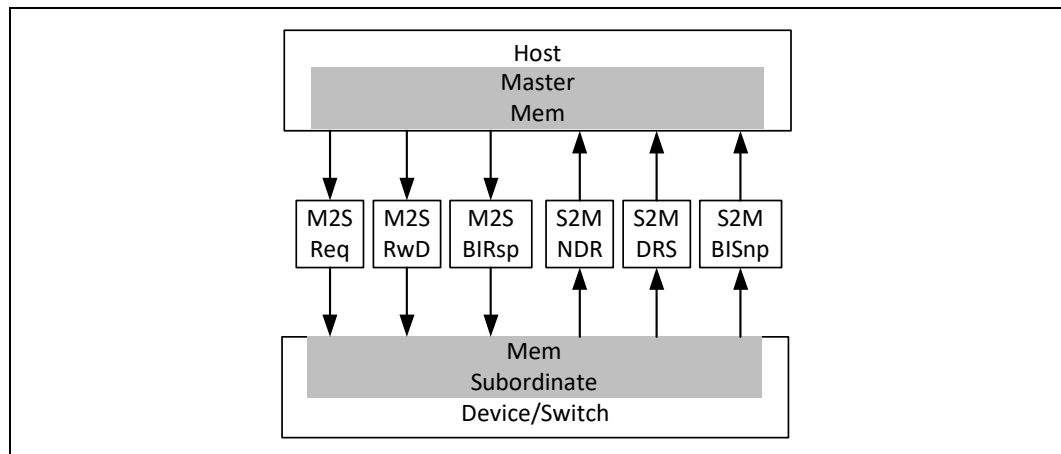
The device interface for CXL.mem defines 6 channels on primary memory protocol and an additional 6 to support direct P2P as shown in [Figure 3-18](#). Devices that support HDM-DB must support the BI* channels (S2M BISnp and M2S BIRsp). Type 2 devices that use the HDM-D memory region may not have the BI* channels. Type 3 devices (Memory Expansion) may support HDM-DB to support direct peer-to-peer on CXL.io. MLD and G-FAM devices may use HDM-DB to enable multi-host coherence and direct peer-to-peer on CXL.mem. The HDM-DB regions will be known by software and programmed as such in the decode registers and these regions will follow the protocol flows, using the BISnp channels as defined in [Appendix C, "Memory Protocol Tables."](#)

Figure 3-18. CXL.mem Channels for Devices



For Hosts, the number of channels are defined in [Figure 3-19](#). The channel definition is the same as for devices.

Figure 3-19. CXL.mem Channels for Hosts



3.3.2.1 Direct P2P CXL.mem for Accelerators

In certain topologies, an accelerator (Type 1, Type 2, or Type 3) device may optionally be enabled to communicate with peer Type 3 memories with CXL.mem protocol. Support for such communication is provided by an additional set of CXL.mem channels, with their directions reversed from conventional CXL.mem as shown in as shown in [Figure 3-18](#). These channels exist only on a link between the device and the switch downstream port to which the link is attached. Ordering requirements, message formats, and channel semantics are the same as for conventional CXL.mem. Topologies supporting Direct P2P.mem require an accelerator (requester device) and a target Type 3 peer memory device which are both directly connected to a PBR Edge DSP. PBR routing is required because not all CXL.mem messages contain sufficient information for an HBR switch to determine whether to route between a device and the host or a device and a peer device. Edge DSPs contain tables (FAST and LDST) which enable routing to the proper destination.

Details related to the device in-out dependence covering standard CXL.mem target and the source of Direct P2P CXL.mem and are covered in [Table 3-58](#).

3.3.2.2 Snoop Handling with Direct P2P CXL.mem

It is possible for a device that is using the Direct P2P CXL.mem interface to receive a snoop on H2D Req for an address that the device had previously requested over its P2P CXL.mem interface. This could occur, for example, if the host has snoop filtering disabled. Conversely, the device could receive an S2M BISnp from a peer for a line that it had acquired over CXL.cache through the host.

As a result, devices that use the Direct P2P CXL.mem interface are required to track which interface was used when a cacheline was requested and respond normally to snoops using this channel. If the device receives a snoop on a different interface, the device shall respond as though it does not have the address cached returning RspIHtI or BIRspI and shall not change the cacheline state.

IMPLEMENTATION NOTE

How the device tracks which interface was used to request each cacheline is implementation dependent. One method of tracking could be for the device to maintain a table of address ranges, programmed by software with an indication for each range whether the CXL.cache or Direct P2P CXL.mem interface should be used. This table could then be looked up when snoops are received. Other methods may also be used.

3.3.3**Back-Invalidate Snoop**

To enable a device to implement an inclusive Snoop Filter for tracking host caching of device memory, a Back-Invalidate Snoop (BISnp) is initiated from the device to change the cache state of the host. The flows related to this channel are captured in [Section 3.5.1](#). The definition of “inclusive Snoop Filter” for the purpose of CXL is a device structure that tracks cacheline granular host caching and is a limited size that is a small subset of the total Host Physical Address space supported by the device.

In 68B flits, only the CXL.cache D2H Request flows can be used for device-attached memory to manage coherence with the host as shown in [Section 3.5.2.3](#). This flow is used for addresses with the HDM-D memory attribute. A major constraint with this flow is that the D2H Req channel can be blocked waiting on forward progress of the M2S Request channel which disallows an inclusive Snoop Filter architecture. For the HDM-DB memory region, the BISnp channel (instead of CXL.cache) is used to resolve coherence. CXL host implementations may have a mix of devices with HDM-DB and HDM-D below a Root Port.

The rules related to Back-Invalidate are spread around in different areas of the specification. The following list captures a summary and pointers to requirements:

- Ordering rules in [Section 3.4](#)
- Conflict detection flows and blocking in [Section 3.5.1](#)
- Protocol Tables in [Section C.1.2](#)
- BI-ID configuration in [Section 9.14](#)
- If an outstanding S2M BISnp is pending to an address the device must block M2S Req to the same address until the S2M BISnp is completed with the corresponding M2S BIRsp
- M2S RwD channel must complete/drain without dependence on M2S Req or S2M BISnp

IMPLEMENTATION NOTE

Detailed performance implications of the implementation of an Inclusive Snoop Filter are beyond the scope of this specification, but high-level considerations are captured here:

- The number of cachelines that are tracked in an Inclusive Snoop Filter is determined based on host-processor caching of the address space. This is a function of the use model and the cache size in the host processor with upsizing of 4x or more. The 4x is based on an imprecise estimation of the unknowns in future host implementations and mismatch in Host cache ways/sectors as compared to Snoop-Filter ways/sectors.
- Device should have the capability to track BISnp messages triggered by Snoop Filter capacity evictions without immediately blocking requests on the M2S Req channel when the Inclusive Snoop Filter becomes full. In the case that the BISnp tracking structure becomes full the M2S Req channel will need to be blocked for functional correctness, but the design should size this BISnp tracker to ensure that blocking of the M2S Req channel is a rare event.
- The state per cacheline could be implemented as 2 states or 3 states. For 2 states, it would track the host in I vs. A, where A-state would represent “Any” possible MESI state in the host. For 3 states, it would add the precision of S-state tracking in which the Host may have at most a shared copy of the cacheline.

3.3.4**QoS Telemetry for Memory**

QoS Telemetry for Memory is a mechanism for memory devices to indicate their current load level (DevLoad) in each response message for CXL.mem requests and each completion for (CXL.io) UIO requests. This enables the host or peer requester to meter the issue rate of CXL.mem requests and UIO requests to portions of devices, individual devices, or groups of devices as a function of their load level, optimizing the performance of those memory devices while limiting fabric congestion. This is especially important for CXL hierarchies containing multiple memory types (e.g., DRAM and persistent memory), Multi-Logical-Device (MLD) components, and/or G-FAM Devices (GFDs).

In addition to use cases with hosts that access memory devices, QoS Telemetry for memory supports the UIO Direct P2P to HDM (see [Section 7.7.9](#)) and Direct P2P CXL.mem for Accelerators (see [Section 7.7.10](#)) use cases. For these, the peer requester for each UIO or .mem request receives a DevLoad indication in each UIO completion or .mem response. For the UIO Direct P2P use case, the peer requester may be native PCIe or CXL. Within this section, “hosts/peers” is a shorthand for referring to host and/or peer requesters that access HDM devices.

Certain aspects of QoS Telemetry are mandatory for current CXL memory devices while other aspects are optional. CXL switches have no unique requirements for supporting QoS Telemetry. It is strongly recommended for Hosts to support QoS Telemetry as guided by the reference model contained in this section. For peer requesters, the importance of supporting QoS Telemetry depends on the device type, its capabilities, and its specific use case(s).

3.3.4.1**QoS Telemetry Overview**

The overall goal of QoS Telemetry is for memory devices to provide immediate and ongoing DevLoad feedback to their associated hosts/peers, for use in dynamically adjusting their request-rate throttling. If a device or set of Devices become overloaded, the associated hosts/peers increase their amount of request rate throttling. If such

Devices become underutilized, the associated hosts/peers reduce their amount of request rate throttling. QoS Telemetry is architected to help hosts/peers avoid overcompensating and/or undercompensating.

Host/peer memory request rate throttling is optional and primarily implementation specific.

Table 3-29. Impact of DevLoad Indication on Host/Peer Request Rate Throttling

DevLoad Indication Returned in Responses	Host/Peer Request Rate Throttling
Light Load	Reduce throttling (if any) soon
Optimal Load	Make no change to throttling
Moderate Overload	Increase throttling immediately
Severe Overload	Invoke heavy throttling immediately

To accommodate memory devices supporting multiple types of memory more optimally, a device is permitted to implement multiple QoS Classes, which are identified sets of traffic, between which the device supports differentiated QoS and significant performance isolation. For example, a device supporting both DRAM and persistent memory might implement two QoS Classes, one for each type of supported memory. Providing significant performance isolation may require independent internal resources (e.g., individual request queues for each QoS Class).

This version of the specification does not provide architected controls for providing bandwidth management between device QoS Classes.

MLDs provide differentiated QoS on a per-LD basis. MLDs have architected controls specifying the allocated bandwidth fraction for each LD when the MLD becomes overloaded. When the MLD is not overloaded, LDs can use more than their allocated bandwidth fraction, up to specified fraction limits based on maximum sustained device bandwidth.

GFDs provide differentiated QoS on a per-host/peer basis. GFDs have architected controls that specify a QoS Limit Fraction value for each host/peer, based on maximum sustained device bandwidth.

HDM-DB devices send BISnp requests and receive BIRsp responses as a part of processing requests that they receive from host/peer requesters. BISnp and BIRsp messages shall not be tracked by QoS Telemetry mechanisms. If a BISnp triggers a host/peer requester writing back cached data, those transactions will be tracked by QoS Telemetry.

The DevLoad indication from CXL 1.1 memory devices will always indicate Light Load, allowing those devices to operate as best they can with hosts/peers that support QoS Telemetry, though they cannot have their memory request rate actively metered by the host/peer. Light Load is used instead of Optimal Load in case any CXL 1.1 devices share the same host/peer throttling range with current memory devices. If CXL 1.1 devices were to indicate Optimal Load, they would overshadow the DevLoad of any current devices indicating Light Load.

3.3.4.2 Reference Model for Host/Peer Support of QoS Telemetry

Host/peer support for QoS Telemetry is strongly recommended but not mandatory.

QoS Telemetry provides no architected controls for mechanisms in hosts/peers. However, if a host/peer implements independent throttling for multiple distinct sets of memory devices through a given port, the throttling must be based on HDM ranges, which are referred to as host/peer throttling ranges.

The reference model in this section covers recommended aspects for how a host/peer should support QoS Telemetry. Such aspects are not mandatory, but they should help maximize the effectiveness of QoS Telemetry in optimizing memory device performance while providing differentiated QoS and reducing CXL fabric congestion.

Each host/peer is assumed to support distinct throttling levels on a throttling-range basis, represented by Throttle[Range]. Throttle[Range] is periodically adjusted by conceptual parameters NormalDelta and SevereDelta. During each sampling period for a given Throttle[Range], the host/peer records the highest DevLoad indication reported for that throttling range, referred to as LoadMax.

Table 3-30. Recommended Host/Peer Adjustment to Request Rate Throttling

LoadMax Recorded by Host/Peer	Recommended Adjustment to Request Rate Throttling
Light Load	Throttle[Range] decremented by NormalDelta
Optimal Load	Throttle[Range] unchanged
Moderate Overload	Throttle[Range] incremented by NormalDelta
Severe Overload	Throttle[Range] incremented by SevereDelta

Any increments or decrements to Throttle[Range] should not overflow or underflow legal values, respectively.

Throttle[Range] is expected to be adjusted periodically, every tH nanoseconds unless a more immediate adjustment is warranted. The tH parameter should be configurable by platform-specific software, and ideally configurable on a per-throttling-range basis. When tH expires, the host/peer should update Throttle[Range] based on LoadMax, as shown in Table 3-30, and then reset LoadMax to its minimal value.

Round-trip fabric time is the sum of the time for a request message to travel from host/peer to Device, plus the time for a response message to travel from Device to host/peer. The optimal value for tH is anticipated to be a bit larger than the average round-trip fabric time for the associated set of devices (e.g., a few hundred nanoseconds). To avoid overcompensation by the host/peer, time is needed for the received stream of DevLoad indications in responses to reflect the last Throttle[Range] adjustment before the host/peer makes a new adjustment.

If the host/peer receives a Moderate Overload or Severe Overload indication, it is strongly recommended for the host/peer to make an immediate adjustment in throttling, without waiting for the end of the current tH sampling period. Following that, the host/peer should reset LoadMax and then wait tH nanoseconds before making an additional throttling adjustment, to avoid overcompensating.

3.3.4.3 Memory Device Support for QoS Telemetry

3.3.4.3.1 QoS Telemetry Register Interfaces

An MLD must support a specified set of MLD commands from the MLD Component Command Set as documented in Section 7.6.7.4. These MLD commands provide access to a variety of architected capability, control, and status registers for a Fabric Manager to use via the FM API.

A GFD must support a specified set of GFD commands from the GFD Component Management Command Set as documented in Section 8.2.9.9.10. These GFD commands provide access to a variety of architected capability, control, and status registers for a Fabric Manager to use via the FM API.

If an SLD supports the Memory Device Command set, it must support a specified set of SLD QoS Telemetry commands. See [Section 8.2.9.9](#). These SLD commands provide access to a variety of architected capability, control, and status fields for management by system software via the CXL Device Register interface.

Each “architected QoS Telemetry” register is one that is accessible via the above mentioned MLD commands, GFD commands, and/or SLD commands.

3.3.4.3.2 Memory Device QoS Class Support

Each CXL memory device may support one or more QoS Classes. The anticipated typical number is one to four, but higher numbers are not precluded. If a device supports only one type of media, it may be common for it to support one QoS Class. If a device supports two types of media, it may be common for it to support two QoS Classes. A device supporting multiple QoS Classes is referred to as a multi-QoS device.

This version of the specification does not provide architected controls for providing bandwidth management between device QoS Classes. Still, it is strongly recommended that multi-QoS devices track and report DevLoad indications for different QoS Classes independently, and that implementations provide as much performance isolation between different QoS Classes as possible.

3.3.4.3.3 Memory Device Internal Loading (IntLoad)

A CXL memory device must continuously track its internal loading, referred to as IntLoad. A multi-QoS device should do so on a per-QoS-Class basis.

A device must determine IntLoad based at least on its internal request queuing. For example, a simple device may monitor the instantaneous request queue depth to determine which of the four IntLoad indications to report. It may also incorporate other internal resource utilizations, as summarized in [Table 3-31](#).

Table 3-31. Factors for Determining IntLoad

IntLoad	Queuing Delay inside Device	Device Internal Resource Utilization
Light Load	Minimal	Readily handles more requests
Optimal Load	Modest to Moderate	Optimally utilized
Moderate Overload	Significant	Limiting throughput and/or degrading efficiency
Severe Overload	High	Heavily overloaded and/or degrading efficiency

The actual method of IntLoad determination is device-specific, but it is strongly recommended that multi-QoS devices implement separate request queues for each QoS Class. For complex devices, it is recommended for them to determine IntLoad based on internal resource utilization beyond just request queue depth monitoring.

Although the IntLoad described in this section is a primary factor in determining which DevLoad indication is returned in device responses, there are other factors that may need to be considered, depending upon the situation (see [Section 3.3.4.3.4](#) and [Section 3.3.4.3.5](#)).

3.3.4.3.4 Egress Port Backpressure

Even under a consistent Light Load, a memory device may experience flow control backpressure at its egress port. This is readily caused if an RP is oversubscribed by multiple memory devices below a switch. Prolonged egress port backpressure usually indicates that one or more upstream traffic queues between the device and the RP are full, and the delivery of responses from the device to the host/peer is significantly

delayed. This makes the QoS Telemetry feedback loop less responsive and the overall mechanism less effective. Egress Port Backpressure is an optional normative mechanism to help mitigate the negative effects of this condition.

IMPLEMENTATION NOTE

Egress Port Backpressure Leading to Larger Request Queue Swings

When the QoS Telemetry feedback loop is less responsive, the device's request queue depth is prone to larger swings than normal.

When the queue depth is increasing, the delay in the host/peer receiving Moderate Overload or Severe Overload indications results in the queue getting more full than normal, in extreme cases filling completely and forcing the ingress port to exert backpressure to incoming downstream traffic.

When the queue depth is decreasing, the delay in the host/peer receiving Light Load indications results in the queue getting more empty than normal, in extreme cases emptying completely, and causing device throughput to drop unnecessarily.

Use of the Egress Port Backpressure mechanism helps avoid upstream traffic queues between the device and its RP from filling for extended periods, reducing the delay of responses from the device to the host/peer. This makes the QoS Telemetry feedback loop more responsive, helping avoid excessive request queue swings.

IMPLEMENTATION NOTE

Minimizing Head-of-Line Blocking with Upstream Responses from MLDs/GFDs

When one or more upstream traffic queues become full between the MLD and one or more of its congested RPs, head-of-line (HOL) blocking associated with this congestion can delay or block traffic targeting other RPs that are not congested.

Egress port backpressure for extended periods usually indicates that the ingress port queue in the Downstream Switch Port above the device is often full. Responses in that queue targeting congested RPs can block responses targeting uncongested RPs, reducing overall device throughput unnecessarily.

Use of the Egress Port Backpressure mechanism helps reduce the average depth of queues carrying upstream traffic. This reduces the delay of traffic targeting uncongested RPs, increasing overall device throughput.

The Egress Port Congestion Supported capability bit and the Egress Port Congestion Enable control bit are architected QoS Telemetry bits, which indicate support for this optional mechanism plus a means to enable or disable it. The architected Backpressure Average Percentage status field returns a current snapshot of the measured egress port average congestion.

QoS Telemetry architects two thresholds for the percentage of time that the egress port experiences flow control backpressure. This condition is defined as the egress port having flits or messages waiting for transmission but is unable to transmit them due to a lack of suitable flow control credits. If the percentage of congested time is greater than or equal to Egress Moderate Percentage, the device may return a DevLoad indication of Moderate Overload. If the percentage of congested time is greater than or

equal to Egress Severe Percentage, the device may return a DevLoad indication of Severe Overload. The actual DevLoad indication returned for a given response may be the result of other factors as well.

A hardware mechanism for measuring Egress Port Congestion is described in [Section 3.3.4.3.9](#).

3.3.4.3.5 Temporary Throughput Reduction

There are certain conditions under which a device may temporarily reduce its throughput. Envisioned examples include a non-volatile memory (NVM) device undergoing media maintenance, a device cutting back its throughput for power/thermal reasons, and a DRAM device performing refresh. If a device is significantly reducing its throughput capacity for a temporary period, it may help mitigate this condition by indicating Moderate Overload or Severe Overload in its responses shortly before the condition occurs and only as long as really necessary. This is a device-specific optional mechanism.

The Temporary Throughput Reduction mechanism can give proactive advanced warning to associated hosts/peers, which can then increase their throttling in time to avoid the device's internal request queue(s) from filling up and potentially causing ingress port congestion. The optimum amount of time for providing advanced warning is highly device-specific, and a function of several factors, including the current request rate, the amount of device internal buffering, the level/duration of throughput reduction, and the fabric round-trip time.

A device should not use the mechanism unless conditions truly warrant its use. For example, if the device is currently under Light Load, it's probably not necessary or appropriate to indicate an Overload condition in preparation for a coming event. Similarly, a device that indicates an Overload condition should not continue to indicate the Overload condition past the point where it's needed.

The Temporary Throughput Reduction Supported capability bit and the Temporary Throughput Reduction Enable control bit are architected QoS Telemetry bits, which indicate support for this optional mechanism plus a means to enable or disable it.

IMPLEMENTATION NOTE

Avoid Unnecessary Use of Temporary Throughput Reduction

Ideally, a device should be designed to limit the severity and/or duration of its temporary throughput reduction events enough to where the use of this mechanism is not needed.

3.3.4.3.6 DevLoad Indication by Multi-QoS and Single-QoS SLDs

For SLDs, the DevLoad indication returned in each response is determined by the maximum of the device's IntLoad, Egress Port Congestion state, and Temporary Throughput Reduction state, as detailed in [Section 3.3.4.3.3](#), [Section 3.3.4.3.4](#), and [Section 3.3.4.3.5](#). For example, if IntLoad indicates Light Load, Egress Port Congestion indicates Moderate Overload, and Temporary Throughput Reduction does not indicate an overload, the resulting DevLoad indication for the response is Moderate Overload.

3.3.4.3.7 DevLoad Indication by Multi-QoS and Single-QoS MLDs

For MLDs, the DevLoad indication returned in each response is determined by the same factors as for SLDs, with additional factors used for providing differentiated QoS on a per-LD basis. Architected controls specify the allocated bandwidth for each LD as a fraction of total LD traffic when the MLD becomes overloaded. When the MLD is not

overloaded, LDs can use more than their allocated bandwidth fraction, up to specified fraction limits based on maximum sustained device bandwidth, independent of overall LD activity.

Bandwidth utilization for each LD is measured continuously based on current requests being serviced, plus the recent history of requests that have been completed.

Current requests being serviced are tracked by ReqCnt[LD] counters, with one counter per LD. The ReqCnt counter for an LD is incremented each time a request for that LD is received. The ReqCnt counter for an LD is decremented each time a response by that LD is transmitted. ReqCnt reflects instantaneous “committed” utilization, allowing the rapid reflection of incoming requests, especially when requests come in bursts.

The recent history of requests completed is tracked by CmpCnt[LD, Hist] registers, with one set of 16 Hist registers per LD. An architected configurable Completion Collection Interval control for the MLD determines the time interval over which transmitted responses are counted in the active (newest) Hist register/counter. At the end of each interval, the Hist register values for the LD are shifted from newer to older Hist registers, with the oldest value being discarded, and the active (newest) Hist register/counter being cleared. Further details on the hardware mechanism for CmpCnt[LD, Hist] are described in [Section 3.3.4.3.10](#).

Controls for LD bandwidth management consist of per-LD sets of registers called QoS Allocation Fraction[LD] and QoS Limit Fraction[LD]. For each LD, QoS Allocation Fraction specifies the fraction of current device utilization allocated for the LD across all its QoS classes. QoS Limit Fraction for each LD specifies the fraction of maximum sustained device utilization as a fixed limit for the LD across all its QoS classes, independent of overall MLD activity.

Bandwidth utilization for each LD is based on the sum of its associated ReqCnt and CmpCnt[Hist] counters/registers. CmpCnt[Hist] reflects recently completed requests, and Completion Collection Interval controls how long this period of history covers (i.e., how quickly completed requests are “forgotten”). CmpCnt reflects recent utilization to help avoid overcompensating for bursts of requests.

Together, ReqCnt and CmpCnt[Hist] provide a simple, fair, and tunable way to compute average utilization. A shorter response history emphasizes instantaneous committed utilization, improving responsiveness. A longer response history smooths the average utilization, reducing overcompensation.

ReqCmpBasis is an architected control register that provides the basis for limiting each LD’s utilization of the device, independent of overall MLD activity. Because ReqCmpBasis is compared against the sum of ReqCnt[] and CmpCnt[], its maximum value must be based on the maximum values of ReqCnt[] and CmpCnt[] summed across all configured LDs. The maximum value of Sum(ReqCnt[*]) is a function of the device’s internal queuing and how many requests it can concurrently service. The maximum value of Sum(CmpCnt[*,*]) is a function of the device’s maximum request service rate over the period of completion history recorded by CmpCnt[], which is directly influenced by the setting of Completion Collection Interval.

The FM programs ReqCmpBasis, the QoS Allocation Fraction array, and the QoS Limit Fraction array to control differentiated QoS between LDs. The FM is permitted to derate ReqCmpBasis below its maximum sustained estimate as a means of limiting power and heat dissipation.

To determine the DevLoad indication to return in each response, the device performs the following calculation:

```
Calculate TotalLoad = max(IntLoad[QoS], Egress Port Congestion state, Temporary
Throughput Reduction state);
```

Calculate ReqCmpTotal and populate ReqCmpCnt[LD] array element

```
ReqCmpTotal = 0;

For each LD

    ReqCmpCnt[LD] = ReqCnt[LD] + Sum(CmpCnt[LD, *]);

ReqCmpTotal += ReqCmpCnt[LD];
```

IMPLEMENTATION NOTE

Avoiding Recalculation of ReqCmpTotal and ReqCmpCnt[] Array

ReqCmpCnt[] is an array that avoids having to recalculate its values later in the algorithm.

To avoid recalculating ReqCmpTotal and ReqCmpCnt[] array from scratch to determine the DevLoad indication to return in each response, it is strongly recommended that an implementation maintains these values on a running basis, only incrementally updating them as new requests arrive and responses are transmitted. The details are implementation specific.

IMPLEMENTATION NOTE

Calculating the Adjusted Allocation Bandwidth

When the MLD is overloaded, some LDs may be over their allocation while others are within their allocation.

- Those LDs under their allocation (especially inactive LDs) contribute to a “surplus” of bandwidth that can be distributed across active LDs that are above their allocation.
- Those LDs over their allocation claim “their fair share” of that surplus based on their allocation, and the load value for these LDs is based on an “adjusted allocated bandwidth” that includes a prorated share of the surplus.

This adjusted allocation bandwidth algorithm avoids anomalies that otherwise occur when some LDs are using well below their allocation, especially if they are idle.

In subsequent algorithms, certain registers have integer and fraction portions, optimized for implementing the algorithms in dedicated hardware. The integer portion is described as being 16 bits unsigned, although it is permitted to be smaller or larger as needed by the specific implementation. It must be sized such that it will never overflow during normal operation. The fractional portion must be 8 bits. These registers are indicated by their name being in *italics*.

IMPLEMENTATION NOTE

Registers with Integer and Fraction Portions

These registers can hold the product of a 16-bit unsigned integer and an 8-bit fraction, resulting in 24 bits with the radix point being between the upper 16 bits and the lower 8 bits. Rounding to an integer is readily accomplished by adding 0000.80h (0.5 decimal) and truncating the lower 8 bits.

If TotalLoad is Moderate Overload or Severe Overload, calculate the adjusted

```

allocated bandwidth:

    ClaimAllocTotal = 0;

    SurplusTotal = 0;

    For each LD

        AllocCnt = QoS Allocation Fraction[LD] * ReqCmpTotal;

        If this LD is the (single) LD associated with the response

            AllocCntSaved = AllocCnt;

        If ReqCmpCnt[LD] > AllocCnt then

            ClaimAllocTotal += AllocCnt;

        Else

            SurplusTotal += AllocCnt - ReqCmpCnt[LD];

    For the single LD associated with the response

        If ReqCmpCnt[LD] > (AllocCntSaved + AllocCntSaved * SurplusTotal /
ClaimAllocTotal) then LD is over its adjusted allocated bandwidth; // Use this
result in the subsequent table

```

IMPLEMENTATION NOTE

Determination of an LD Being Above its Adjusted Allocated Bandwidth

The preceding equation requires a division, which is relatively expensive to implement in hardware dedicated for this determination. To enable hardware making this determination more efficiently, the following derived equivalent equation is strongly recommended:

$$\text{ReqCmpCnt[LD]} > (\text{AllocCntSaved} + \text{AllocCntSaved} * \text{SurplusTotal} / \text{ClaimAllocTotal})$$

$$(\text{ReqCmpCnt[LD]} * \text{ClaimAllocTotal}) > (\text{AllocCntSaved} * \text{ClaimAllocTotal} + \text{AllocCntSaved} * \text{SurplusTotal})$$

$$(\text{ReqCmpCnt[LD]} * \text{ClaimAllocTotal}) > (\text{AllocCntSaved} * (\text{ClaimAllocTotal} + \text{SurplusTotal}))$$

```
// Perform the bandwidth limit calculation for this LD
```

```
If ReqCmpCnt[LD] > QoS Limit Fraction [LD] * ReqCmpBasis then LD is over its limit
BW;
```

Table 3-32. Additional Factors for Determining DevLoad in MLDs (Sheet 1 of 2)

TotalLoad	LD over Limit BW?	LD over Adjusted Allocated BW?	Returned DevLoad Indication
Light Load or Optimal Load	No	-	TotalLoad
	Yes	-	Moderate Overload

Table 3-32. Additional Factors for Determining DevLoad in MLDs (Sheet 2 of 2)

TotalLoad	LD over Limit BW?	LD over Adjusted Allocated BW?	Returned DevLoad Indication
Moderate Overload	No	No	Optimal Load
	No	Yes	Moderate Overload
	Yes	-	Moderate Overload
Severe Overload	-	No	Moderate Overload
	-	Yes	Severe Overload

The preceding table is based on the following policies for LD bandwidth management:

- The LD is always subject to its QoS Limit Fraction
- For TotalLoad indications of Light Load or Optimal Load, the LD can exceed its QoS Allocation Fraction, up to its QoS Limit Fraction
- For TotalLoad indications of Moderate Overload or Severe Overload, LDs with loads up to QoS Allocation Fraction get throttled less than LDs with loads that exceed QoS Allocation Fraction

3.3.4.3.8 DevLoad Indication by Multi-QoS and Single-QoS GFDs

DevLoad indication for GFDs is similar to that for MLDs, with the exception that 12-bit GFD host/peer requester PIDs (RPIDs) scale much higher than the 4-bit LDs for MLDs, and the QoS Allocation Fraction mechanism (based on current device utilization) is not supported for GFDs due to architectural scaling challenges. However, the QoS Limit Fraction mechanism (based on a fixed maximum sustained device utilization) is supported for GFDs, and architected controls specify the fraction limits.

Bandwidth utilization for each RPID is measured continuously based on current requests being serviced, plus the recent history of requests that have been completed.

Current requests that are being serviced are tracked by ReqCnt[RPID] counters, with one counter per RPID. The ReqCnt counter for an RPID is incremented each time a request from that RPID is received. The ReqCnt counter for an RPID is decremented each time a response to that RPID is transmitted. ReqCnt reflects instantaneous “committed” utilization, allowing the rapid reflection of incoming requests, especially when requests come in bursts.

The recent history of requests completed is tracked by CmpCnt[RPID, Hist] registers, with one set of 16 Hist registers per RPID. An architected configurable Completion Collection Interval control for the GFD determines the time interval over which transmitted responses are counted in the active (newest) Hist register/counter. At the end of each interval, the Hist register values for the RPID are shifted from newer to older Hist registers, with the oldest value being discarded, and the active (newest) Hist register/counter being cleared. Further details on the hardware mechanism for CmpCnt[RPID, Hist] are described in [Section 3.3.4.3.10](#).

Controls for RPID bandwidth management consist of per-RPID sets of registers called QoS Limit Fraction[RPID]. QoS Limit Fraction for each RPID specifies the fraction of maximum sustained device utilization as a fixed limit for the RPID across all its QoS classes, independent of overall GFD activity.

Bandwidth utilization for each RPID is based on the sum of its associated ReqCnt and CmpCnt[Hist] counters/registers. CmpCnt[Hist] reflects recently completed requests, and Completion Collection Interval controls how long this period of history covers (i.e., how quickly completed requests are “forgotten”). CmpCnt reflects recent utilization to help avoid overcompensating for bursts of requests.

Together, ReqCnt and CmpCnt[Hist] provide a simple, fair, and tunable way to compute average utilization. A shorter response history emphasizes instantaneous committed utilization, thus improving responsiveness. A longer response history smooths the average utilization, thus reducing overcompensation.

ReqCmpBasis is an architected control register that provides the basis for limiting each RPID's utilization of the device, independent of overall GFD activity. Because ReqCmpBasis is compared against the sum of ReqCnt[] and CmpCnt[], its maximum value must be based on the maximum values of ReqCnt[] and CmpCnt[] summed across all configured RPIDs. The maximum value of Sum(ReqCnt[*]) is a function of the device's internal queuing and how many requests it can concurrently service. The maximum value of Sum(CmpCnt[*,*]) is a function of the device's maximum request service rate over the period of completion history recorded by CmpCnt[], which is directly influenced by the setting of Completion Collection Interval.

The FM programs ReqCmpBasis and the QoS Limit Fraction array to control differentiated QoS between RPIDs. The FM is permitted to derate ReqCmpBasis below its maximum sustained estimate as a means of limiting power and heat dissipation.

To determine the DevLoad indication to return in each response, the device performs the following calculation:

```
Calculate TotalLoad = max(IntLoad[QoS], Egress Port Congestion state, Temporary
Throughput Reduction state);

// Perform the bandwidth limit calculation for this RPID

If ReqCmpCnt[RPID] > QoS Limit Fraction[RPID] * ReqCmpBasis then the RPID is over
its limit BW;
```

Table 3-33. Additional Factors for Determining DevLoad in MLDs/GFDs

TotalLoad	RPID over Limit BW?	Returned DevLoad Indication
Light Load or Optimal Load	No	TotalLoad
	Yes	Moderate Overload
Moderate Overload	No	Moderate Overload
	Yes	Severe Overload
Severe Overload	-	Severe Overload

Table 3-33 is based on the following policies for RPID bandwidth management:

- The RPID is always subject to its QoS Limit Fraction
- For TotalLoad indications of Moderate Overload, RPIDs with loads up to QoS Limit Fraction get throttled less than RPIDs with loads that exceed QoS Limit Fraction

3.3.4.3.9 Egress Port Congestion Measurement Mechanism

This hardware mechanism measures the average egress port congestion on a rolling percentage basis.

FCBP (Flow Control Backpressured): this binary condition indicates the instantaneous state of the egress port. It is true if the port has messages or flits available to transmit but is unable to transmit any of them due to a lack of suitable flow control credits.

Backpressure Sample Interval register: this architected control register specifies the fixed interval in nanoseconds at which FCBP is sampled. It has a range of 0-31. One hundred samples are recorded, so a setting of 1 yields 100 ns of history. A setting of 31 yields 3.1 us of history. A setting of 0 disables the measurement mechanism, and it must indicate an average congestion percentage of 0.

BPhist[100] bit array: this stores the 100 most-recent FCBP samples. It is not accessible by software.

Backpressure Average Percentage: when this architected status register is read, it indicates the current number of Set bits in BPhist[100]. It ranges in value from 0 to 100.

The actual implementation of BPhist[100] and Backpressure Average Percentage is device specific. Here is a possible implementation approach:

- BPhist[100] is a shift register
- Backpressure Average Percentage is an up/down counter
- With each new FCBP sample:
 - If the new sample (not yet in BPhist) and the oldest sample in BPhist are both 0 or both 1, no change is made to Backpressure Average Percentage.
 - If the new sample is 1 and the oldest sample is 0, increment Backpressure Average Percentage.
 - If the new sample is 0 and the oldest sample is 1, decrement Backpressure Average Percentage.
- Shift BPhist[100], discarding the oldest sample and entering the new sample

3.3.4.3.10 Recent Transmitted Responses Measurement Mechanism

This hardware mechanism measures the number of recently transmitted responses on a per-host/peer basis in the most recent 16 intervals of a configured time period. Hosts are identified by a Requester ID (ReqID), which is the LD-ID for MLDs and the RPID for GFDs.

Completion Collection Interval register: this architected control register specifies the interval over which transmitted responses are counted in an active Hist register. It has a range is 0-127. A setting of 1 yields 16 nanoseconds of history. A setting of 127 yields about 2 us of history. A setting of 0 disables the measurement mechanism, and it must indicate a response count of 0.

CmpCnt[ReqID, 16] registers: these registers track the total of recent transmitted responses on a per-host/peer basis. CmpCnt[ReqID, 0] is a counter and is the newest value, while CmpCnt[ReqID, 1:15] are registers. These registers are not directly visible to software.

For each ReqID, at the end of each Completion Collection Interval:

- The 16 CmpCnt[ReqID, *] register values are shifted from newer to older
- The CmpCnt[ReqID, 15] Hist register value is discarded
- The CmpCnt[ReqID, 0] register is cleared and it is armed to count transmitted responses in the next interval

3.3.5 M2S Request (Req)

The Req message class generically contains reads, invalidates, and signals going from the Master to the Subordinate.

Table 3-34. M2S Request Fields (Sheet 1 of 2)

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The valid signal indicates that this is a valid request
MemOpcode	4			Memory Operation: This specifies which, if any, operation needs to be performed on the data and associated information. Details in Table 3-35 .
Snptype	3			Snoop Type: This specifies what snoop type, if any, needs to be issued by the DCOH and the minimum coherency state required by the Host. Details in Table 3-38 . This field is used to indicate the Length Index for the TEUpdate opcode.
MetaField	2			Metadata Field: Up to 3 Metadata Fields can be addressed. This specifies which, if any, Metadata Field needs to be updated. Details of Metadata Field in Table 3-36 . If the Subordinate does not support memory with Metadata, this field will still be used by the DCOH for interpreting Host commands as described in Table 3-37 .
MetaValue	2			Metadata Value: When MetaField is not No-Op, this specifies the value to which the field needs to be updated. Details in Table 3-37 . If the Subordinate does not support memory with Metadata, this field will still be used by the device coherence engine for interpreting Host commands as described in Table 3-37 . For the TEUpdate message, this field carries the TE state change value where 00b is TE cleared and 01b is TE set.
Tag	16			The Tag field is used to specify the source entry in the Master which is pre-allocated for the duration of the CXL.mem transaction. This value needs to be reflected with the response from the Subordinate so the response can be routed appropriately. The exceptions are the MemRdFwd and MemWrFwd opcodes as described in Table 3-35 . Note: The Tag field has no explicit requirement to be unique.
Address[5]	1	0		Address[5] is provisioned for future usages such as critical chunk first for 68B flit, but this is not included in a 256B flit.
Address[51:6]	46			This field specifies the Host Physical Address associated with the MemOpcode.
LD-ID[3:0]	4		0	Logical Device Identifier: This identifies a Logical Device within a Multiple-Logical Device. Not applicable in PBR mode where SPID infers this field.
SPID	0		12	Source PID
DPID	0		12	Destination PID
CKID	0	13		Context Key ID: Optional key ID that references preconfigured key material utilized for device-based data-at-rest encryption. If the device has been configured to utilize CKID-based device encryption and locked utilizing the CXL Trusted Execution Environment (TEE) Security Protocol (TSP), then this field shall be valid for Data Read access types (MemRd/MemRdTEE/MemRdData*/MemSpecRd*) and treated as reserved for other messages.

Table 3-34. M2S Request Fields (Sheet 2 of 2)

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
RSVD	6	7		Reserved
TC	2			Traffic Class: This can be used by the Master to specify the Quality of Service associated with the request. This is reserved for future usage.
Total	87	100	120	

Table 3-35. M2S Req Memory Opcodes (Sheet 1 of 2)

Opcode	Description	Encoding
MemInv	Invalidation request from the Master. Primarily for Metadata updates. No data read or write required. If SnpType field contains valid commands, perform required snoops.	0000b
MemRd	Normal memory data read operation. If MetaField contains valid commands, perform Metadata updates. If SnpType field contains valid commands, perform required snoops.	0001b
MemRdData	Normal Memory data read operation. MetaField has no impact on the coherence state. MetaValue is to be ignored. Instead, update Meta0-State as follows: If initial Meta0-State value = 'I', update Meta0-State value to 'A' Else, no update required If SnpType field contains valid commands, perform required snoops. MetaField encoding of Extended Meta-State (EMS) follows the rules for it in Table 3-36.	0010b
MemRdFwd	This is an indication from the Host that data can be directly forwarded from device-attached memory to the device without any completion to the Host. This is only sent as a result of a CXL.cache D2H read request to device-attached memory that is mapped as HDM-D. The Tag field contains the reflected CQID sent along with the D2H read request. The SnpType is always No-Op for this Opcode. The caching state of the line is reflected in the Meta0-State value. Note: This message is not sent to devices that have device-attached memory that is mapped only as HDM-H or HDM-DB.	0011b
MemWrFwd	This is an indication from the Host to the device that it owns the line and can update it without any completion to the Host. This is only sent as a result of a CXL.cache D2H write request to device-attached memory that is mapped as HDM-D. The Tag field contains the reflected CQID sent along with the D2H write request. The SnpType is always No-Op for this Opcode. The caching state of the line is reflected in the Meta0-State value. Note: This message is not sent to devices that have device-attached memory that is mapped only as HDM-H or HDM-DB.	0100b
MemRdTEE ¹	Same as MemRd but with the Trusted Execution Environment (TEE) attribute. See Section 11.5.4.5 for description of TEE attribute handling.	0101b
MemRdDataTEE ¹	Same as MemRdData but with the Trusted Execution Environment (TEE) attribute. See Section 11.5.4.5 for description of TEE attribute handling.	0110b
MemSpecRd	Memory Speculative Read is issued to start a memory access before the home agent has resolved coherence to reduce access latency. This command does not receive a completion message. The Tag, MetaField, MetaValue, and SnpType are reserved. See Section 3.5.3.1 for a description of the use case.	1000b
MemInvNT	This is similar to the MemInv command except that the NT is a hint that indicates the invalidation is non-temporal and the writeback is expected soon. However, this is a hint and not a guarantee.	1001b
MemClnEvct	Memory Clean Evict is a message that is similar to MemInv, but intent to indicate host going to I-state and does not require Meta0-State return. This message is supported only to the HDM-DB address region.	1010b

Table 3-35. M2S Req Memory Opcodes (Sheet 2 of 2)

Opcode	Description	Encoding
MemSpecRdTEE ¹	Same as MemSpecRd but with Trusted Execution Environment (TEE) attribute. See Section 11.5.4.5 for description of TEE attribute handling.	1100b
TEUpdate ¹	Update of the TE state for the memory region. The memory region update is defined by the length-index field (passed in SnpType bits). The lower address bits in the message may be set to allow routing of the message to reach the correct interleave set target; however, the lower bits are masked to the natural alignment of the length when updating TE state. The MetaValue field defines the new TE state that supports 00b to clear and 01b to set. See details of the use of this message in Section 11.5.4.5.3 .	1101b
Reserved	Reserved	<Others>

1. Supported only in 256B and PBR Flit messages and considered Reserved in 68B Flit messages.

Table 3-36. Metadata Field Definition

MetaField	Description	Encoding
Meta0-State	Update the Metadata bits with the value in the Metadata Value field. Details of MetaValue associated with Meta0-State in Table 3-37 .	00b
Extended Meta-State (EMS)	This encoding has different interpretation in different channels: - M2S Req usage indicates that the request requires the Extended MetaValue to be returned from the device in the response unless an error condition occurs. - M2S Rwd and S2M DRS use this to indication that the Extended MetaValue is attached to the message as a Trailer. This size of the MetaValue is configurable up to 32 bits. - Other channels do not use this encoding and it should be considered Reserved. For HDM-DB, the MetaValue is defined in Table 3-37 for coherence resolution, Reserved for HDM-H. This encoding is not used for HDM-D.	01b
Reserved	Reserved	10b
No-Op	No Metadata operation. The MetaValue field is Reserved. For NDR/DRS messages that would return Metadata, this encoding can be used in case of an error in Metadata storage (standard 2-bits or EMD) or if the device does not store Metadata.	11b

Table 3-37. Meta0-State Value Definition (HDM-D/HDM-DB Devices)¹

Description	Encoding
Invalid (I): Indicates the host does not have a cacheable copy of the line. The DCOH can use this information to grant exclusive ownership of the line to the device. Note: When paired with a MemOpcode = MemInv and SnpType = SnpInv, this is used to communicate that the device should flush this line from its caches, if cached, to device-attached memory resulting in all caches ending in I.	00b
Explicit No-Op: Used only when MetaField is Extended Meta-State in HDM-DB requests to indicate that a coherence state update is not requested. For all other cases this is considered a Reserved.	01b
Any (A): Indicates the host may have a shared, exclusive, or modified copy of the line. The DCOH can use this information to interpret that the Host likely wants to update the line and the device should not be given a copy of the line without resolving coherence with the host using the flow appropriate for the memory type.	10b
Shared (S): Indicates the host may have at most a shared copy of the line. The DCOH can use this information to interpret that the Host does not have an exclusive or modified copy of the line. If the device wants a shared or current copy of the line, the DCOH can provide this without informing the Host. If the device wants an exclusive copy of the line, the DCOH must resolve coherence with the Host using the flow appropriate for the memory type.	11b

1. HDM-H use case in Type 3 devices have Meta0-State definition that is host specific, so the definition in this table does not apply for the HDM-H address region in devices.

Table 3-38. Snoop Type Definition

SnpType Description	Description	Encoding
No-Op	No snoop needs to be performed	000b
SnpData	Snoop may be required - the requester needs at least a Shared copy of the line. Device may choose to give an exclusive copy of the line as well.	001b
SnpCur	Snoop may be required - the requester needs the current value of the line. Requester guarantees the line will not be cached. Device need not change the state of the line in its caches, if present.	010b
SnpInv	Snoop may be required - the requester needs an exclusive copy of the line.	011b
Reserved	Reserved	1xxb

Valid uses of M2S request semantics are described in [Table 3-39](#) but are not the complete set of legal flows. For a complete set of legal combinations, see [Appendix C](#).

Table 3-39. M2S Req Usage

M2S Req	MetaField	Meta Value	SnpType	S2M NDR	S2M DRS	Description
MemRd	Meta0-State	A	SnpInv	Cmp-E	MemData	The Host wants an exclusive copy of the line
MemRd	Meta0-State	S	SnpData	Cmp-S or Cmp-E	MemData	The Host wants a shared copy of the line
MemRd	No-Op	N/A ¹	SnpCur	Cmp	MemData	The Host wants a non-cacheable but current value of the line
MemRd	No-Op	N/A ¹	SnpInv	Cmp	MemData	The Host wants a non-cacheable value of the line and the device should invalidate the line from its caches
MemInv	Meta0-State	A	SnpInv	Cmp-E	N/A	The Host wants ownership of the line without data
MemInvNT	Meta0-State	A	SnpInv	Cmp-E	N/A	The Host wants ownership of the line without data. However, the Host expects this to be non-temporal and may do a writeback soon.
MemInv	Meta0-State	I	SnpInv	Cmp	N/A	The Host wants the device to invalidate the line from its caches
MemRdData	No-Op	N/A ¹	SnpData	Cmp-S or Cmp-E	MemData	The Host wants a cacheable copy in either exclusive or shared state
MemClnEvct	Meta0-State	I	No-Op	Cmp	N/A	Host is dropping E or S state from its cache and leaving the line in I-state. This message allows the Device to clean the Snoop Filter (or BIAS table).

1. N/A in the MetaValue indicates that the entire field is considered Reserved (cleared to 0 by sender and ignored by receiver).

3.3.6 M2S Request with Data (RwD)

The Request with Data (RwD) message class generally contains writes from the Master to the Subordinate.

Table 3-40. M2S RwD Fields (Sheet 1 of 2)

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The valid signal indicates that this is a valid request.
MemOpcode	4			Memory Operation: This specifies which, if any, operation needs to be performed on the data and associated information. Details in Table 3-41 .
Snptype	3			Snoop Type: This specifies what snoop type, if any, needs to be issued by the DCOH and the minimum coherency state required by the Host. Details in Table 3-38 .
MetaField	2			Metadata Field: Up to 3 Metadata Fields can be addressed. This specifies which, if any, Metadata Field needs to be updated. Details of Metadata Field in Table 3-36 . If the Subordinate does not support memory with Metadata, this field will still be used by the DCOH for interpreting Host commands as described in Table 3-37 .
MetaValue	2			Metadata Value: When MetaField is not No-Op, this specifies the value the field needs to be updated to. Details in Table 3-37 . If the Subordinate does not support memory with Metadata, this field will still be used by the device coherence engine for interpreting Host commands as described in Table 3-37 .
Tag	16			The Tag field is used to specify the source entry in the Master which is pre-allocated for the duration of the CXL.mem transaction. This value needs to be reflected with the response from the Subordinate so the response can be routed appropriately. For BICConflict, the tag encoding must use the same value as the pending M2S Req message (if one exists) which the BISnp found to be in conflict. This requirement is necessary to use Tag for fabric ordering of S2M NDR (Cmp* and BICConflictAck ordering for same tag). Note: The Tag field has no explicit requirement to be unique.
Address[51:6]	46			This field specifies the Host Physical Address associated with the MemOpcode.
Poison	1			The Poison bit indicates that the data contains an error. The handling of poisoned data is device specific. See Chapter 12.0 for more details.
TRP (formerly BEP)	0	1		Trailer Present: Indicates that a trailer is included on the message. The trailer size for RwD is defined in Table 3-43 . The trailer is observed in the Link Layer as a G-Slot following a 64B data payload. The baseline requirement for this bit is to enable only Byte Enables for partial writes (MemWrPtl). This bit is also optionally extended for Extend-Metadata indication. Note: This bit was formerly referred to as Byte-Enables Present (BEP), but has been redefined as part of an optional extension to support message trailers.
LD-ID[3:0]	4		0	Logical Device Identifier: This identifies a logical device within a multiple-logical device. Not applicable in PBR messages where SPID infers this field.
SPID	0		12	Source PID
DPID	0		12	Destination PID

Table 3-40. M2S Rwd Fields (Sheet 2 of 2)

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
CKID	0	13		Context Key ID: Optional key ID that references preconfigured key material utilized for device-based data-at-rest encryption. If the device has been configured to utilize CKID-based device encryption and locked utilizing the CXL Trusted Execution Environment (TEE) Security Protocol (TSP), then this field shall be valid for accesses that carry a non-reserved payload or cause a memory read to occur (MemWr*, MemRdFill*) and reserved for other cases (BICnflct).
RSVD	6	9		Reserved
TC		2		Traffic Class: This can be used by the Master to specify the Quality of Service associated with the request. This is reserved for future usage.
Total	87	104	124	

Table 3-41. M2S Rwd Memory Opcodes

Opcode	Description	Encoding
MemWr	Memory write command. Used for full cacheline writes. If MetaField contains valid commands, perform Metadata updates. If SnpType field contains valid commands, perform required snoops. If the snoop hits a Modified cacheline in the device, the DCOH will invalidate the cache and write the data from the Host to device-attached memory.	0001b
MemWrPtl	Memory Write Partial. Contains 64 byte enables, one for each byte of data. If MetaField contains valid commands, perform Metadata updates. If SnpType field contains valid commands, perform required snoops. If the snoop hits a Modified cacheline in the device, the DCOH will need to perform a merge, invalidate the cache, and write the contents back to device-attached memory. Note: This command cannot be used with host-side memory encryption unless byte-enable encodings are aligned with encryption boundaries (32B aligned is an example which may be allowed).	0010b
BICnflct	Part of conflict flow for BISnp indicating that the host observed a conflicting coherent request to the same cacheline address. See Section 3.5.1 for details. This message carries a 64B payload as required by the Rwd channel, but the payload bytes are reserved (cleared to all 0s). This message is sent on the Rwd channel because the dependence rules on this channel allow for a low-complexity flow from a deadlock-avoidance point of view.	0100b
MemRdFill ¹	This is a simple read command equivalent to MemRd but never changes coherence state (MetaField=No-Op, SnpType=No-Op). The use of this command is intended for partial write data that is merging in the host with host-side encryption. With host-side encryption, it is not possible to merge partial data in the device as an attribute of the way encryption works. This message carries a 64B payload as required by the Rwd channel; however, the payload bytes are reserved (i.e., cleared to all 0s). This message is sent on the Rwd channel because the dependence rules on this channel allow for a low-complexity flow from a deadlock-avoidance point of view.	0101b
MemWrTEE ¹	Same as MemWr but with the Trusted Execution Environment (TEE) attribute. See Section 11.5.4.5 for description of TEE attribute handling.	1001b
MemWrPtlTEE ¹	Same as MemWrPtl but with the Trusted Execution Environment (TEE) attribute. See Section 11.5.4.5 for description of TEE attribute handling.	1010b
MemRdFillTEE ¹	Same as MemRdFill but with the Trusted Execution Environment (TEE) attribute. See Section 11.5.4.5 for description of TEE attribute handling.	1101b
Reserved	Reserved	<Others>

1. Supported only in 256B and PBR Flit messages and considered reserved in 68B Flit messages.

The definition of other fields are consistent with M2S Req (see [Section 3.3.11](#)). Valid uses of M2S Rwd semantics are described in [Table 3-42](#) but are not complete set of legal flows. For a complete set of legal combinations, see [Appendix C](#).

Table 3-42. M2S Rwd Usage

M2S Req	MetaField	Meta Value	SnpType	S2M NDR	Description
MemWr	Meta0-State	I	No-Op	Cmp	The Host wants to write the cacheline back to memory and does not retain a cacheable copy.
MemWr	Meta0-State	A	No-Op	Cmp	The Host wants to write the cacheline back to memory and retains a cacheable copy in shared, exclusive or modified state.
MemWr	Meta0-State	I	SnpInv	Cmp	The Host wants to write the cacheline to memory and does not retain a cacheable copy. In addition, the Host did not get ownership of the cacheline before doing this write and needs the device to snoop-invalidate its caches before performing the writeback to memory.
MemWrPtl	Meta0-State	I	SnpInv	Cmp	Same as the above row except the data being written is partial and the device needs to merge the data if it finds a copy of the cacheline in its caches.

3.3.6.1 Trailer Present for Rwd (256B Flit)

In 256B Flit mode, a Trailer Present bit (TRP; formerly BEP, Byte-Enables Present) bit is included with the message header that indicates whether a Trailer slot is included at the end of the message. The trailer can be up to 96 bits.

Byte Enables field is 64 bits wide and indicates which of the bytes are valid for the contained data.

The Extended Metadata (EMD) trailer can be up to 32 bits. [Section 8.2.4.31](#) describes the registers that aid in discovery of device's EMD capability and EMD related configuration of the device. The mechanism for discovering the host's EMD capabilities and EMD related configuration of the host is host-specific. The host and the device must be configured in a consistent manner.

Table 3-43. Rwd Trailers

Opcode/Message	MetaField	TRP	Trailer Size Required	Description
MemWr/ MemWrTEE	EMS	1	32 bits	Trailer bits[31:0] defined as EMD.
	No-Op/MS0	0	No Trailer	
MemWrPtl/ MemWrPtlTEE	EMS	1	96 bits	Trailer bits[63:0] defined as Byte Enables. Trailer bits[95:64] defined as EMD.
	No-Op/MS0		64 bits	Trailer bits[63:0] defined as Byte Enables.
<Others>	N/A	0	No Trailer	Other combinations do not encode trailers.

3.3.7 M2S Back-Invalidate Response (BIRsp)

The Back-Invalidate Response (BIRsp) message class contains response messages from the Master to the Subordinate as a result of Back-Invalidate Snoops. This message class is not supported in 68B Flit mode.

Table 3-44. M2S BIRsp Fields

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	N/A	1		The valid signal indicates that this is a valid response.
Opcode		4		Response type with encodings in Table 3-45 .
BI-ID		12	0	BI-ID of the device that is the destination of the message. See Section 9.14 for details on how this field is assigned to devices. Not applicable in PBR messages where DPID infers this field.
BITag		12		Tracking ID from the device.
LowAddr		2		The lower 2 bits of Cacheline address (Address[7:6]). This is needed to differentiate snoop responses when a Block Snoop is sent and receives snoop response for each cacheline. For block response (opcode names *Blk), this field is reserved.
SPID		0	12	Source PID
DPID		0	12	Destination PID
RSVD		9		
Total		40	52	

Table 3-45. M2S BIRsp Memory Opcodes

Opcode	Description	Encoding
BIRspI	Host completed the Back-Invalidate Snoop for one cacheline and the host cache state is I.	0000b
BIRspS	Host completed the Back-Invalidate Snoop for one cacheline and the host cache state is S.	0001b
BIRspE	Host completed the Back-Invalidate Snoop for one cacheline and the host cache state is E.	0010b
BIRspIBlk	Same as BIRspI except that the message applies to the entire block of cachelines. The size of the block is explicit in the BISnp*Blk message for which this is a response.	0100b
BIRspSBlk	Same as BIRspS except that the message applies to the entire block of cachelines. The size of the block is explicit in the BISnp*Blk message for which this is a response.	0101b
BIRspEBlk	Same as BIRspE except that the message applies to the entire block of cachelines. The size of the block is explicit in the BISnp*Blk message for which this is a response.	0110b
Reserved	Reserved	<Others>

3.3.8 S2M Back-Invalidate Snoop (BISnp)

The Back-Invalidate Snoop (BISnp) message class contains Snoop messages from the Subordinate to the Master. This message class is not supported in 68B Flit mode.

Table 3-46. S2M BISnp Fields

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	N/A	1		The valid signal indicates that this is a valid request.
Opcode		4		Snoop type with encodings in Table 3-47.
BI-ID		12	0	BI-ID of the device that issued the message. See Section 9.14 for details on how this field is assigned. Not applicable in PBR messages where SPID infers this field.
BITag		12		Tracking ID from the device.
Address[51:6]		46		Host Physical Address. For *Blk opcodes, the lower 2 bits (Address[7:6]) are encoded as defined in Table 3-48. Used for all other opcodes that represent the standard definition of Host Physical Address.
SPID		0	12	Source PID
DPID		0	12	Destination PID
RSVD		9		
Total		84	96	

Table 3-47. S2M BISnp Opcodes

Opcode	Description	Encoding
BISnpCur	Device requesting Current copy of the line but not requiring caching state.	0000b
BISnpData	Device requesting Shared or Exclusive copy.	0001b
BISnpInv	Device requesting Exclusive Copy.	0010b
BISnpCurBlk	Same as BISnpCur except covering 2 or 4 cachelines that are naturally aligned and contiguous. The Block Enable encoding is in Address[7:6] and defined in Table 3-48. The host may give per cacheline response or a single block response applying to all cachelines in the block. More details are in Section 3.3.8.1.	0100b
BISnpDataBlk	Same as BISnpData except covering 2 or 4 cachelines that are naturally aligned and contiguous. The Block Enable encoding is in Address[7:6] and defined in Table 3-48. The host may give per cacheline response or a single block response applying to all cachelines in the block. More details are in Section 3.3.8.1.	0101b
BISnpInvBlk	Same as BISnpInv except covering 2 or 4 cachelines that are naturally aligned and contiguous. The Block Enable encoding is in Address[7:6] and defined in Table 3-48. The host may give per cacheline response or a single block response applying to all cachelines in the block. More details are in Section 3.3.8.1.	0110b
Reserved	Reserved	<Others>

3.3.8.1 Rules for Block Back-Invalidate Snoops

A Block Back-Invalidate Snoop applies to multiple naturally aligned contiguous cachelines (2 or 4 cachelines). The host must ensure that coherence is resolved for each line and may send combined or individual responses for each in arbitrary order. In the presence of address conflicts, it is necessary that the host resolve conflicts for each cacheline separately. This special address encoding applies only to BISnp*Blk messages.

Table 3-48. Block (Blk) Enable Encoding in Address[7:6]

Addr[7:6]	Description
00b	Reserved
01b	Lower 128B block is valid, Lower is defined as Address[7]=0
10b	Upper 128B block, Upper is defined as Address[7]=1
11b	256B block is valid

3.3.9 S2M No Data Response (NDR)

The NDR message class contains completions and indications from the Subordinate to the Master.

Table 3-49. S2M NDR Fields

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The valid signal indicates that this is a valid request.
Opcode	3			Memory Operation: This specifies which, if any, operation needs to be performed on the data and associated information. Details in Table 3-50 .
MetaField	2			Metadata Field: For devices that support memory with Metadata, this field may be encoded with Meta0-State in response to an M2S Req. For devices that do not support memory with Metadata or in response to an M2S Rwd, this field must be set to the No-Op encoding. No-Op may also be used by devices if the Metadata is unreliable or corrupted in the device.
MetaValue	2			Metadata Value: If MetaField is No-Op, this field is don't care; otherwise, it is Metadata Field as read from memory.
Tag	16			Tag: This is a reflection of the Tag field sent with the associated M2S Req or M2S Rwd.
LD-ID[3:0]	4		0	Logical Device Identifier: This identifies a logical device within a multiple-logical device. Not applicable in PBR messages where DPID infers this field.
DevLoad	2			Device Load: Indicates device load as defined in Table 3-51 . Values are used to enforce QoS as described in Section 3.3.4 .
DPID	0		12	Destination PID
RSVD	0	10		
Total	30	40	48	

Opcodes for the NDR message class are defined in [Table 3-50](#).

Table 3-50. S2M NDR Opcodes (Sheet 1 of 2)

Opcode	Description	Encoding
Cmp	Completions for Writebacks, Reads and Invalidates.	000b
Cmp-S	Indication from the DCOH to the Host for Shared state.	001b
Cmp-E	Indication from the DCOH to the Host for Exclusive ownership.	010b
Cmp-M	Indication from the DCOH to the Host for Modified state. This is optionally supported by host implementations and devices must support disabling of this response.	011b

Table 3-50. S2M NDR Opcodes (Sheet 2 of 2)

Opcode	Description	Encoding
BI-ConflictAck	Completion of the Back-Invalidate conflict handshake.	100b
CmpTEE	Completion for Writes (MemWr*) with TEE intent. Does not apply to any M2S Req.	101b
Reserved	Reserved	<Others>

Table 3-51 defines the DevLoad value used in NDR and DRS messages. The encodings were assigned to allow CXL 1.1 backward compatibility such that the 00b value would cause the least impact in the host.

Table 3-51. DevLoad Definition

DevLoad Value	Queuing Delay inside Device	Device Internal Resource Utilization	Encoding
Light Load	Minimal	Readily handles more requests	00b
Optimal Load	Modest to Moderate	Optimally utilized	01b
Moderate Overload	Significant	Limiting request throughput and/or degrading efficiency	10b
Severe Overload	High	Heavily overloaded and/or degrading efficiency	11b

Definition of other fields are the same as for M2S message classes.

3.3.10 S2M Data Response (DRS)

The DRS message class contains memory read data from the Subordinate to the Master.

The fields of the DRS message class are defined in Table 3-52.

Table 3-52. S2M DRS Fields (Sheet 1 of 2)

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
Valid	1			The valid signal indicates that this is a valid request.
Opcode	3			Memory Operation: This specifies which, if any, operation needs to be performed on the data and associated information. Details in Table 3-53.
MetaField	2			Metadata Field: For devices that support memory with Metadata, this field can be encoded as Meta0-State. For devices that do not, this field must be encoded as No-Op. No-Op encoding may also be used by devices if the Metadata is unreliable or corrupted in the device.
MetaValue	2			Metadata Value: If MetaField is No-Op, this field is don't care; otherwise, it must encode the Metadata field as read from Memory.
Tag	16			Tag: This is a reflection of the Tag field sent with the associated M2S Req or M2S Rwd.
Poison	1			The Poison bit indicates that the data contains an error. The handling of poisoned data is Host specific. See Chapter 12.0 for more details.
LD-ID[3:0]	4		0	Logical Device Identifier: This identifies a logical device within a multiple-logical device. Not applicable in PBR mode where DPID infers this field.
DevLoad	2			Device Load: Indicates device load as defined in Table 3-51. Values are used to enforce QoS as described in Section 3.3.4.

Table 3-52. S2M DRS Fields (Sheet 2 of 2)

Field	Width (Bits)			Description
	68B Flit	256B Flit	PBR Flit	
DPID	0		12	Destination PID
TRP	0	1		Trailer Present: Indicates that a trailer is included after the 64B payload. The Trailer size and legal encodings for DRS are defined in Table 3-54.
RSVD	9	8		
Total	40	40	48	

Table 3-53. S2M DRS Opcodes

Opcode	Description	Encoding
MemData	Memory read data. Sent in response to Reads.	000b
MemData-NXM	Memory Read Data to Non-existent Memory region. This response is only used to indicate that the device or the switch was unable to positively decode the address of the MemRd as either HDM-H or HDM-D*. Must encode the payload with all 1s and set poison if poison is enabled. This special opcode is needed because the host will have expectation of a DRS only for HDM-H or a DRS+NDR for HDM-D*, and this opcode allows devices/switches to send a single response to the host, allowing a deallocation of host tracking structures in an otherwise ambiguous case.	001b
MemDataTEE	Same as MemData but in response to MemRd* with TEE attribute.	010b
Reserved	Reserved	<Others>

3.3.10.1 Trailer Present for DRS (256B Flit)

In 256B Flit mode, a Trailer Present (TRP) bit is included with the message header that indicates whether a trailer slot is included with the message. The trailer can be up to 32 bits for DRS.

The TRP bit can be inferred by other field decode as defined in Table 3-54 for DRS. It is included to enable simple decode in the Link Layer.

The Extended Metadata (EMD) trailer is the only trailer supported. The Extended Metadata (EMD) trailer can be up to 32 bits. Section 8.2.4.31 describes the registers that aid in discovery of device's EMD capability and EMD related configuration of the device. The mechanism for discovering the host's EMD capabilities and EMD related configuration of the host is host-specific. The host and the device must be configured in a consistent manner.

Table 3-54. DRS Trailers

Opcode/Message	MetaField	TRP	Trailer Size Required	Description
MemData/ MemDataTEE	EMS	1	32 bits	Trailer bits[31:0] defined as EMD.
	No-OP/MS0	0	No Trailer	
<Others>	N/A	0	No Trailer	

3.3.11 Forward Progress and Ordering Rules

- Req may be blocked by BISnp to the Host, but Rwd cannot be blocked by BISnp to the Host.
 - This rule impacts Rwd MemWr* to Shared FAM HDM-DB uniquely requiring SnpType=No-Op to avoid causing BISnp to other requesters that are sharing the memory which could deadlock. The resulting is a requirement that the requester must first get ownership of the cacheline using M2S Req message referred to as a 2-phase write as described in [Section 2.4.4](#).
- A CXL.mem Request in the M2S Req channel must not pass a MemRdFwd or a MemWrFwd, if the Request and MemRdFwd or MemWrFwd are to the same cacheline address.
 - Reason: As described in [Table 3-35](#), MemRdFwd and MemWrFwd opcodes, sent on the M2S Req channel are, in fact, responses to CXL.cache D2H requests. The reason the response for certain CXL.cache D2H requests are on CXL.mem M2S Req channel is to ensure subsequent requests from the Host to the same address remain ordered behind it. This allows the host and device to avoid race conditions. Examples of transaction flows using MemRdFwd are shown in [Figure 3-35](#) and [Figure 3-40](#). Apart from the above, there is no ordering requirement for the Req, Rwd, NDR, and DRS message classes or for different addresses within the Req message class.
- NDR and DRS message classes, each, need to be pre-allocated at the request source. This guarantees that the responses can sink and ensures forward progress.
- On CXL.mem, write data is only guaranteed to be visible to a later access after the write is complete.
- CXL.mem requests need to make forward progress at the device without any dependency on any device initiated request except for BISnp messages. This includes any request from the device on CXL.io or CXL.cache.
- S2M and M2S Data transfer of a cacheline must occur with no interleaved transfers.

IMPLEMENTATION NOTE

There are two cases of bypassing with device-attached memory where messages in the M2S Rwd channel may pass messages for the same cacheline address in M2S Req channel.

1. Host generated weakly ordered writes (as showing in [Figure 3-32](#)) may bypass MemRdFwd and MemWrFwd. The result is the weakly ordered write may bypass older reads or writes from the Device.
2. For Device initiated RdCurr to the Host, the Host will send a MemRdFwd to the device after resolving coherency (as shown in [Figure 3-35](#)). After sending the MemRdFwd the Host may have an exclusive copy of the line (because RdCurr does not downgrade the coherency state at the target) allowing the Host to subsequently modify this line and send a MemWr to this address. This MemWr will not be ordered with respect to the previously sent MemRdFwd.

Both examples are legal because weakly ordered stores (in Case #1) and RdCurr (in Case #2) do not guarantee strong consistency.

3.3.11.1 Buried Cache State Rules for HDM-D/HDM-DB

Buried Cache state for CXL.mem protocol refers to the state of the cacheline registered by the Master's Home Agent logic (HA) for a cacheline address when a new Req or Rwd message is being sent. This cache state could be a cache that is controlled by the host,

but does not cover the cache in the device that is the owner of the HDM-D/HDM-DB memory. These rules are applicable to only HDM-D/HDM-DB memory where the device is managing coherence.

For implementations that allow multiple outstanding requests to the same address, the possible future cache state must be included as part of the buried cache state. To avoid this complexity, it is recommended to limit to one Req/RwD per cacheline address.

Buried Cache state rules for Master-issued CXL.mem Req/RwD messages:

- Must not issue a MemRd/MemInv/MemInvNT (MetaValue=I) if the cacheline is buried in Modified, Exclusive, or Shared state.
- Shall not issue a MemRd/MemInv/MemInvNT (MetaValue=S) or MemRdData if the cacheline is buried in Modified or Exclusive state, but is allowed to issue when the host has Shared or Invalid state.
- May issue a MemRd/MemInv/MemInvNT (MetaValue = A) from any state.
- May issue a MemRd/MemInv/MemInvNT (MetaField = No-Op) from any state. Note that the final host cache state may result in a downgraded state such as Invalid when initial buried state exists and conflicting BISnp results in the buried state being downgraded.
- May issue MemClnEvct from Shared or Exclusive state.
- May issue MemWr with SnpType=SnpInv only from I-state. Use of this encoding is not allowed for HDM-DB memory regions in which coherence extends to multiple hosts (e.g., Coherent Shared FAM as described in [Section 2.4.4](#)).
- MemWr with SnpType=No-Op may be issued only from Modified state.

Note:

The Master may silently degrade clean cache state (E to S, E to I, S to I) and as such the Subordinate may have more conservative view of the Master's cache state. This section is discussing cache state from the Master's view.

[Table 3-55](#) summarizes the Req message and RwD message allowance for Buried Cache state. MemRdFwd/MemWrFwd/BICoConflict are excluded from this table because they are response messages.

Table 3-55. Allowed Opcodes for HDM-D/HDM-DB Req and RwD Messages per Buried Cache State

CXL.mem Req/RwD				Buried Cache State			
Opcodes	MetaField	MetaValue	SnpType	Modified	Exclusive	Shared	Invalid
MemRdData	All Legal Combinations		All Legal Combinations			X	X
MemClnEvct					X	X	
MemRd/ MemInv/ MemInvNT	MS0/EMD	A		X ¹	X	X	X
		S				X	X
		I					X
	No-Op	N/A		X ¹	X	X	X
	EMD	Explicit No-Op					
MemWr	All Legal Combinations		No-Op	X			
			SnpInv				X
MemRdFill/ MemRdFillTEE/ MemRdDataTEE/ MemRdTEE/ MemWrTEE	N/A (Commands not supported for HDM-D/HDM-DB)						

1. Requesters that have active reads with buried-M state must expect data return to be stale. It is up to the requester to ensure that possible stale data case is handled in all cases including conflicts with BISnp.

3.4 Transaction Ordering Summary

This section presents CXL ordering rules in a series of tables and descriptions. [Table 3-56](#) captures the upstream ordering cases. [Table 3-57](#) captures the downstream ordering cases.

For CXL.mem and CXL.cache, the term upstream describes traffic on all S2M and D2H message classes, and the term downstream describes traffic on all M2S and H2D message classes, regardless of the physical direction of travel.

Where upstream and downstream traffic coexist in the same physical direction within PBR switches and on Inter Switch Links (ISLs) or on links from a device that issues direct P2P CXL.mem, the upstream and downstream Ordering Tables each apply to their corresponding subset of the traffic and each subset shall be independent and not block one another.

[Table 3-58](#) lists the Device in-out dependence. [Table 3-59](#) lists the Host in-out dependence. Additional detail is provided in [Section 3.2.2.1](#) for CXL.cache and in [Section 3.3.11](#) for CXL.mem.

In [Table 3-56](#) and [Table 3-57](#), the columns represent a first-issued message and the rows represent a subsequently issued message. The table entry indicates the ordering relationship between the two messages. The table entries are defined as follows:

- Yes: The second message (row) must be allowed to pass the first message (column) to avoid deadlock. (When blocking occurs, the second message is required to pass the first message.)
- Y/N: There are no ordering requirements. The second message may optionally pass the first message or may be blocked by it.
- No: The second message must not be allowed to pass the first message. This is required to support the protocol ordering model.

Note: Passing, where permitted, must not be allowed to cause the starvation of any message class.

Table 3-56. Upstream Ordering Summary

Row Pass Column?	CXL.io TLPs (Col 2-5)	S2M NDR/DRS D2H Rsp/Data (Col 6)	D2H Req (Col 7)	S2M BISnp (Col 13)
CXL.io TLPs (Row A-D)	PCIe Base	Yes(1)	Yes(1)	Yes(1)
S2M NDR/DRS D2H Rsp/Data (Row E)	Yes(1)	a. No(3) b. Y/N	Yes(2)	Yes(2)(4)
D2H Req (Row F)	Yes(1)	Y/N	Y/N	Y/N
S2M BISnp (Row M)	Yes(1)(4)	Y/N	Yes(4)	Y/N

Explanation of row and column headers:

M7 requires BISnp to pass D2H Req in accordance with dependence relationship: D2H Req depends on M2S Req depends on S2M BISnp.

E6a requires that within the NDR channel, BICConflictAck must not pass prior Cmp* messages with the same Cacheline Address (implied by the tag field).

E6b other cases not covered by rule E6a are Y/N.

Color-coded rationale for cells in Table 3-56	
Yes(1)	CXL architecture requirement for ARB/MUX.
Yes(2)	CXL.cachemem: Required for deadlock avoidance.
No(3)	Type 2/3 devices where BICConflictAck must not pass prior Cmp* to the same address.
Yes(4)	Required for deadlock avoidance with the introduction of the BISnp channel. For CXL.io Unordered I/O, this is necessary because Unordered I/O can trigger BISnp.

Table 3-57. Downstream Ordering Summary

Row Pass Column?	CXL.io TLPs (Col 2-5)	M2S Req (Col 8)	M2S Rwd (Col 9)	H2D Req (Col 10)	H2D Rsp (Col 11)	H2D Data (Col 12)	M2S BIRsp (Col 14)
CXL.io TLPs (Row A-D)	PCIe Base	Yes(1)	Yes(1)	Yes(1)	Yes(1)	Yes(1)	Yes(1)
M2S Req (Row G)	Yes(1)	a. No(5)	Y/N	Y/N(3)	Y/N	Y/N	Y/N
		b. Y/N					
M2S Rwd (Row H)	Yes(1)(6)	a. Yes(6)	Y/N	Yes(3)	Y/N	Y/N	Y/N
		b. Y/N					
H2D Req (Row I)	Yes(1)	Yes(2)(6)	a. Yes(2)	Y/N	a. No(4)	Y/N(3)	Y/N
			b. Y/N		b. Y/N		
H2D Rsp (Row J)	Yes(1)	Yes(2)	Yes(2)	Yes(2)	Y/N	Y/N	Y/N
H2D Data (Row K)	Yes(1)	Yes(2)	Yes(2)	Yes(2)	Y/N	Y/N	Y/N
M2S BIRsp (Row N)	Yes(1)(6)	Yes(2)	Yes(2)	Yes(2)	Y/N	Y/N	Y/N

Explanation of row and column headers:

In Downstream direction pre-allocated channels are kept separate because of unique ordering requirements in each.

Color-coded rationale for cells in Table 3-57	
Yes(1)	CXL architecture requirement for ARB/MUX.
Yes(2)	CXL.cachemem: Required for deadlock avoidance.
Yes(3)	CXL.cachemem: Performance optimization.
Y/N(3)	CXL.cachemem: Non-blocking recommended for performance optimization.
No(4)	Type 1/2 device: Snoop push GO requirement.
No(5)	Type 2 device: MemRd*/MemInv* push Mem*Fwd requirement.
Yes(6)	Required for deadlock avoidance with the introduction of the BISnp channel.

Explanation of table entries:

G8a MemRd*/MemInv* must not pass prior Mem*Fwd messages to the same cacheline address. This rule is applicable only for HDM-D memory regions in devices which result in receiving Mem*Fwd messages (Type 3 devices with no HDM-D don't need to implement this rule). This rule does not apply to Type 2 devices that implement the HDM-DB memory region which use the BI* channels because they do not support Mem*Fwd.

G8b All other cases not covered by rule G8a do not have ordering requirements (Y/N).

H8a applies to components that support the BISnp/BIRsp message classes to ensure that the Rwd channel can drain to the device even if the Req channel is blocked.

H8b applies to components that do not support the BISnp/BIRsp message classes.

I9a applies for PBR-capable switches, for ISLs, and for devices that can initiate P2P CXL.mem. (Possible future use case for Host-to-Host CXL.mem will require host to apply this ordering rule.)

I9b applies to all other cases.

I11a Snoops must not pass prior GO* messages to the same cacheline address. GO messages do not carry the address, so implementations where address cannot be inferred from UQID in the GO message will need to strictly apply this rule across all messages.

I11b Other case not covered by I11a are Y/N.

Table 3-58. Device In-Out Ordering Summary

Row (in) Independent of Column (out)?	CXL.io TLPs (Col A-D)	S2M NDR/DRS D2H Rsp/Data (Col E)	D2H Req (Col F)	S2M BISnp (Col M)	M2S Req (Col N) ¹	M2S Rwd (Col O) ¹	M2S BIRsp (Col P) ¹
CXL.io TLPs (Row 2-5)	PCIe Base	Y/N(1)	Y/N(1)	Y/N(1)	Y/N(1)	Y/N(1)	Y/N(1)
		Yes(3)	Yes(3)	Yes(3)	Yes(3)	Yes(3)	Yes(3)
M2S Req (Row 8)	Yes(1)	Y/N	Yes(2)	Y/N	Yes(2)	Y/N	Y/N
M2S Rwd (Row 9)	Yes(1)(2)	Y/N	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Y/N
H2D Req (Row 10)	Yes(1)	Y/N	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Y/N
H2D Rsp (Row 11)	Yes(1)	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Yes(2)
H2D Data (Row 12)	Yes(1)	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Yes(2)
M2S BIRsp (Row 14)	Yes(1)(2)	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Yes(2)
S2M NDR/DRS (Row 15)¹	Yes(1)	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Yes(2)	Y/N
S2M BISnp (Row 16)¹	Yes(1)	Y/N	Yes(2)	Yes(2)	Yes(2)	Y/N	Y/N

1. These rows and columns are supported only by devices that have Direct P2P CXL.mem enabled.

In the device ordering, the row represents incoming message class and the column represents the outgoing message class. The cases in this table show when incoming must be independent of outgoing (Yes) and when it is allowed to block incoming based on outgoing (Y/N).

Color-coded rationale for cells in Table 3-58	
Yes(1)	CXL.cachemem is independent of outgoing CXL.io.
Y/N(1)	CXL.io traffic, except UIO Completions, may be blocked by CXL.cachemem.
Yes(2)	CXL.cachemem: Required for deadlock avoidance.
Yes(3)	CXL UIO completions are independent of CXL.cachemem.

Table 3-59. Host In-Out Ordering Summary

Row (in) Independent of Column (out)?	CXL.io TLPs (Col A-D)	M2S Req (Col G)	M2S Rwd (Col H)	H2D Req (Col I)	H2D Rsp (Col J)	H2D Data (Col K)	M2S BIRsp (Col N)
CXL.io TLPs (Row 2-5)	PCIe Base	Y/N(1)	Y/N(1)	Y/N(1)	Y/N(1)	Y/N(1)	Y/N(1)
		Yes(3)	Yes(3)	Yes(3)	Yes(3)	Yes(3)	Yes(3)
S2M NDR/DRS D2H Rsp/Data (Row 6)	Yes(1)(2)	Yes(2)	Yes(2)	Yes(2)	Y/N	Y/N	Y/N
D2H Req (Row 7)	Yes(1)	Y/N	Y/N	Y/N	Y/N	Y/N	Y/N
S2M BISnp (Row 13)	Yes(1)(2)	Yes(2)	Y/N	Y/N	Y/N	Y/N	Y/N

In the host ordering, the row represents incoming message class and the column represents the outgoing message class. The cases in this table show when incoming must be independent of outgoing (Yes) and when it is allowed to block incoming based on outgoing (Y/N).

Color-coded rationale for cells in Table 3-59	
Yes(1)	Incoming CXL.cachemem must not be blocked by outgoing CXL.io.
Y/N(1)	Incoming CXL.io may be blocked by outgoing CXL.cachemem.
Yes(2)	CXL.cachemem: Required for deadlock avoidance.
Yes(3)	CXL UIO completions are independent of CXL.cachemem.

3.5 Transaction Flows to Device-attached Memory

3.5.1 Flows for Back-Invalidate Snoops on CXL.mem

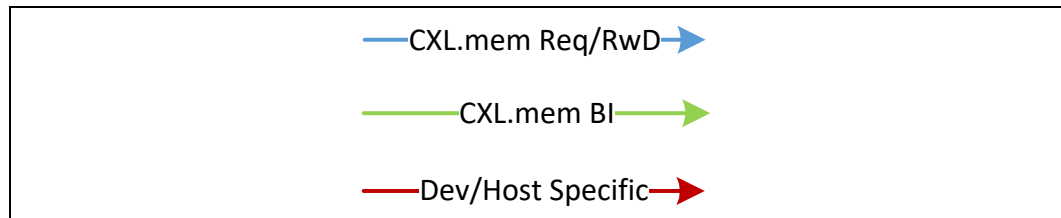
3.5.1.1 Notes and Assumptions

The Back-Invalidate Snoop (BISnp) channel provides a dedicated channel S2M to allow the owner of an HDM region to snoop a host that may have a cached copy of the line. The forward progress rules as defined in [Section 3.4](#) ensure that the device can complete the BISnp while blocking new requests (M2S Req).

The term Snoop Filter (SF) in the following diagrams is a structure in the device that is inclusively tracking any host caching of device memory and is assumed to have a size that may be less than the total possible caching in the host. The Snoop Filter is kept

inclusive of host caching by sending “Back-Invalidate Snoops” to the host when it becomes full. This full trigger that forces the BISnp is referred to as “SF Victim”. In the diagrams, an “SF Miss” that is caused by an M2S request implies that the device must also allocate a new SF entry if the host is requesting a cached copy of the line. When allocating an SF entry, it may also trigger an SF Victim for a different cacheline address if the SF is full. Figure 3-20 provides the legend for the Back-Invalidate Snoop flow diagrams that appear in the subsections that follow. The “CXL.mem BI” type will cover the BI channel messages and any conflict message/flow (e.g., BIConglict) that flow on the RxD channels. Note that the “Dev/Host Specific” messages are just short-hand flows for the type of flow expected in the host or device.

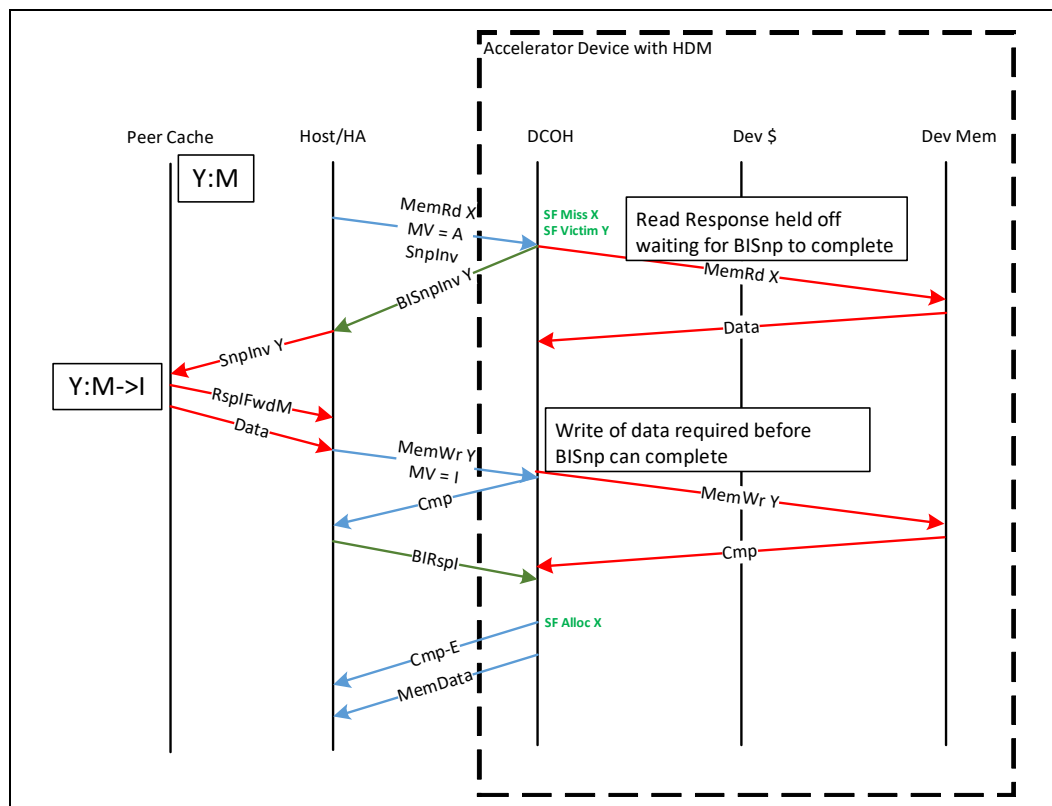
Figure 3-20. Flows for Back-Invalidate Snoops on CXL.mem Legend



3.5.1.2 BISnp Blocking Example

Figure 3-21 starts out with MemRd that is an SF Miss in the device. The SF is full, which prevents SF allocation; thus, the device must create room in the SF by triggering an SF Victim for Address Y before it can complete the read. In this example, the read to device memory Address X is started in parallel with the BISnpInv to Address Y, but the device will be unable to complete the MemRd until it can allocate an SF which requires the BISnp to Y to complete. As part of the BISnpInv, the host finds modified data for Y which must be flushed to the device before the BISnpInv can complete. The device completes the MemWr to Y, which allows the host to complete the BISnpInv to Y with the BIRspI. That completion allows the SF allocation to occur for Address X, which enables the Cmp-E and MemData to be sent.

Figure 3-21. Example BISnp with Blocking of M2S Req

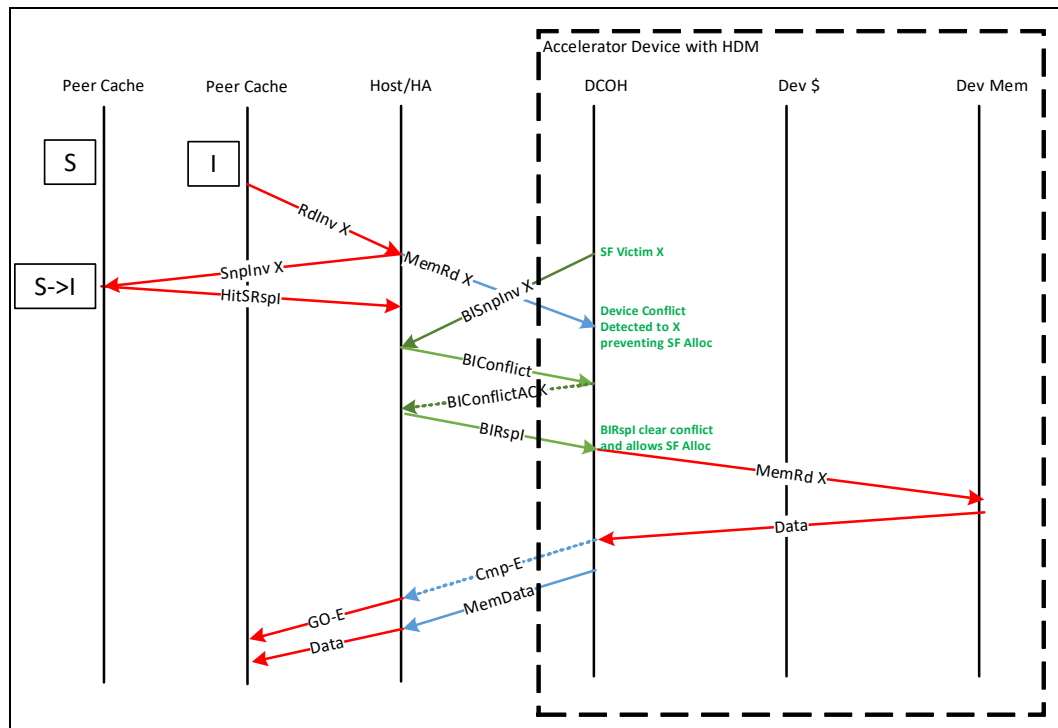


3.5.1.3 Conflict Handling

A conflict is defined as a case where S2M BISnp and M2S Req are active at the same time to the same address. There are two cases to consider: Early Conflict and Late Conflict. The two cases are ambiguous to the host side of the link until observation of a Cmp message relative to BIConglictAck. The conflict handshake starts when by the host detecting a BISnp to the same address as a pending Req. The host sends a BIConglict with the Tag of the M2S Req and device responds to a BIConglict with a BIConglictAck which must push prior Cmp* messages within the NDR channel. This ordering relationship is fundamental to allow the host to correctly resolve the two cases.

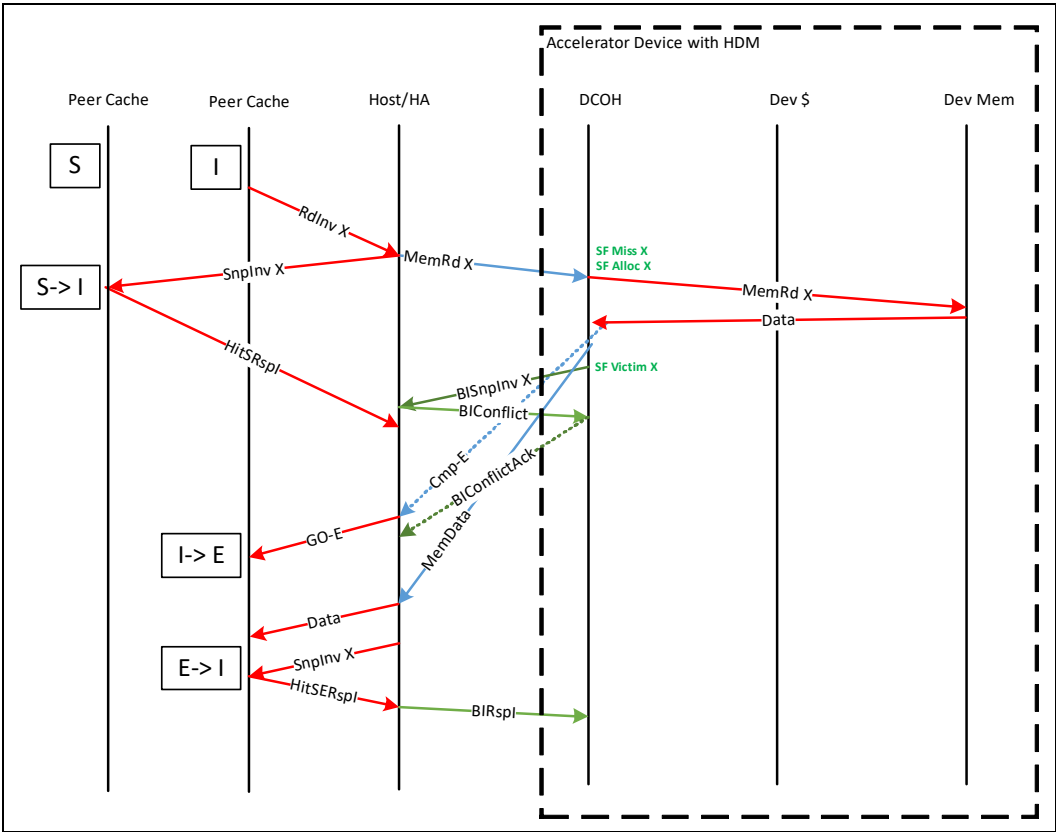
The Early Conflict case in Figure 3-22 is defined as a case where M2S Req is blocked (or in flight) at the device while S2M BISnp is active. The host observing BIConglictAck before Cmp-E determines the M2S MemRd is still pending so that it can reply with RspI.

Figure 3-22. BISnp Early Conflict



Late conflict is captured in Figure 3-23 and is defined as the case where M2S Req was processed and completions are in flight when BISnp is started. In the example below, the Cmp-E message is observed at the host before BICnflictAck, so the host must process the BISnpInv with E-state ownership, which requires it to degrade E to I before completing the BISnpInv with BIRsplt. Note that MemData has no ordering requirement and can be observed either before or after the BICnflictAck, although this example shows it after which delays the host's ability to immediately process the internal Snplnv X.

Figure 3-23. BISnp Late Conflict



3.5.1.4 Block Back-Invalidate Snoops

To support increased efficient snooping the BISnp channel defines messages that can Snoop multiple cachelines in the host in a single message. These messages support either 2 or 4 cachelines where the base address must be naturally aligned with the length (128B or 256B). The host is allowed to respond with either a single block response or individual snoop responses per cacheline.

Figure 3-24 is an example of a Block response case. In this example the host receives the BISnpInvBlk for Y, which is a 256B block. Internally the host logic is showing resolving coherence by snooping Y0 and Y2 and the host HA tracker knows the other portions of the block Y1 and Y3 are already in the invalid state, so it does not need to snoop for that portion of the 256B block. Once snoop responses for Y0 and Y2 are completed, the Host HA can send the BIRspIBlk indicating that the entire block is in I-state within the host, thereby allowing the device to have Exclusive access to the block. This results in the SF in I-state for the block and the device cache in E-state.

Figure 3-24. Block BISnp with Block Response

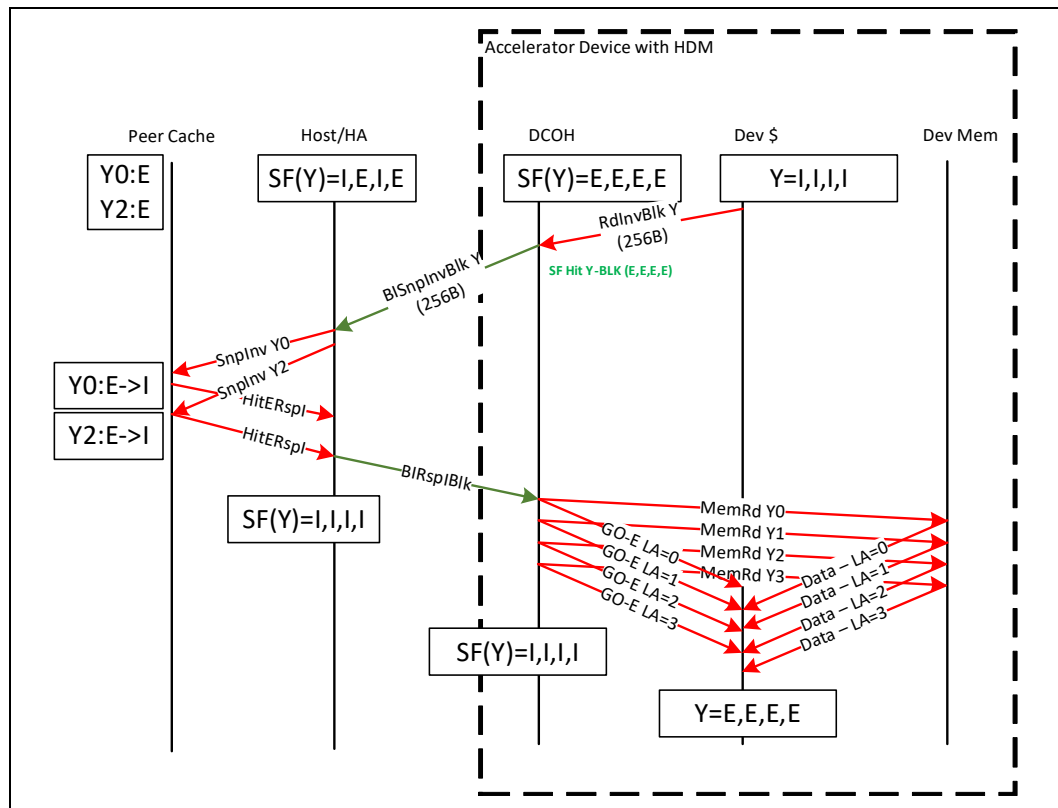
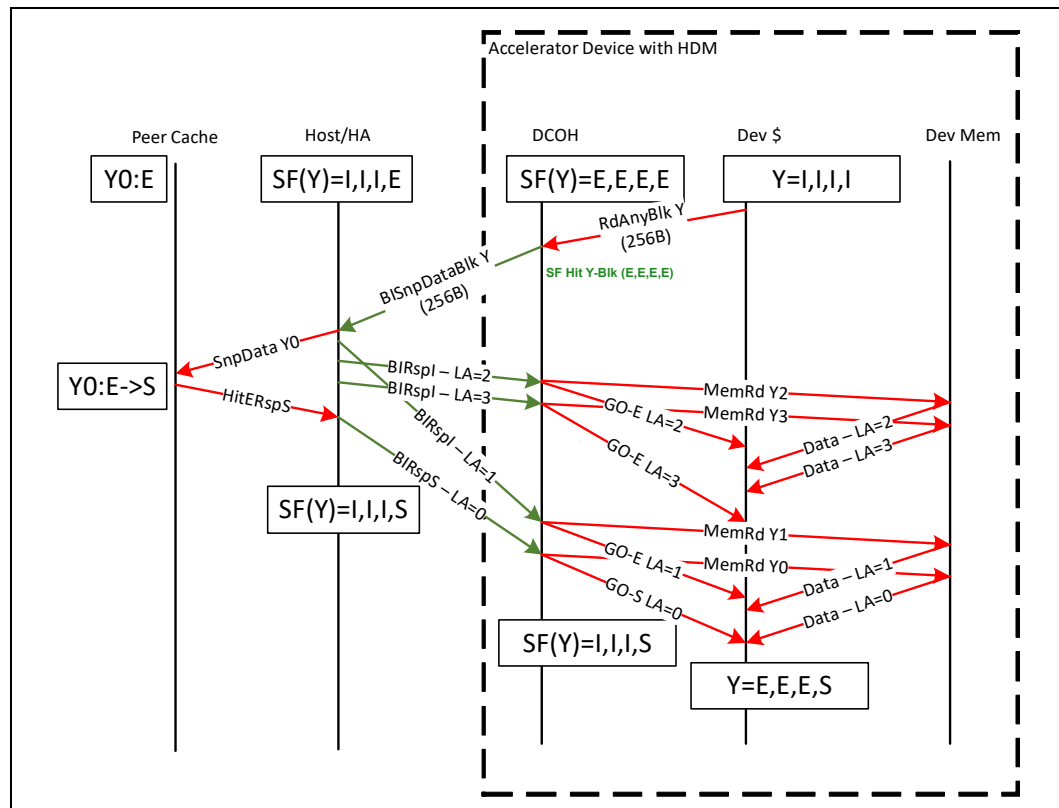


Figure 3-25 is an example where the host sends individual cacheline responses on CXL.mem for each cacheline of the block. The host encodes the 2-bit Lower Address (LowAddr) of the cacheline (Address[7:6]) with each cacheline response to allow the device to determine for which portion of the block the response is intended. The device may see the response messages in any order, which is why LA must be explicitly sent. In a Block, BISnp Address[7:6] is used to indicate the offset and length of the block as defined in Table 3-48 and is naturally aligned to the length.

Figure 3-25. Block BISnp with Cacheline Response



3.5.2 Flows for Type 1 Devices and Type 2 Devices

3.5.2.1 Notes and Assumptions

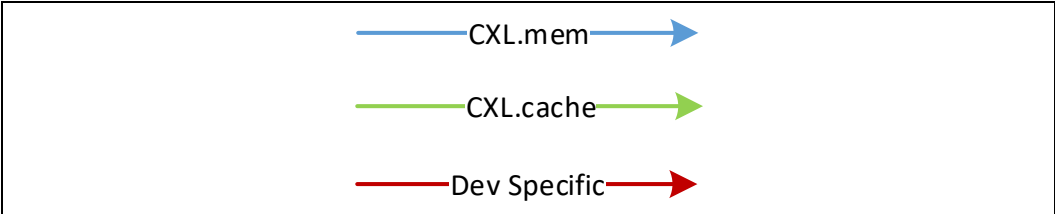
The transaction flow diagrams below are intended to be illustrative of the flows between the Host and device for access to device-attached Memory using the Bias-Based Coherency mechanism described in [Section 2.2.2](#). However, these flows are not comprehensive of every Host and device interaction. The diagrams below make the following assumptions:

- The device contains a coherency engine which is called DCOH in the diagrams below.
- The DCOH contains a Snoop Filter which tracks any caches (called Dev cache) implemented on the device. This is not strictly required, and the device is free to choose an implementation specific mechanism as long as the coherency rules are obeyed.
- The DCOH contains host coherence tracking logic for the device-attached memory. This tracking logic is referred to as a Bias Table in the context of the HDM-D memory region. For HDM-DB, it is referred to as a Directory or a Host Snoop Filter. The implementation of this is device specific.
- The device-specific aspects of the flow, illustrated using red flow arrows, need not conform exactly to the diagrams below. These can be implemented in a device-specific manner.
- Device-attached Memory exposed in a Type 2 device can be either HDM-D or HDM-DB. HDM-D will resolve coherence using a request that is issued on CXL.cache and

the Host will send a Mem*Fwd as a response on the CXL.mem Req channel. The HDM-DB region uses the separate CXL.mem BISnp channel to manage coherence with detailed flows covered in [Section 3.5.1](#). This section will indicate where the flows differ.

Figure 3-26 provides the legend for the diagrams that follow.

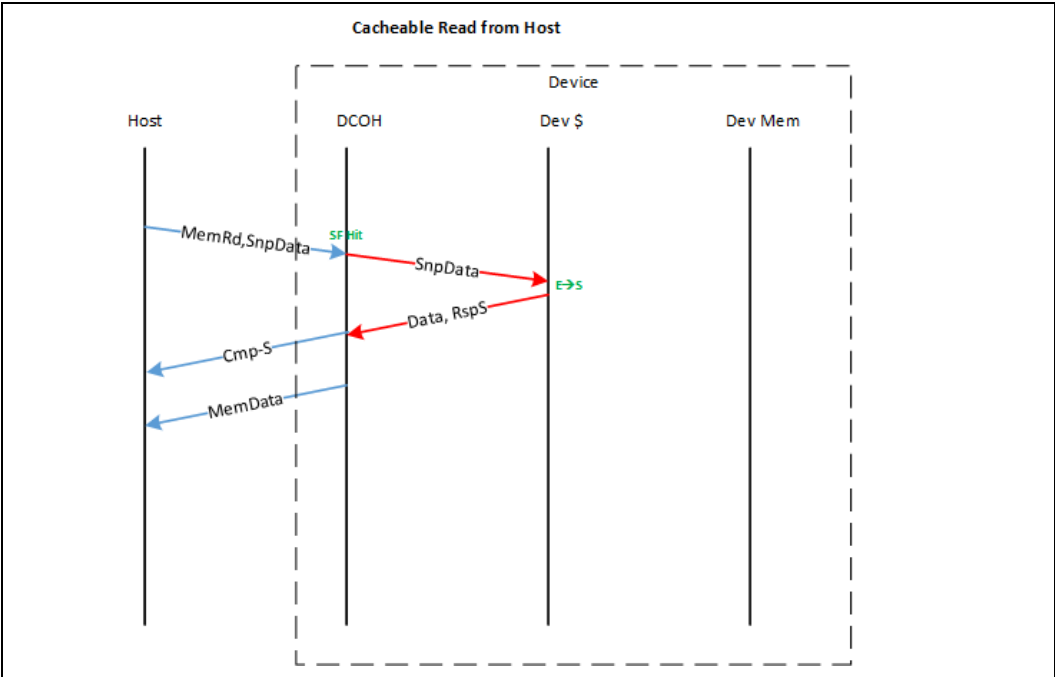
Figure 3-26. Flows for Type 1 Devices and Type 2 Devices Legend



3.5.2.2 Requests from Host

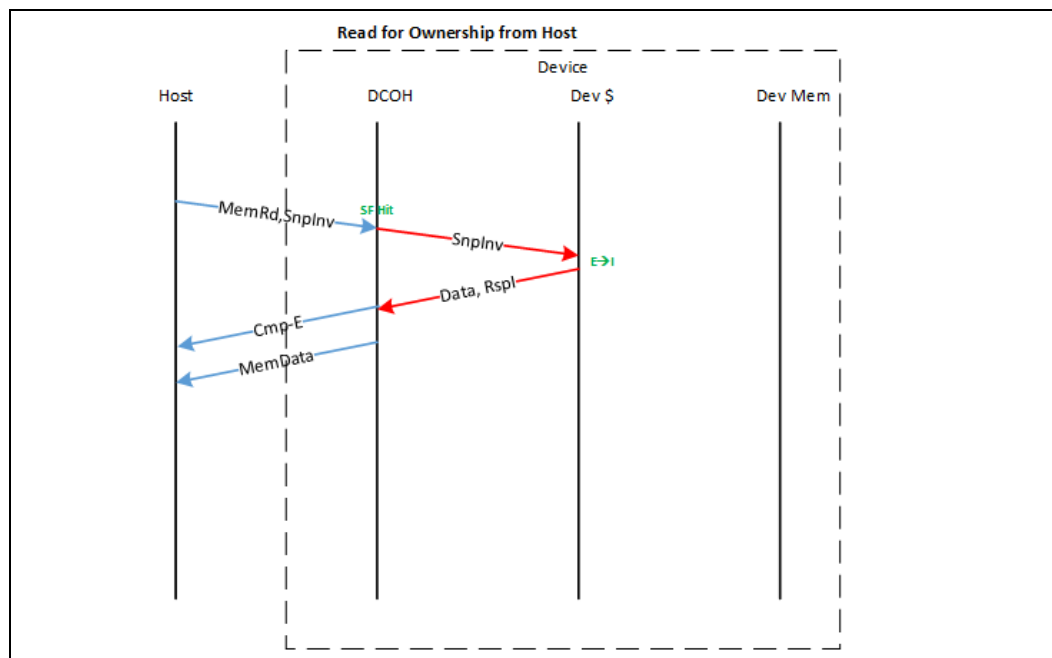
Please note that the flows shown in this section ([Requests from Host](#)) do not change on the CXL interface regardless of the bias state of the target region. This effectively means that the device needs to give the Host a consistent response, as expected by the Host and shown in [Figure 3-27](#).

Figure 3-27. Example Cacheable Read from Host



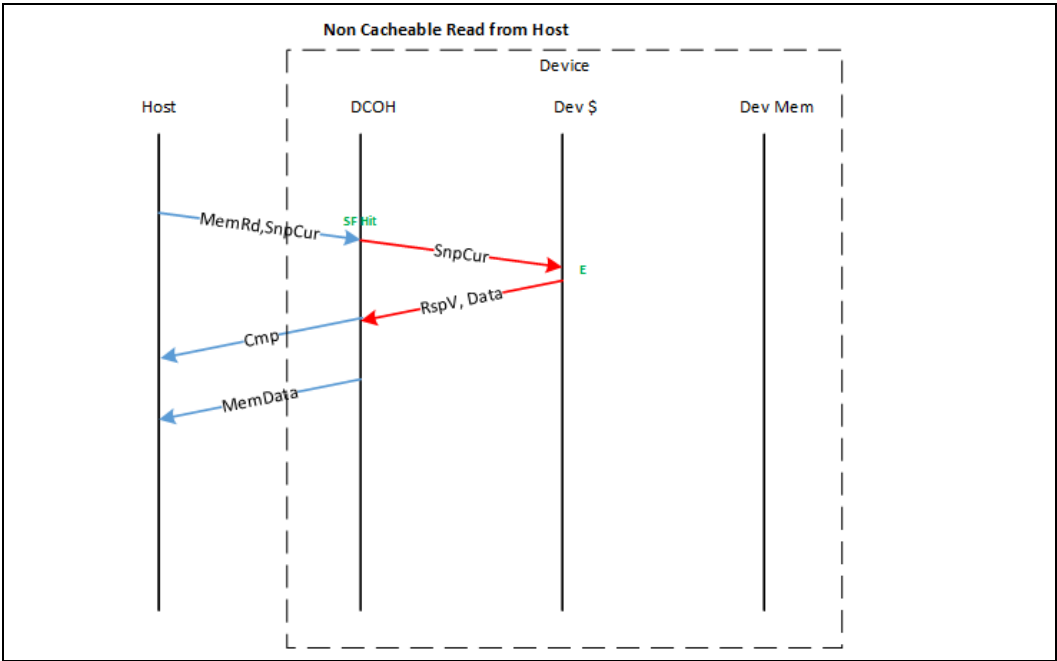
In the above example, the Host requested a cacheable non-exclusive copy of the line. The non-exclusive aspect of the request is communicated using the "SnpData" semantic. In this example, the request got a snoop filter hit in the DCOH, which caused the device cache to be snooped. The device cache downgraded the state from Exclusive to Shared and returned the Shared data copy to the Host. The Host is told of the state of the line using the Cmp-S semantic.

Figure 3-28. Example Read for Ownership from Host



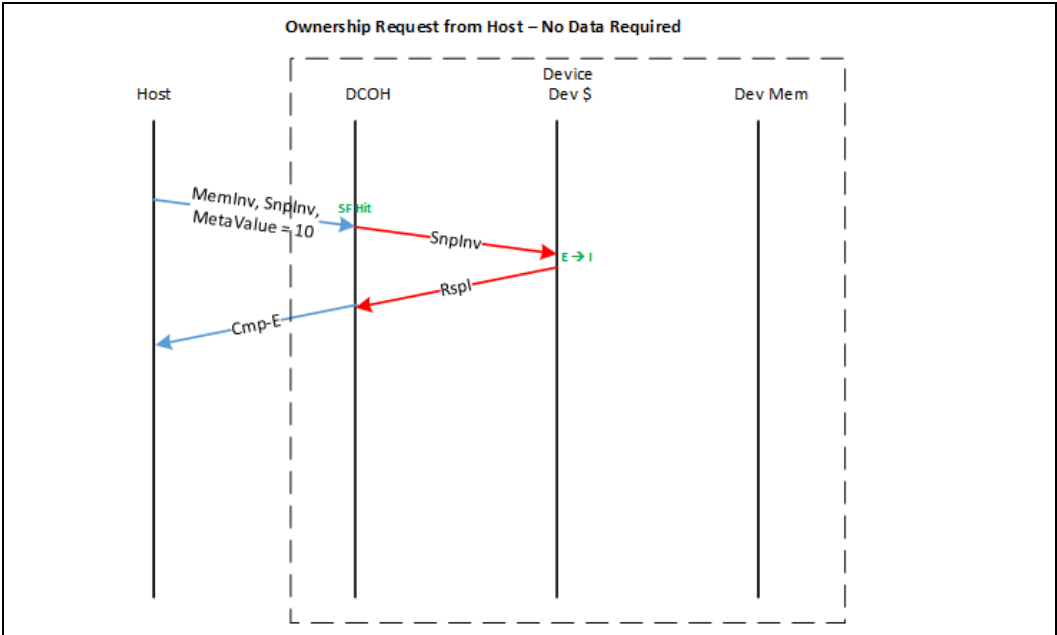
In the above example, the Host requested a cacheable exclusive copy of the line. The exclusive aspect of the request is communicated using the "SnpInv" semantic, which asks the device to invalidate its caches. In this example, the request got a snoop filter hit in the DCOH, which caused the device cache to be snooped. The device cache downgraded the state from Exclusive to Invalid and returned the Exclusive data copy to the Host. The Cmp-E semantic is used to communicate the line state to the Host.

Figure 3-29. Example Non Cacheable Read from Host



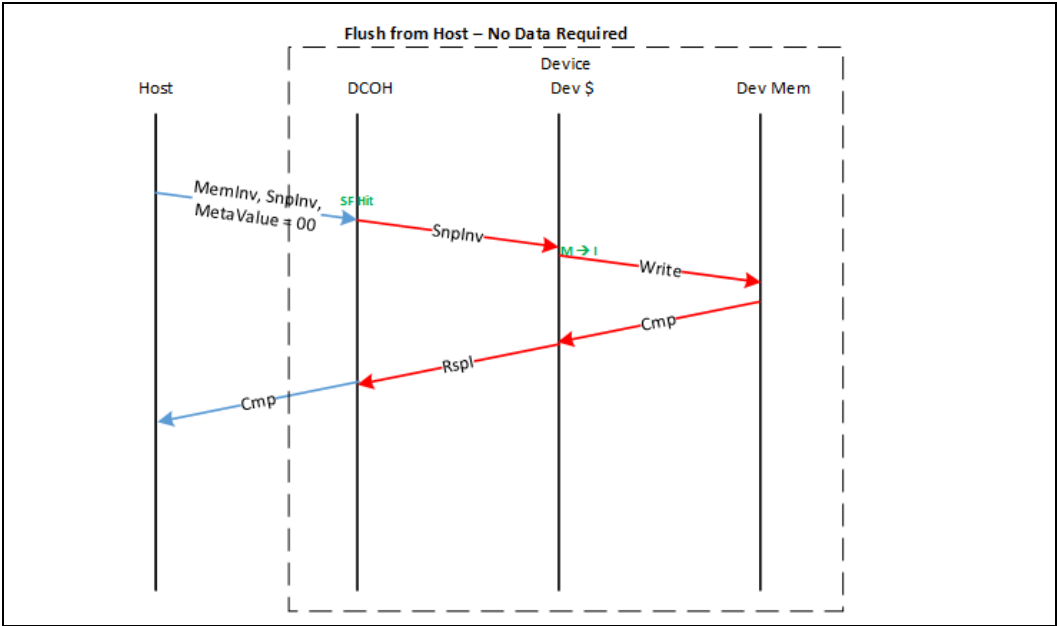
In the above example, the Host requested a non-cacheable copy of the line. The non-cacheable aspect of the request is communicated using the “SnpCur” semantic. In this example, the request got a snoop filter hit in the DCOH, which caused the device cache to be snooped. The device cache did not need to change its caching state; however, it gave the current snapshot of the data. The Host is told that it is not allowed to cache the line using the Cmp semantic.

Figure 3-30. Example Ownership Request from Host - No Data Required



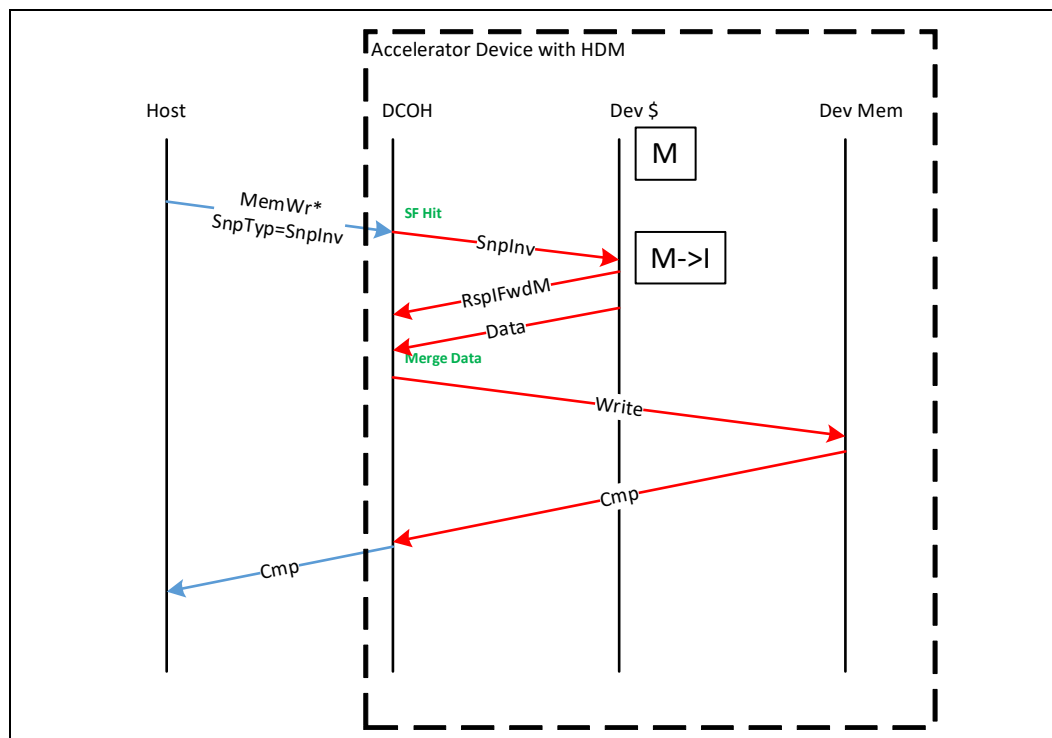
In the above example, the Host requested exclusive access to a line without requiring the device to send data. It communicates that to the device using an opcode of MemInv with a MetaValue of 10b (Any), which is significant in this case. It also asks the device to invalidate its caches with the SnpInv command. The device invalidates its caches and gives exclusive ownership to the Host as communicated using the Cmp-E semantic.

Figure 3-31. Example Flush from Host

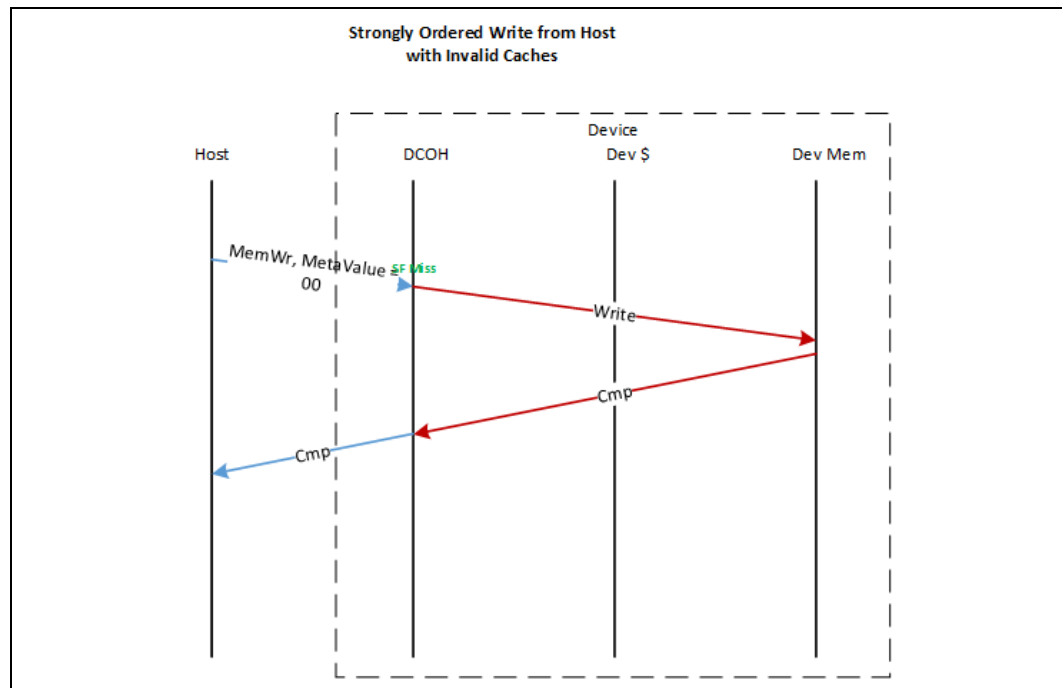


In the above example, the Host wants to flush a line from all caches, including the device’s caches, to device memory. To do so, it uses an opcode of MemInv with a MetaValue of 00b (Invalid) and a SnpInv. The device flushes its caches and returns a Cmp indication to the Host.

Figure 3-32. Example Weakly Ordered Write from Host

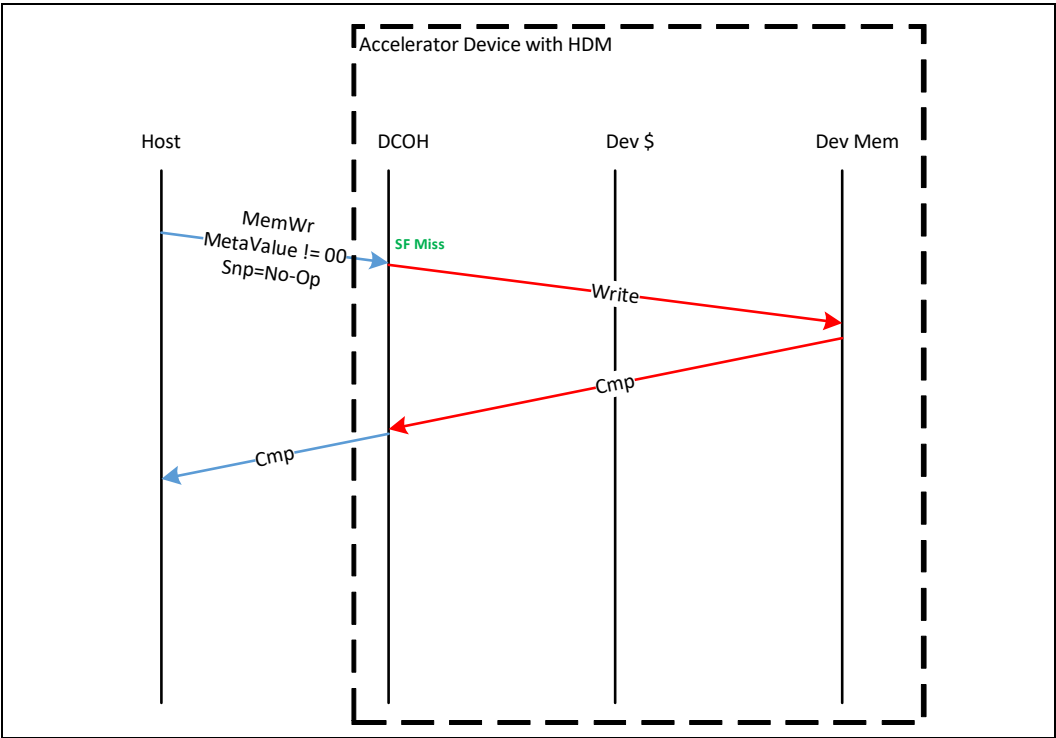


In the above example, the Host issues a weakly ordered write (partial or full line). The weakly ordered semantic is communicated by the embedded `SnpInv`. In this example, the device had a copy of the line cached. This resulted in a merge within the device before writing it back to memory and sending a `Cmp` indication to the Host. The term “weakly ordered” in this context refers to an expected-use model in the host CPU in which ordering of the data is not guaranteed until after the `Cmp` message is received. This is in contrast to a “data visibility is guaranteed with the host” CPU cache in M-state.

Figure 3-33. Example Write from Host with Invalid Host Caches

In the above example, the Host performed a write while guaranteeing to the device that it no longer has a valid cached copy of the line. The fact that the Host didn't need to snoop the device's caches means that the Host previously acquired an exclusive copy of the line. The guarantee on no valid cached copy is indicated by a MetaValue of 00b (Invalid).

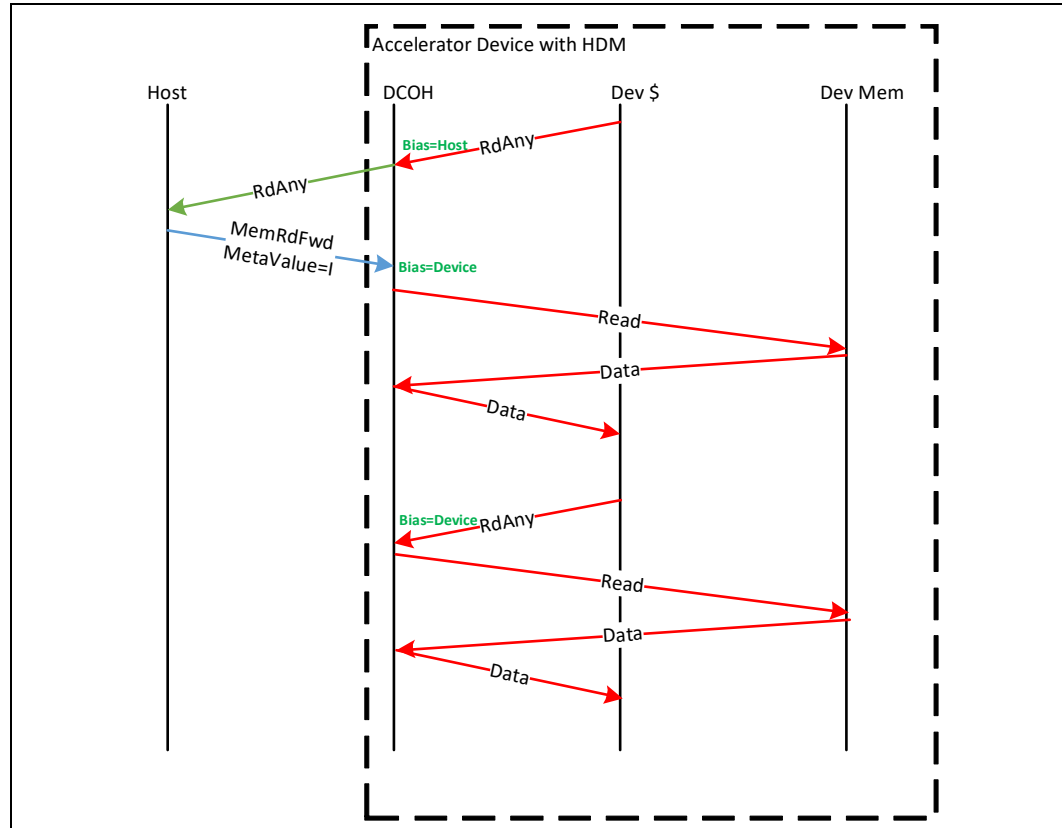
Figure 3-34. Example Write from Host with Valid Host Caches



The above example is the same as the previous one except that the Host chose to retain a valid cacheable copy of the line after the write. This is communicated to the device using a MetaValue of not 00b (Invalid).

3.5.2.3 Requests from Device in Host and Device Bias

Figure 3-35. Example Device Read to Device-attached Memory (HDM-D)



The two flows in Figure 3-35 both start with an internal CXL.cache request (RdAny) that targets the device's HDM-D address region.

In the first flow in Figure 3-35, a device read to device attached memory happened to find the line in Host bias. Because it is in Host bias, the device needs to send the request to the Host to resolve coherency. The Host, after resolving coherency, sends a MemRdFwd on CXL.mem to complete the transaction, at which point the device can internally complete the read.

In the second flow in Figure 3-35, the device read happened to find the line in Device Bias. Because it is in Device Bias, the read can be completed entirely within the device itself and a request doesn't need to be sent to the Host.

The same device request is shown in Figure 3-36, but in this case the target is the HDM-DB address region, meaning that the BISnp channel is used to resolve coherence with the host. In this flow, the difference is that the SF Hit (similar to BIAS=host) indicates that the host could have a cached copy, so BISnpData is sent to the host to resolve coherence. After the host resolves coherence, the host responds with BIRspI indicating that the host is in I-state and that the device can proceed to access its data.

Figure 3-36. Example Device Read to Device-attached Memory (HDM-DB)

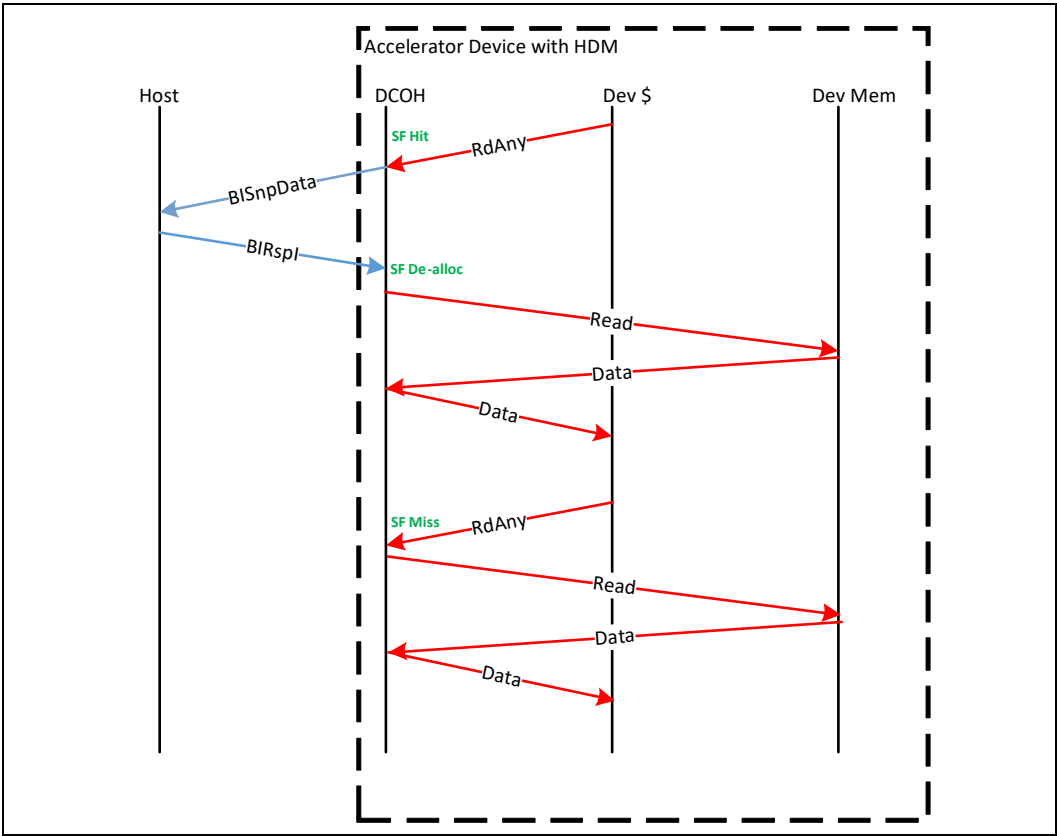
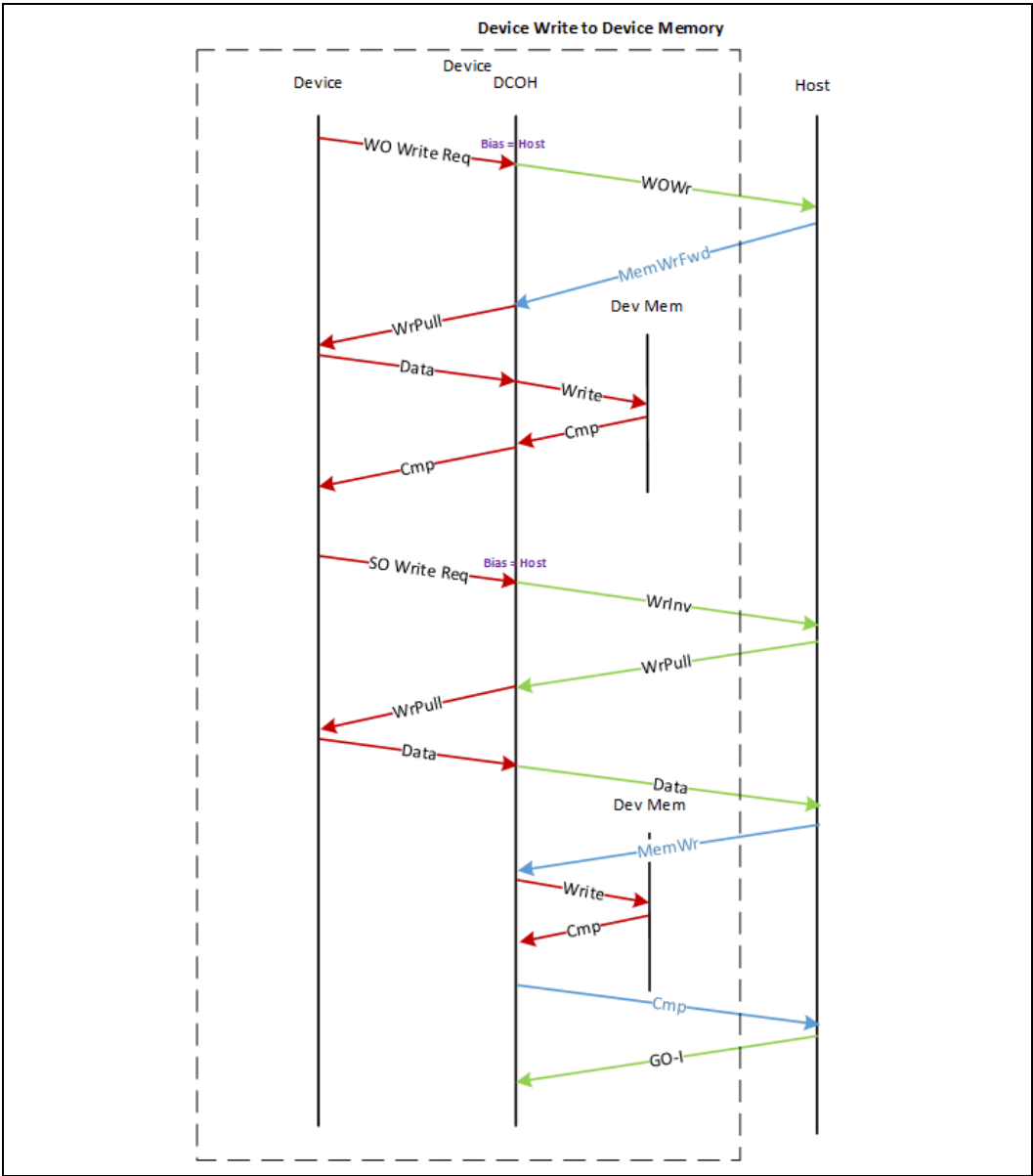


Figure 3-37. Example Device Write to Device-Attached Memory in Host Bias (HDM-D)



There are two flows shown above in Figure 3-37 for the HDM-D region. Both start with the line in Host Bias: a weakly ordered write request and a strongly ordered write request.

In the case of the weakly ordered write request, the request is issued by the device to the Host to resolve coherency. The Host resolves coherency and sends a CXL.mem MemWrFwd opcode, which carries the completion for the WOWrInv* command on CXL.cache. The CQID associated with the CXL.cache WOWrInv* command is reflected in the Tag of the CXL.mem MemWrFwd command. At this point, the device is allowed to complete the write internally. After sending the MemWrFwd, because the Host no longer prevents future accesses to the same line, this is considered a weakly ordered write.

In the second flow, the write is strongly ordered. To preserve the strongly ordered semantic, the Host can prevent future accesses to the same line while this write completes. However, as can be seen, this involves two transfers of the data across the link, which is inefficient. Unless strongly ordered writes are absolutely required, better performance can be achieved with weakly ordered writes.

Figure 3-38. Example Device Write to Device-attached Memory in Host Bias (HDM-DB)

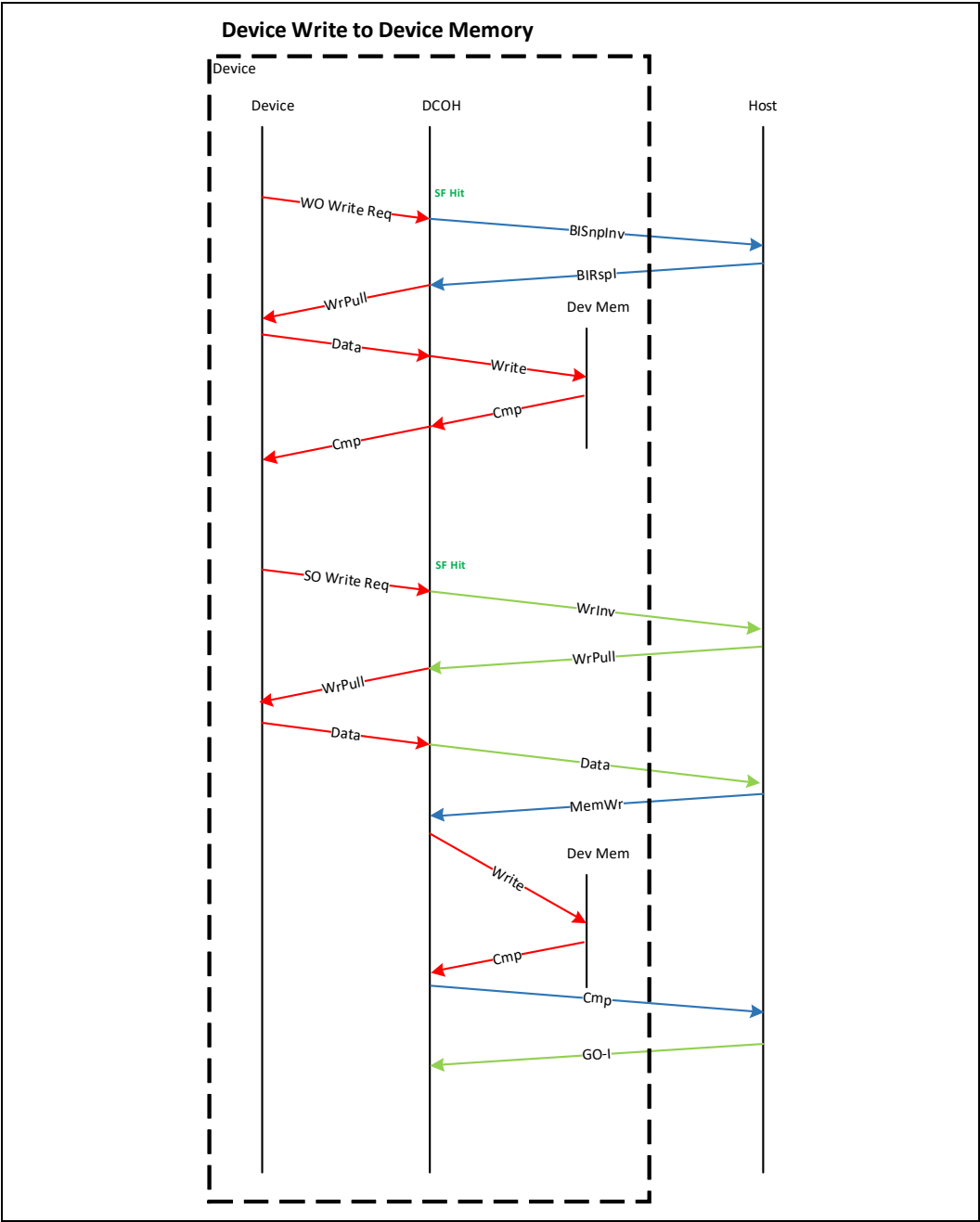
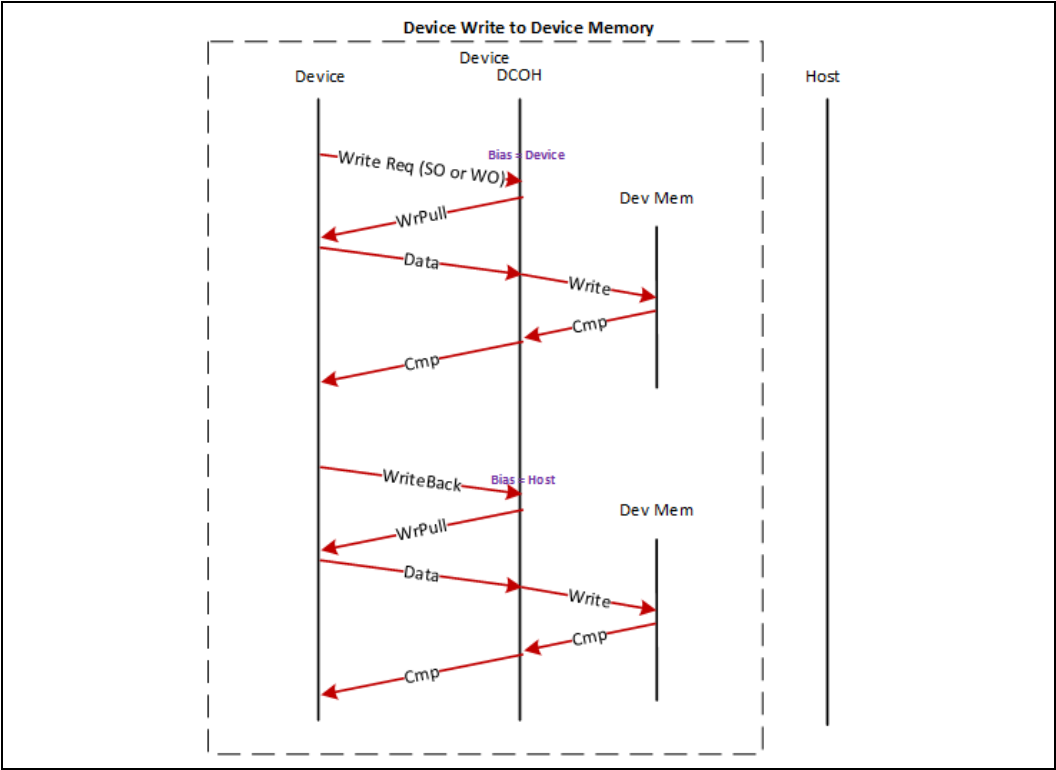


Figure 3-38 for HDM-DB is in contrast to Figure 3-37 for the HDM-D region. In the HDM-DB flow, the BISnp channel in the CXL.mem protocol is used to resolve coherence with the host for the internal weakly ordered write. The strongly ordered write follows the same flow for both HDM-DB and HDM-D.

Figure 3-39. Example Device Write to Device-attached Memory



Again, two flows are shown above in Figure 3-39. In the first case, if a weakly or strongly ordered write finds the line in Device Bias, the write can be completed entirely within the device without having to send any indication to the Host.

The second flow shows a device writeback to device-attached memory. Note that if the device is doing a writeback to device-attached memory, regardless of bias state, the request can be completed within the device without having to send a request to the Host.

The HDM-DB vs. HDM-D regions have the same basic assumption in these flows such that no interaction is required with the host.

Figure 3-40. Example Host to Device Bias Flip (HDM-D)

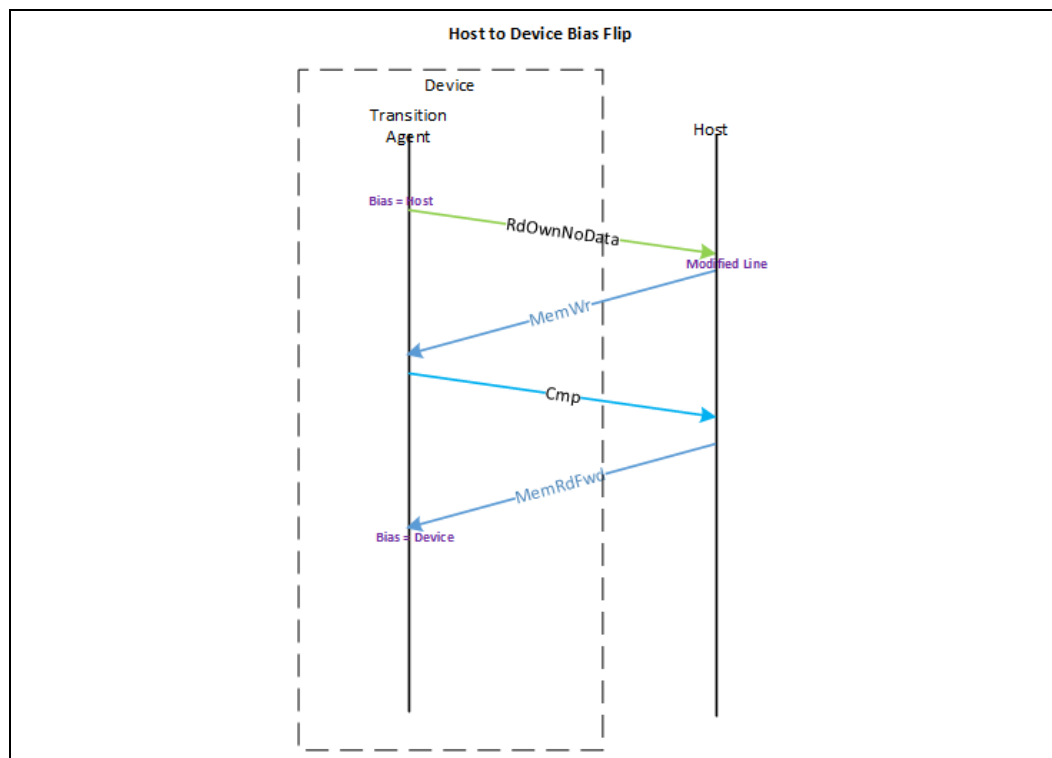


Figure 3-40 captures the “Bias Flip” flows for HDM-D memory. For the HDM-DB memory region, see [Section 3.3.3](#) for details regarding how this case is handled. Please note that the MemRdFwd will carry the CQID of the RdOwnNoData transaction in the Tag. The reason for putting the RdOwnNoData completion (MemRdFwd) on CXL.mem is to ensure that subsequent M2S Req Channel requests from the Host to the same address are ordered behind the MemRdFwd. This allows the device to assume ownership of a line as soon as the device receives a MemRdFwd without having to monitor requests from the Host.

3.5.3 Type 2 Memory Flows and Type 3 Memory Flows

3.5.3.1 Speculative Memory Read

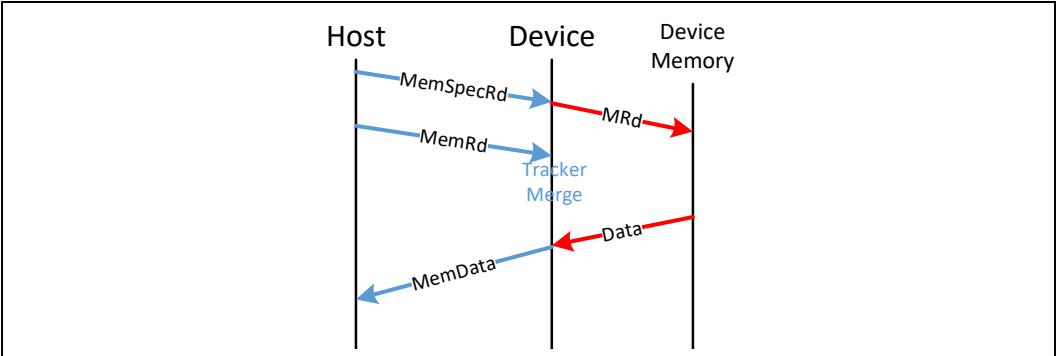
To support latency saving, CXL.mem includes a speculative memory read command (MemSpecRd) which is used to start memory access before the home agent has resolved coherence. This command does not receive a completion message and can be arbitrarily dropped. The host, after resolving coherence, may issue a demand read (i.e., MemRd or MemRdData) that the device should merge with the earlier MemSpecRd to achieve latency savings. See [Figure 3-41](#) for an example of this type of flow.

The MemSpecRd command can be observed while another memory access is in progress in the device to the same cacheline address. In this condition, it is recommended that the device drops the MemSpecRd.

To avoid performance impact, it is recommended that MemSpecRd commands are treated as low priority to avoid adding latency to demand accesses. Under loaded conditions the MemSpecRd can hurt performance because of the extra bandwidth it

consumes and should be dropped when loading of memory or loading of the CXL link is detected. QoS Telemetry data as indicated by the DevLoad field is one way that loading of memory can be detected in the host or switch.

Figure 3-41. Example MemSpecRd

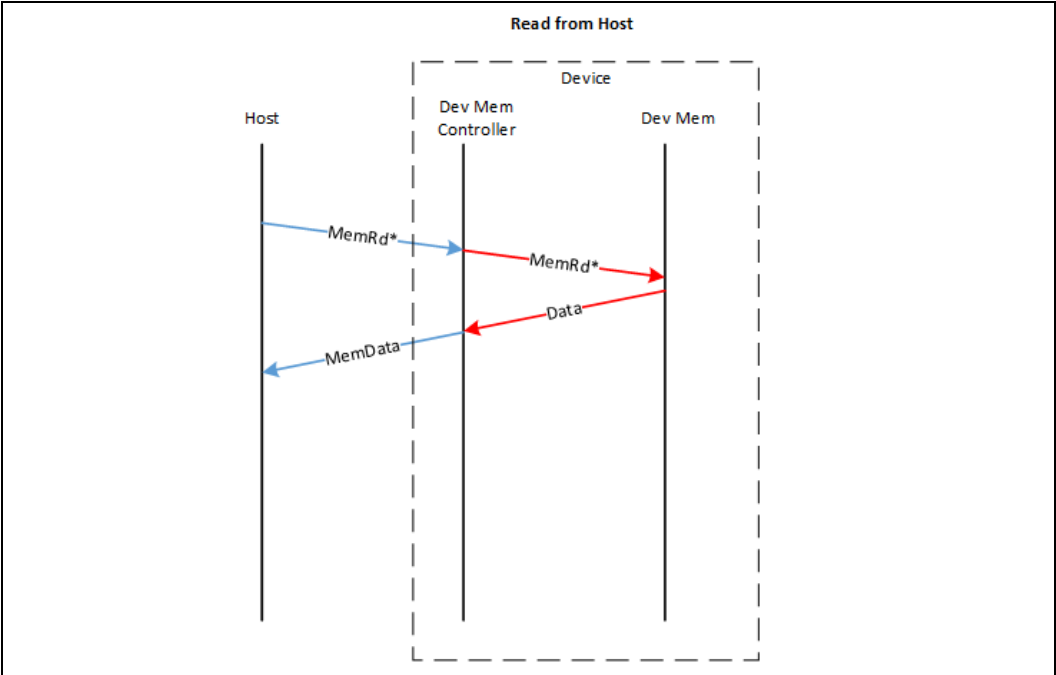


3.6 Flows to HDM-H in a Type 3 Device

The HDM-H address region in a Type 3 device is used as a memory expander or for Shared FAM device with software coherence where the device does not require active management of coherence with the Host. Thus, access to HDM-H does not use a DCOH agent. This allows the transaction flows to HDM-H to be simplified to just two classes, reads and writes, as shown below.

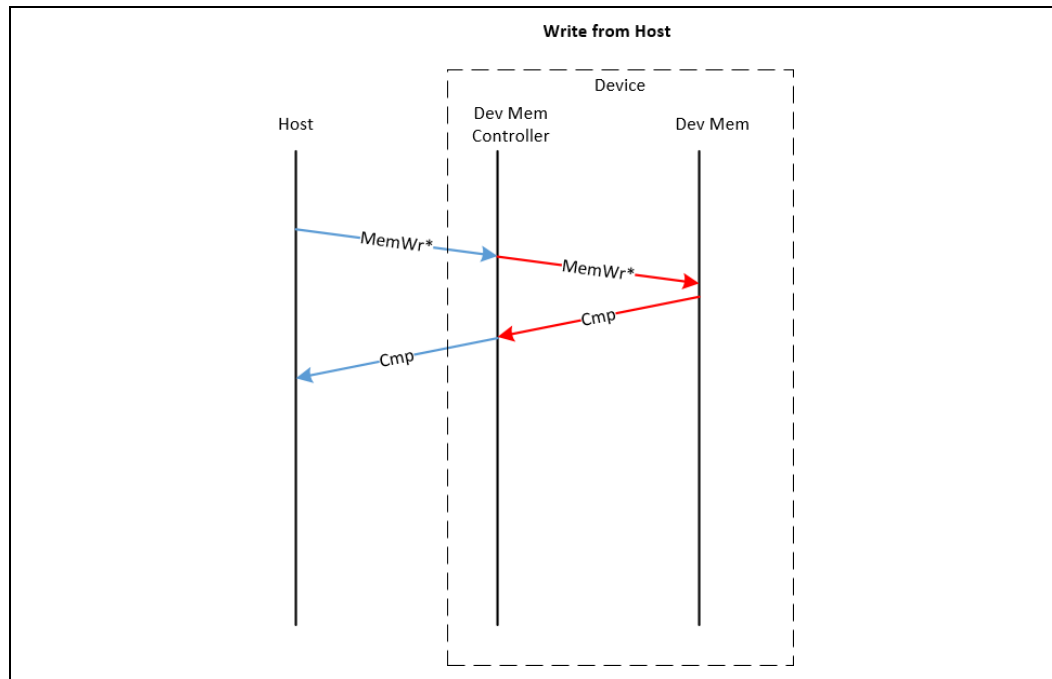
In Figure 3-42, the optimized read flow is shown for the HDM-H address region. In this flow, only a Data message is returned. In contrast, in the HDM-D/HDM-DB address region, both NDR and Data are returned. The legend shown in Figure 3-26 also applies to the transaction flows.

Figure 3-42. Read from Host to HDM-H



Unlike reads, writes to the HDM-H region use the same flow as the HDM-D/HDM-DB region and always complete with an S2M NDR Cmp message. This common write flow is shown in [Figure 3-43](#).

Figure 3-43. Write from Host to All HDM Regions



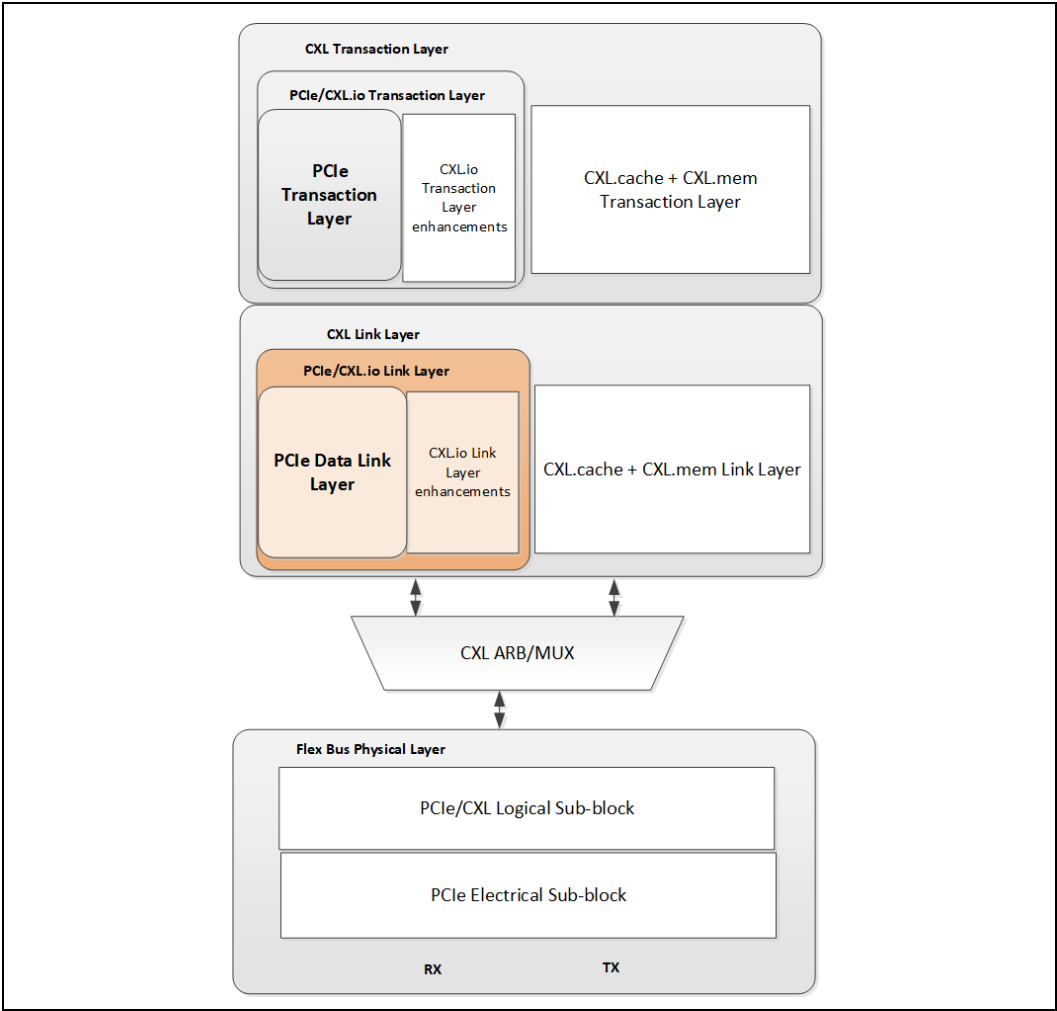
§ §

4.0 CXL Link Layers

4.1 CXL.io Link Layer

The CXL.io link layer acts as an intermediate stage between the CXL.io transaction layer and the Flex Bus Physical layer. Its primary responsibility is to provide a reliable mechanism for exchanging transaction layer packets (TLPs) between two components on the link. The PCIe* Data Link Layer is utilized as the link layer for CXL.io Link layer. Please refer to chapter titled "Data Link Layer Specification" in PCIe Base Specification for details. In 256B Flit mode, the PCIe-defined PM and Link Management DLLPs are not applicable for CXL.io and must not be used.

Figure 4-1. Flex Bus Layers - CXL.io Link Layer Highlighted



In addition, for 68B Flit mode, the CXL.io link layer implements the framing/deframing of CXL.io packets. CXL.io uses the Encoding for 8 GT/s, 16 GT/s, and 32 GT/s data rates only (see “128b/130b Encoding for 8.0 GT/s, 16.0 GT/s, and 32.0 GT/s Data Rates” in PCIe Base Specification for details).

This chapter highlights the notable framing and application of symbols to lanes that are specific for CXL.io. Note that when viewed on the link, the framing symbol-to-lane mapping will be shifted as a result of additional CXL framing (i.e., two bytes of Protocol ID and two reserved bytes) and of interleaving with other CXL protocols.

For CXL.io, only the x16 Link transmitter and receiver framing requirements described in PCIe Base Specification apply regardless of the negotiated link width. The framing related rules for $N = 1, 2, 4,$ and 8 do not apply. For downgraded Link widths, where number of active lanes is less than x16, a single x16 data stream is formed using x16 framing rules and transferred over x16/(degraded link width) degraded link width streams.

The CXL.io link layer forwards a framed I/O packet to the Flex Bus Physical layer. The Flex Bus Physical layer framing rules are defined in [Chapter 6.0](#).

For 256B Flit mode, NOP-TLP alignment rules from PCIe Base Specification for PCIe Flit mode are shifted as a result of two bytes of Flit Type at the beginning of the flit.

The CXL.io link layer must guarantee that if a transmitted TLP ends exactly at the flit boundary, there must be a subsequent transmitted CXL.io flit. Please refer to [Section 6.2.2.7](#) for more details.

4.2

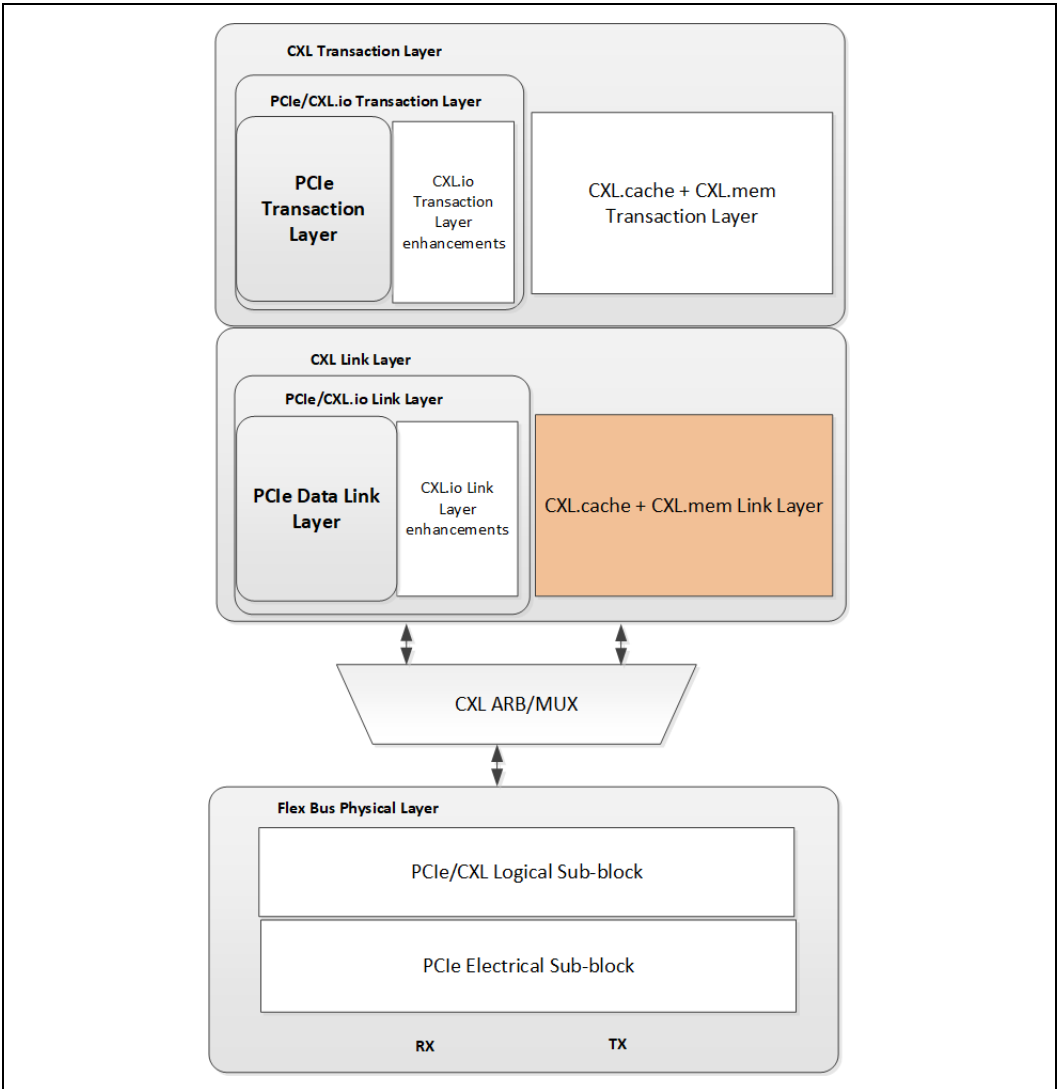
CXL.cache and CXL.mem 68B Flit Mode Common Link Layer

4.2.1

Introduction

[Figure 4-2](#) shows where the CXL.cache and CXL.mem link layer exists in the Flex Bus layered hierarchy. The link layer has two modes of operation: 68B flit and 256B flit. 68B flit, which defines 66B in the link layer and 2B in the ARB/MUX, supports the physical layer up to 32 GT/s. To support higher speeds a flit definition of 256B is defined; the reliability flows for that flit definition are handled in the Physical layer, so retry flows from 68B Flit mode are not applicable. 256B flits can support any legal transfer rate, but are required for >32 GT/s. The 256B flit definition and requirements are captured in [Section 4.3](#). There are Transaction Layer features that require 256B flits and those features include CacheID, Back-Invalidate Snoop (BISnp), and Port Based Routing (PBR).

Figure 4-2. Flex Bus Layers - CXL.cache + CXL.mem Link Layer Highlighted



As previously mentioned, CXL.cache and CXL.mem protocols use a common Link Layer. This chapter defines the properties of this common Link Layer. Protocol information, including definition of fields, opcodes, transaction flows, etc., can be found in [Section 3.2](#) and [Section 3.3](#), respectively.

4.2.2 High-Level CXL.cachemem Flit Overview

The CXL.cachemem flit size is a fixed 528b. There are 2B of CRC code and 4 slots of 16B each as shown below.

Figure 4-3. CXL.cachemem Protocol Flit Overview

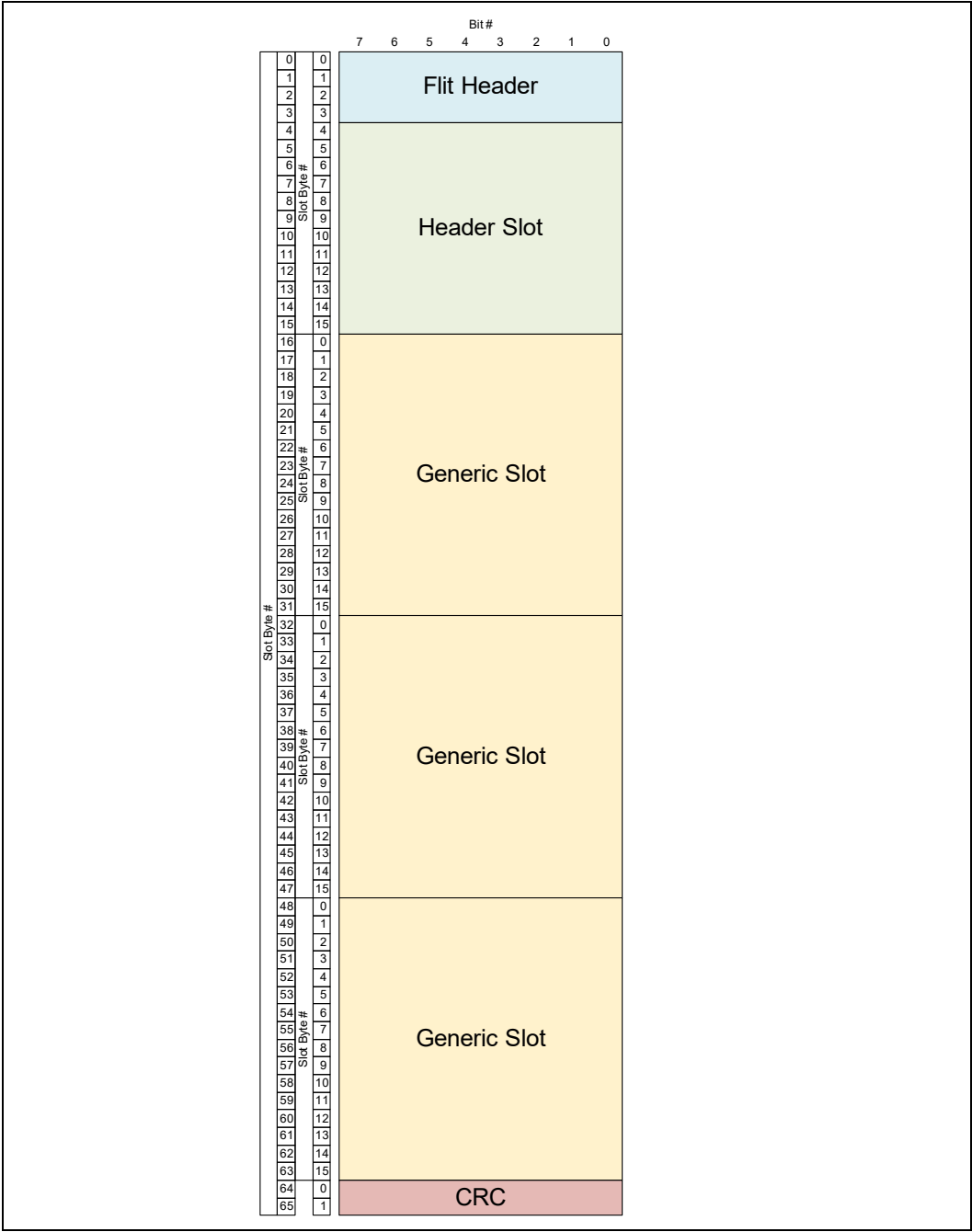
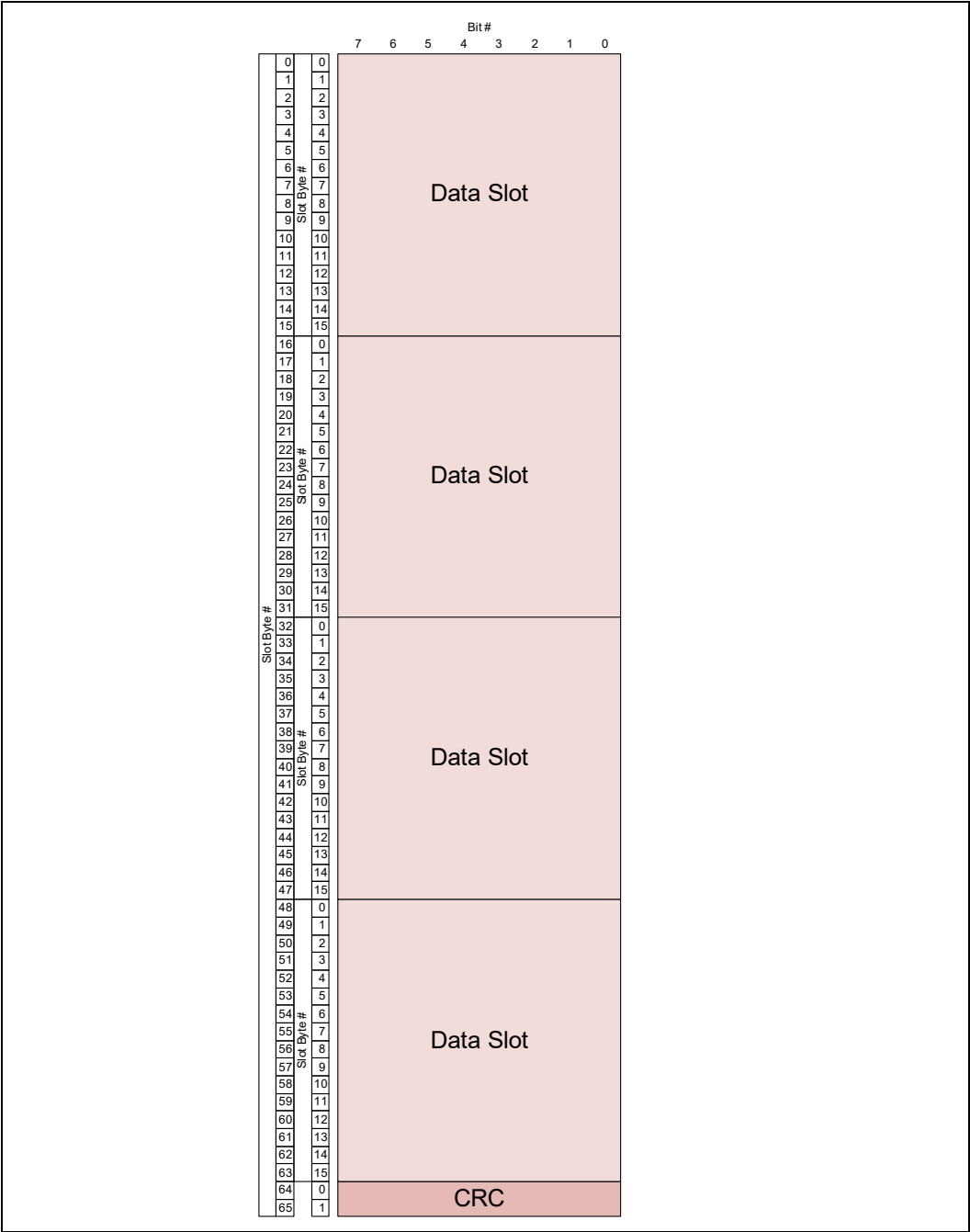
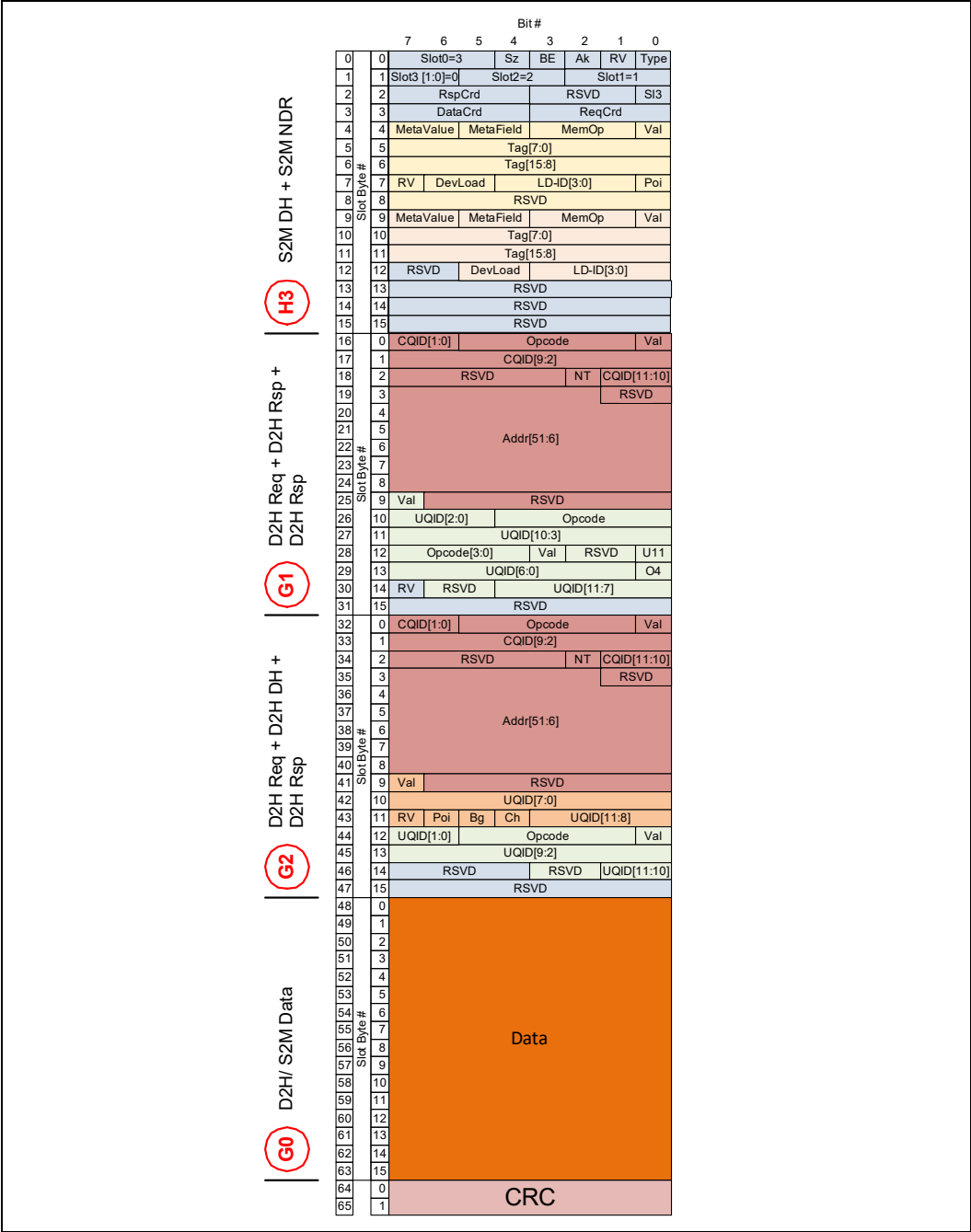


Figure 4-4. CXL.cachemem All Data Flit Overview



An example of a Protocol flit in the device to Host direction is shown below. For detailed descriptions of slot formats, see [Section 4.2.3](#).

Figure 4-5. Example of a Protocol Flit from Device to Host



A "Header" Slot is defined as one that carries a "Header" of link-layer specific information, including the definition of the protocol-level messages contained in the remainder of the header as well as in the other slots in the flit.

A “Generic” Slot can carry one or more request/response messages or a single 16B data chunk.

The flit can be composed of a Header Slot and 3 Generic Slots or four 16B Data Chunks.

The Link Layer flit header uses the same definition for both the Upstream Ports, as well as the Downstream Ports, as summarized in [Table 4-1](#).

Table 4-1. CXL.cachemem Link Layer Flit Header Definition

Field Name	Brief Description	Length in Bits
Type	This field distinguishes between a Protocol or a Control flit.	1
Ak	This is an acknowledgment of 8 successful flit transfers. Reserved for RETRY, and for INIT control flits.	1
BE	Byte Enable (Reserved for control flits).	1
Sz	Size (Reserved for control flits).	1
ReqCrd	Request Credit Return. Reserved for RETRY, and for INIT control flits.	4
DataCrd	Data Credit Return. Reserved for RETRY, and for INIT control flits.	4
RspCrd	Response Credit Return. Reserved for RETRY, and for INIT control flits.	4
Slot 0	Slot 0 Format Type (Reserved for control flits).	3
Slot 1	Slot 1 Format Type (Reserved for control flits).	3
Slot 2	Slot 2 Format Type (Reserved for control flits).	3
Slot 3	Slot 3 Format Type (Reserved for control flits).	3
RSVD	Reserved	4
Total		32

In general, bits or encodings that are not defined will be marked “Reserved” or “RSVD” in this specification. These bits should be cleared to 0 by the sender of the packet and the receiver should ignore them. Please also note that certain fields with static 0/1 values will be checked by the receiving Link Layer when decoding a packet. For example, Control flits have several static bits defined. A Control flit that passes the CRC check but fails the static bit check should be treated as a standard CRC error or as a fatal error when in “RETRY_LOCAL_NORMAL state of the LRSM. Logging and reporting of such errors is device specific. Checking of these bits reduces the probability of silent error under conditions where the CRC check fails to detect a long burst error. However, link layer must not cause fatal error whenever it is under shadow of CRC errors (i.e., its LRSM is not in RETRY_LOCAL_NORMAL state). This is prescribed because all-data-flit can alias to control messages after a CRC error and those alias cases may result in static bit check failure.

The following describes how the flit header information is encoded.

Table 4-2. Type Encoding

Value	Flit Type	Description
0	Protocol	This is a flit that carries CXL.cache or CXL.mem protocol-related information.
1	Control	This is a flit inserted by the link layer only for link layer-specific functionality. These flits are not exposed to the upper layers.

The Ak field is used as part of the link layer retry protocol to signal CRC-passing receipt of flits from the remote transmitter. The transmitter sets the Ak bit to acknowledge successful receipt of 8 flits; a cleared Ak bit is ignored by the receiver.

The BE (Byte Enable) and Sz (Size) fields have to do with the variable size of data messages. To reach its efficiency targets, the CXL.cachemem link layer assumes that generally all bytes are enabled for most data, and that data is transmitted at the full cacheline granularity. When all bytes are enabled, the link layer does not transmit the byte enable bits, but instead clears the Byte Enable field of the corresponding flit header. When the receiver decodes that the Byte Enable field is cleared, it must regenerate the byte enable bits as all 1s before passing the data message on to the transaction layer. If the Byte Enable bit is set, the link layer Rx expects an additional data chunk slot that contains byte enable information. Note that this will always be the last slot of data for the associated request.

Similarly, the Sz field reflects the fact that the CXL.cachemem protocol allows transmission of data at the half cacheline granularity. When the Size bit is set, the link layer Rx expects four slots of data chunks, corresponding to a full cacheline. When the Size bit is cleared, it expects only two slots of data chunks. In the latter case, each half cacheline transmission will be accompanied by its own data header. A critical assumption of packing the Size and Byte Enable information in the flit header is that the Tx flit packet may begin at most one data message per flit.

Note: Multi-Data-Headers are not allowed to be sent when Sz=0 or BE=1 as described in the flit packing rules in [Section 4.2.5](#).

[Table 4-3](#) describes legal values of Sz and BE for various data transfers. For cases where a 32B split transfer is sent that includes Byte Enables, the trailing Byte Enables apply only to the 32B sent. The Byte Enable bits that are applicable to that transfer are aligned based on which half of the cacheline is applicable to the transfer (BE[63:32] for Upper half of the cacheline or BE[31:0] for the lower half of the cacheline). This means that each of the split 32B transfers that are used to form a cacheline of data will include Byte Enables if Byte Enables are needed. Illegal use will cause an uncorrectable error. The reserved bits included in the BE slot may not be preserved when passing through a switch.

Table 4-3. Legal Values of Sz and BE Fields

Type of Data Transfer	32B Transfer Permitted in 68B Flit? ¹	BE Permitted?
CXL.cache H2D Data	Yes	No
CXL.mem M2S Data	No	Yes
CXL.cache D2H Data	Yes	Yes
CXL.mem S2M Data	Yes	No

1. The 32B transfer allowance is only defined for 68B flit definition and does not apply for 256B flit.

The transmitter sets the CRD fields to indicate freed resources that are available in the co-located receiver for use by the remote transmitter. Credits are given for transmission per message class, which is why the flit header contains independent Request, Response, and Data CRD fields. Note that there are no Requests sourced in the S2M direction, and that there are no Responses sourced in the M2S direction. Details of the channel mapping are captured in [Table 4-5](#). Credits returned for channels not supported by the device or the host should be silently discarded. The granularity of credits is per message. These fields are encoded exponentially, as delineated in [Table 4-4](#).

Note: Messages sent on Data channels require a single data credit for the entire message. This means that 1 credit allows for one data transfer, including the header of the message, regardless of whether the transfer is 64B, or 32B, or contains Byte Enables.

The transaction layer requires all messages that carry payload to send 64B and the link layer allows for those to be sent as independent 32B messages to optimize latency for implementation-specific cases in which only 32B of data is ready to send.

Table 4-4. CXL.cachemem Credit Return Encodings

Credit Return Encoding[3]	Protocol
0	CXL.cache
1	CXL.mem
Credit Return Encoding[2:0]	Number of Credits
000b	0
001b	1
010b	2
011b	4
100b	8
101b	16
110b	32
111b	64

Table 4-5. ReqCrd/DataCrd/RspCrd Channel Mapping

Credit Field	Credit Bit 3 Encoding	Link Direction	Channel
ReqCrd	0 - CXL.cache	Upstream	D2H Request
		Downstream	H2D Request
	1 - CXL.mem	Upstream	Reserved
		Downstream	M2S Request
DataCrd	0 - CXL.cache	Upstream	D2H Data
		Downstream	H2D Data
	1 - CXL.mem	Upstream	S2M DRS
		Downstream	M2S Rwd
RspCrd	0 - CXL.cache	Upstream	D2H Rsp
		Downstream	H2D Rsp
	1 - CXL.mem	Upstream	S2M NDR
		Downstream	Reserved

Finally, the Slot Format Type fields encode the Slot Format of both the header slot and of the other generic slots in the flit (if the Flit Type bit specifies that the flit is a Protocol flit). The subsequent sections detail the protocol message contents of each slot format, but [Table 4-6](#) provides a quick reference for the Slot Format field encoding.

Note:

Format H6 is defined for use with Integrity and Data Encryption. See details of requirements for its use in [Chapter 11.0](#).

Table 4-6. Slot Format Field Encoding

Slot Format Encoding	H2D/M2S		D2H/S2M	
	Slot 0	Slots 1, 2, and 3	Slot 0	Slots 1, 2, and 3
000b	H0	G0	H0	G0
001b	H1	G1	H1	G1
010b	H2	G2	H2	G2
011b	H3	G3	H3	G3
100b	H4	G4	H4	G4
101b	H5	G5	H5	G5
110b	H6	RSVD	H6	G6
111b	RSVD	RSVD	RSVD	RSVD

Table 4-7 and Table 4-8 describe the slot format and the type of message contained by each format for both directions.

Table 4-7. H2D/M2S Slot Formats

Format to Req Type Mapping	H2D/M2S	
	Type	Length in Bits
H0	CXL.cache Req + CXL.cache Rsp	96
H1	CXL.cache Data Header + 2 CXL.cache Rsp	88
H2	CXL.cache Req + CXL.cache Data Header	88
H3	4 CXL.cache Data Header	96
H4	CXL.mem Rd Header	87
H5	CXL.mem Req Only	87
H6	MAC slot used for link integrity	96
G0	CXL.cache/ CXL.mem Data Chunk	128
G1	4 CXL.cache Rsp	128
G2	CXL.cache Req + CXL.cache Data Header + CXL.cache Rsp	120
G3	4 CXL.cache Data Header + CXL.cache Rsp	128
G4	CXL.mem Req + CXL.cache Data Header	111
G5	CXL.mem Rd Header + CXL.cache Rsp	119

Table 4-8. D2H/S2M Slot Formats

Format to Req Type Mapping	D2H/S2M	
	Type	Length in Bits
H0	CXL.cache Data Header + 2 CXL.cache Rsp + CXL.mem NDR	87
H1	CXL.cache Req + CXL.cache Data Header	96
H2	4 CXL.cache Data Header + CXL.cache Rsp	88
H3	CXL.mem DRS Header + CXL.mem NDR	70
H4	2 CXL.mem NDR	60
H5	2 CXL.mem DRS Header	80
H6	MAC slot used for link integrity	96
G0	CXL.cache/ CXL.mem Data Chunk	128
G1	CXL.cache Req + 2 CXL.cache Rsp	119
G2	CXL.cache Req + CXL.cache Data Header + CXL.cache Rsp	116
G3	4 CXL.cache Data Header	68
G4	CXL.mem DRS Header + 2 CXL.mem NDR	100
G5	2 CXL.mem NDR	60
G6	3 CXL.mem DRS Header	120

4.2.3 Slot Format Definition

Slot diagrams in this section include abbreviations for bit field names to allow them to fit into the diagram. In the diagrams, most abbreviations are obvious, but the following abbreviation list ensures clarity:

- Bg = Bogus
- Ch = ChunkValid
- LA0 = LowerAddr[0]
- LA1 = LowerAddr[1]
- LI3 = LD-ID[3]
- MV0 = MetaValue[0]
- MV1 = MetaValue[1]
- O4 = Opcode[4]
- Op0 = Opcode[0]
- Poi = Poison
- R11 = RspData[11]
- RSVD = Reserved
- RV = Reserved
- SL3 = Slot3[2]
- Tag15 = Tag[15]
- U11 = UQID[11]
- Val = Valid

4.2.3.1 H2D and M2S Formats

Figure 4-6. H0 - H2D Req + H2D Rsp

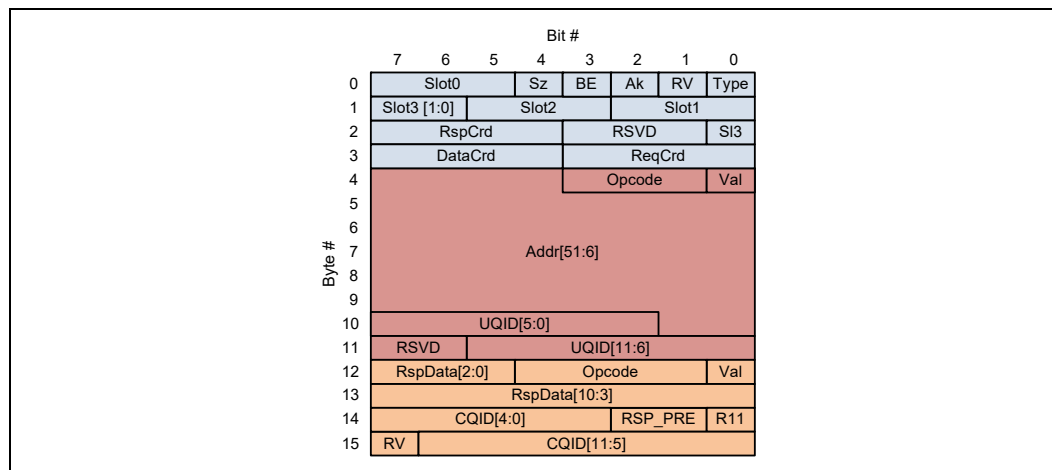


Figure 4-7. H1 - H2D Data Header + H2D Rsp + H2D Rsp

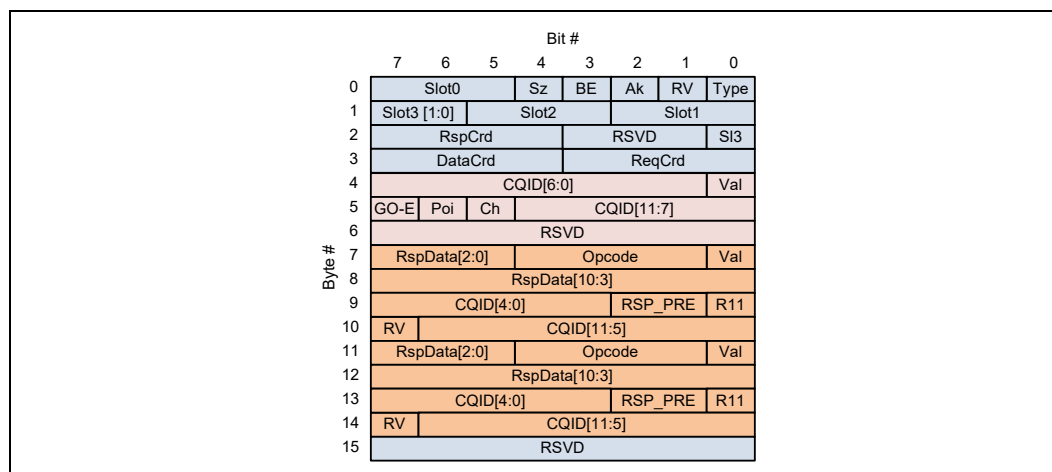


Figure 4-8. H2 - H2D Req + H2D Data Header

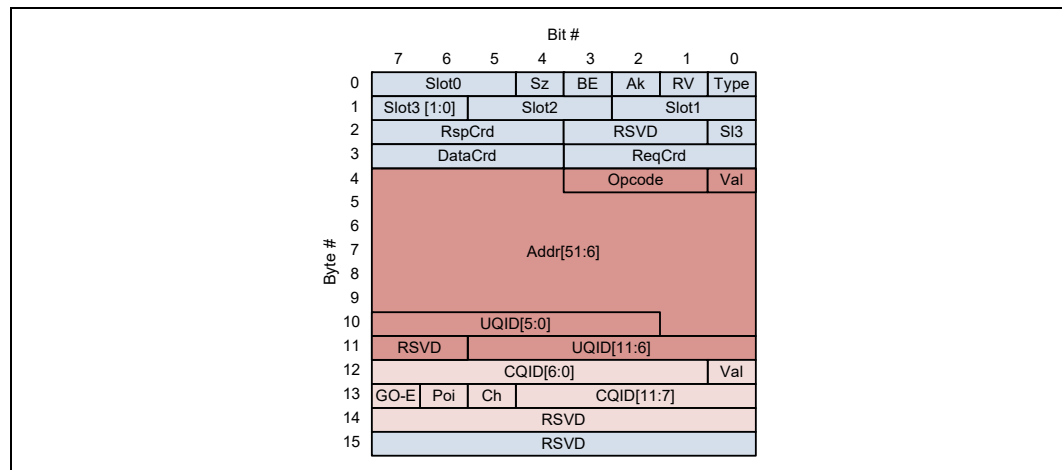


Figure 4-9. H3 - 4 H2D Data Header

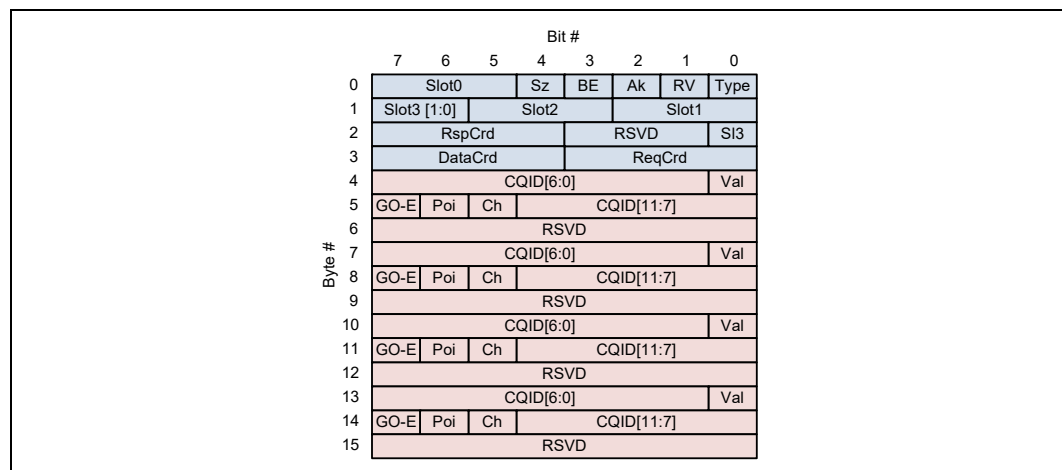


Figure 4-10. H4 - M2S Rwd Header

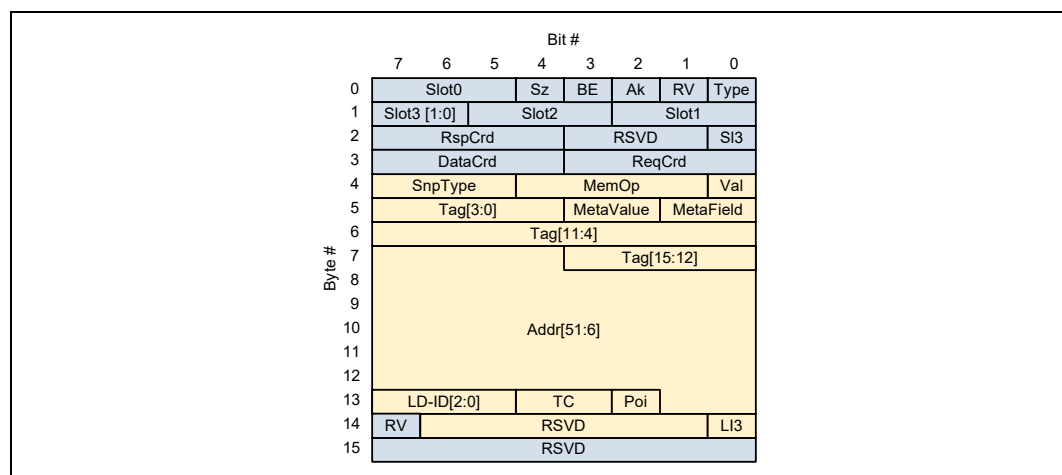


Figure 4-11. H5 - M2S Req

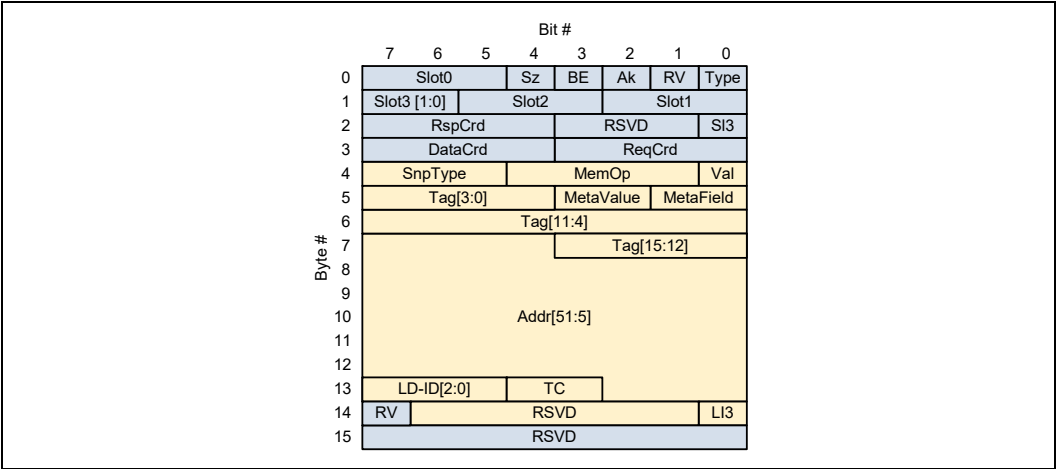


Figure 4-12. H6 - MAC

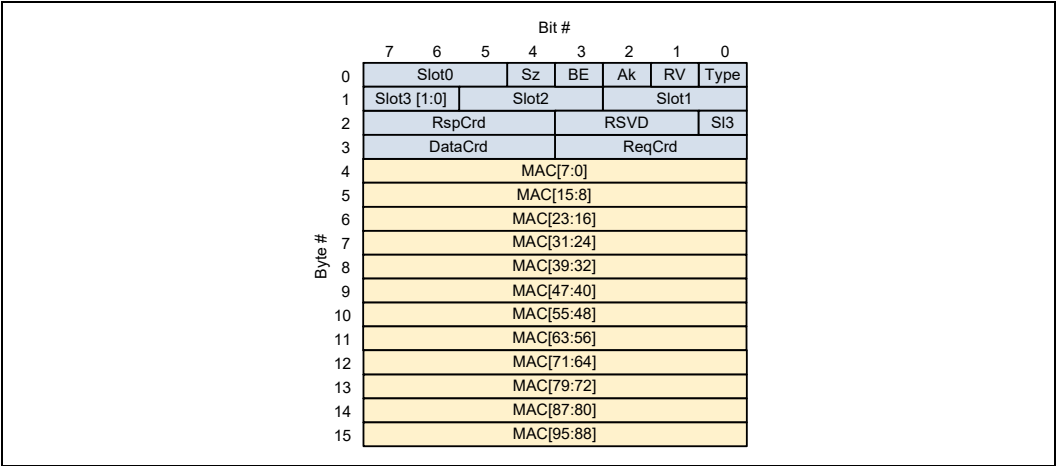


Figure 4-13. G0 - H2D/M2S Data

